AUTOMATA FORMAL LANGUAGES AND LOGIC

Lecture Notes
on
Sets

Prepared by :
Prof. Preet Kanwal
Associate Professor

Department of Computer Science & Engineering
PES UNIVERSITY
(Established under Karnataka Act No.16 of 2013)
100-ft Ring Road, BSK III Stage, Bangalore – 560 085

Table of Contents

1.What is a Set?

Why Sets :

Sets are the fundamental property of mathematics. Now as a word of warning, sets, by themselves, seem pretty pointless. But it's only when we apply sets in different situations do they become the powerful building block of mathematics that they are.

Math can get amazingly complicated quite fast. Graph Theory, Abstract Algebra, Real Analysis, Complex Analysis, Linear Algebra, Number Theory, and the list goes on. But there is one thing that all of these share in common: Sets.

Definition :

A set is an unordered collection of distinct objects, which may be anything (including other sets).

For Example : Here, we have a set of Indian Coins.

We represent Sets in Curly braces with commas separating out the elements

2. Representing Sets

We can represent Sets in 3 different ways namely,

1. Descriptive Form
2. Set-builder Notation
3. Roster Form

1) Descriptive form:

In descriptive form we provide an english description of what is contained in the set. For example : “The set of all even numbers “ or “The set of all real numbers less than 150 and so on”

2) Set builder notation :

Set-builder notation is a mathematical notation for describing a set by enumerating its elements or stating the properties that its members must satisfy.

Set-builder notation has three parts: a variable, a colon or vertical bar separator, and a logical rule. These three parts are contained in curly brackets.

Here if we are talking about set of even natural numbers, in set builder notation we can specify it as:
The set of all n, where or we can say such that

The set of all n such that n is a natural number and n is even.

3) Roster form:

In the roster form, we list all elements of a set inside a pair of braces

For example,

Let say we define A as set of all x where x is an integer and its values is greater than and equal to -1 but less than 5 }

then In roster form we can say A = { -1, 0,1, 2, 3, 4 }

3. Set Membership

Given a set S and an object x, we say x is contained in S, if x is one of the elements of S and is denoted by,

$$x \in S$$

x is not a member of S if x is not an element of S and is denoted by,

$$x \notin S$$

4. Order of Sets / Cardinality :

The order of a set or cardinality defines the number of elements a set is having. It describes the size of a set.

For Examples:

1) if Set A = {a, b, c, d, e}

then order of A is 5

2) Here Set B is a set of Sets {{a, b}, {c, d, e, f, g}, {h}} and the size of B = 3

3) if Set C = $\{ n \in N \mid n < 137\}$

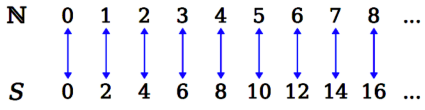
then |C| = 137

For finite sets the order (or cardinality) is the number of elements

For infinite sets, all we can say is that the order is infinite.. We can ask What is |N|? where N denotes a set of natural number? There are infinitely many natural numbers. Hence order of N can't be a natural number, since it's infinitely large.

Consider the set S, where S is the set of all n such that n is a natural number and is even then, What is order of S? Is it less than or equal or greater than the order of N? Basically we are boiling down to the question , Which infinity is larger?

if we see for every natural number n we can have a corresponding even number 2n, this means the order of S and N is same but can't be a natural number, since it's infinitely large.

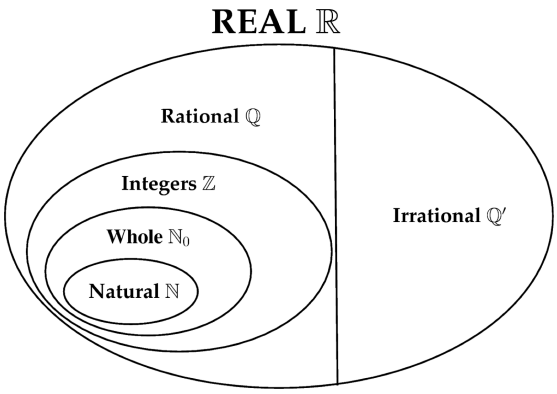


$$n \leftrightarrow 2n$$

$$S = \{ n \mid n \in \mathbb{N} \text{ and } n \text{ is even} \}$$

$$|S| = |\mathbb{N}| = \aleph_0$$

Also, There are infinitely many whole numbers {0,1,2,3,4,…}, But there are more real numbers (such as 12.308 or 1.1111115) because there are infinitely many possible variations after the decimal place as well. But both are infinite, so the question is Do all infinite sets have the same cardinality?



Oddly enough, we can say with sets that some infinities are larger than others, but this is a more advanced topic in sets, we will talk about this in detail in Unit 4.

5.Types of Sets :

In this section we discuss about the following types of Sets:

13. Empty Set or Null Set
14. Singleton Set
15. Finite Set
16. Infinite Set
17. Equivalent Set
18. Equal Sets
19. Disjoint Sets
20. Subsets
21. Proper Subset
22. Super Set
23. Universal Set
24. Power Set

1) Empty Set : A null set or an empty set is a valid set with no member. We can use empty braces({ }) or ϕ to denote an empty set.

Does $\phi = \{\phi\}$?
ϕ indicates an empty set , this set contains no elements. Whereas $\{\phi\}$ indicates a set that has one element which happens be an empty set.

Hence they are not equal.

Does 2 = {2}?
2 indicates that it is a number, whereas {2} indicates It is a set that contains a number.
Hence they are not equal.

2) Singleton Set : If a set contains only one element it is called to be a singleton set.

Example :

Set A = {1}
Set B ={a}
Set C ={4}
Set D ={car}

3) Finite Sets : A set in which the number of elements are finite (can be counted) are called Finite Sets.
Example

Set A = {4, 8, 12, 16}

Set B = {90, 180}

4) Infinite Sets : A set in which the number of elements are infinite are called Infinite Sets.

Example

N = Set of all natural no's

= {1, 2, 3, … }

Z = Set of all integers

= {..,-3,-2,-1,0,1,2,3..}

5) Equivalent Sets If the number of elements are same for two different sets, then they are called equivalent sets.

Example:

If A = {1,2,3,4} and

B = {Red, Blue, Green, Black}

Here, Set A and B are equivalent,

since |A| = |B|= 4

6) Equal Sets : The two sets A and B are said to be equal if they have exactly the same elements, order of elements do not matter.

Example:

if A = {1,2,3,4} and

B = {4,3,2,1}

Then, Set A and B are equal

& can be denoted as :

A = B

7) Disjoint Sets : The two sets A and B are said to be disjoint if the set does not contain any common element.

Example:

Set A = {1,2,3,4} and

Set B = {5,6,7,8} are disjoint sets, because there is no common element between them.

8) Subsets : A set S is a subset of a set T (denoted $S \subseteq T$) if all elements of S are also elements of T.

Examples:

1. $\{ 1, 2, 3 \} \subseteq \{ 1, 2, 3, 4 \}$
2. $N \subseteq Z$ (every natural number is an integer)
3. $Z \subseteq R$ (every integer is a real number)

We can ask ourselves a question whether there exists any Set S where Ø is a subset of S?

Since the empty set has no elements, ϕ is a subset of every set. Infact all the sets are subsets of itself

9) Proper Subset : If B is a proper subset of A (denoted as $B \subset A$), then all elements of B are in A but A contains atleast one element that is not in B.

Example :
A = {1, 3, 5}
B = { 1, 5}
C = { 1, 3, 5}

then,
$B \subset A$
But, $C \subseteq A$

10) Superset
if $B \subseteq A$ (B is a subset of A) then,
$A \supseteq B$ (A is a superset of B)

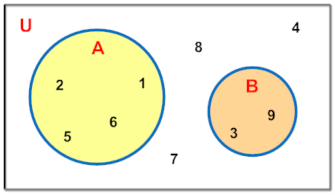
Here,
if $B \subset A$ (B is a proper subset of A) then,
$A \supset B$ (A is a proper superset of B)

Consider Example :
$A = \{1, 3, 5\}$
$B = \{ 1, 5\}$
$C = \{ 1, 3, 5\}$

Here, $A \supseteq C$ or $C \supseteq A$
and $A \supset B$

So we can say every set is a subset and superset of itself.

11) Universal Set : A universal set is a set of all the elements, or members, of any group under consideration, including its own elements. In the given example U contains elements that belong to Set A and B, plus the elements 7, 8 and 4.



12) Power Set : Power set of any set S is the set of all subsets of S, including the empty set and S itself.

For example,

Consider a Set S = {a, b, c, d} then Power Set of S would be

P(S) = {Ø,{a}, {b}, {c}, {d},{a, b}, {a, c}, {a, d}, {b, c},

{b, d}, {c, d},

{a, b, c}, {a, b, d},

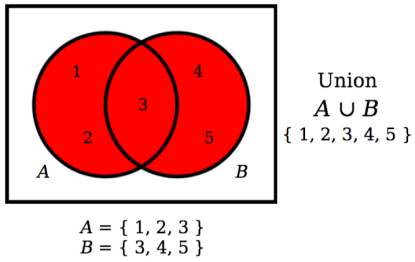
{a, c, d}, {b, c, d},

{a, b, c, d}}

|S| < |P(S)|

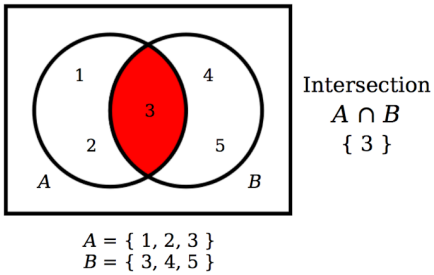
6. Operations on Sets

1)Union : Union of two sets which will contain elements from both the sets.



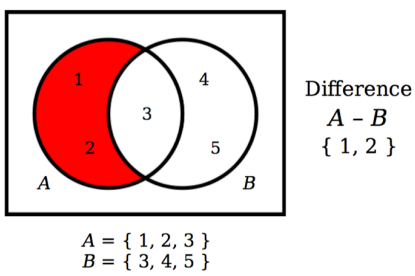
$$A \cup B = \{ x : x \in A \text{ or } x \in B\}$$

2) Intersection : Intersection of two sets will only contain elements that belong to both the sets.



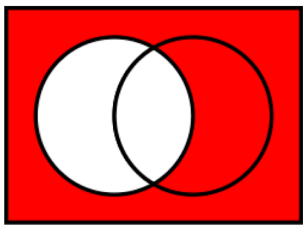
$$A \cap B = \{ x : x \in A \text{ and } x \in B\}$$

3) Difference : Difference between Set A and Set B will only contain those elements that belong to Set A, any element in common will be discarded.



$$A - B = \{ x : x \in A \text{ and } x \notin B\}$$

4) Complement : Complement of a Set A contains all the elements which do not belong to A.



$A' = \{ x : x \in U \text{ and } x \notin A\}$

7. List of Set Identities:

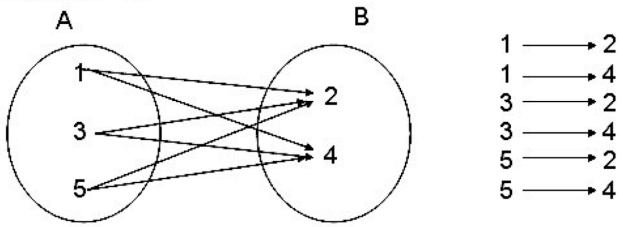
Identity	*Name*
$A \cap U = A$ $A \cup \emptyset = A$	Identity laws
$A \cup U = U$ $A \cap \emptyset = \emptyset$	Domination laws
$A \cup A = A$ $A \cap A = A$	Idempotent laws
$\overline{(\overline{A})} = A$	Complementation law
$A \cup B = B \cup A$ $A \cap B = B \cap A$	Commutative laws
$A \cup (B \cup C) = (A \cup B) \cup C$ $A \cap (B \cap C) = (A \cap B) \cap C$	Associative laws
$A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$ $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$	Distributive laws
$\overline{A \cap B} = \overline{A} \cup \overline{B}$ $\overline{A \cup B} = \overline{A} \cap \overline{B}$	De Morgan's laws
$A \cup (A \cap B) = A$ $A \cap (A \cup B) = A$	Absorption laws
$A \cup \overline{A} = U$ $A \cap \overline{A} = \emptyset$	Complement laws

8. Cartesian Product of two sets

The Cartesian product of two sets A and B, denoted A × B, is the set of all possible ordered pairs where ,

the elements of A are first and
the elements of B are second.
A X B = { (a, b) : a ∈ A and b ∈ B}
A = {1, 3, 5} and B ={2, 4} then,
A X B = { (a, b) : a ∈ A and b ∈ B}
A X B = { (1,2), (1,4), (3,2), (3, 4), (5,2), (5,4) }

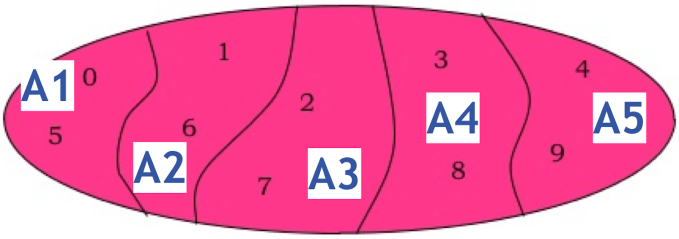
Arrow Diagram of A ×B :



9. Partition of a Set :

A partition of a set is a grouping of its elements into non-empty subsets, in such a way that every element is included in exactly one subset.

For example the regions A1, A2, A3, A4, A5 form partitions of Set A as shown below



A1, A2, A3, A4, A5 are disjoint subsets of A.

A = A1 U A2 U A3 U A4 U A5

AUTOMATA FORMAL LANGUAGES AND LOGIC

Lecture Notes
on
Functions & Relations

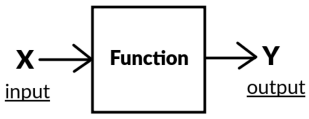
Prepared by :
Prof. Preet Kanwal
Associate Professor

Department of Computer Science & Engineering
PES UNIVERSITY
(Established under Karnataka Act No.16 of 2013)
100-ft Ring Road, BSK III Stage, Bangalore – 560 085

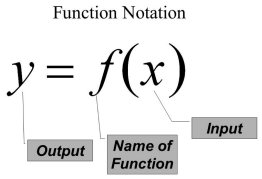
Table of Contents

1.What is a Function?

Functions are central to mathematics. A function can be thought of as a rule that assigns a set of inputs to a set of outputs.



We use the notation f(x) where f is the function (a general notation) x is the input. For example, we could define f(x) = {x + 4 for 0 < x < 5}. So, if we give the input as 2 then , f(2) = 2 + 4 = 6. We denote the output as y, so we can say y is a function of x which is equal to {x + 4 for 0 < x < 5}.



A function also is called a mapping, and, if f(x) = y, we say that f maps x to y.

In a function, it is imperative that each input is assigned to only one output. The reason for this is because if an input were assigned to two different outputs, then given the input, we will not be able to determine the output. Given an input, we must be able to determine the output. Thus, each input of a function has only one output.

2.Domain & Range of a Function

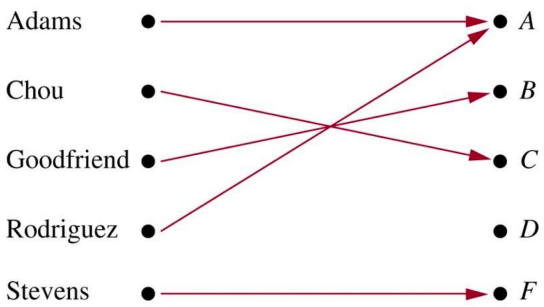
The set of inputs of a function is the domain of the function and the set of outputs of a function the range of a function.

For example, consider the function defined by the rule
f(x) = {x + 4 for 0 < x < 5}

Here domain = 1, 2, 3, 4
and range = 5, 6,7,8

The Notation :
f : D -> R
A function maps a set of inputs (domain) to set of outputs(range).

A function may not necessarily use all elements in a range. Consider the following example:
f : Student -> Grades



Here we are mapping Set of students to their corresponding grade. Each student will receive a grade for sure although few students might receive the same grade. However, there could be a possibility that none of the students got a particular grade, in this example, no student got grade D. So we have not used all the elements in a range.

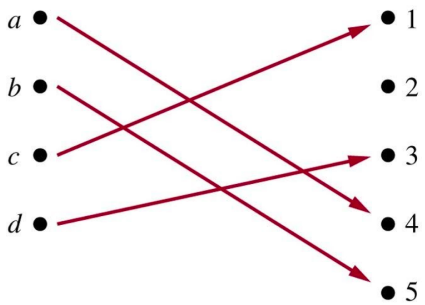
3.Different types of functions

We discuss the following types of functions:

1) One to One (injective)
2) Onto (Surjective)
3) Bijective

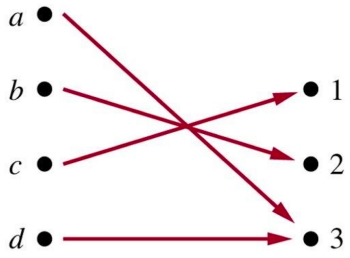
1)One to One Function :

A function f: A → B is One to One if for each element of A there is a distinct element of B. It is also known as Injective. or we can say every element in A has a unique image.
Here, Image means to which value does an element map to.

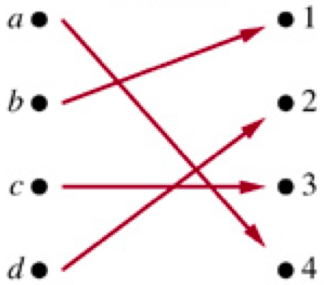


In the given example, Image(a) = 4 and we can in turn say pre-image(4) = a.

2)Onto Function :
A function where, for every element of set B there exists a pre-image in set A, it is called Onto Function. Onto is also referred to as Surjective Function.
For example:



3)Bijective Function :
A function, Bijective if the function is both One to One and Onto function. In other words, the function associates each element of A with a distinct element of B and every element of B has a pre-image in A.
For example :



4.What is a Relation?

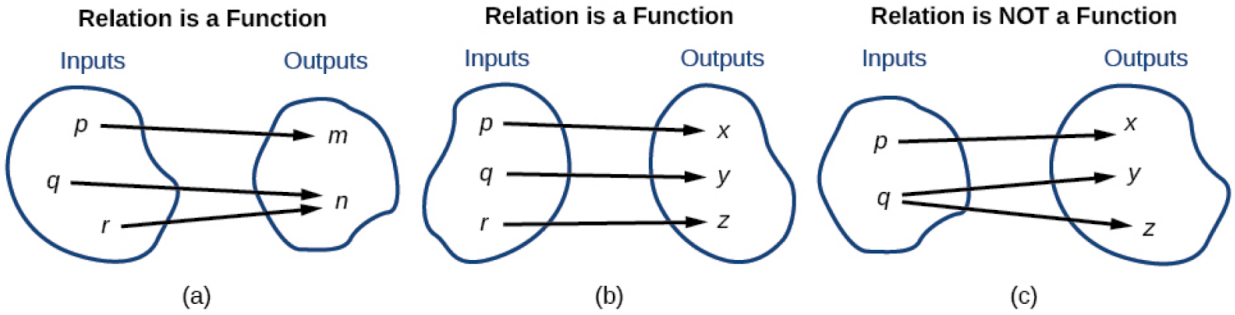
A relation between two sets is a collection of ordered pairs (x,y), containing one object from each set. Here x is from the first set and y is from the second set.

The set of the first components of each ordered pair is called the domain and the set of the second components of each ordered pair is called the range.
For example :
If we define relation R as,
R = {(1,2), (2,4), (3,6)}

then
Domain is {1,2,3}
Range is {2,4,6}

5.When is a Relation called a Function?

A function is a special relation in which each possible input maps to exactly one output value.
Given the following relations lets see which one is a function.



In example a, Each input maps to exactly one output hence this relation is a function.

In example b, again Each input element maps to exactly one output hence this relation is a function.

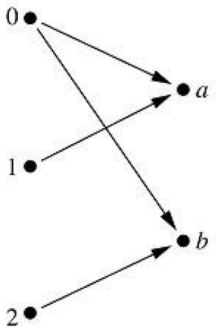
In example c, input q maps to two outputs y and z, which is not desirable from function point of view, where the output for each input is determined and is a single value with no confusion. Hence this one is not a function.

6.Binary Relation

A binary relation is a relationship between two sets lets say X and Y and formally is a subset of the Cartesian product of X and Y;

We denote an Element a is related to element b as aRb. which means the pair (a, b) $\in$ R

Here if we consider Set A as containing {0, 1, 2} and Set B as {a, b} then the cartesian product of A and B will contain elements like
A x B = { (0, a), (1, a), (0, b), (1, b), (2, a), (2, b) }
We can define a relation as containing only few pairs as shown in the figure below and is a subset of the cartesian product of A and B



R	a	b
0	×	×
1	×	
2		×

7.Representing a Relation

We can represent a relation in two ways -

1) Matrix form
2) Directed Graph (Digraph)

Let's Consider a Set A = {a, b, c, d}

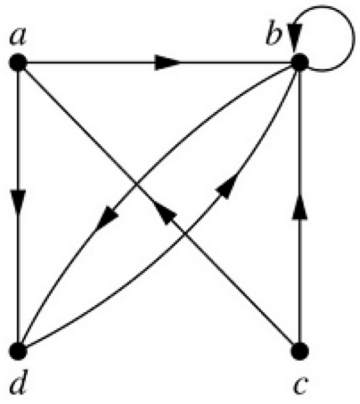
Relation R on Set A is a relation from A to A ($\subseteq$ A x A)

R = {(a, d),(a, b),(b, b),(b, d), (c, b), (c, a), (d, b)}

We can represent this relation as a matrix where the row and the column headings are the elements of the set. Value 1 for a pair indicates they are related. For example here (a,b) belongs to R hence is marked 1 whereas (a,a) and (a,c) does not and therefore these entries are marked 0.

	a	b	c	d
a	0	1	0	1
b	0	1	0	1
c	1	1	0	0
d	0	1	0	0

In the form of digraph, an edge is drawn from element x to y for each ordered pair (x,y) as shown in the figure below for R = {(a, d),(a, b),(b, b),(b, d), (c, b), (c, a), (d, b)}.



8.Properties of a Relation :

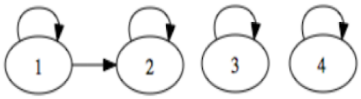
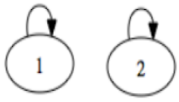
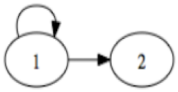
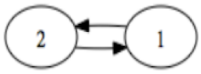
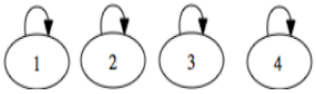
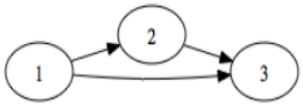
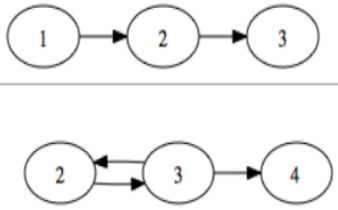
A relation R on a set A (A = {1, 2, 3, 4}) is

Reflexive : iff (a,a)∈R for every element a∈A.
R = {(1, 1), (2, 2), (3, 3), (4, 4), (1, 2)} is reflexive

Symmetric : iff (b,a)∈R whenever (a,b)∈R, for all a,b∈A.
R = {(1, 2), (2, 1), (1, 4), (4, 1), (3, 3)} symmetric

Transitive : iff (a,c)∈R whenever (a,b)∈R and (b,c)∈R, for all a,b,c∈R.
R = {(1,2), (2,1), (1,1)}

Provided are a few examples of relation with the properties identified:

Relation	Reflexive	Symmetric	Transitive
	Y	N	Y
	Y	Y	Y
	N	N	Y
	N	Y	N
	Y	Y	Y
	N	N	Y
	N	N	N
	N	N	N

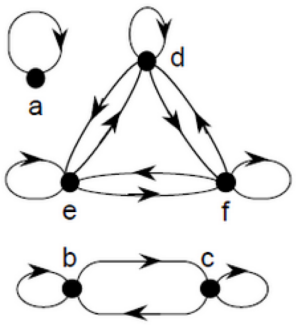
9.Equivalence Relation

Denoted by x ≡ y

A relation R on a set A is called an equivalence relation if it is reflexive, symmetric, and transitive.

Example: Let S = {a,b,c,d,e,f}.

Relation R on set S be
{(a,a),(b,b), (b,c),(c,b),(c,c),(d,d),(d,e),(d,f),(e,d),(e,e),(e,f),(f,d),(f,e),(f,f)}



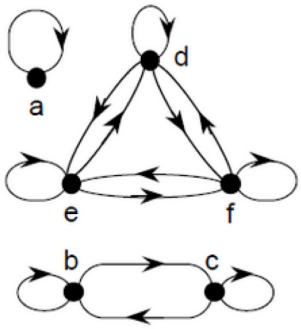
	a	b	c	d	e	f
a	1					
b		1	1			
c		1	1			
d				1	1	1
e				1	1	1
f				1	1	1

10.Equivalence Class

Let R be an equivalence relation on a set A.

The set of all the elements that are related to an element ‘a’ of A is called the equivalence class of ‘a’.

Denoted by $[a]_R$ or [a] when the relation is implicit.
Given the relation :



	a	b	c	d	e	f
a	1					
b		1	1			
c		1	1			
d				1	1	1
e				1	1	1
f				1	1	1

We can say,
[b] = {b, c}
[d] = {d, e, f}

AUTOMATA FORMAL LANGUAGES AND LOGIC

Lecture Notes on Deterministic Finite Acceptor/Automata(DFA)

Prepared by :
Prof. Sangeeta V I
Assistant Professor

Department of Computer Science & Engineering
PES UNIVERSITY
(Established under Karnataka Act No.16 of 2013)
100-ft Ring Road, BSK III Stage, Bangalore – 560 085

Table of Contents

Examples Solved

8	The language of strings over Σ ={a,b} where every string must contain “aa” as the substring.	**17**
9	The language of strings over Σ ={a,b} of the form $a^n\ b^m$ |n,m>=1.	**18**
10	The language of strings over Σ={a,b} of the form $a^n\ b^m$ |n,m>=0.	**19**
11	The language of strings over Σ={a,b} where, n_a(w)mod 2=0	**21**
12	The language of strings over Σ={a,b} where, n_a(w)mod 2=0 and n_b(w)mod 2=0	**22**
13	The language of strings over Σ={a,b} where, n_a(w)mod 3=0 and n_b(w)mod 2=0	**23**
14	The language of strings over Σ={a,b} where, n_a(w)mod 3=0 and n_b(w)mod 3=0	**24**
15	The language of strings over Σ={a,b} where, n_a(w)mod 3 = **2** and n_b(w)mod 3 = **1**	**25**
16	DFA for binary number divisible by 2.	**26**
17	DFA for binary number divisible by 3.	**28**

1.Formal Definition of a DFA

A DFA can be represented by a 5-tuple $(Q, \sum, \delta, q_0 F)$ where –

- Q: a finite set of internal states
- Σ: a set of symbols called the input alphabet
- δ: a transition function from Q X Σ to Q
- q_0: the initial state
- F: a subset of Q representing the final states

In a DFA, for a particular input character, the machine goes to one state only. A transition function is defined on every state for every input symbol.

2.Language Accepted or Recognized by a DFA :

The language accepted or recognized by a DFA M is the set of all strings accepted by M , and is denoted by

$L(M)=\{w \epsilon \Sigma | M \text{ accepts } w\}$

3.Constructing a DFA

We can construct automata for every string in the finite language.So,every finite language has a finite acceptor .

3.1Graphical Representation of a DFA

A DFA is represented by digraphs called state diagrams.

- The vertices represent the states.
- The arcs labeled with an input alphabet show the transitions.
- The initial state is denoted by an empty single incoming arc.
- The final state is indicated by double circles.

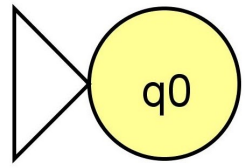
Length of a string : The number of symbols in a string w is called its length, denoted by |w|.

Example : | 011 | = 4, |11| = 2, | b | = 1

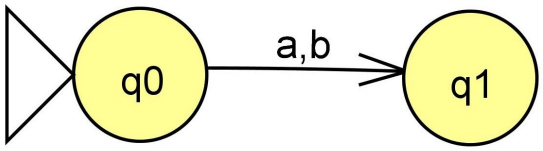
Example 1:

Assume Σ={a,b}
Consider language L1={strings of length 2 over Σ }={w | |w|=2}
Minimum strings:{ab,ba,aa,bb}
L1={ab,ba,aa,bb} The language is finite and we can construct a DFA.
We will construct the automata for the minimum string.

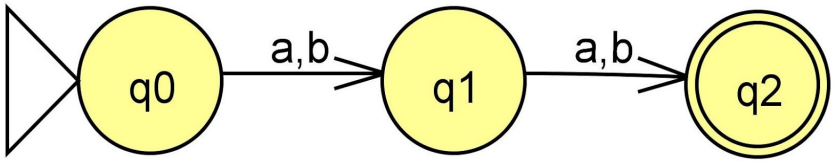
- Start state q0 with an arrow.



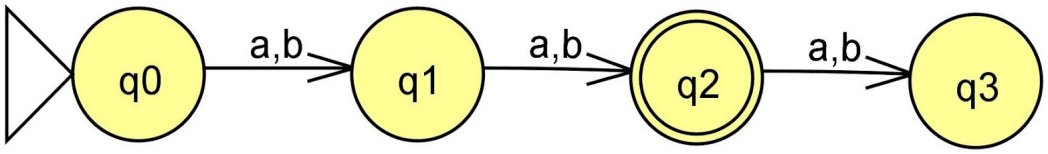
- The first symbol could be either a or b, we accept it and move to the next state q1.



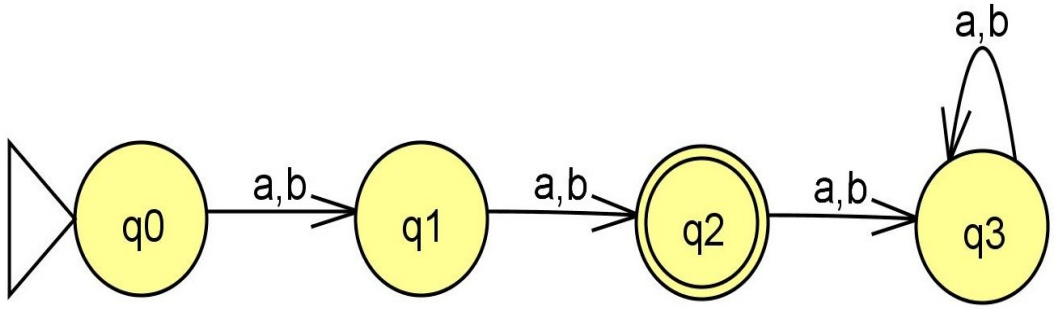
- The second symbol could be either a or b,we accept it and move to the next state q2.



- When we reach q2 we have covered all possible combinations ab,ba,aa,bb so we can make q2 as the final state.
- If we encounter any strings of length greater than 2 ,we reject such strings!

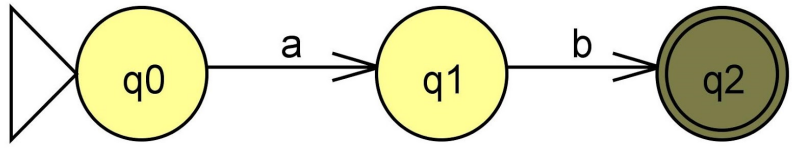


- On state q2 when we get any symbol we reject it and send it to q3 , a non accepting state/non final state/reject state/trap state/dead state.



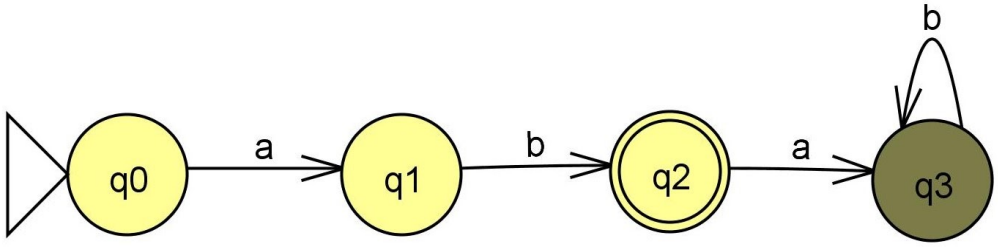
- On state q3 when we get any symbol they remain in q3.
- Any string which lands in a non-accepting state is rejected .

❖ Tracing the string ab.



String ab is accepted as it lands in state q2.

❖ Tracing the string abab



abab is rejected as it lands in state q3 which is a trap state/dead state.

Example 2:

Assume Σ={a,b}
Consider language L2={strings of length <=2 }={w | |w|<=2}

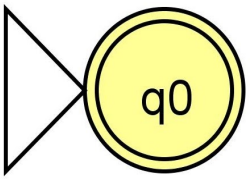
We will enumerate the elements in the language.

- Minimum Strings:
  - String of length 0=λ(empty string)
  - Strings of length 1={a,b}
  - Strings of length 2={ab,ba,aa,bb}

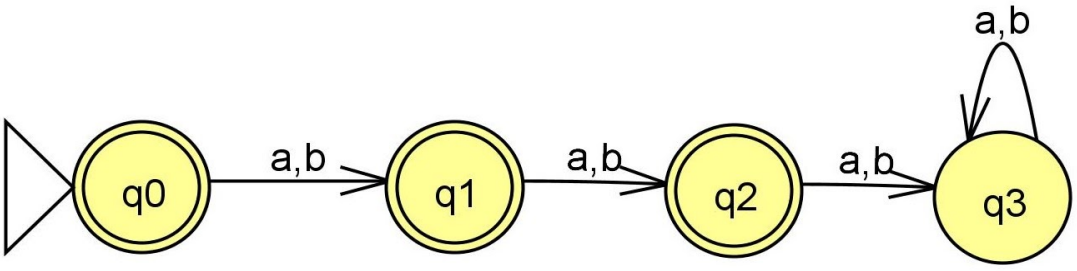
L2={λ,a,b,ab,ba,aa,bb}

We will construct the automata for the minimum string.

- Whenever λ (empty string) is a part of the language ,the start state itself will be the final state.



- On state q0, if we see a or b ,we accept it as a and b belong to the language and move to state q1.
- On state q1, if we see a or b we accept it as strings ab,ba,aa,bb are accepted and move to state q2.
- On state q2, if we see a or b (strings of length >2) we move to q3 which is dead or a trap state.



Example 3:

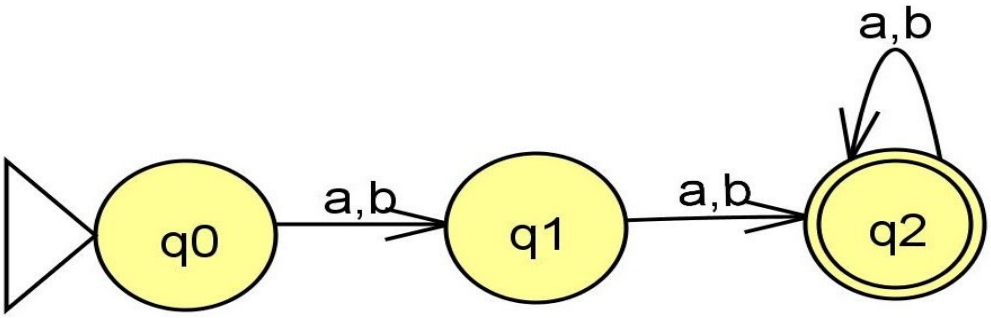
Assume Σ={a,b}
Consider language L3={strings of length >=2 }={w | |w|>=2}

We will enumerate the elements in the language.

- Minimum Strings:
  - String of length 2={ab,ba,aa,bb}
  - Strings of length 3={aaa,aab,aba,abb,.....}
  - Strings of length 4={aaaa,bbbb,.....} and so on.

L3={ab,ba,aa,bb,aaa,aab,aba,abb,.....,aaaa,bbbb,.....}
We will first construct the automata for the minimum string .

- q0 is the start state .
- On q1, which is a non accepting state,if we see a or b we move to state q2.
- q2 is accepting state as we have encountered strings of length 2 .
- On q2 ,we have a jolly ride /self loop on a or b as we can accept strings of length >2.
  We accept all strings of length >=2 and reject strings of length <2.



Example 4:

Assume Σ={a,b}
Consider language L4={strings which start with a }

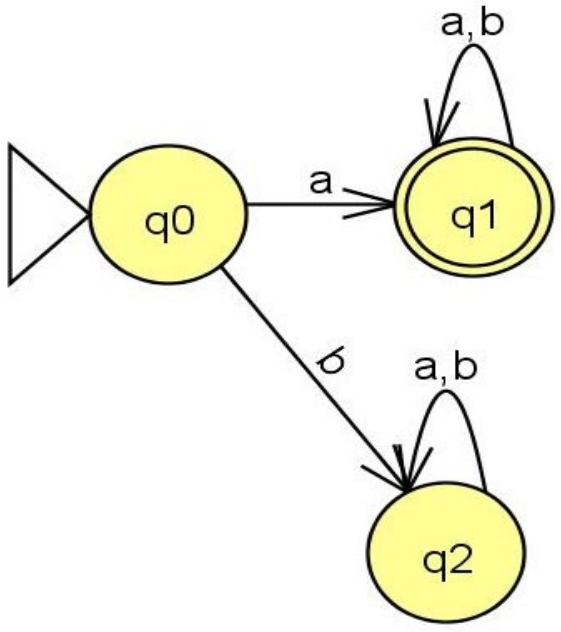
We will enumerate the elements in the language.

- Minimum Strings:
  - String of length 1:{a}
  - Strings of length 2:{ab,ba,aa,bb}
  - Strings of length 3:{aaa,abb,aba,aab} =a (a/b) (a/b)
  - String of length 4 and so on .

L4={a,ab,ba,bb,aaa,abb,aba,aab,.....}

We will first construct the automata for the minimum string .

- q0 is the start state .
- On q0, if we encounter a we move to q1 ,if we encounter b we reject it(reject strings starting with b).
- On q1, we will have a jolly ride/self loop on a,b.(accept string of length >1).
- All the strings starting with b are rejected.q2 is a trap/dead state.
- On q2, if we encounter a or b we will have a jolly eide/self loop .



3.2 Representation of DFA with Transition Table:

It is basically a tabular representation of the transition function that takes two arguments (a state and a symbol) and returns a value ("the next state").

- Rows correspond to states,
- Columns correspond to input symbols,
- Entries correspond to next states
- The start state is marked with an arrow
- The accept states are marked with a star (*).

Transition table for Example 4:

	a	b
→q0	q1	q2
q1	q1	q1
✱q2	q2	q2

In the transition table for a DFA ,we must specify the transition of each state and every input symbol from the alphabet exactly once.
Every state on every input symbol will go only to one state.

3.3 Representation of DFA with Transition Function δ:

The transition function describes how the machine changes its internal state.
It is a function

$$\delta: Q \times \Sigma \rightarrow Q$$

from (state,input symbol)pait to state.If the machine is in state q and the present input symbol is a then the machine changes its internal state to $\delta(q,a)$ and moves to the next input symbol.

The basic transition function δ gives the new state of the machine after a single symbol/letter is read. The extended function (that we will first denote by δ^*) gives the new state after an arbitrary string of symbols is read. In other words, $\delta^*$is a function.

- δ^*: $Q \times \Sigma^* \rightarrow Q$. For every state q and word w the value of $\delta^*(q,w)$ is the state that the DFA reaches if in state q it reads the input word w.

The language accepted or recognized by a DFA M is the set of all strings accepted by M , and is denoted by

$L(M)=\{w \in \Sigma^* \mid \delta^*(q0,w) \in F\}$

Transition function for Example 4:

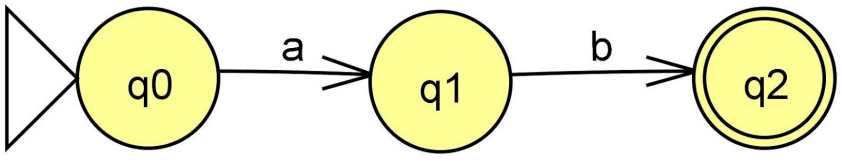
- $\delta(q0,a)=q1$
- $\delta(q0,b)$-q2
- $\delta(q1,a)=q1$
- $\delta(q1,b)=q1$
- $\delta(q2,a)=q2$
- $\delta(q2,b=)q2$

Example 5:

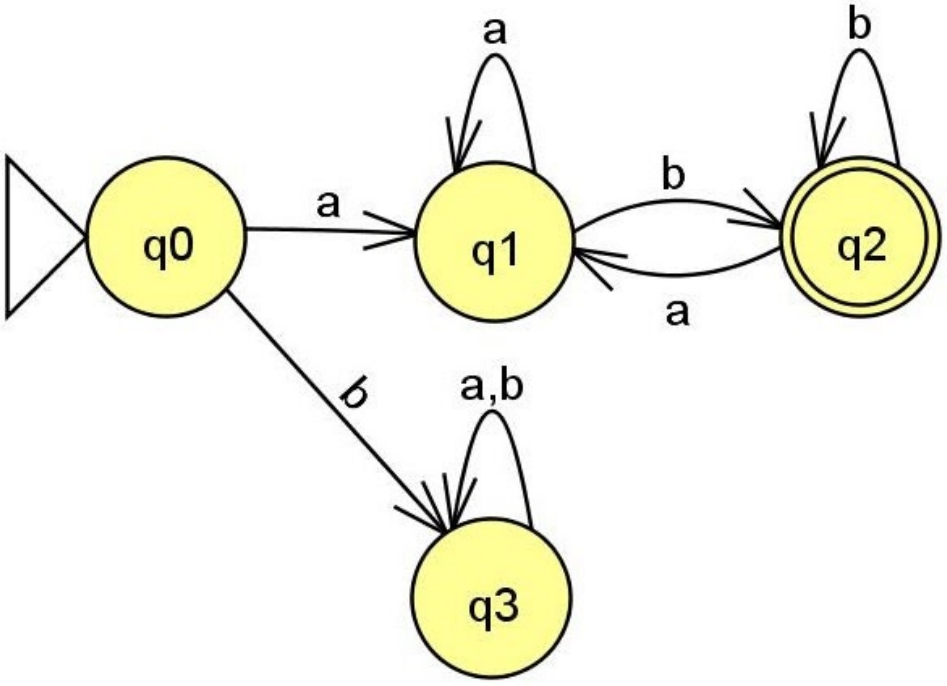
L5={w|awb,wϵ{a,b}*}
The language defines strings which start with a and end in b.
We will enumerate the elements in the language.

- Minimum String:ab

L5={ab,a.(followed by any number of a's or b's). b}
We will first construct the automata for the minimum string ab.



- DFA for the minimum string ab.



- On q0,if we see b re will reject and send to satte q3, it would violate the condition.
- On q1, we can have a jolly ride/self loop on a.
- On q2, we can have a jolly ride/self loop on b.
- On q2,if we see the symbol a we move to state q1 so that we do not miss on string like ababa,abaaab etc

Example 6:

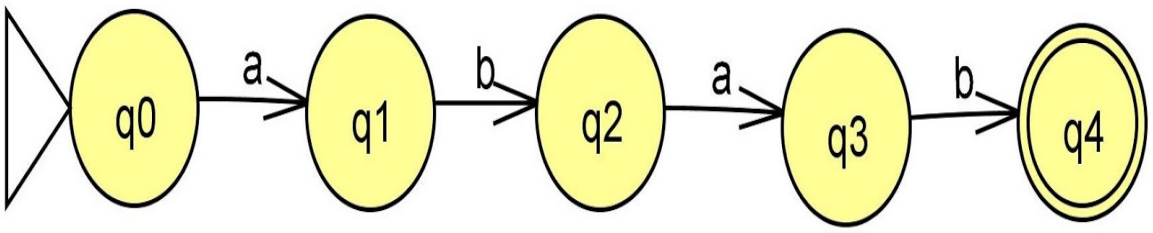
L6={w|abwab,w**ϵ**{a,b}*}

We will enumerate the elements in the language.

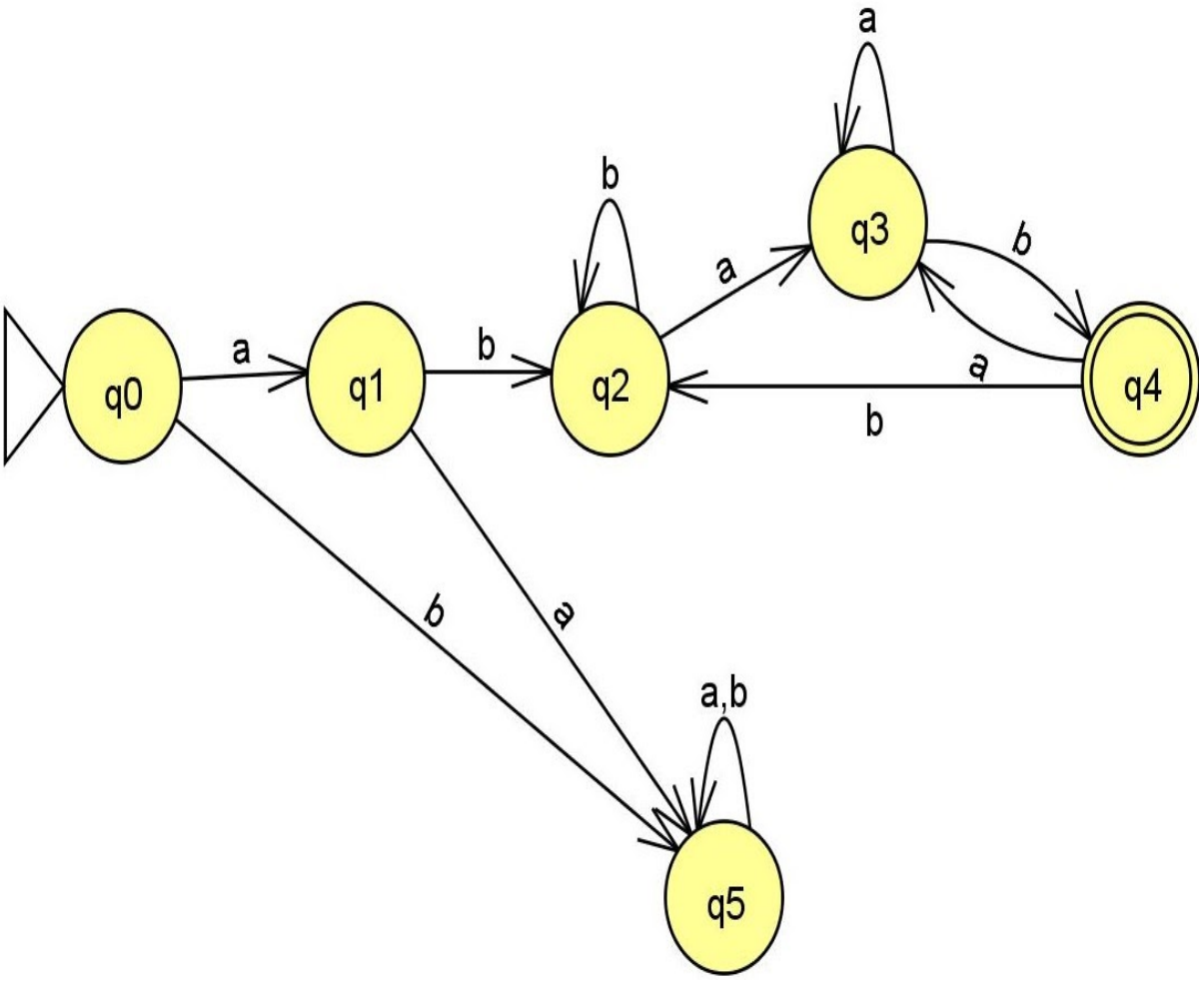
- Minimum String:abab

L6={abab,ab (any number of a's or b's) ab}

We will first construct the automata for the minimum string abab



- DFA for the minimum string abab.



- On state q0,if we see ‘b’ we move to state q5(dead/trap state) as we reject strings starting with b.
- On q1, if we see ‘a’ we move to dead/trap state as we accept strings starting with ab.
- On q2 we can have a self loop on ‘b’ .
- On q3,we can have a jolly ride/self loop on ‘a’.
- On state q4,if we see the symbol ‘a’ we move back to state q3 ,so that we accept strings of the form :ababab
- On state q4 ,if we see the symbol ‘b’ we move back to state q2 ,so that we accept strings of the form:ababbab.

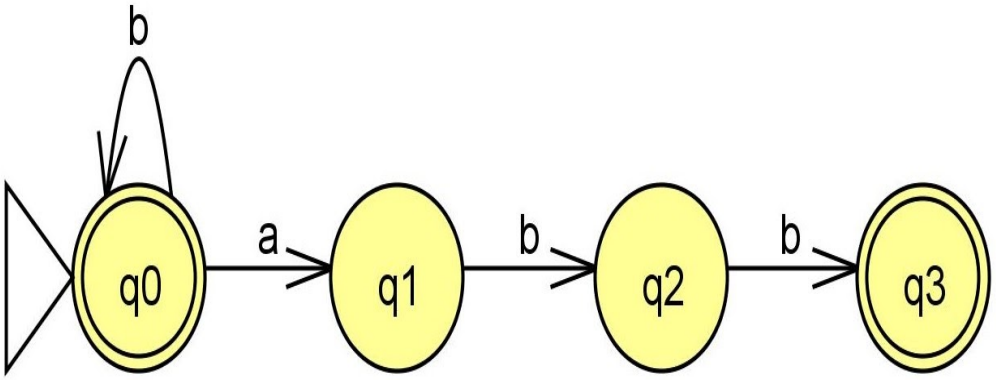
Example 7:

L7={Every a followed by bb}
We will enumerate the elements in the language.

L7={λ,b,bb……,abb,b….babb….abb…...}

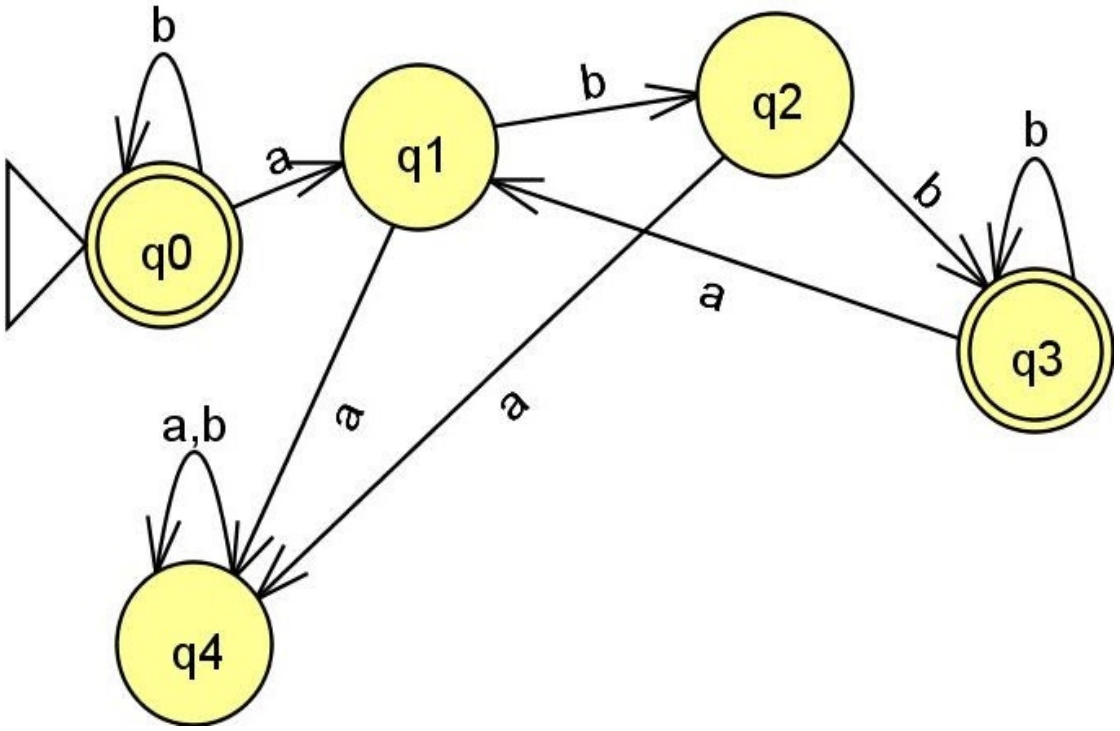
We will first construct DFA for the minimum string.

- Since λ is a per of the language,start state itself is the final state.



- On q0,if we see a ‘b’ ,we can have a jolly ride on ‘b’ ,since there is no restriction on the number of b’s.
- On q0,if we see an ‘a’ it has to be followed by ‘bb’

.



- On state q1 if we see ‘a’,we move t o dead/trap state.
- On state q2,if we see ‘a’,we move to dead/trap state.
- On state q3,if we see ‘a’ we move back to q1,so we take care of strings like abbabb
- On state q3,since there is no restriction on ‘b’ ,we have a jolly ride/self loop on ‘b’.

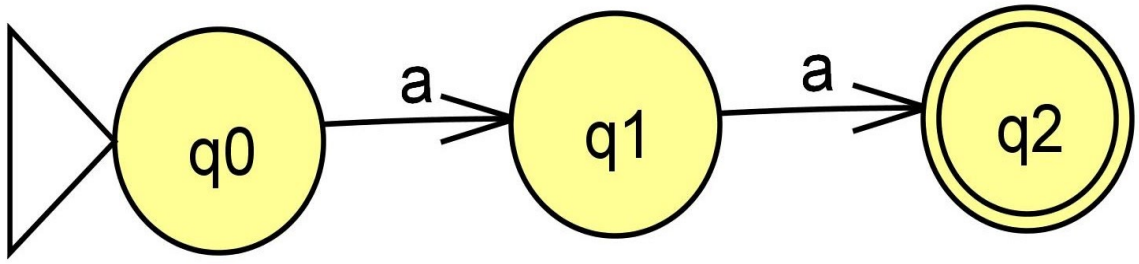
Example 8:

L8={Every string must contain “aa” as the substring}

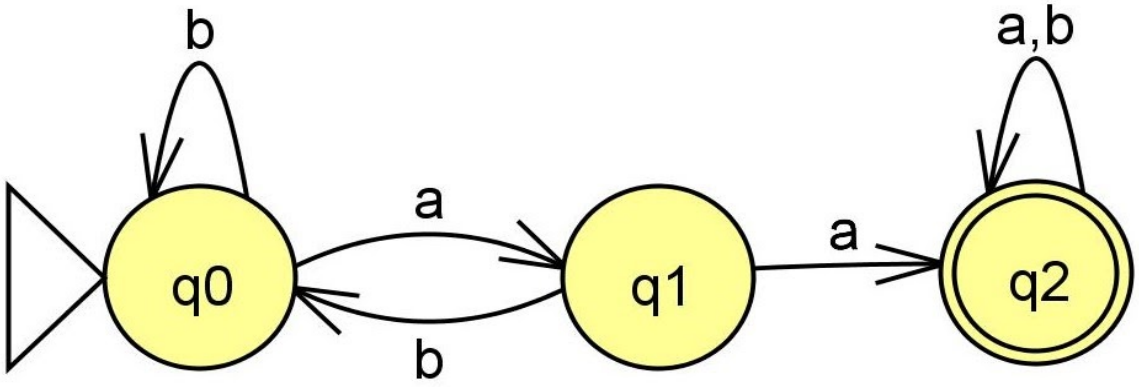
We will enumerate the elements in the language.

L8={aa, (any number of a’s or b’s) aa (any number of a’s or b’s) }

We will first construct DFA for the minimum string.



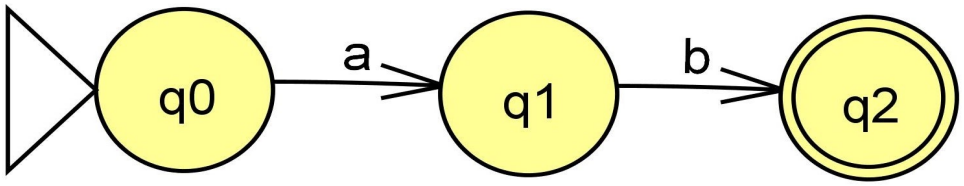
- DFA for the minimum string aa.



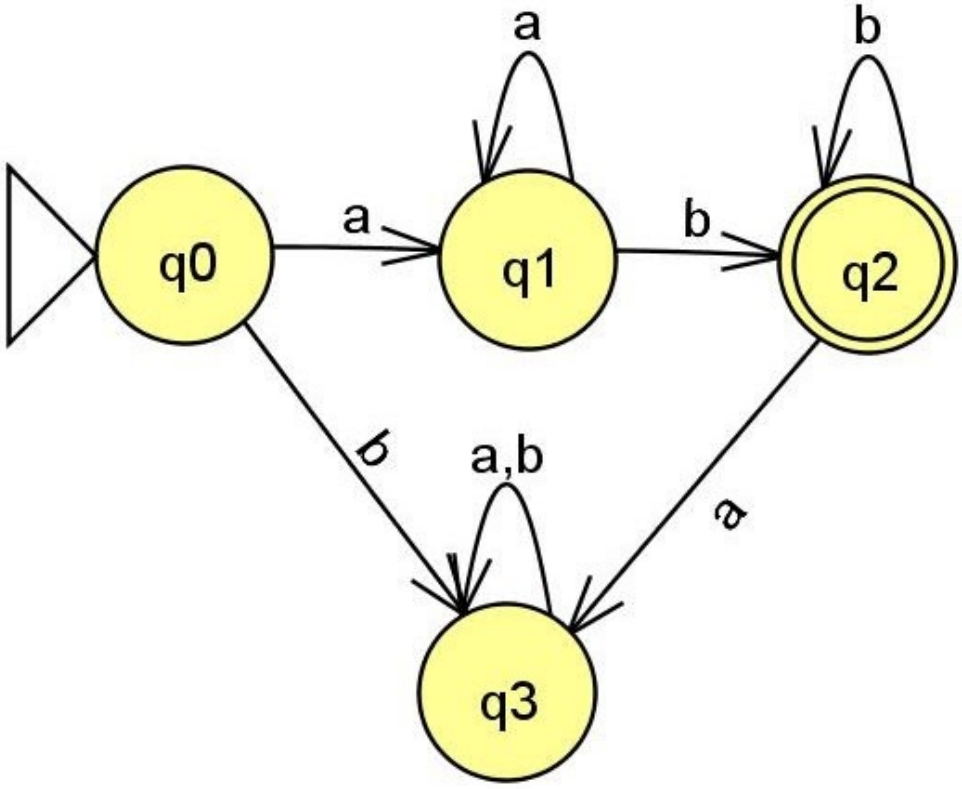
- On state q0,if we see ‘b’ we can have a jolly ride/self loop .
- On q1,if we see ‘b’ we will go back to q0 ,so that we can take care of strings like abaab,abbabaabb etc
- On q2,we will have seen “aa” ,now can see any number of a’s or b’s so we can have a jolly ride/self loop on q2.

Example 9:

L9={ a^n b^m |n,m>=1}
We will enumerate the elements in the language.
L9={ab,aab,aaaabbbbbbbbb,..... }
Minimum String: ab
We will first construct DFA for the minimum string.



- DFA for the minimum string ab.



- On state q0 ,if we see symbol 'b' we move to dead/trap state as the language is a's followed by b's.
- On q1 ,we can have a jolly ride/self loop on 'a'.
- On q2,we see symbol 'a' we will move to dead/trap state as the language is a's followed by b's,and we cannot have a's after b's.

Example 10:

L10={ a^n b^m |n,m>=0}

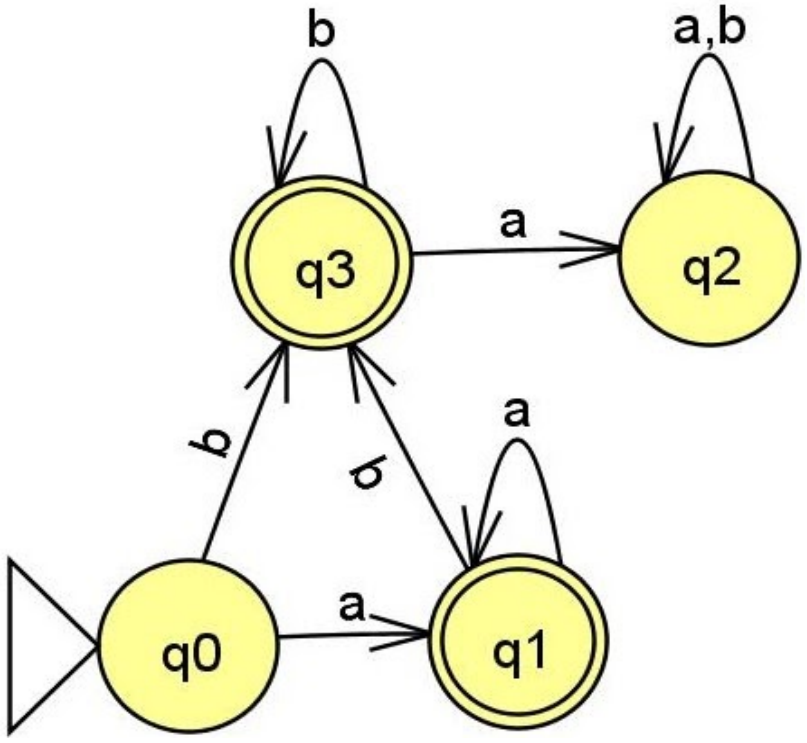
We will enumerate the elements in the language.

n (number of a's)	m (number of b's)	
0	> 0	any number of b's
> 0	0	any number of a's
0	0	λ (empty string-no a's ,no b's)
> 0	> 0	some a's followed by some b's

L10={λ , any number of a's , any number of b's , some a's followed by some b's }
Minimum String: ab

We will first construct DFA for the minimum string.

Since λ is a part of the language,start state itself is a final state.



- On q0,if we see input symbol b we move to q3 ,as the string can have only b's.

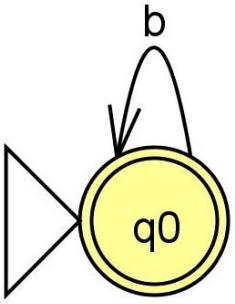
- We reach q3 from q0 on symbol 'b'.If we see the symbol 'a' in q3 we move it to the dead/trap state as it would not belong to the language.

- On q1,we see the symbol 'b' and we move to state q3.

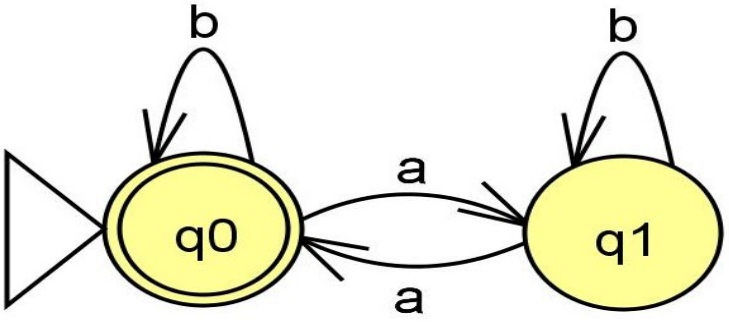
Example 11:

L11={n_a(w)mod 2=0}
We will enumerate the elements in the language.
Note that there is no restriction on the number of b's.
L11={ $\boldsymbol{\lambda}$,any number of b's,even number of a's and any number of b's in between}

We will first construct DFA for the minimum string.
Since $\boldsymbol{\lambda}$ is a part of the language,start state itself is a final state.



- Since,there is no restriction on the number of b's so we can have a jolly ride/self loop on b in q0.

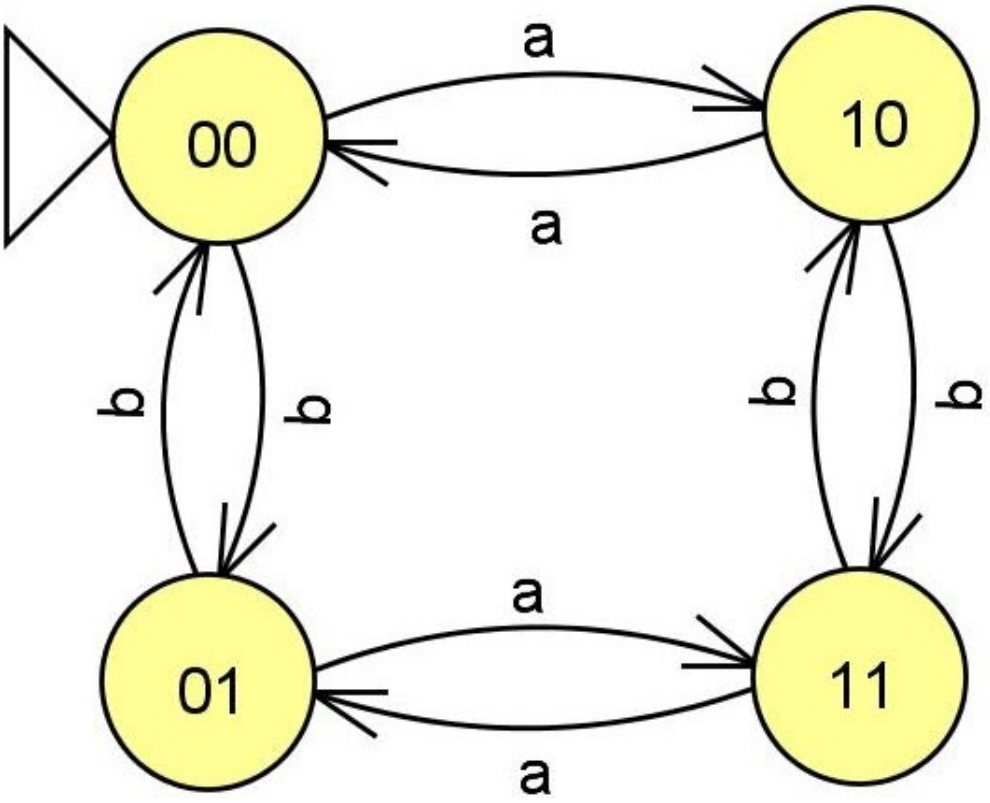


- On state q0 ,if we see the symbol 'a' we move to state q1,we will not accept a single 'a' so we will not make q1 as an accepting state.On q1,if we see the symbol 'a' we move back to q0. So only when we see 2 a's we reach the final state.

- The DFA accepts only even numbers of a's.

Example 12:

L12={n_a(w)mod 2=0 and n_b(w)mod 2=0 }



- 1 indicates odd
- 0 indicates even
- 00 indicates even a's and even b's
- 10 indicates odd a's and even b's
- 01 indicates even a's and odd b's
- 11 indicates odd a's and odd b's

Number of states in a machine for modulo questions:

- Single symbol
  n_a(w)mod n=0
  The number of states in the DFA is 'n' i.e the number of remainders for mod n.

- Two symbols
  If the language is n_a(w)mod n=0 and n_b(w)mod n=0 then the number of states in the DFA is m✖n.

Example 13:

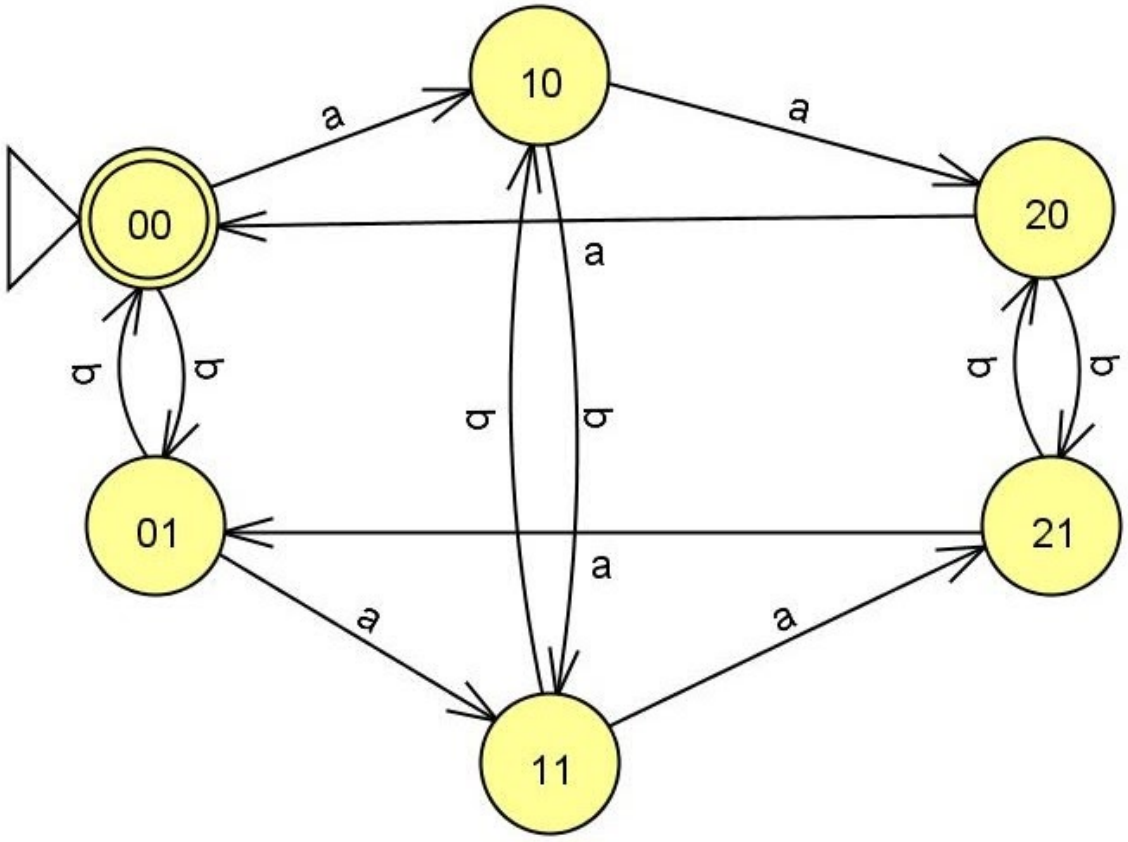
L13={n_a(w)mod 3=0 and n_b(w)mod 2=0 }

The remainder on mod 3 is 0,1,2.
So in DFA for n_a(w)mod 3=0 there will be 3 states .

The remainders on mod 2 is 0,1
So in DFA for n_b(w)mod 2=0 there will be 2 states

Therefore,the DFA for the language L13 will have 3✖2=6 states.

- In the DFA we have all the a's horizontally and all the b's vertically.



- The states are named based on the remainder on the number of a's and b's.

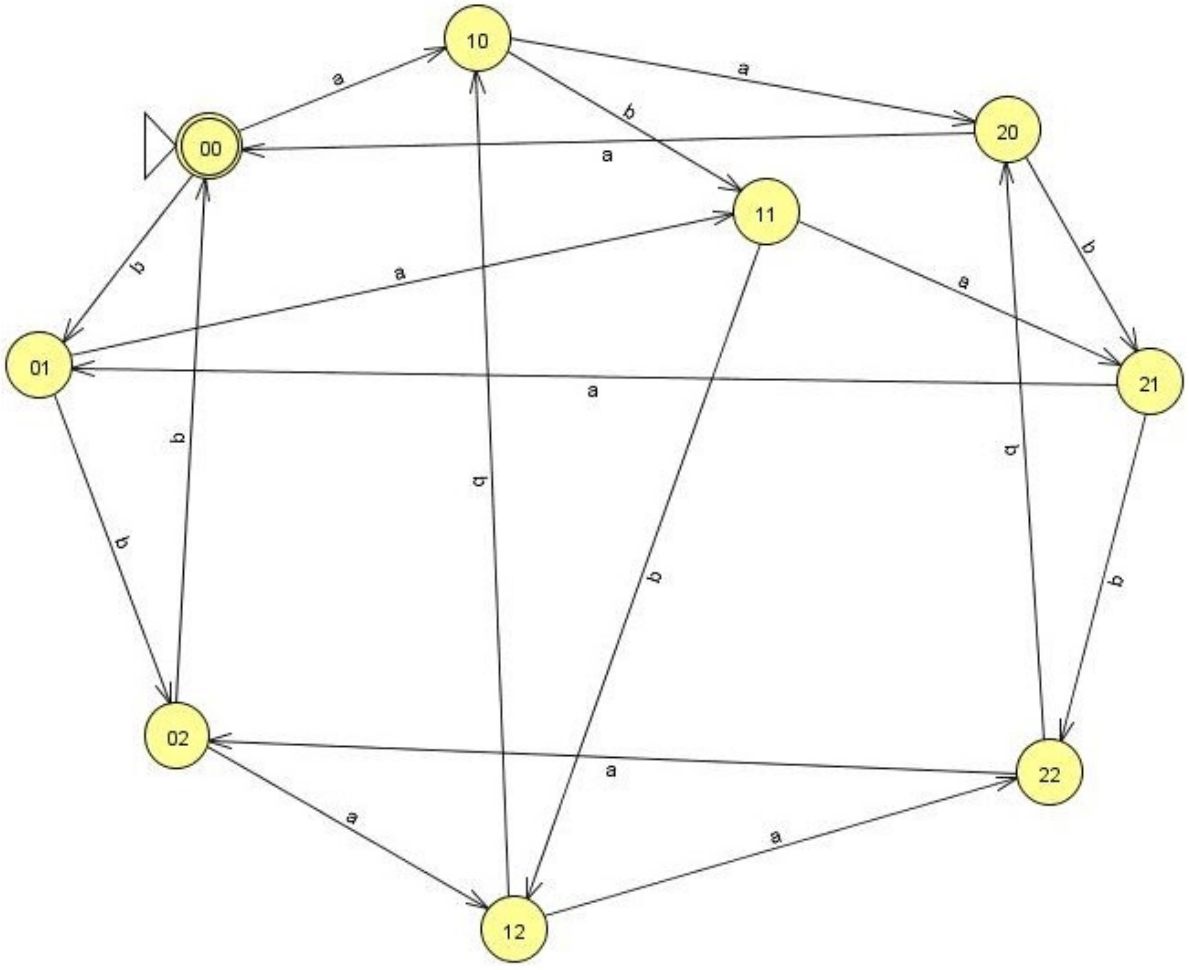
Example 14:

L14={n_a(w)mod 3=0 and n_b(w)mod 3=0 }

The remainder on mod 3 is 0,1,2.
So in DFA for n_a(w)mod 3=0 there will be 3 states .

Also in DFA for n_a(w)mod 3=0 there will be 3 states .

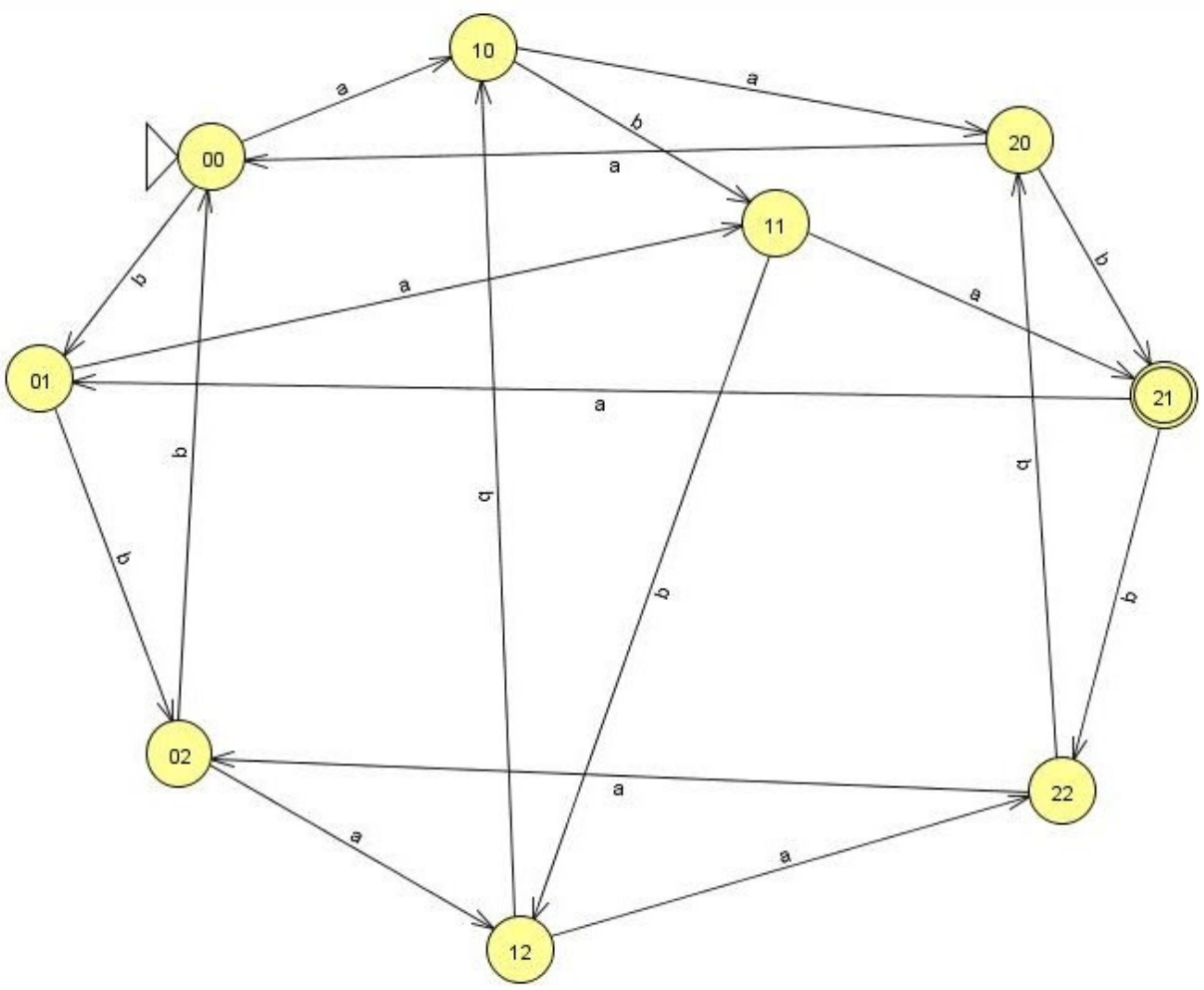
Therefore,the DFA for the language L14 will have 3✖3=9 states..



- The states are named based on the remainder on the number of a's and b's.

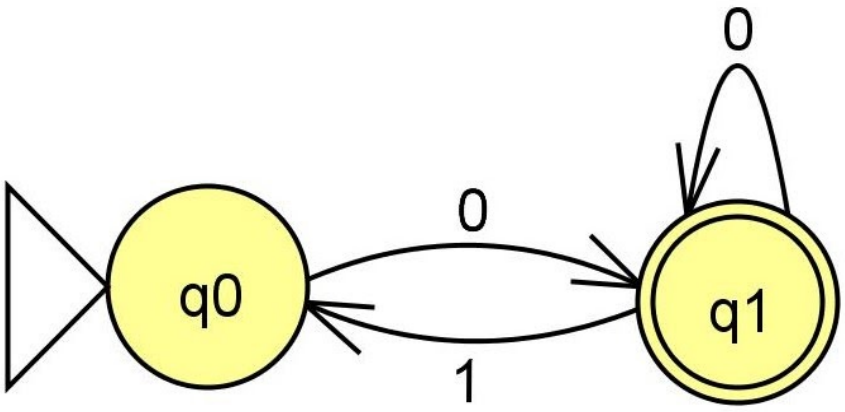
Example 15:

L15={n_a(w)mod 3 = **2** and n_b(w)mod 3 = **1** }
We can refer to the same DFA constructed for the language L14.
The DFA for L15 would be the same as DFA for L14,but there will be change in the final state.
The final state for L15 will be the state numbered **21**.



Example 16:

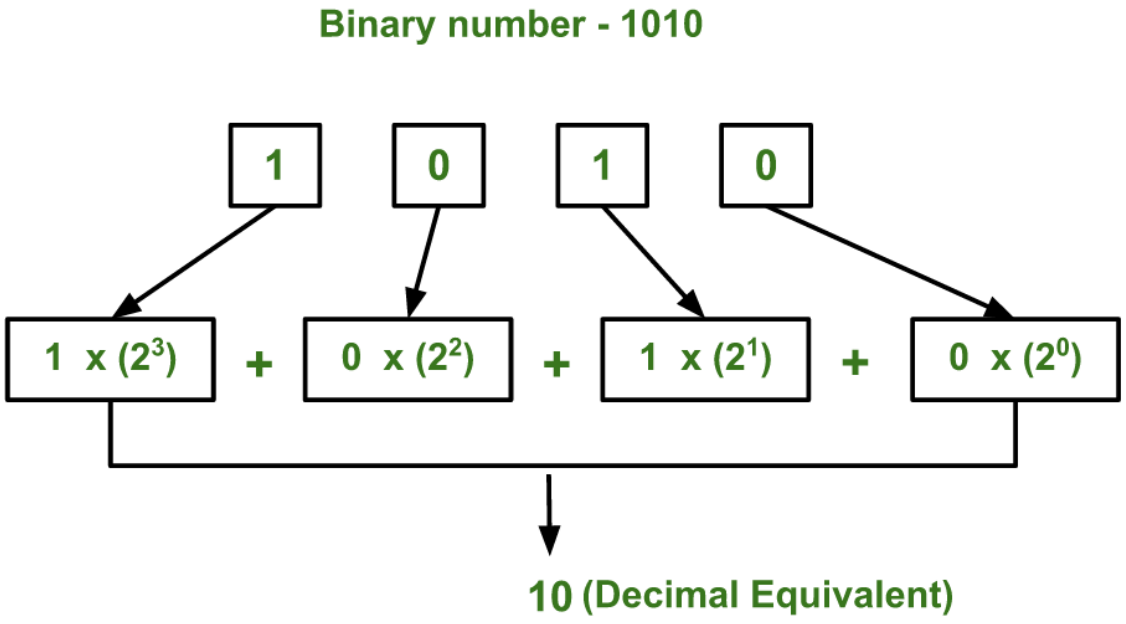
Constructing a DFA for binary number divisible by 2 or w mod 2 =0
In the binary system,any number divisible by 2 would end in 0.



Constructing a DFA for binary number divisible by 3 or w mod 3=0
The remainder on mod 3 is 0,1,2.
So in DFA for w mod 3=0 there will be 3 states .

Methods to convert binary to decimal :

One method:



Another method:

Example: 101

.

101

2*1=2 Multiply base i.e 2 with the most significant bit

2+0=2 Add the previous result to the next bit

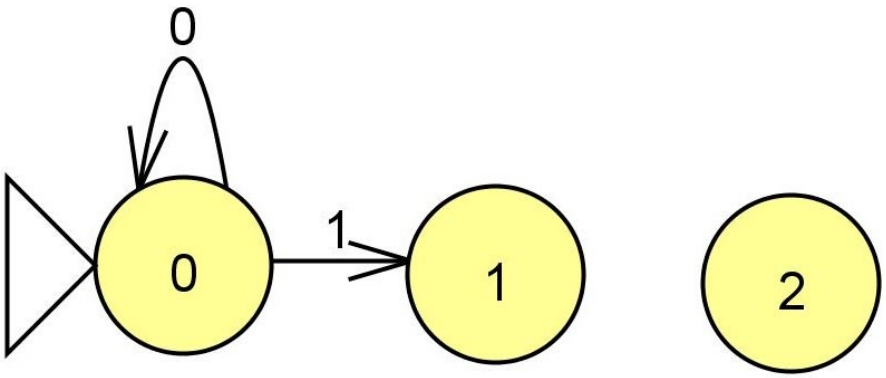
2*2=4 Multiply the previous result with base i.e 2

4+1=5 Add the next bit to the precious result.

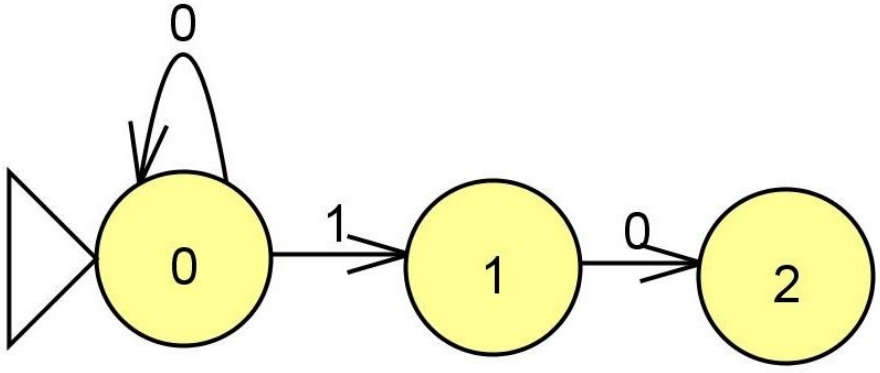
Example 17:

Constructing a DFA for binary number divisible by 3 or w mod 3 =0
The remainder on mod 3 is 0,1,2.

Let us construct DFA ,



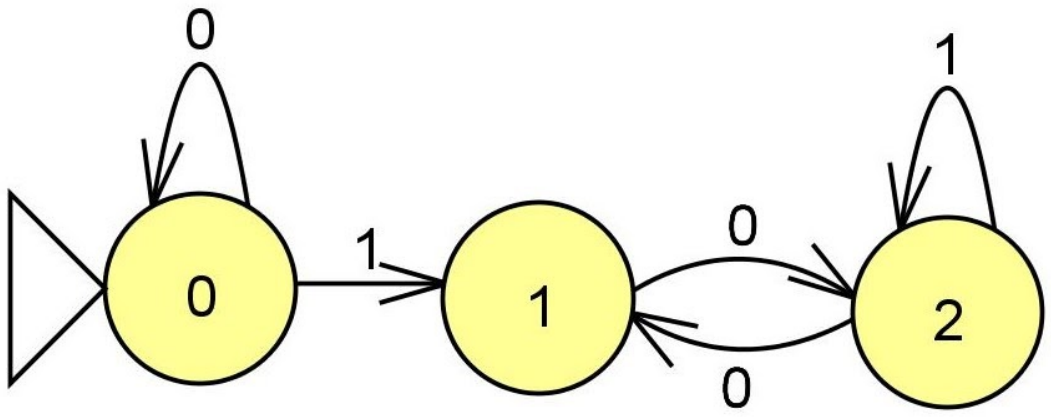
- The remainders on mod 3 0,1,2 are the states in DFA.
- If x is the input number to the DFA, x mod 3= either 0,1,2,it ends up in the state depending on what the remainder is. .
- 0 mod 3 = 0,hence a self loop on state 0.
- 1 mod 3 = 1 so we move to state 1.



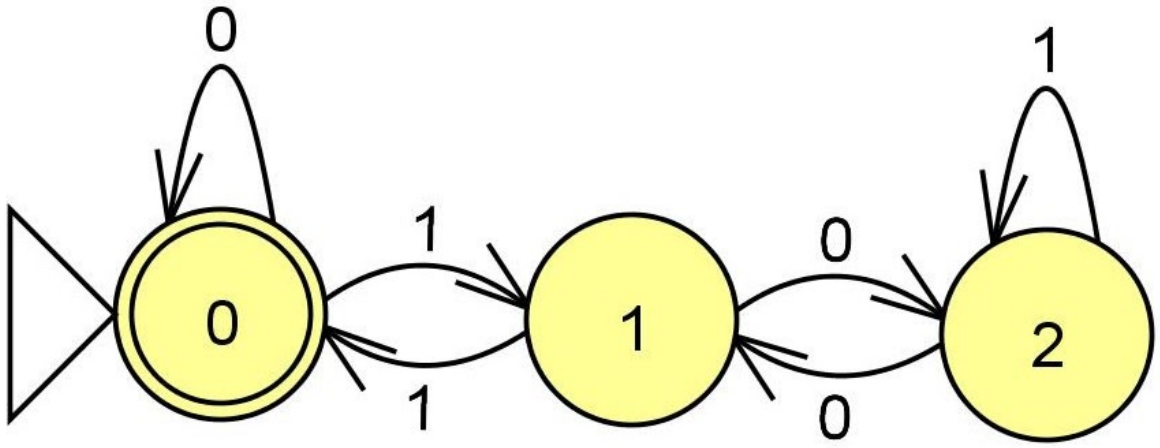
- 2 mod 3 =2
- The binary equivalent of 2 is 10 .

In state 2 if we get a '1' ,what do we do ?
For example ,if the input is 101, the decimal equivalent is 5.
Now, 5mod3=2.
So ,in state 2 if we see a '1' it will remain in state2.

In state 2 if we see a ‘0’,what do we do?
For example ,if the input is 100,the decimal equivalent is 4.
Now,4 mod 3=1
So ,in state 2 if we see symbol ’1’ we move to state 1.



In state 0 and state 2 we know what happens on 0,1 .
In state 1 we have to figure out what happens if we see symbol 1.
For example,if the input is 11.
The decimal equivalent of 11 is 3.
Now, 3 mod 3 = 0.
So in state 1 if we see symbol ‘1’ we move to state 0.



- This is the DFA for w mod 3=0

AUTOMATA FORMAL LANGUAGES AND LOGIC

Lecture Notes on
Non-deterministic Finite Acceptors
(NFA/λ-NFA)

Prepared by :
Prof. Sangeeta V I
Assistant Professor

Department of Computer Science & Engineering
PES UNIVERSITY
(Established under Karnataka Act No.16 of 2013)
100-ft Ring Road, BSK III Stage, Bangalore – 560 085

Table of Contents

Examples Solved

1.Non-deterministic Finite Accepters/Automata(NFA)

An NFA can be represented by a 5-tuple $(Q, \sum, \delta, q_0, F)$ where –

- Q is a finite set of states.
- $\sum$ is a finite set of symbols called the alphabets.
- δ is the transition function where $\delta: Q \times \sum \rightarrow 2^Q$
  (Here the power set of Q (2^Q) has been taken because in case of NFA, from a state, transition can occur to any combination of Q states)
- q_0 is the initial state from where any input is processed ($q_0 \in Q$).
- F is a set of final state/states of Q ($F \subseteq Q$).

In an NFA, a (state, symbol) combination may lead to several states simultaneously.

If a transition is labeled with the empty string as its input symbol, the NFA may change states without consuming input.

An NFA may have undefined transitions.

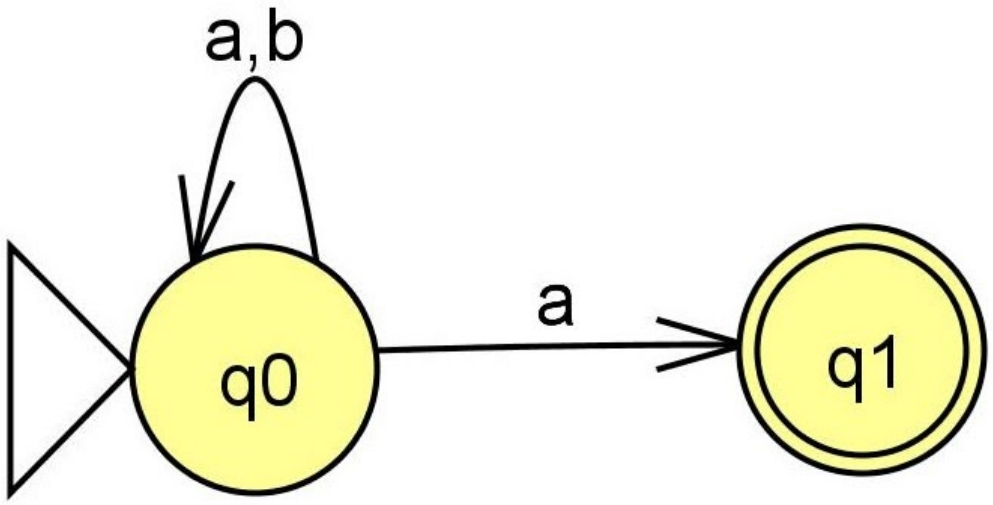
In DFA we have to be determined about the transition of every input symbol on every state.

IN NFA we do not have this restriction.

Example 1:

Assume Σ={a,b}
Consider language L1={set of string that end in ‘a’}
Minimum string : a

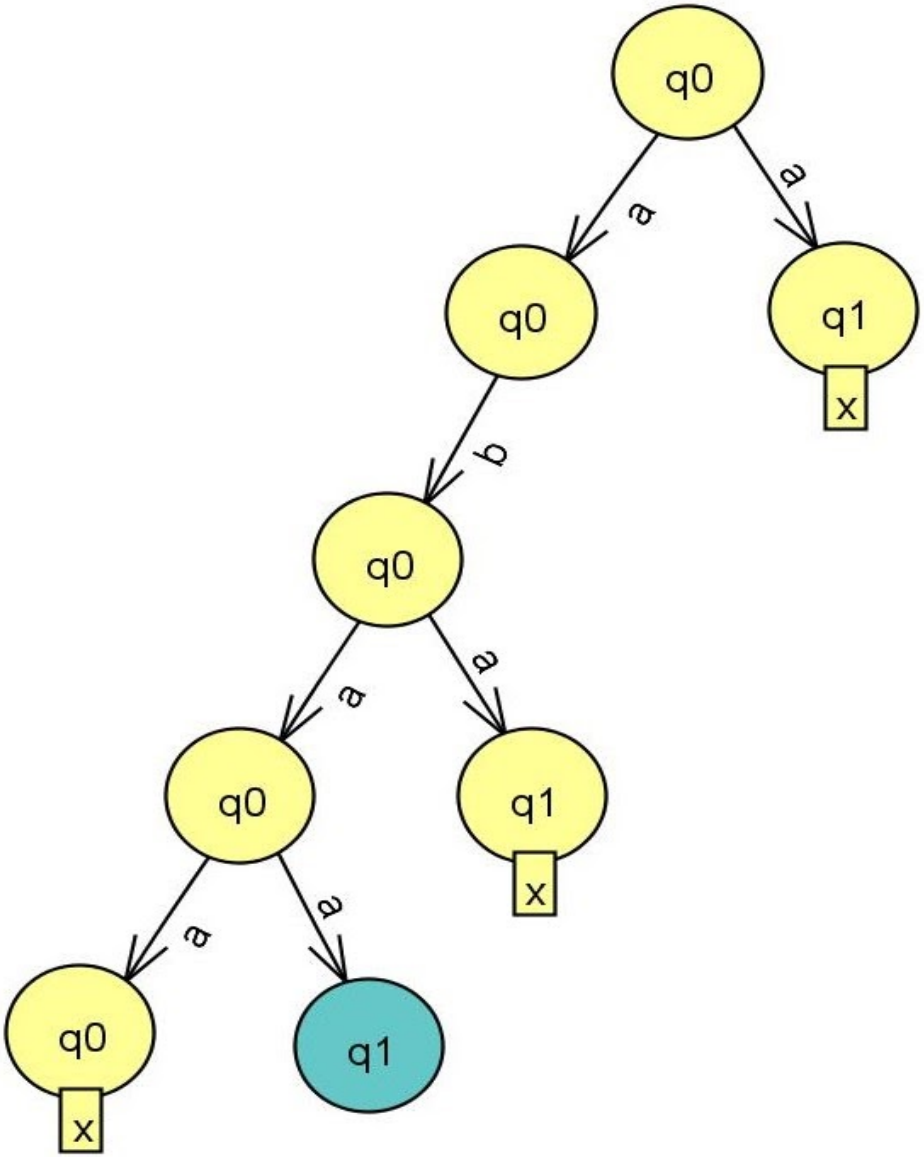
L1={a,(a’s or b’s) a}
We will first construct the NFA for the set of strings ending in a.



- In NFA we can have multiple transitions on the same symbol from a given state or there can be no transition on a symbol from a given state.
- We have two transitions on ‘a’ in q0,one remains in the same state and the other moves to state q1.
- In q1, there is no transition on a and b.This is allowed in NFA but not in DFA.
- It is easier to construct a NFA ,since we don't have to worry about the transition of every symbol in every state.
- We can convert NFA to DFA using an algorithm .

- Computation in NFA for the string abaa.
  We will try out simultaneously all the possible paths the string can take ,even if one of the paths leads us to the final state then we will accept the string.

  abaa



Tracing the string “abaa”

- On q0,if we see **‘a’** there are two possibilities,one it remains in q0 ,other it goes to state q1.We will check both the possibilities and see which path takes us to the final state.

- On q0, we can go to state q1 after seeing ‘a’ but in q1 we do not have any transition ,hence we do not choose this path(we haven't reached the end of the string) .

- We will pick the other path where on q0,after seeing ‘a’ we remain in q0.

- Now,we are in q0,the next symbol from the string “abaa” is **‘b’.**

- In q0 ,on ‘b’ we remain in q0 .

- Now,we are still in q0,the next symbol from the string “abaa” is **‘a’.**

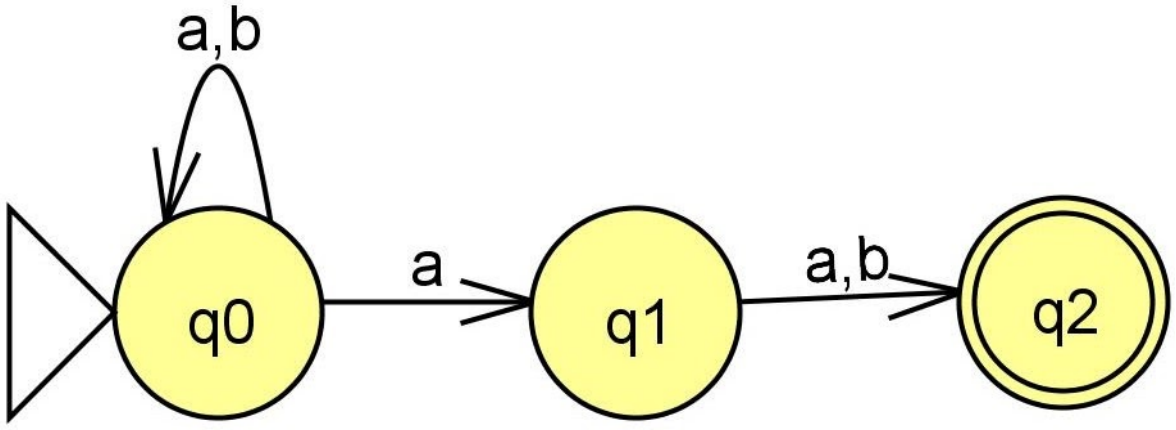
- On q0, we can go to state q1 after seeing ‘a’ but in q1 we do not have any transition ,hence we do not choose this path.

- We will pick the other path where on q0,after seeing ‘a’ we remain in q0.

Example 2:

Assume Σ={a,b}

Consider language L2={Strings where the second symbol from RHS is 'a'}
(a's or b's) a (a or b)

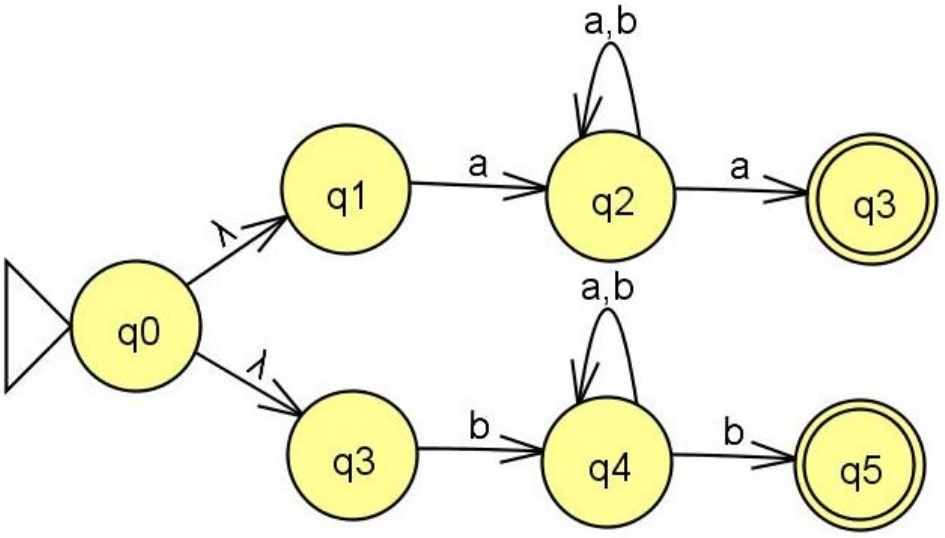


λ-Non-deterministic Finite Acceptors/Automata(λ-NFA)

A nondeterministic finite automata with $\boldsymbol{\lambda}$–transitions is a 5-tuple $(Q, \Sigma, q0, A, \delta)$, where Q and Σ are finite sets, $q0 \in QQ$, $A \subseteq Q$, and $\delta : Q \times (\Sigma \cup \{\boldsymbol{\lambda}\}) \rightarrow 2\ Q$.
When we have a $\boldsymbol{\lambda}$–transition we jump to states without consuming any input.

Example 1:

Assume Σ={a,b}
Consider language L1={Strings that start and end with the same symbol}
L1={awa or bwb |w∈{a,b}*}
We construct two NFA one each for awa and bwb and combine the NFA with a single new start state .



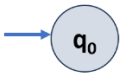
Department of Computer Science and Engineering
PES University, Bangalore, India

UE23CS243A: Automata Formal Language and Logic

Prakash C O, Associate Professor, Department of CSE

FSM Components

- **Input** - A string fed to a machine which the machine will determine whether it is part of the language that the machine was designed for. The string must only be made up of symbols from the machine's alphabet. An input is read by a machine in a forward fashion, one symbol at a time.
- **Return** - The results of running the machine on a given input. Initially, this will either be **accepted** or **rejected**, indicating whether the input string is respectively part of the language or not.
- **State** - A resting place while the machine reads more input, if more input is available. States are typically named. Canonical names for states (which we will get to later) commonly consist of the lowercase letter 'q' followed by a number, e.g. q_0. State names must be unique. Graphically, states are represented by a circle with the name inside.
- **Start State** - For programmers, this is known as the *program entry point*. It is the state that the machine naturally starts in before it reads any input. The name for the start state will usually either be *S* or, canonically, q_0 (it is numbered with the lowest number of all the states). Graphically, the start state is represented as a state with an unlabeled arrow pointing from nothing to it.



- **Accepting State** or **Final State** - **A set of states which the machine may halt in, provided it has no input left, in order to accept the string as part of the language**. Graphically, accepting states are indicated distinctly from other states using a double circle instead of a single.

It is worth noting that a machine with no accepting states does not accept any string as part of the language (not even the empty string) - it is essentially the Empty Language, ∅.

- **Rejecting State** - **Any state in the machine which is not denoted as an accepting state**. The string is only rejected if the machine *halts* in a rejecting state with no more input left. Rejecting states have no special representation, because **every state is either accepting or rejecting.**

- **Dead State** - A rejecting state that is essentially a dead end. Once the machine enters a dead state, there is no way for it to reach an accepting state, so we already know that the string is going to be rejected.

  A machine may have multiple dead states, but at most only one dead state is needed per machine.

- **Transition** - A way for a machine to go from one state to another given a symbol from the input. Graphically, **a transition is represented as an arrow pointing from one state to another**, **labeled with the symbol or symbols that it can read** in order to move the machine from the state at the tail of the arrow to the tip of the arrow.

  Transitions may even point back to the same state that they came from, which is called a self **loop**.

Using this terminology, we can logically deduce that a **Finite State Automaton** or **Finite State Machine** (FSM) is a machine that has a finite number of states.

Why NFA?

DFA may be cumbersome to specify for complex systems and may even be unknown.
NFA provides a nice way of 'abstraction of information' - a concept applied in so many aspects of computing.

NFA are often more convenient to design than DFA. Examples below can be easily designed using NFA than DFA.

1. Strings containing 1 in the third last position.
2. Strings accepting multiples of 2 or 2. (General union)

Nondeterministic finite automaton (NFA)

Formal Definition of an NFA

Formally, an NFA is a 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$

where:

- Q is a finite set of states
- Σ is the input alphabet (symbols which are part of input alphabet)
- δ is a transition function which maps $Q \times \Sigma \rightarrow 2^Q$
- q_0 is the initial (or start) state
- F is the set of accepting (or final) states ($F \subseteq Q$). If any state of F is reached, input string is accepted.

In automata theory, a finite-state machine is called a deterministic finite automaton (DFA), if

- each of its transitions is *uniquely* determined by its source state and input symbol, and
- reading an input symbol is required for each state transition.

A nondeterministic finite automaton (NFA), does not need to obey these restrictions. In particular, every DFA is also an NFA.

NFA provides a nice way of 'abstraction of information'. **In an NFA, a (state, symbol) combination may lead to several states simultaneously.**

DFAs have been generalized to nondeterministic finite automata (NFA) which may have several arrows of the same label starting from a state.

Using subset construction method, every NFA can be translated to a DFA that recognizes the same language. DFAs, and NFAs as well, recognize exactly the set of regular languages.

For an input string, NFA will create a 'computation tree' rather than a 'computation sequence' in case of DFA.
An NFA accepts a string if any one of the paths in the tree leads to some accept state.
Nondeterminism causes multiple runs. There can be several or no runs for a given input word.

Computation or Run of an NFA

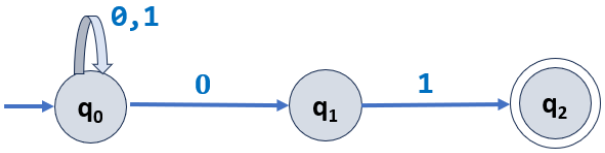
For an input string, NFA will create a 'computation tree' rather than a 'computation sequence' in case of DFA.

An NFA accepts a string if any one of the paths in the tree leads to some accept state.

Nondeterminism causes multiple runs. There can be several or no runs for a given input word.

If there is at least one run that leads to an accepting state, the machine will accept the input string.

Example 1: Consider L = { x ∈ {0, 1}* | where x ends with 01}



Let 10101 be an input word. The following are potential runs.

$q_0 \xrightarrow{1} q_0 \xrightarrow{0} q_1 \xrightarrow{1} q_2 \xrightarrow{0}$ stuck — **Unfinished runs are unacceptable**

$q_0 \xrightarrow{1} q_0 \xrightarrow{0} q_0 \xrightarrow{1} q_0 \xrightarrow{0} q_1 \xrightarrow{1} q_2$ — **Accepted run/Computation**

$q_0 \xrightarrow{1} q_0 \xrightarrow{0} q_0 \xrightarrow{1} q_0 \xrightarrow{0} q_0 \xrightarrow{1} q_0$ — **Rejected run/Computation**

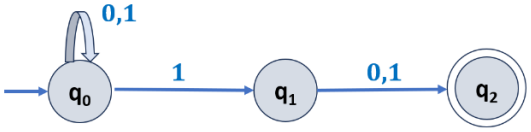
If the word has 01 at the end, there exists an accepting run!

Exercise 1:

Give runs for word 1110.

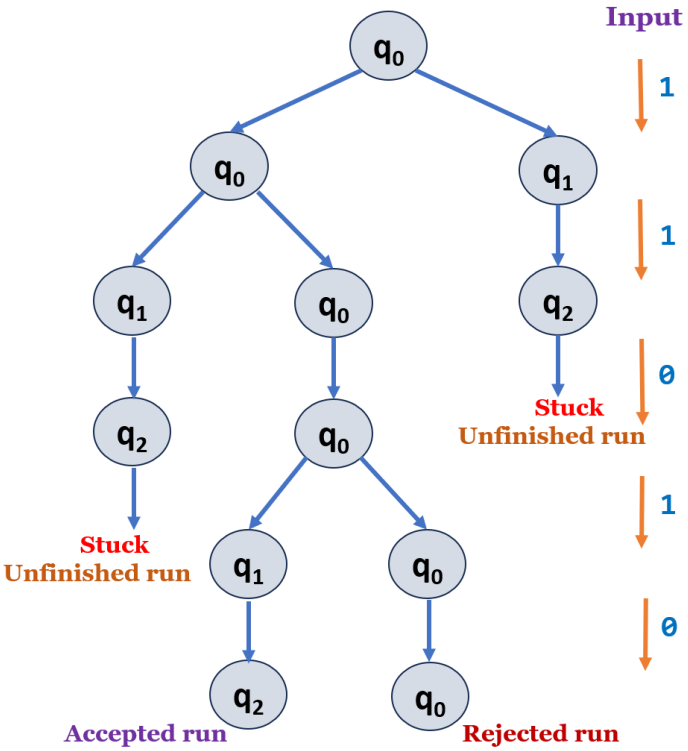
Example 2:

Consider L = {x ∈ {0, 1}* | 2nd symbol from the right is 1}



In the above NFA, the movement from state q_0 when input is '1' is nondeterministic.

Computation tree for the input string 11010



The NFA consumes a string of input symbols, one by one. In each step, whenever two or more transitions are applicable, it "clones" itself into appropriately many copies, each one following a different transition.

If no transition is applicable, the current copy is in a dead end (or stuck), it "dies". If, after consuming the complete input, any of the copies is in an accept state, the input is accepted, else, it is rejected.

Equivalence of DFA and NFA

Surprisingly, for any NFA N there is a DFA D, such that L(D) = L(N), and vice versa. Given an NFA N we will construct a DFA D such that L(D) = L(N)

NFAs and DFAs are equivalent in that if a language is recognized by an NFA, it is also recognized by a DFA and vice versa. The establishment of such equivalence is important and useful. It is useful because constructing an NFA to recognize a given language is sometimes much easier than constructing a DFA for that language.

It is important because NFAs can be used to reduce the complexity of the mathematical work required to establish many important properties in the theory of computation.
For example, it is much easier to prove closure properties of regular languages using NFAs than DFAs.

Note:

- **NFAs have been generalized in multiple ways**, e.g., nondeterministic finite automata with ε-moves, finite-state transducers, pushdown automata, alternating automata, ω-automata, and probabilistic automata.
- Every NFA has an equivalent DFA (accepting the same language).
- The languages accepted by Finite State Machines (DFA, NFA, **ε-NFA**) are referred to as Regular languages.

NFA with ε-transitions (or λ-transitions)

Nondeterministic finite automaton with ε-moves (ε-NFA) is a further generalization to NFA. In this kind of automaton, the transition function is additionally defined on the empty string ε. A transition without consuming an input symbol is called an ε-transition.

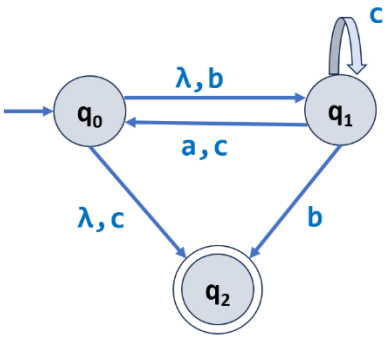
Let us add another feature to our automaton.

Let us allow it to jump states without reading inputs. We will call such moves ε-transitions (or λ-transitions).

ε-closure (or λ-closure) of a state or set of states

For a state q, let ε-closure(q) (or λ-closure(q)) denote the set of states that are reachable from q by ε-transitions (or λ-transitions) including the state q itself.

Example:



λ-closure(q_0) = {q_0 q_1 q_2}
λ-closure(q_1) = { q_1 }
λ-closure(q_2) = { q_2 }

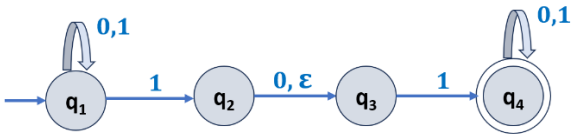
Formal Definition of an ε-NFA (or λ-NFA)

ε-NFA can be represented by a 5-tuple M = (Q, ∑, δ, q_0, F) where

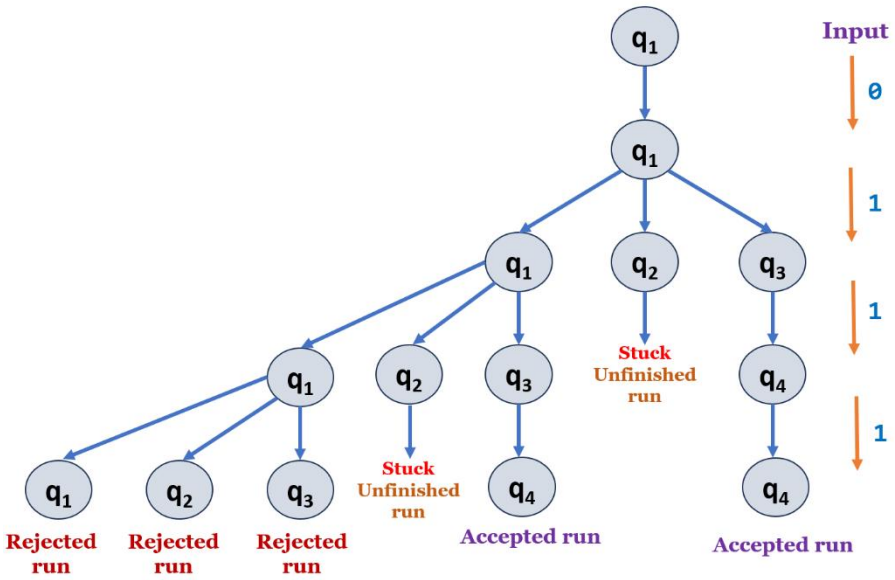
- Q is a finite set of states.
- ∑ is a finite non-empty set of input symbols called the alphabet.
- δ is the transition function where δ: Q × (**Σ** ∪ {ε}) → 2^Q or δ: Q × (**Σ** ∪ {λ}) → 2^Q
- q_0 is the initial state from where any input is processed (q_0 ∈ Q).
- F is a set of final state/states (F ⊆ Q).

Computation or Run of an ε-NFA

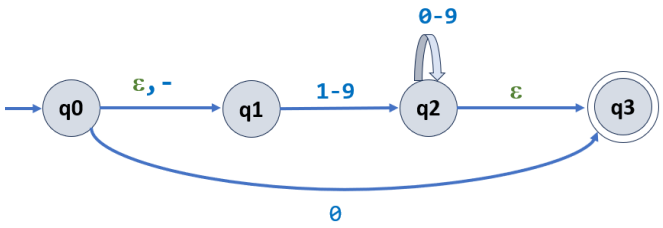
Example 1: Consider L = { x ∈ {0, 1}* | where x contains the substring 101 or 11}



Computation tree for the input string 0111



Example 1: Consider the following automaton with ε -transitions that recognizes integers without leading zeros.

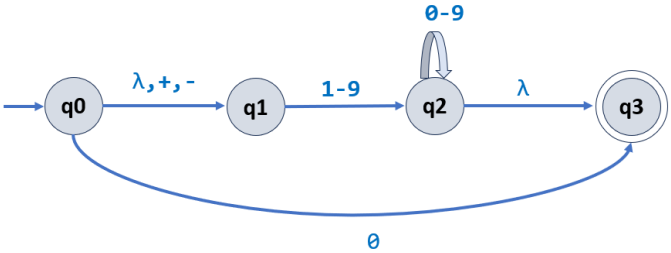


Note:

- '-' is used to specify range of symbols.
- 0-9 means {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}

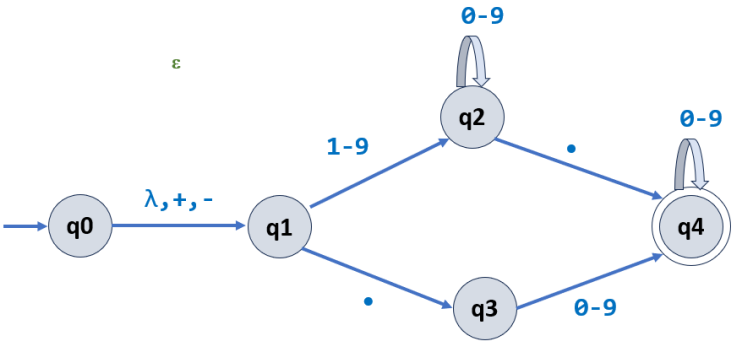
Construct ε-NFA for

- L1 is the set of all strings that are decimal integer numbers. Specifically, L1 consists of strings that start with an optional sign, followed by one or more digits.
  Examples of strings in L1 are "22", "+9", and "-241".

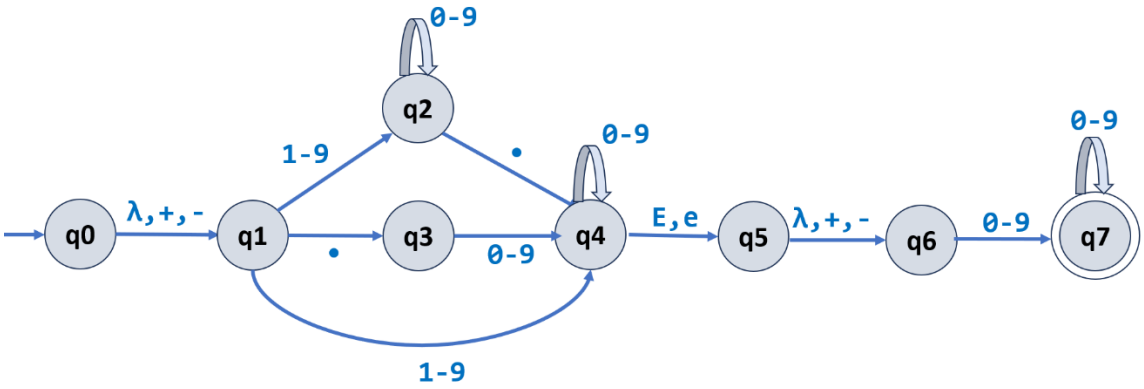


- L2 is the set of all strings that are floating-point numbers that are not in exponential notation. Specifically, L2 consists of strings that start with an optional sign, followed by zero or more digits, followed by a decimal point, and end with zero or more digits, where there must be at least one digit in the string.
  Examples of strings in L2 are "13.231", "-28." and ".124". All strings in L2 have exactly one decimal point.

Σ = {0,1,2,3,4,5,6,7,8,9, . , +, - }



- L3 is the set of all strings that are floating-point numbers in exponential notation. Specifically, L3 consists of strings that start with a string from L1 or L2, followed by "E" or "e", and end with a string from L1.
  Examples of strings in L3 are "-80.1E-083", "+8.E5" and "1e+31".



Converting an NFA to a DFA

Given: A non-deterministic finite state machine (NFA)

Goal: Convert to an equivalent deterministic finite state machine (DFA)

Why?

- Faster recognizer!

IDEA: Each state in the DFA will correspond to a set of NFA states.

Worst-case:

- There can be an exponential number $\mathbf{O}(2^N)$ of sets of states.
- **The DFA can have exponentially many more states than the NFA... but this is rare.**

The idea behind subset construction method:

- Given an NFA N = (Q, Σ, δ, q0, F) we construct the equivalent DFA D:
- **The basic idea of this construction is to define a DFA whose states are sets of NFA states.**
- **A set of possible NFA states thus becomes a single DFA state.**
- **The DFA transition function is given by considering all reachable NFA states for each of the current possible NFA states for each input symbol.**
- The resulting set of possible NFA states is again just a single DFA state.
- **A DFA state is final if that set contains at least one final NFA state**.

The subset construction method (for NFA to DFA) follows the following steps:

Convert the given NFA transition diagram to its equivalent DFA transition table by following the below steps.

Step 1: Make the start state of NFA as the start state of DFA (in DFA transition table)

Step 2: Find the transition of the start state to another set of states on a set of alphabets (set of input symbols).

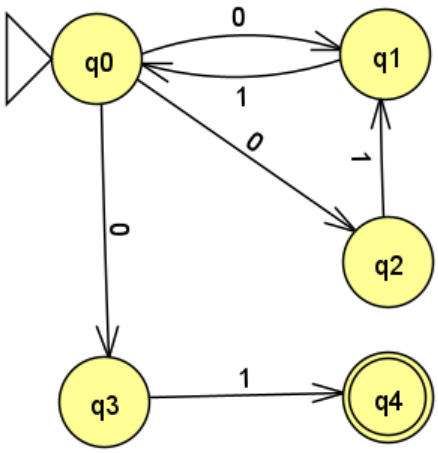
Step 3: Add the newly emerged state as a state of DFA and find the combination of states which the current state transit on the input alphabets.

Step 4: This process is repeated until no new states emerge.

Step 5: Make the state that contains any of the final states of NFA as the final state of its equivalent DFA.

Examples:

1. Convert the following NFA to DFA using subset construction method.



Solution:

Transition table for NFA:

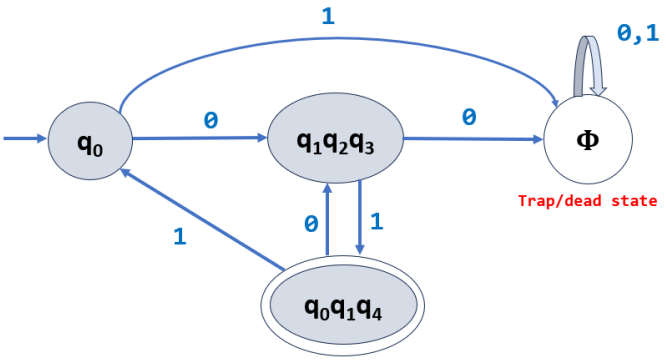
δ	0	1
→q_0	q_1q_2 q_3	Φ
q_1	Φ	q_0
q_2	Φ	q_1
q_3	Φ	q_4
*q_4	Φ	Φ

Note: Φ in transition table represents empty set and Φ in FSM represents Trap state and Φ in formal languages represents empty language.

Transition table for DFA:

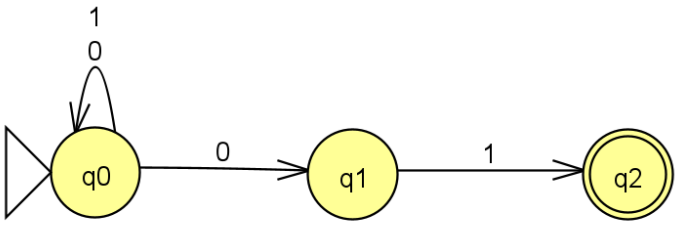
δ	0	1
→q_0	{q_1 q_2 q_3}	Φ
{q_1 q_2 q_3}	Φ	{q_0 q_1 q_4}
*{q_0 q_1 q_4}	{q_1 q_2 q_3}	q_0

DFA:

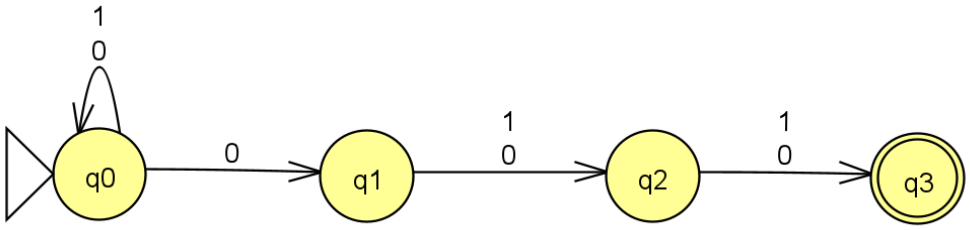


Exercise:

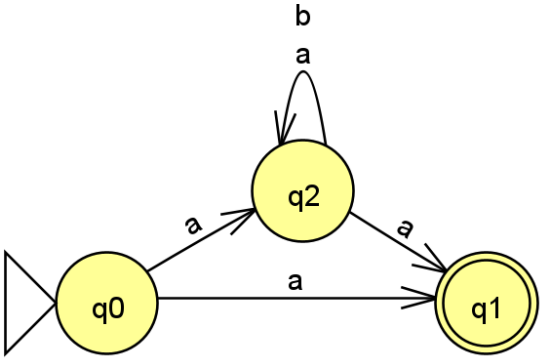
1. Convert the following NFA to DFA using subset construction method.



2. Convert the following NFA to DFA using subset construction method.



3. Convert the following NFA to DFA using subset construction method.



Converting an ε-NFA (or λ -NFA) to a DFA

The subset construction method (for ε-NFA to DFA) follows the following steps:

Convert the given ε-NFA (or λ-NFA) transition diagram to its equivalent DFA transition table by following the below steps.

Step 1: Make the ε-closure of start state of ε-NFA as the start state of DFA (in DFA transition table)

Step 2: Find ε-closure of the transition of the start state to another set of states on a set of alphabets.

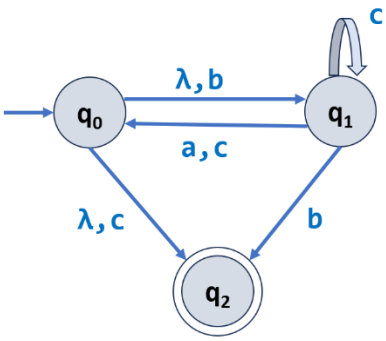
Step 3: Add the newly emerged state as a state of DFA and find the combination of states which the current state transit on the input alphabets.

Step 4: This process is repeated until no new states emerge.

Step 5: Make the state that contains any of the final states of ε-NFA as the final state of its equivalent DFA.

Examples:

1. Convert the following λ-NFA to DFA using subset construction method.



Solution:

λ-NFA Transition table:

δ	a	b	c	λ-closure
→q_0	Φ	q_1	q_2	{q_0 q_1 q_2}
q_1	q_0	q_2	{q_0 q_1 }	{ q_1 }
*q_2	Φ	Φ	Φ	{ q_2 }

DFA Transition table:

The ε-closure of start state of ε-NFA is the start state of DFA.

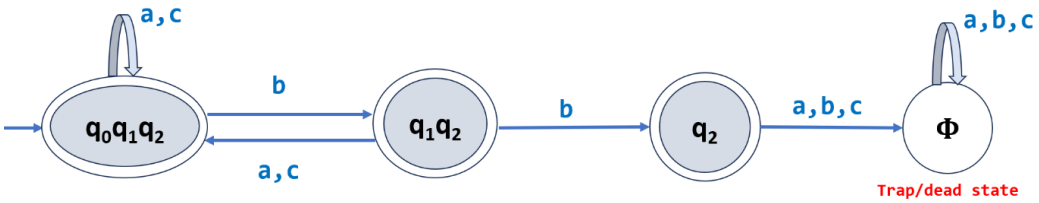
δ	a	b	c
→*{q_0 q_1 q_2}	=λ-closure(δ({q_0 q_1 q_2}, a)) = λ-closure(q_0) = {q_0 q_1 q_2}	=λ-closure(δ({q_0 q_1 q_2}, b)) = λ-closure({q_1 q_2}) = {q_1 q_2} ←new state	=λ-closure(δ({q_0 q_1 q_2}, c)) = λ-closure({q_0 q_1 q_2}) = {q_0 q_1 q_2}
*{q_1 q_2}	=λ-closure(δ({q_1 q_2}, a)) = λ-closure(q_0) = {q_0 q_1 q_2}	=λ-closure(δ({q_1 q_2}, b)) = λ-closure(q_2) = q_2 ←new state	=λ-closure(δ({q_1 q_2}, c)) = λ-closure({q_0 q_1}) = {q_0 q_1 q_2}
*q_2	=λ-closure(δ(q_2, a)) = λ-closure(Φ) = Φ	=λ-closure(δ(q_2, b)) = λ-closure(Φ) = Φ	=λ-closure(δ(q_2, c)) = λ-closure(Φ) = Φ

Note:

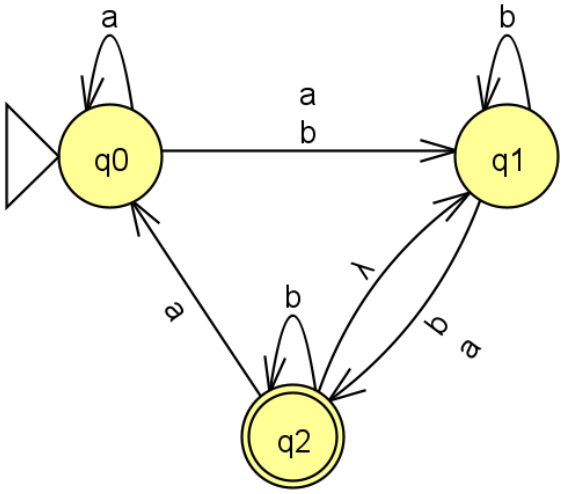
$\delta(\{q_0\ q_1\ q_2\},a) = \delta(q_0,a) \cup \delta(q_1,a) \cup \delta(q_2,a)$

$\lambda\text{-closure}(\{q_0\ q_1\ q_2\}) = \lambda\text{-closure}(q_0) \cup \lambda\text{-closure}(q_1) \cup \lambda\text{-closure}(q_2)$

DFA:



2. Convert the following λ-NFA to DFA using subset construction method.



Solution:

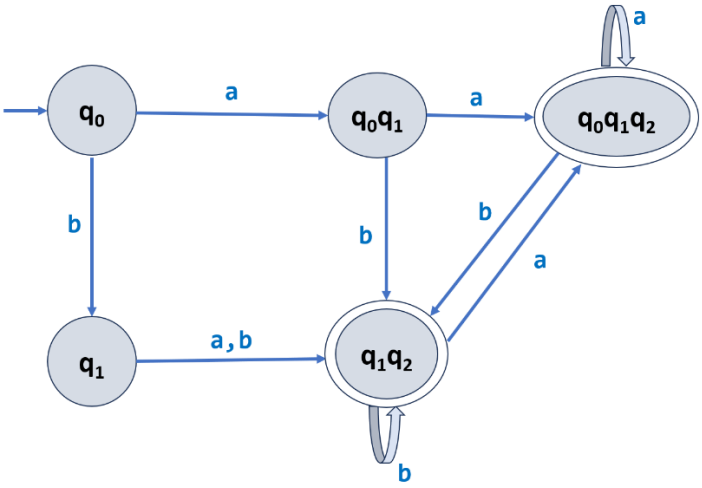
λ-NFA Transition table:

δ	a	b	λ-closure
→q_0	q_0q_1	q_1	{ q_0 }
q_1	q_2	q_1q_2	{ q_1 }
*q_2	q_0	q_2	{ q_1 q_2 }

DFA Transition table:

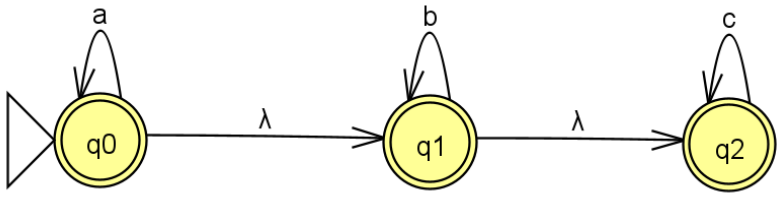
δ	a	b
→{q_0}	=λ-closure(δ({q_0}, a)) = λ-closure({ q_0 q_1 }) = {q_0 q_1} ←new state	=λ-closure(δ({q_0}, b)) = λ-closure(q_1) = { q_1 } ←new state
{q_0 q_1}	=λ-closure(δ({q_0 q_1}, a)) = λ-closure({q_0 q_1 q_2}) = {q_0 q_1 q_2} ←new state	=λ-closure(δ({q_0 q_1}, b)) = λ-closure({q_1 q_2}) = { q_1 q_2} ←new state
{q_1 }	=λ-closure(δ(q_1, a)) = λ-closure(q_2) = { q_1 q_2}	=λ-closure(δ(q_1, b)) = λ-closure({q_1 q_2}) = { q_1 q_2}
*{q_0 q_1 q_2}	=λ-closure(δ({q_0 q_1 q_2}, a)) = λ-closure({q_0 q_1 q_2}) = {q_0 q_1 q_2}	=λ-closure(δ({q_0 q_1 q_2}, b)) = λ-closure({q_1 q_2}) = { q_1 q_2}
*{q_1 q_2}	=λ-closure(δ({q_1 q_2}, a)) = λ-closure({q_0 q_2}) = {q_0 q_1 q_2}	=λ-closure(δ({ q_1 q_2}, b)) = λ-closure({q_1 q_2}) = { q_1 q_2}

DFA:



Exercise:

1. Convert the following λ-NFA to DFA using subset construction method.



DFA Minimization by Table filling Algorithm:

Let the DFA be D= (Q, Σ, δ, q_0, F),

I. Remove the unreachable states, if any.

II. The table filling algorithm for minimizing the given DFA is as follows:

1) **Construct a table (triangular matrix table) of pairs (x, y) where x and y are the states in DFA, x,y ∈ Q, initially all the entries in the table are unmarked.**
   Note: While writing state names horizontally (i.e., from left to right) in table, consider only first state to last but one state, and while writing state names vertically (i.e., from top to bottom) in table, consider only from second state to last state.

2) **Repeat the following computation upwards in table for all the cells.**
   Mark the state pair (x, y) as distinguishable (using the symbol ×), iff x ∈ F and y ∉ F or the other way around. This marked pairs are called distinguishable (i.e., not same or not similar) pairs.

3) **Repeat the below computation upwards in table for all the unmarked cells following the step-2 process.**
   If pair (x, y) is unmarked and there exists a symbol a ∈ Σ such that a pair {δ(x, a), δ(y, a)} is marked then mark the pair (x, y) as distinguishable.

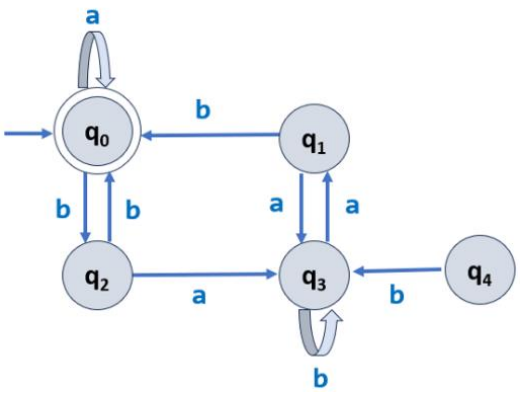
4) **After step-3 completion, if any pair (x,y) is not marked (i.e., indistinguishable), that pair can be merged and considered as a single state.**

Note:

- **If pairs (x1, y1), (x1, y2) and (x2, y2) are indistinguishable and they have common states among them, then we make single merged new state as (x1, y1, x2, y2)**
- If 2 states are not distinguished by table-filling algorithm, then they are equivalent.
- **DFA Minimization by Table filling Algorithm is known as Myhill-Nerode Theorem.**

Example 1:

1) Minimize the following DFA using table filling algorithm (aka Myhill-Nerode Theorem). Clearly mention the start and final state(s).



Solution:

DFA transition table:

δ	a	b
→ *q0	q0	q2
q1	q3	q0
q2	q3	q0
q3	q1	q3

I. Eliminate unreachable state q4.

II. The table filling algorithm for minimizing the given DFA is as follows: (4-Marks)

1) Construct a table (triangular matrix table) of pairs (x, y) where x and y are the states in DFA, x,y ∈ Q, initially all the entries in the table are unmarked.

Note: While writing state names horizontally (i.e., from left to right) in table, consider only first state to last but one state, and while writing state names vertically (i.e., from top to bottom) in table, consider only from second state to last state.

q1			
q2			
q3			
	q0	q1	q2

2) Repeat the following computation upwards in table for all the cells.

Mark the state pair (x, y) as distinguishable (using the symbol ×), iff x ∈ F and y ∉ F or the other way around. This marked pairs are called distinguishable (i.e., not same or not similar) pairs.

q1	X		
q2	X		
q3	X		
	q0	q1	q2

3) **Repeat the below computation upwards in table for all the unmarked cells following the step-2 process.**
If pair (x, y) is unmarked and there exists a symbol a ∈ Σ such that a pair {δ(x, a), δ(y, a)} is marked then mark the pair (x, y) as distinguishable.

Consider unmarked cell (q1, q3):

{δ(q1, a), δ(q3, a)} = **(q3, q1)** [Getting the same pair(just ignore), then check with other input symbols]

{δ(q1, b), δ(q3, b)} = **(q0, q3)**

The pair (q0, q3) is already marked distinguishable then mark the pair (q1, q3) also as distinguishable.

Consider unmarked cell (q2, q3):

{δ(q2, a), δ(q3, a)} = **(q3, q1)**

The pair (q3, q1) is already marked distinguishable then mark the pair (q2, q3) also as distinguishable.

We came to know that the pair (q2, q3) is already distinguishable, no need to check the pair (q2, q3) for other input symbols.

Consider unmarked cell (q1, q2):

{δ(q1, a), δ(q2, a)} = **(q3, q3)** ← **this pair does not exist, just ignore**

{δ(q1, b), δ(q2, b)} = **(q0, q0)** ← **this pair does not exist, just ignore**

q1	X		
q2	X		
q3	X	X	X
	q0	q1	q2

4)After step-3 completion, if any pair (x, y) is not marked (i.e., indistinguishable), that pair can be merged and considered as a single state in the minimized DFA.

The pair (q1, q2) is not marked (i.e., indistinguishable), that pair can be merged and considered as a single state.

q1	X		
q2	X		
q3	X	X	X
	q0	q1	q2

DFA Transition table:

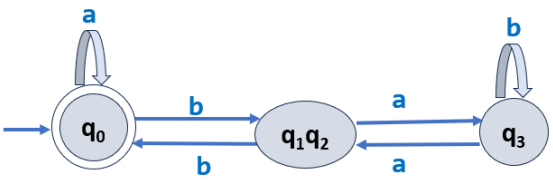
δ	a	b
→ *q0	q0	q2
q1	q3	q0
q2	q3	q0
q3	q1	q3

The indistinguishable pair (q1, q2) is merged as a single state and its transitions are also merged. The all occurrences of q1 and q2 in other state transitions are replaced by the pair {q1 q2}

Minimized DFA Transition table:

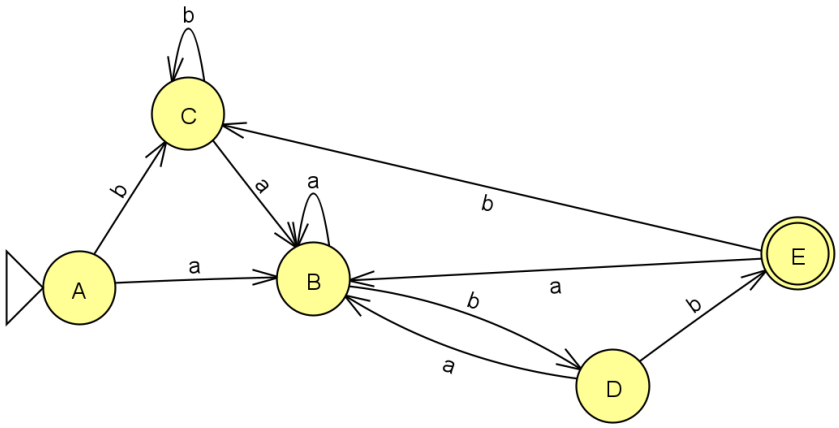
δ	a	b
→ *q0	q0	q1q2
{q1q2}	q3	q0
q3	q1q2	q3

Minimized DFA:

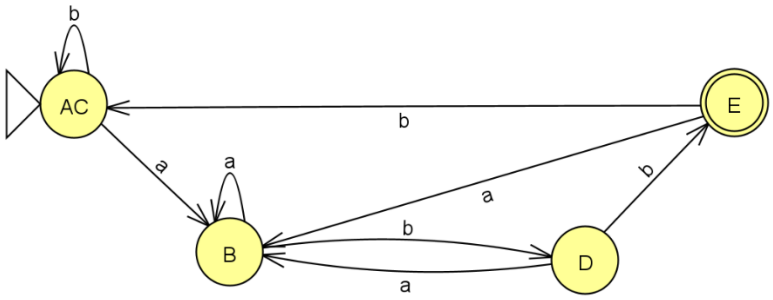


Exercise:

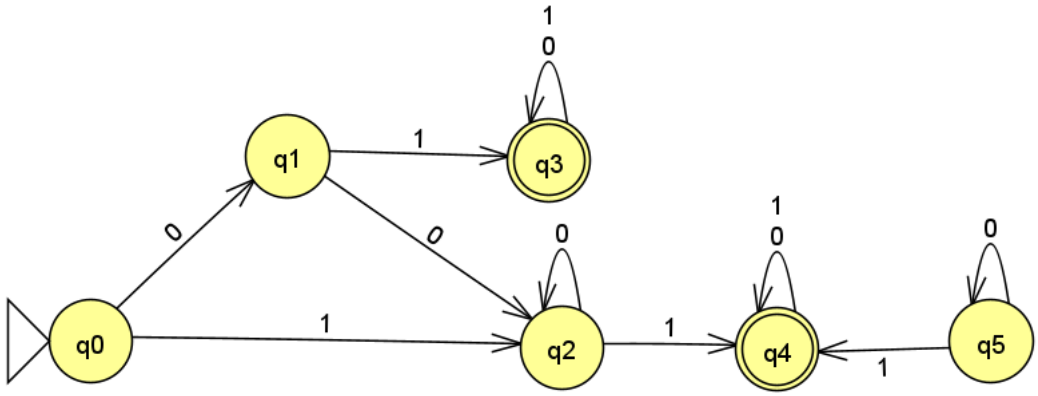
1) Minimize the following DFA using table filling algorithm (aka Myhill-Nerode Theorem). Clearly mention the start and final state(s).



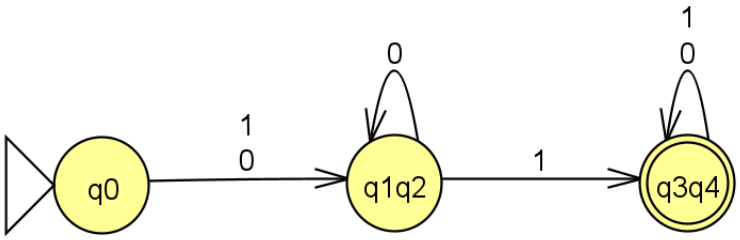
Minimized DFA:



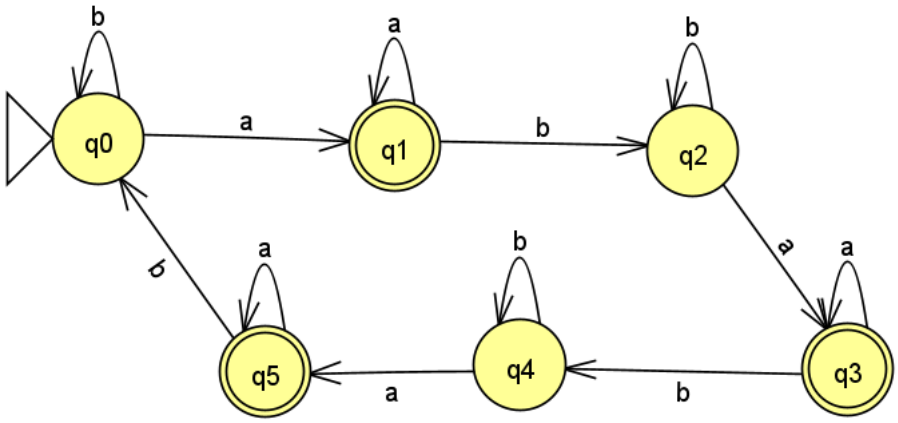
2) Minimize the following DFA using table filling algorithm (aka Myhill-Nerode Theorem). Clearly mention the start and final state(s).



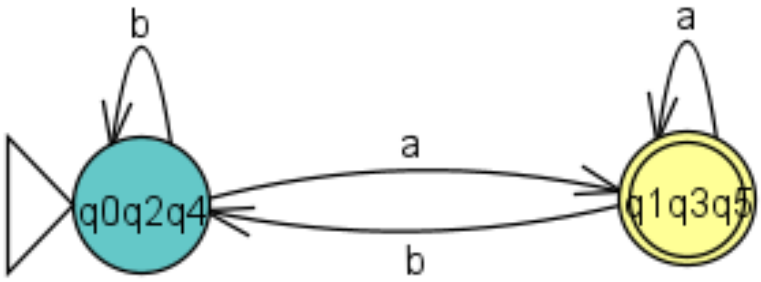
Minimized DFA:



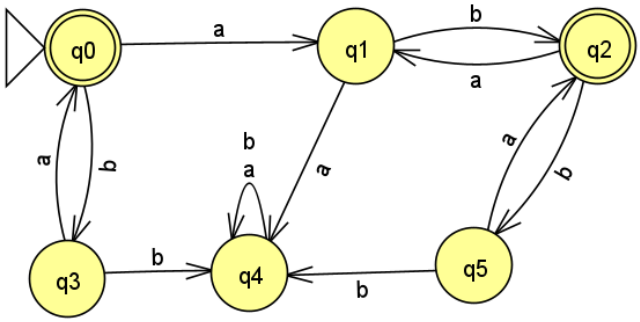
3) Minimize the following DFA using table filling algorithm (aka Myhill-Nerode Theorem). Clearly mention the start and final state(s).



Minimized DFA:



4) Minimize the following DFA using table filling algorithm (aka Myhill-Nerode Theorem). Clearly mention the start and final state(s).



Equivalence of Finite State Machines

The two finite automata (FA) are said to be equivalent if both accept the same set of strings over an input set Σ.

When two FA's are equivalent then, there is some string w over Σ. On acceptance of that string, both FA's should be in final state.

Method

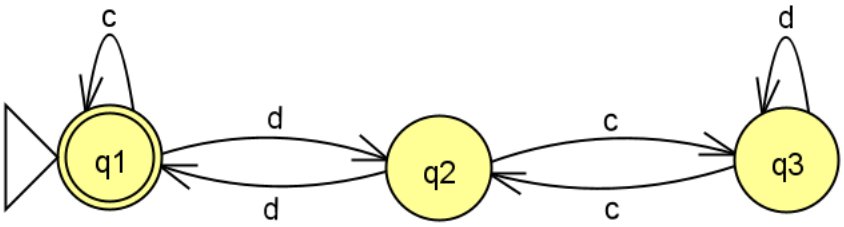
The method for comparing two FA's is explained below:

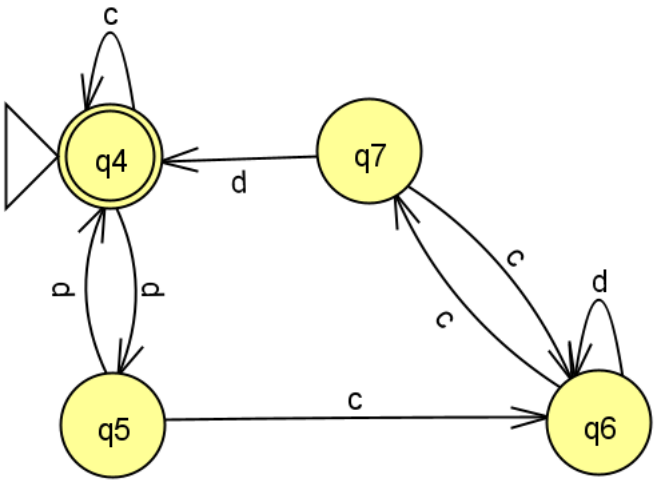
Let M1 and M2 be the two FA's and Σ be a set of input symbols.

- **Step 1: Construct a transition table that has pairwise entries (p, q) where p ∈ M1 and q ∈ M2 for each input symbol. Start from the start state pair.**
- **Step 2: If we get in a pair as one final state(FS) and other non-final state(NFS) then we terminate the construction of transition table declaring that two FA's are not equivalent**
- **Step 3: The construction of the transition table gets terminated when there is no new pair appearing in the transition table.**

Example:

Consider the two Deterministic Finite Automata (DFA) and check whether they are equivalent or not.





The transition table is as follows:

States	c	d
{q1, q4}	{q1, q4} FS FS	{q2, q5} NFS NFS
{q2, q5}	{q3, q6} NFS NFS	{q1, q4} FS FS

States	c	d
{q3, q6}	{q2, q7} NFS NFS	{q3, q6} NFS NFS
{q2, q7}	{q3, q6} NFS NFS	{q1, q4} FS FS

Here, FS is the Final State and NFS is the Non-final State.

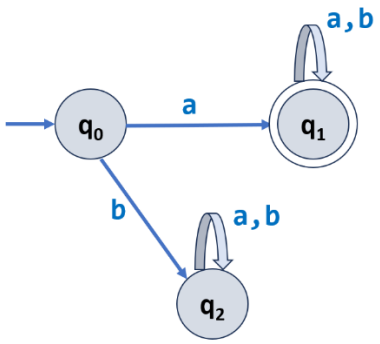
Therefore, by seeing the above table it is clear that we don't get one Final State and one Non-final State in any pair. Hence, we can declare that two DFA's are equivalent.

Complement of FSM's

Example:

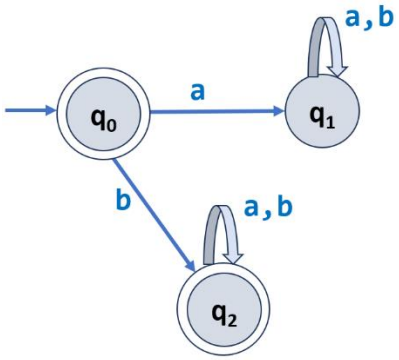
1) Find the complement of a given Deterministic Finite Automata (DFA).

M:



Solution: **Toggle final and non-final states.**

M^C:



Note:

1) Complement of machine should also complement language recognised by FSM.

L={ a, ab, aa, aba, abb, aaa, aab, ...}

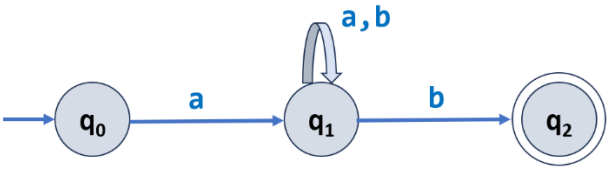
L^C={ ε, b, ba, bb, bba, bbb, baa, bab, ...}

2) Complement of NFA may or may not work.

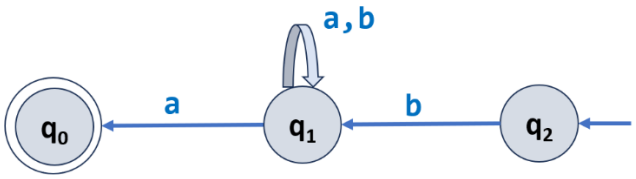
Reversal of a language through Finite Automata

Example: Find the reversal of a below language through Finite Automata.

a) L = { awb | w ∈ {a, b}* }

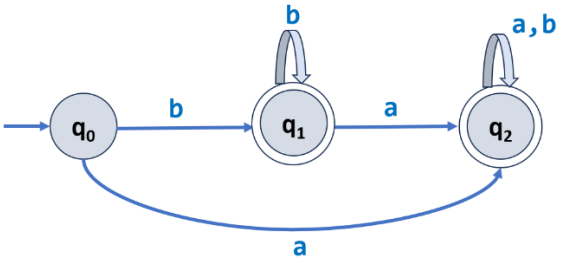


Solution: Toggle initial and final state and reverse the arrows.



L^R = { bwa | w ∈ {a, b}* }

Exercise: Find the language reversal of below Finite Automata.



Note 1: Do finite automata have memory?

- **FSM doesn't have memory, It can't remember the things unlike Push Down Automata (i.e nothing but a FSM with Memory). Anything which require memory can't accepted by FSM.**
- All finite automata have "state" which is a form of memory. Any memory in addition to the state, make the finite automaton belong to a different class. So, strictly speaking DFAs and NFAs, which is what most people mean by **finite automata have no memory other than their state and in that sense are considered to have no memory.**
- Each state in a finite automaton represents a specific condition or configuration of the system. When an input is given, the automaton transitions from one state to another according to predefined rules. This transition is influenced by the current state and the input.
- While finite automata have memory in the form of states, they have limited memory capacity. They can only store information about their current state and cannot remember details about past states. This limited memory makes finite automata efficient for solving specific computational problems that do not require extensive memory.
- It's important to note that there are different types of finite automata with varying memory capabilities. For example, deterministic finite automata (DFA) have a fixed memory and can only be in one state at a time. On the other hand, nondeterministic finite automata (NFA) can be in multiple states simultaneously, allowing for more flexible memory representation.

- While finite automata have memory in the form of states, their memory capacity is limited and restricted to the current state.

Note 2:

- A scanner (i.e., lexical analyser) is a concrete realization of a finite automaton, a type of formal machine.
- A parser (i.e., syntax analyser) is a concrete realization of a push-down automaton. Just as context-free grammars add recursion to regular expressions, push-down automata add a stack to the memory of a finite automaton.
- It can be proven, constructively, that regular expressions and finite automata are equivalent: one can construct a finite automaton that accepts the language defined by a given regular expression, and vice versa.

AUTOMATA FORMAL LANGUAGES AND LOGIC

Lecture notes
Regular Expression

Prepared by:
Kavitha K N
Assistant Professor

Department of Computer Science & Engineering
PES UNIVERSITY
(Established under Karnataka Act No.16 of 2013)
100-ft Ring Road, BSK III Stage, Bangalore – 560 085

Table of Contents:

Examples Solved:

12	Length of the string is greater than or equal to 2(at least 2)	**9**
13	Length of the string is at most 2	**9**
14	Length of the string is even	**9**
15	Length of the string is odd	**9**
16	Length of the string |w| mod 3=0	**9**
17	Length of the string |w| mod 3=2	**9**
18	Number of a's in the strings is equal to 2	**9**
19	Number of a's in the string should be greater than or equal to 2	**10**
20	Number of a's in the string should be utmost 2	**10**
21	L = { n_a (w) mod 2 = 0}	**10**
22	L = { n_a (w) mod 2 = 1}	**10**
23	L=($a^n b^m$: n+m is even}	**10**
24	L=($a^{2n} b^{2m}$: n,m >=2 }	**10**
25	L=($a^n b^m$: n,m >=1 and n*m >=3}	**10**
26	L=($a^n b^m$: n>=4 and m<=3}	**11**
27	L={$a^n b^m c^k$: n+m is odd and k is even}	**11**
28	L={$a^n b^m c^k$: n<=4, m>=2, k<=2}	**11**
29	L={aba, a^5 ,aba^7, a^{11},aba^{13} , …}	**11**
30	L ={VUV : U,V ∈ {a,b}* and |V| =2}	**11**
31	If the binary string is even then the length of the string is even, and if the binary string is odd then the length of the string is odd.	**11**

32	**L={ Binary string whose decimal equivalent is twice an odd number}**	**12**
33	L={binary strings that do not have two consecutive zero}	**12**
34	L= {binary strings that have exactly one pair of 0's}	**12**
35	L= (Strings over the alphabet a and where it does not have 2 consecutive a's and 2 consecutive b's}	**12**

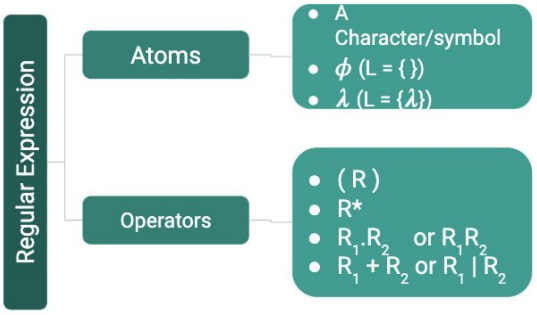
1. Regular expression

Regular expressions provide a convenient and concise notation for describing regular languages. They are Used as the basis for numerous software systems (Perl, flex, grep, etc.)

A regular expression is a pattern consisting of a sequence of characters that are matched against a text, more formally a string. If the string and the pattern match, the string belongs to the language denoted by the Regular expression.

More formally we can say regular expressions are an algebraic way to describe regular languages. It is like a mathematical expression.

Like how a mathematical expression is made of operands (data) and operators.
A regular expression is made of atoms and operators.



The atom specifies what we are looking for and where in the text the match is to be made.
The operator combines atoms into complex expressions.

Atomic Regular Expressions :
The regular expressions begin with three simple building blocks.
A character or a symbol for example (a, b, 0, 1 etc)
● The symbol Ø is a regular expression that represents the empty language Ø.
● The symbol lambda is a regular expression that represents the language which contains empty string{ λ }

We can make complex/Compound Regular Expressions by combining these atomic regular expressions using operators in four ways:

1) If R is a regular expression, (R) is a regular expression with the same meaning as R.
2) If R is a regular expression, R* is a regular expression for the Kleene closure of the language of R.
3) If R1 and R2 are regular expressions, R1 | R2 is a regular expression for the union of the languages of R1 and R2 .
4) If R1 and R2 are regular expressions, R1R2 is a regular expression for the concatenation of the languages of R1 and R2 .

The precedence of the operators is as follows:

- Brackets have the highest precedence(R)
- Then kleene closure R*
- Followed by concatenation R1R2
- And lastly union has the least precedence, that means it will be evaluated at last.R1 | R2

1.2 Algebraic laws of Regular Expressions :

a) **Identity of Regular Expression:**
An Identity of an operator is a value that when the operator is applied to the identity and some other value, the result is the other value
-> Union:
Identity of union operator is Ø
Ø + A = A + Ø = A
-> Concatenation:
Identity of Concatenation is λ
λ .A = A. λ = A

-> R . ϕ = ϕ . R = ϕ :

Let L_1, L_2 be languages, then the concatenation $L_1 \circ L_2 = \{w \mid w = xy, x \in L_1, y \in L_2\}$. If $L_2 = \emptyset$, then there is no string $y \in L_2$ and so there is no possible w such that $w = xy$. Thus for any L_1, we'll have $L_1 \circ \emptyset = \emptyset$.

b) **Associativity Laws for Regular Expressions**
-> Union:
Union operator is associative
A + (B + C) = (A + B) + C
-> Concatenation:
Concatenate operator is associative
A.(B.C) = (A.B).C

c) **Commutativity for Regular Expressions**
-> Union:
Union operator is Commutative
A + B = B + A
-> Concatenation:
Concatenate operator is not Commutative
AB ≠ BA

1.3 Basic regular expressions

1) **a**

If the regular expression is 'a' then the language has only one single String and can be represented as L={ a}

2) **a***

This regular expression means 0 or more occurrences of a

L={ λ , a,aa,aaa,aaaa…}

3) **a+**

1 or more more occurrences of a

L={a,aa,aaa,aaaa…}

4) **(a+b) can also be represented as (a|b)**

This regular expression means either a or b

L={a,b}

5) **(a+b)***

Set of strings greater than equal to 0 over the alphabet a or b

L={ λ ,a,b,aa,bb,ab,ba.aab…}

$(a+b)^0$

String of length 0

L={ ε }

= Σ^0

$(a+b)^1$

Set of all strings with length 1

L={a,b}

$(a+b)^2$

Set of all strings of length 2

(a+b)(a+b) or L={aa,ab,ba,bb}

1.2 Construction of regular expressions

-> Obtain regular expression for the following over the alphabet Σ {a,b}:

1) **Strings ending with ab**

The string should end with ab, it can start with any number of a's and b's in any order of any length

RE= (a+b)*ab

2) **Strings starts with ab**

The string should start with ab, it can end with any number of a's and b's in any order of any length

RE=ab(a+b)*

3) **String contains ab**

The string should mandatorily contain the substring ab, before the substring ab, there can be an any number of a's and b's in any order of any length and after the

substring ab, there can be an any number of a's and b's in any order of any length

RE= (a+b)* ab (a+b)*

4) **String should end with ab or ba**

The string should either end with ab or ba, it can start with any number of a's and b's in any order of any length

RE=(a+b)*(ab+ba)

5) **Strings that start and end with same symbol**

The string can either start and end with a or it can start and end with b and can have any number of a's and b's in any order of any length in between.

RE=a (a+b)* a + b(a+b)*b

6) **Strings that start and end with different symbol**

The string can either start with a and end with b or it can with start b and end with a and can have any number of a's and b's in any order of any length in between.

RE=a(a+b)*b + b(a+b)*a

7) **String where the 3rd symbol from the left (from starting) is 'a'(LHS)**

The first and second symbol can be either a or b 3 rd symbol should be a after that the string can have any number of a's and b's in any order of any length

RE=(a+b)(a+b)a(a+b)*

8) **String where the 2nd symbol from the right (from ending) is a(RHS)**

The last symbol can be either a or b, last but one symbol should be 'a' , the string can start with number of a's and b's in any order of any length

RE=(a+b)*a (a+b)

9) **String where at least one 'a' in the first three symbols and at least one 'b' in the last two Symbols.**

'a' can be 1st , 2nd or 3rd symbol in the first three symbols and be can be last or last but one symbol in the last two symbols , between first three and last 2 symbols there can be any number of a's and b's in any order of any length

RE= (a(a+b)(a+b)) + ((a+b) a(a+b)) + (a+b)(a+)a (a+b)* (b(a+b)) + (a+b)b

10) Strings contain at least one a and at least one b.

The minimum length of the string is 2,

RE=(a+b)* a (a+b)* b (a+b)* + (a+b)* b (a+b)* a (a+b)*

11) Length of the string is exactly equal to 2

|w|=2

(a+b)(a+b) or (aa+ab+ba+bb)

12) Length of the string is greater than or equal to 2(at least 2)

|w|>=2

(a+b)(a+b)(a+b)*

13) Length of the string is at most 2

|w|<=2

Here you can have strings of length {0,1,2}

(λ +a+b+aa+ab+ba+bb)

=(λ +(a+b))(λ +(a+b))

Mod counter questions:

14) Length of the string is even

|w| mod 2=0

Generate strings in multiples of 2,

RE = ((a+b)(a+b))*

15) Length of the string is odd

|w| mod 2=1

RE = ((a+b)(a+b))*(a+b)

16) Length of the string |w| mod 3=0

RE = ((a+b)(a+b)(a+b))*

17) Length of the string |w| mod 3=2

RE = ((a+b)(a+b)(a+b))*(a+b)(a+b)

Regular expressions based on restriction to the individual symbols Σ = {a , b}

18) Number of a's in the strings is equal to 2

L = { n_a (w) =2, w $\in$ {a,b}*}

Number of a's in the string should be exactly 2, and no restriction on the number of b's in the starting or in the middle or in the end.

RE = b* a b* a b*

19) Number of a's in the string should be greater than or equal to 2

L = { n_a (w) >=2}

We must have at least 2 a's and later we can have any number of a's, there is no restriction on number of b's

RE = b* a b* a b* (a+b)*

20) Number of a's in the string should be at most 2

L = { n_a (w) <=2}

RE = b*+b*ab*+b*ab*ab*

0r

RE = b* (a+ λ)b* (a+ λ)b*

21) L = { n_a (w) mod 2 = 0}

You can have a's which is multiple of 2 any number of times, or you can have only b's

RE = (b* a b* a b*)* + b*

22) L = { n_a (w) mod 2 = 1}

RE = (b*ab*ab*)* b* a b*

23) L=($a^n b^m$: n+m is even}

Length of the string is even, We can get even length if number of a's is even and number of b's is even or number of a's is odd and number of b's is odd so we have two cases,

RE = (aa)*(bb)* + (aa)*a(bb)*b

24) L=($a^{2n} b^{2m}$: n,m >=1 }

The minimum string is aabb

RE = aa(aa)* bb(bb)*

25) L=($a^n b^m$: n,m >=1 and n*m >=3}

Here we have to consider different cases to get the product n * m >=3. If the number of a's is one and number of b;s is greater than or equal to 3 then the product would be greater than or equal to 3 or, number of a's can be greater than or equal to 3 and number of b's is 1 them the product is

greater than or equal to 3, or number a's and b's is greater than or equal to 2 then the product is greater than 3, So,

RE = abbbb* + aaaa*b + aaa*bbb*

26) $L=\{a^n b^m : n>=4 \text{ and } m<=3\}$

Number of a's is at least 4 and the number of b's is at most 3

RE = aaaaa* (λ + b+bb+bbb)

Or

RE = aaaaa* ((λ + b)(λ + b)(λ + b))

27) $L=\{a^n b^m c^k : n+m \text{ is odd and } k \text{ is even}\}$

Note: odd+even=odd and even+odd=odd

RE = ((aa)*a(bb)* + (aa)* (bb)*b) (cc)*

28) $L=\{a^n b^m c^k : n<=4, m>=2, k<=2\}$

Number of a's is at most 4, number of b's is at least 2 and number of c's is at most 2

RE = (λ +a)(λ +a)(λ +a)(λ +a) bbb* (λ +c)(λ +c)

29) $L=\{aba, a^5, aba^7, a^{11}, aba^{13}, \ldots\}$

RE = $(aba + a^5)(a^6)$

Or

(aba + aaaaa) (aaaaaa)*

30) $L=\{VUV : U,V \in \{a,b\}^* \text{ and } |V| =2\}$

RE = (a+b)(a+b)(a+b)*(a+b)(a+b)

Regular expressions on binary strings Σ ={0,1}

31) If the binary string is even then the length of the string is even, and if the binary string is odd then the length of the string is odd.

RE = ((0+1)(0+1))*(0+1)0 + ((0+1)(0+1))*1

Recognize pattern

32) L={ Binary string whose decimal equivalent is twice an odd number}

2* Odd no	Decimal value	Binary string
2*1	2	10
2*3	6	110
2*5	10	1010
2*7	14	1110
2*9	18	10010

The pattern what we can observe here is all the binary strings ends in 10

RE = (0+1)*(10)

33) L={binary strings that do not have two consecutive zero}

-> 01 as the building block:

RE = (1+01)*(0+ λ)

-> 10 as the building block:

RE = (0+ λ)(1+10)*

34) L= {binary strings that have exactly one pair of 0's}

RE = (1+01)* 00 (1+10)*

35) L= (Strings over the alphabet a,b and where it does not have 2 consecutive a's and 2 consecutive b's}

RE = (b+ λ)ab (a+ λ)

Or

RE =(a+ λ) ba (b+ λ)

Department of Computer Science and Engineering
PES University, Bangalore, India

UE23CS243A: Automata Formal Language and Logic

Prakash C O, Associate Professor, Department of CSE

Regular Language:

- **A regular language is a formal language that can be defined by a regular expression.**
- **A formal language is called regular if it is accepted by a finite state automaton.**

A Regular language can be specified either

- **by a regular expression that expresses a regular language as a pattern to which every string in the regular language conforms.**
- **by a regular grammar that generates the strings of the regular language, or**
- **by a finite-state automaton that accepts (or recognizes) the regular language.**

Note:

1. **There is a unique minimal DFA for every regular language.**
2. **Regular expressions and finite automata are equivalent: one can construct a finite automaton that accepts the language defined by a given regular expression, and vice versa.**

Regular Expression (shortened as Regex)

Regular expressions originated in 1951, when mathematician Stephen Cole Kleene described regular languages using his mathematical notation called *regular events*.

Regex Definitions:

- A **regular expression** is **a concise way to describe a set of strings in a language**.
- **A regular expression is an algebraic way to describe a regular language.**
- A **regular expression** is **a sequence of characters that define a search/match pattern in text.** Usually, such patterns are used by string-searching algorithms for "find" or "find and replace" operations on strings, or for input validation

Regex Uses:

Regular expressions are used in

1. **search engines,**
2. **search and replace dialogs of word processors and text editors,**
3. **text processing utilities such as sed, grep and AWK,** and
4. **Lexical analysis.**

Note:

Regular expressions are widely used to find patterns in texts. All programming languages provide libraries to handle regular expressions.

1. **Sed** (**stream editor-most common use of SED command in UNIX is for substitution or for find and replace**) is a **Unix utility that parses and transforms text**.
2. **AWK** (Aho, Weinberger, and Kernighan) is a **domain-specific language designed for text processing and typically used as a data extraction and reporting tool**. Like sed and grep, it is a filter, and is a standard feature of most Unix-like operating systems.
   AWK Operations:
   (a) Scans a file line by line
   (b) Splits each input line into fields
   (c) Compares input line/fields to pattern
   (d) Performs action(s) on matched lines
3. The **grep** (**"global regular expression print"**) **filter searches a file for a particular pattern of characters**, and displays all lines that contain that pattern.)

Regex Construction:

The **regular expressions are built recursively out of smaller regular expressions**. **Each regular expression r denotes a language L(r)**, which is also defined recursively from the languages denoted by r's subexpressions.

Here are the rules that define the regular expressions over some alphabet ∑ and the languages that those expressions denote.

BASIS: There are two rules that form the basis:

1. ϵ is a regular expression, and L(ϵ) is {ϵ}, that is, the language whose sole member is the empty string.
2. If a is a symbol in ∑, then **a** is a regular expression, and L(a) = {a}, that is, the language with one string, of length one, with a in its one position.

Regex Common Operators

1) The Concatenation Operator (. or No-character)

- ### . (dot) Character (in older representation):

  This operator concatenates two regular expressions a and b as a.b

  The concatenation symbol . (dot) often is implicit in regular expressions.

- ### No Character:

  This operator concatenates two regular expressions a and b as ab

  No character represents this operator; you simply put *b* after *a*. The result is a regular expression that will match a string if *a* matches its first part and *b* matches the rest. For example, 'xy' matches xy.

2) Repetition Operators

Repetition operators repeat the preceding regular expression a specified number of times.

i) The zero-or-more Operator (*)

- **This operator repeats the smallest possible preceding regular expression as many times as necessary (including zero) to match the pattern. `*' represents this operator.**
- For example, `a*' matches any string made up of zero or more a's. Since **this operator operates on the smallest preceding regular expression**, 'fa*' has a repeating `a', not a repeating `fa'. So, `fa*' matches `f', `fa', `faa', and so on.
- The **zero-or-more operator is a suffix operator**, it may be useless as such when no regular expression precedes it. This is the case when it is first in a regular expression, or follows a match-beginning-of-line, open-group, or alternation operator.

ii) The one-or-more Operator (+)

- **This operator is similar to the match-zero-or-more operator except that it repeats the preceding regular expression at least once.**
- For example, supposing that '+' represents the match-one-or-more operator; then 'ca+r' matches, e.g., 'car' and 'caaaar', but not 'cr'.

iii) The zero-or-one Operator (?)

- **This operator is similar to the match-zero-or-more operator except that it repeats the preceding regular expression once or not at all**.
- For example, supposing that '?' represents the match-zero-or-one operator; then 'ca?r' matches both 'car' and 'cr', but nothing else.

Note:

1. **The operators *, + and ? are also known as counters or quantifiers.**
2. **The operators * and + are Greedy: Match as Many As Possible (longest match)**
    - By default, quantifiers * and + tells the engine to match as many symbols of its preceding subexpression as possible. This behaviour is called greedy.
    - The matcher processes a match-zero-or-more operator (i.e., *) by first matching as many repetitions of the smallest preceding regular expression as it can. Then it continues to match the rest of the pattern. If it can't match the rest of the pattern, it backtracks (as many times as necessary), each time discarding one of the matches until it can either match the entire pattern or be certain that it cannot get a match.
    - For example, when matching 'ca*ar' against 'caaar', the matcher first matches all three 'a's of the string with the 'a*' of the regular expression. However, it cannot then match the final 'ar' of the regular expression against the final 'r' of the string. So it backtracks, discarding the match of the last 'a' in the string. It can then match the remaining 'ar'.

3) The Alternation (or Union or OR) Operator (| or +(in older representation))

- **This operator match one of a choice of regular expressions:** if you put the character(s) representing the alternation operator between any two regular expressions *a* and *b*, **the result matches the union of the strings that *a* and *b* match**.

  For example, '|' is the alternation operator, then '**cat|dog|fox**' would match any of 'cat', 'dog' or 'fox'.

- **The alternation operator operates on the largest possible surrounding regular expressions.** (Put another way, it has the lowest precedence of any regular expression operator.) **Thus, the only way you can delimit its arguments is to use grouping.**

  For example, 'fo(o|b)ar' would match either 'fooar' or 'fobar'. ('foo|bar' would match 'foo' or 'bar'.)

- **The matcher usually tries all combinations of alternatives so as to match the longest possible string.**

  For example, when matching '**(fooq|foo)*(qbarquux|bar)**' against 'fooqbarquux', it cannot take, say, the first combination it could match, since then it would be content to match just 'fooqbar'.

4) Grouping Operators (...)

- **A group, also known as a subexpression, consists of an open-group operator (left parenthesis), any number of other operators, and a close-group operator (right parenthesis).**

- **Regex treats this sequence as a unit**, just as mathematics and programming languages treat a parenthesized expression as a unit.

- **Therefore, using *groups*, you can:**
  - **delimit the argument(s) to an alternation operator** or **a repetition operator.**
  - keep track of the indices of the substring that matched a given group.

Regex operators precedence

The following table gives the order of RE operator precedence, from highest precedence to lowest precedence.

Highest	**[]**	**Character class**
	()	**Grouping**
	*** + ? {m, n}**	**Repetition Operators**
	. or **No-character operator**	**Concatenation**
	^ $	**Position Anchors**
Lowest	**|**	**Alternation (or Union)**

Note:

- Anchors (^ and $) are special characters that anchor regular expressions to particular places in a string. The most common anchors are the caret ^ and the dollar sign $.
- \ (backlash) is used to suppress the special meaning of a character when matching. For example: \$ matches the character '$'.
- **Regex Characters**: These characters have special meaning in regex . + * ? ^ $ ( ) [ ] { } | \

Regex in Practice

Character classes ([...])

- A regular expression $a_1|a_2|\bullet\bullet\bullet|a_n$, where the a_i's are each symbol of the alphabet, can be replaced by the shorthand $[a_1a_2 \ldots a_n]$
- When $a_1, a_2, \ldots, a_n$ form a logical sequence, we can replace them by a_1-a_n.
  Thus, **[abc] is shorthand for a|b|c**, and **[a-z] is shorthand for a|b|....|z**

Expression	Description
[...]	**Denotes the character class. Matches one of the characters in the brackets.** **Example: T[ao]p Sample match: Tap or Top**
[x-z]	**Matches one of the characters in the range from x to z** **Matches x, or y, or z. Shorthand for x|y|z**
[xyz]	**Matches x, or y, or z**
[x\-z]	**Matches x, or -, or z**
[A-Za-z0-9]+	**One or more alphanumeric characters**
[^x]	**Matches any character except x**
[^x-z]	**Matches one of the characters not in the range from x to z** **(Anything except x or y or z)** **The ^ prefix in a character class creates a *complement class*.**
[x^z]	**One of x, ^, z**
[x|z]	**One of x, |, z**
.	**Matches any (single) character except newline**
^x	**(Caret.) Matches x at the start of the string or after newline.** **(^ indicates start of string or after newline)**
z$	**Matches z at the end of a string just before the newline at the end of the string.** **($ indicates end of string or before newline)**
^$	**Matches empty string**

Note: The caret ^ has three uses:

- to match at the start of a line or string,

- to indicate a negation, if it is the first character inside square brackets(i.e., character class), and
- just to mean a caret.

Expression	Description
x{n}	Matches exactly n occurrences of x
x{m,n}	Matches m through n occurrences of x.
x{n,}	Matches n or more occurrences of x Examples: a{0, } - same as a*, a{1, } - same as a+
xx|yy	Matches either xx or yy
(x)	Matches x
x/y	Matches x but only if followed by y

The programming languages provide macros (or meta-characters)

Expression	Description
.	Any one character except newline. Same as [^\n]
\d	Matches any digit between [0-9] Examples: \d{4} matches any 4 digits \d{3,4} matches 3 or 4 digits
\w	Matches any ASCII letter, digit or underscore
\s	Matches any space, tab, newline, carriage return, vertical tab

Operators	Description
?	Zero or one copy of the preceding expression
*	Zero or more copies of the preceding expression
+	One or more copies of the preceding expression

Examples:

Expression	Description
a|b	a or b (alternating)
(ab)+	One or more copies of ab (grouping)
abc	abc
abc*	ab abc abcc abccc abcccc ...
Abc+	abc abcc abccc abcccc ...

a(bc)+	**abc abcbc abcbcbc ...**
a(bc)*	**a abc abcbc abcbcbc ...**
a(bc)?	**a abc (none or once)**

Example 1:

Let Σ = {a, b, c}. Regular expressions for

a) single letter words

b) words that contain at least one a.

c) words that contain at least one a or one b.

d) words that contain at least one a and one b.

Solution:

a) **a|b|c or Σ or** [abc]

b) (a|b|c)* a (a|b|c)* **or** Σ*aΣ* **or** [abc]*a[abc]*

c) (a|b|c)* (a|b) (a|b|c)* **or** Σ* (a | b)Σ*

d) (a|b|c)* a (a|b|c)* b (a|b|c)* | (a|b|c)* b (a|b|c)* a (a|b|c)* **or** Σ* aΣ* bΣ* | Σ* bΣ* aΣ*

Example 2:

Write regex to find/validate C-language

a) Identifier

b) Integers (with ±0), floating-point numbers and E-notation numbers

Solution:

a) ^[a-zA-Z_] ([a-zA-Z0-9_])*$

b) Integers: [+-]? 0 | [+-]?[1-9][0-9]* or [+-]? (0 | [1-9][0-9]*) or [+-]? 0 | [+-]?[1-9]\d*

We did not use *position anchors* ^ and $ in this regex. Hence, it can match any parts of the input string. For examples,
If the input string is "abc123xyz", it matches the substring "123".
If the input string is "abcxyz", it matches nothing.
If the input string is "abc123xyz456_0", it matches substrings "123", "456" and "0" (three matches).
If the input string is "0012300", it matches substrings: "0", "0" and "12300" (three matches)!!!

^[+-]? 0 | [+-]?[1-9][0-9]*$ or ^[+-]? (0 | [1-9][0-9]*)$ or ^[+-]? 0 | [+-]?[1-9]\d*$

This regex match an Integer literal (for entire string with the *position anchors*), both positive, negative and zero.

Floating point numbers: ^[+-]? (0|[1-9][0-9]*) \. [0-9]+$

E-notation numbers: ^[0-9] \. [0-9]+ E [+-]? [0-9]+$

Note: Signed zero is zero with an associated sign. In ordinary arithmetic, the number 0 does not have a sign, so that −0, +0 and 0 are equivalent. However, in computing, some number representations allow for the existence of two zeros, often denoted by −0 (negative zero) and +0 (positive zero).

Exercise:

1. Let Σ = {0, 1}. Write regular expressions to find

a) strings that contain only 0's and string length longer than 2
b) strings that do not contain 11.

Solution:

a) `0{3,}`

b) Many solutions

- `(0|10)* | (0|01)* | 1(0|01)*`
- `(ε|1)(0|01)*`
- `(1|ε)(0(1|ε))*`
- `((1|ε)0)*(1|ε)`
- `(1|ε)(00*1)*0*`
- `0*(100*)*(1|ε)`
- `0*(100*)*10*|0*`
- `0*|0*1(0*01)*0*`
- `0*(100*)*1|0*(100*)*`

2. Write regex to validate

a) PAN Number (A ten-character alphanumeric identifier, the first five characters are letters (in uppercase by default), followed by four numerals, and the last (tenth) character is a uppercase letter)
b) Adhaar Number with space after 4-digits (It should have 12 digits. It should not start with 0 and 1. It should not contain any alphabet and special characters.)
c) Mobile Number with Optional Country Code (All mobile phone numbers are 10 digits long, In India, mobile numbers start with either 9, 8, 7 or 6.)
d) **Validate**
  i) Date format **dd/mm/yyyy** or **dd-mm-yyyy**, and days between 1-31, months between 1-12 and 4-digits year)
  ii) Year 2024 dates

e) Email Address

f) IPv4 Address

Solution:

a) `^[A-Z]{5}[0-9]{4}[A-Z]$`

b) `^[2-9]\d{3}\s\d{4}\s\d{4}$`

c) `^(0|\+91)?[6-9]\d{9}$`

d) (i) `^(0?[1-9]|[12][0-9]|3[01])[\/\-](0?[1-9]|1[012])[\/\-]\d{4}$`

(ii) …

e) `^\w+([.\-]?\w+)*@\w+([.\-]?\w+)*(\.\w{2,3})+$`

Explanation:

- The *position anchors* ^ and $ match the beginning and the ending of the input string, respectively. That is, this regex shall match the entire input string, instead of a part of the input string (substring).
- \w+ matches 1 or more word characters (same as [a-zA-Z0-9_]+).
- [.-]? matches an optional character . or -. Although dot (.) has special meaning in regex.
  In addition to ^,] and \ , the following characters must be escaped in character classes if they represent literal characters: (,) , [, { , } , / , - , |
- The sub-expression \w+([.-]?\w+)* is used to match the username in the email, before the @ sign. It begins with at least one word character [a-zA-Z0-9_], followed by more word characters or . or -. However, a . or - must follow by a word character [a-zA-Z0-9_]. That is, the input string cannot begin with . or -; and cannot contain "..", "--", ".-" or "-.". Example of valid string are "a.1-2-3".
- The @ matches itself. In regex, all characters other than those having special meanings matches itself, e.g., a matches a, b matches b, and etc.
- Again, the sub-expression \w+([.-]?\w+)* is used to match the email domain name, with the same pattern as the username described above.
- The sub-expression \.\w{2,3} matches a . followed by two or three word characters, e.g., ".com", ".edu", ".us", ".uk", ".co".
- (\.\w{2,3})+ specifies that the above sub-expression could occur one or more times, e.g., ".com", ".co.uk", ".edu.sg" etc.

f) `^(([0-9]|[1-9][0-9]|1[0-9]{2}|2[0-4][0-9]|25[0-5])\.){3}([0-9]|[1-9][0-9]|1[0-9]{2}|2[0-4][0-9]|25[0-5])$`

3. Let Σ = {0, 1}. Give regular expressions for

a) words such that no prefix have the difference between the number of 0s and 1s more than 1.
b) words such that no prefix have the difference between the number of 0s and 1s more than 2

Regular Expression to Finite Automata

Converting regular expressions to finite automata using Thompson Construction Method:

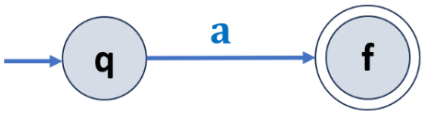
Thompson Construction Method:

The algorithm works recursively by splitting an expression into its constituent subexpressions, from which the NFA will be constructed using a set of rules.

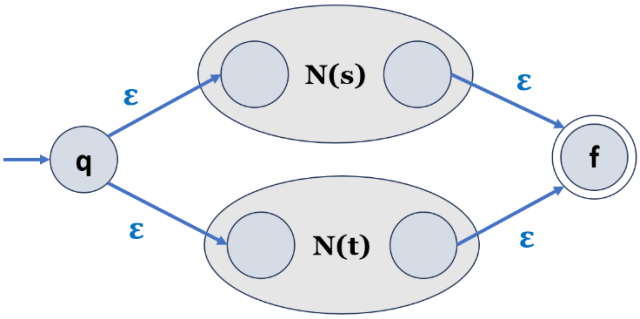
Following are the rules:

- **If subexpression operand is epsilon(ε), then our FA has two states, q (the start state) and f (the final, accepting state), and an ε-transition from q to f.**

- **If subexpression operand is a character a, then our FA has two states, q (the start state) and f (the final, accepting state), and a transition from q to f with label a.**

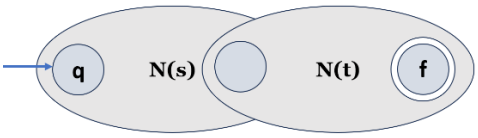


- **If subexpression is the alternation (or union) expression s|t, then our FA is**



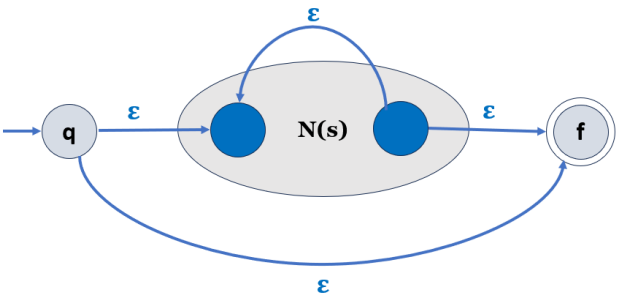
State q goes via ε either to the initial state of Automata of s and t. N(s) or N(t). Their final states become intermediate states of the whole NFA and merge via two ε-transitions into the final state of the NFA.

- **If subexpression is the concatenation expression st, then our FA is**



The initial state of N(s) is the initial state of the whole NFA. The final state of N(s) becomes the initial state of N(t). The final state of N(t) is the final state of the whole NFA.

- **If subexpression is the Kleene star expression s*, then our FA is**



An ε-transition connects the initial and final state of the NFA with the sub-NFA. N(s) Automata for s in between. Another ε-transition from the inner final to the inner initial state of N(s) allows for repetition of expression s according to the star operator.

- **The parenthesized expression (s) is converted to N(s) itself.**

With these rules, using the empty expression and symbol rules as base cases, it is possible to prove with mathematical induction that any regular expression may be converted into an equivalent NFA

Exercise: Construct Finite Automata for the following Regular Expressions using Thompson construction method.

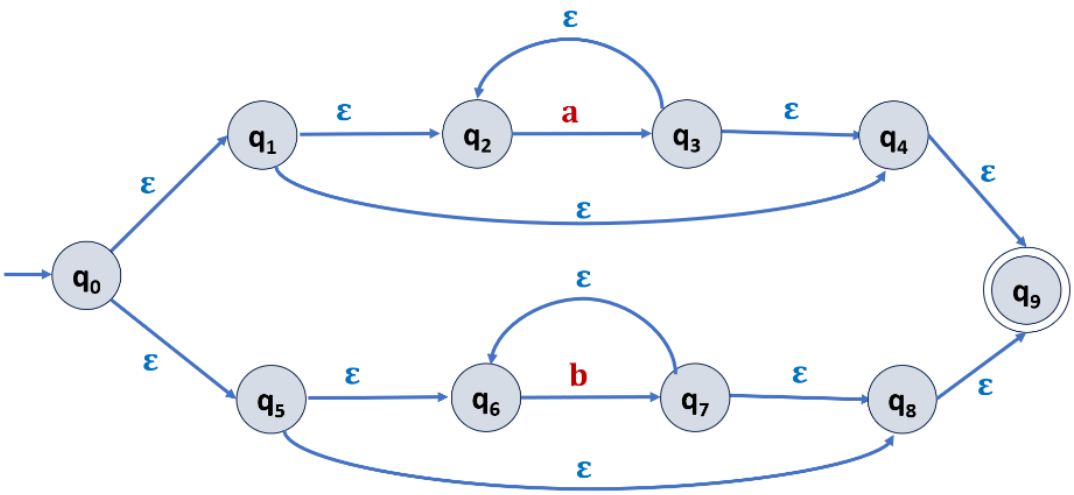
1) (ab|c)*

2) (ab|ba)*

3) a*|b*

4) (0|01*0)*0

Solution for a*|b*



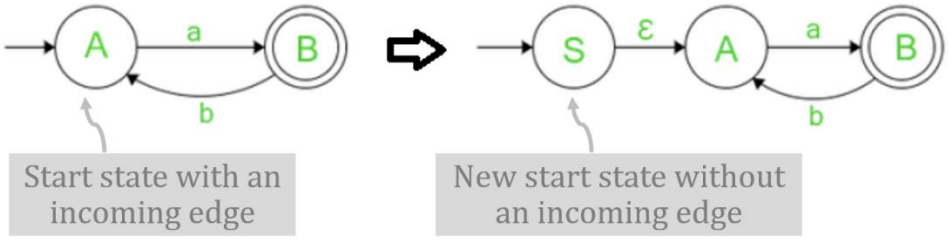
DFA to Regex: State Elimination Method

This method involves the following steps in finding the regular expression for any given DFA.

Step-01:

- If there are no incoming edges to the start state proceed further to check other rules.
- If there are any incoming edges (or transitions) to the start state, to get rid of the incoming edges, make new start state with no incoming edges and then make an outgoing edge from new start state to the old start state with Ɛ-transition.
  The initial state before is now normal state with added incoming Ɛ-transition.

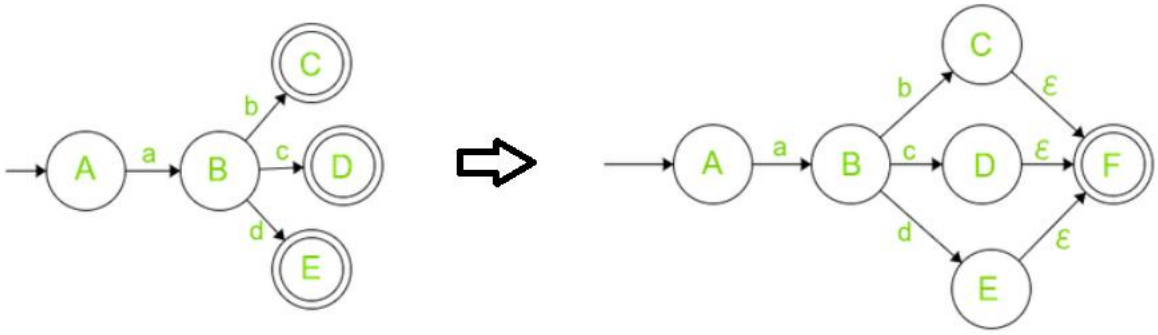
Example:



Step-02:

- If there exist multiple final states in the DFA, then convert all the final states into non-final states and create a new single final state and then make an outgoing edge from all the previous final states to the new final state with Ɛ-transition.
- The previous final states are now normal non-final states with added outgoing Ɛ-transition.

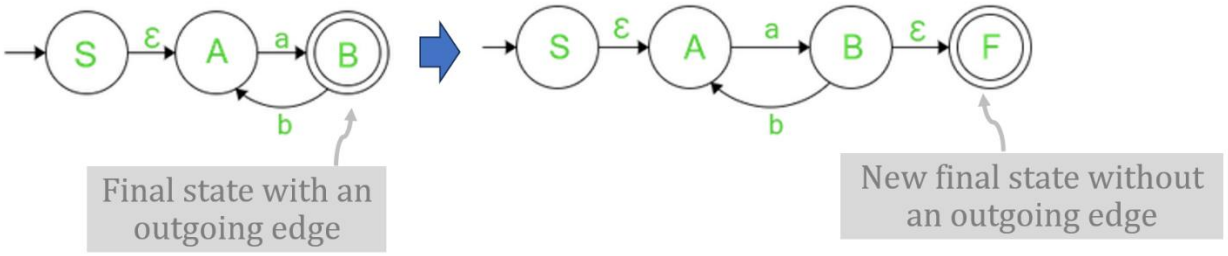
Example:



Step-03:

- If there exists one final state and if there are any outgoing edges from the final state, then create a new final state (having no outgoing edge from it) and then make an outgoing edge from the previous final state to the new final state with Ɛ-transition.
- The previous final state is now normal non-final state with an added outgoing Ɛ-transition.

Example:



Step-04:

- Eliminate all the intermediate states one by one.
- These states may be eliminated in any order.

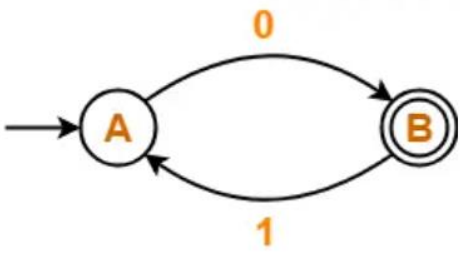
In the end,

- Only an initial state going to the final state will be left.
- The cost of this transition is the required regular expression.

Examples:

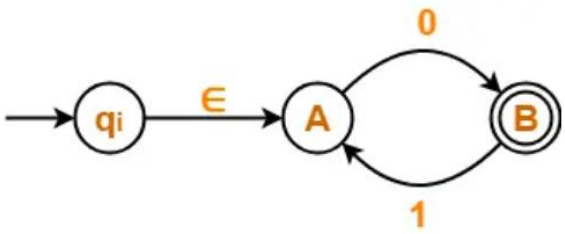
1) Find regular expression for the following DFA using state elimination method.

Note: The State elimination order is A and then B.

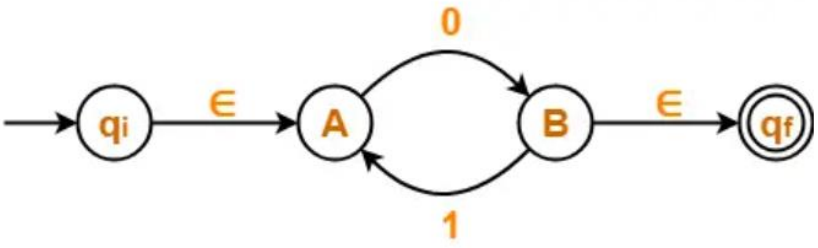


Solution:

Step-01: Initial state A has an incoming edge. So, we create a new initial state q_i. The resulting DFA is



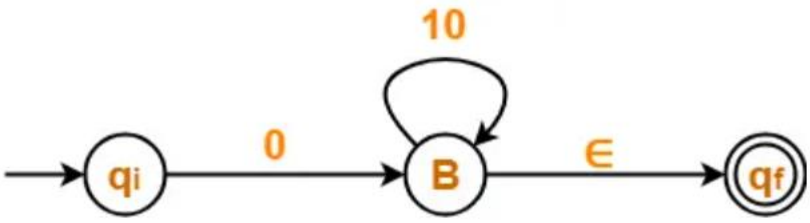
Step-02: Final state B has an outgoing edge. So, we create a new final state q_f. The resulting DFA is



Step-03: Now, we start eliminating the intermediate states. First, let us eliminate state A.

- There is a path going from state q_i to state B via state A.
- So, after eliminating state A, we put a direct path from state q_i to state B having cost **ε.0 = 0**
- There is a loop on state B using state A.
- So, after eliminating state A, we put a direct self-loop on state B having cost **1.0 = 10**

Eliminating state A, we get



Step-04:

Now, let us eliminate state B.

- There is a path going from state q_i to state q_f via state B.
- So, after eliminating state B, we put a direct path from state q_i to state q_f having cost 0.(10)*.ε = 0(10)*

Eliminating state B, we get

From here,

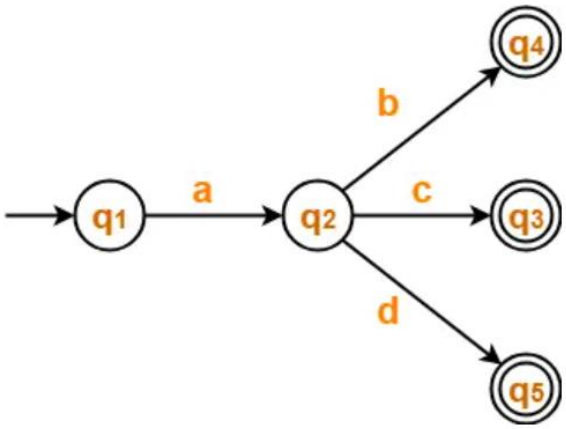
Regular Expression = 0(10)*

Note:

In the above question, If we first eliminate state B and then state A, then regular expression would be = (01)*0. This regex is also representing the same DFA and it also correct.

2) Find regular expression for the following DFA using state elimination method.

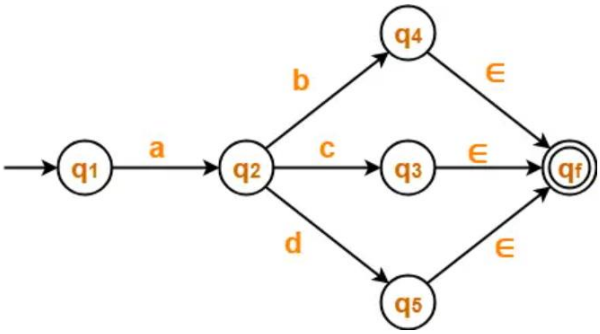
Note: The State elimination order is q4, q3, q5 and then q2.



Solution:

Step-01: There exist multiple final states. So, we convert them into a single final state.

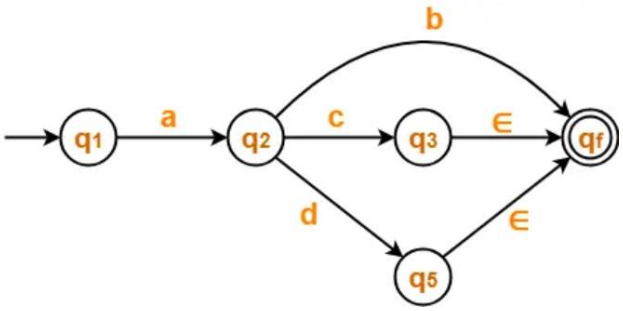
The resulting DFA is



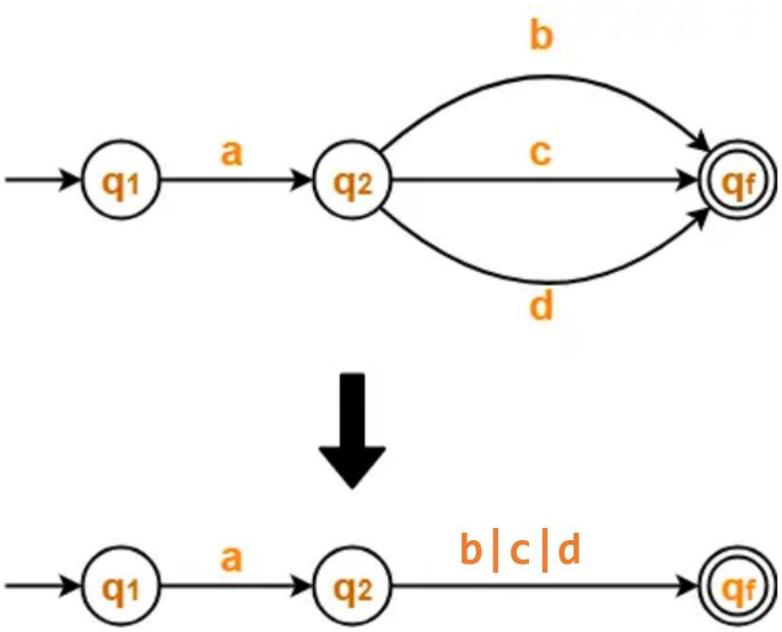
Step-02: Now, we start eliminating the intermediate states.

First, let us eliminate state q_4.

- There is a path going from state q_2 to state q_f via state q_4.
- So, after eliminating state q_4 , we put a direct path from state q_2 to state q_f having cost $b.\varepsilon = b$.



Similarly, eliminate q3 and q5



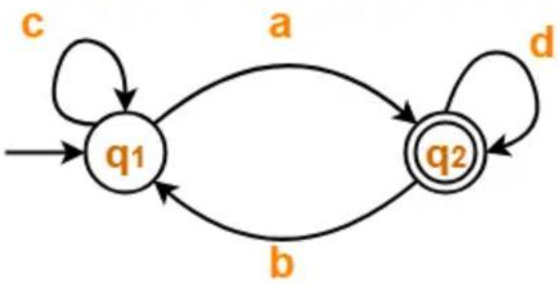
Step-05: Now, let us eliminate state q_2.

- There is a path going from state q_1 to state q_f via state q_2.
- So, after eliminating state q_2 , we put a direct path from state q_1 to state q_f having cost a.(b|c|d).

Regular Expression = a(b|c|d)

3) Find regular expression for the following DFA using state elimination method.

Note: The State elimination order is q1 and then q2.

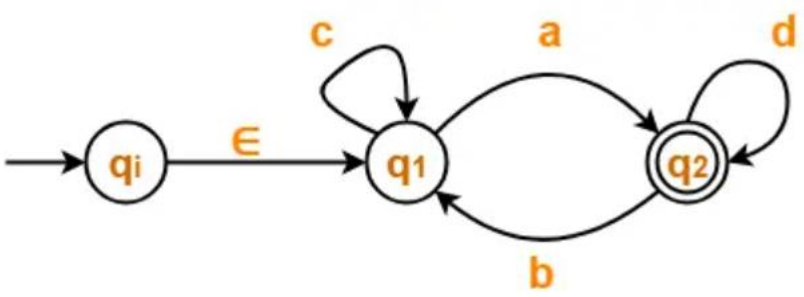


Solution:

Step-01:

- Initial state q_1 has an incoming edge.
- So, we create a new initial state q_i.

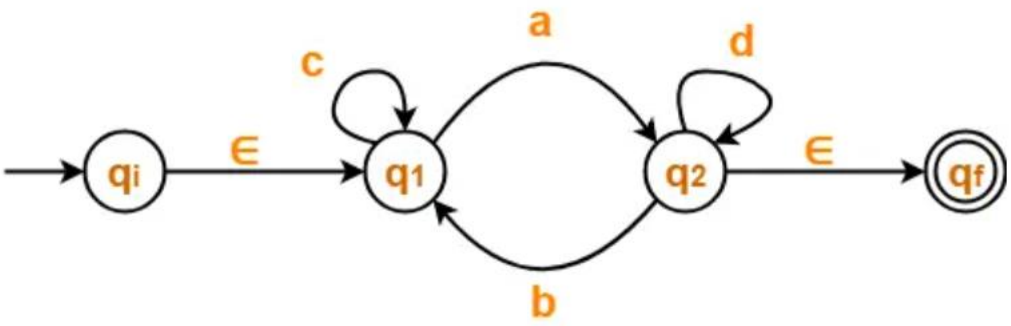
The resulting DFA is



Step-02:

- Final state q_2 has an outgoing edge.
- So, we create a new final state q_f.

The resulting DFA is



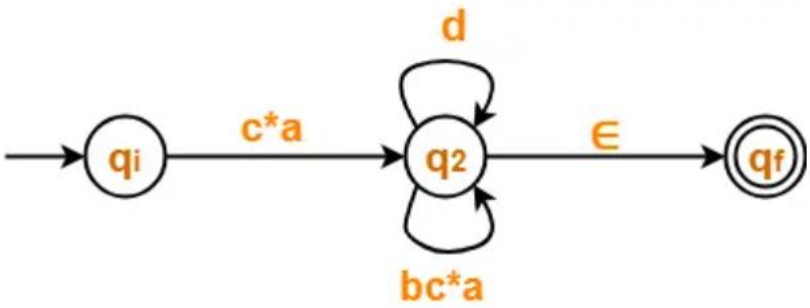
Step-03:

Now, we start eliminating the intermediate states.

First, let us eliminate state q_1.

- There is a path going from state q_i to state q_2 via state q_1 .
- So, after eliminating state q_1, we put a direct path from state q_i to state q_2 having cost $\varepsilon.c^*.a = c^*a$
- There is a loop on state q_2 using state q_1 .
- So, after eliminating state q_1 , we put a direct loop on state q_2 having cost $b.c^*.a = bc^*a$

Eliminating state q_1, we get

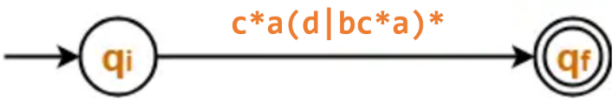


Step-04:

Now, let us eliminate state q_2.

- There is a path going from state q_i to state q_f via state q_2 .
- So, after eliminating state q_2, we put a direct path from state q_i to state q_f having cost $c^*a(d|bc^*a)^*\varepsilon = c^*a(d|bc^*a)^*$

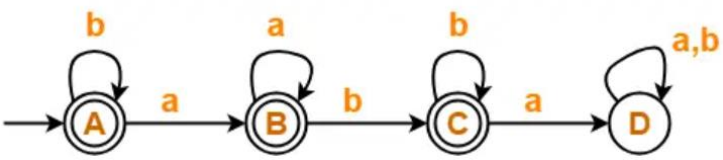
Eliminating state q_2, we get



Regular Expression = c*a(d|bc*a)*

4) Find regular expression for the following DFA using state elimination method.

Note: The State elimination order is C, B and then A.



- State D is a dead state as it does not reach to any final state.
- So, we eliminate state D and its associated edges.

Solution:

Step-01:

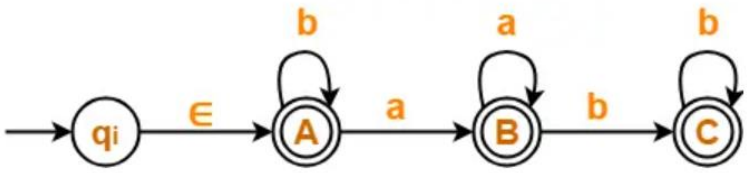
- State D is a dead state as it does not reach to any final state.
- So, we eliminate state D and its associated edges.

The resulting DFA is

Step-02:

- Initial state A has an incoming edge (self loop).
- So, we create a new initial state q_i.

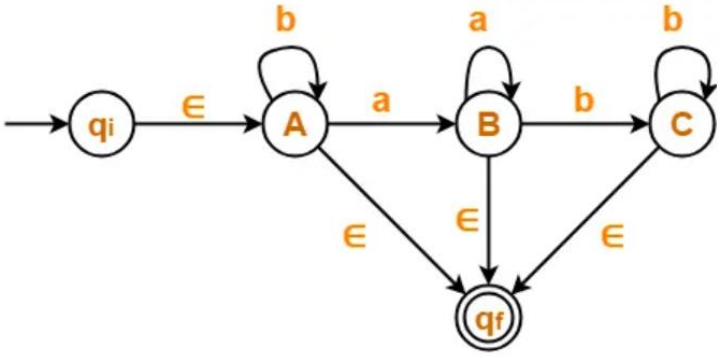
The resulting DFA is



Step-03:

- There exist multiple final states.
- So, we convert them into a single final state.

The resulting DFA is



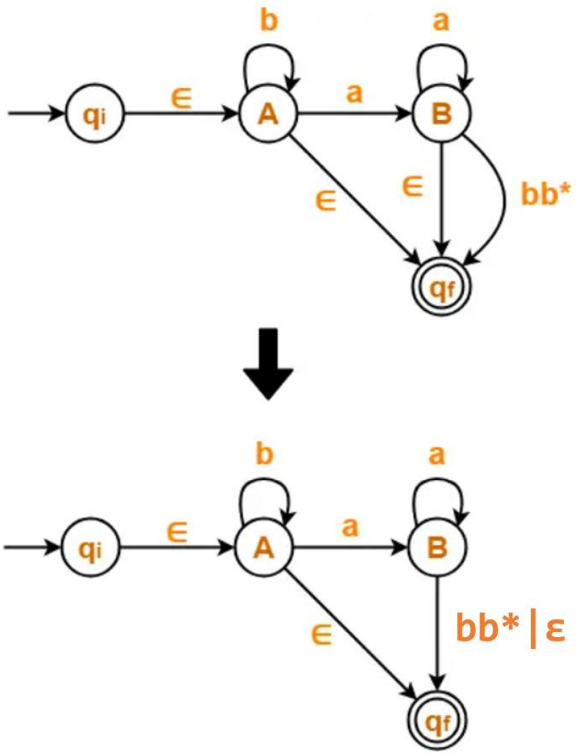
Step-04:

Now, we start eliminating the intermediate states.

First, let us eliminate state C.

- There is a path going from state B to state q_f via state C.
- So, after eliminating state C, we put a direct path from state B to state q_f having cost b.b*.ε = bb*

Eliminating state C, we get

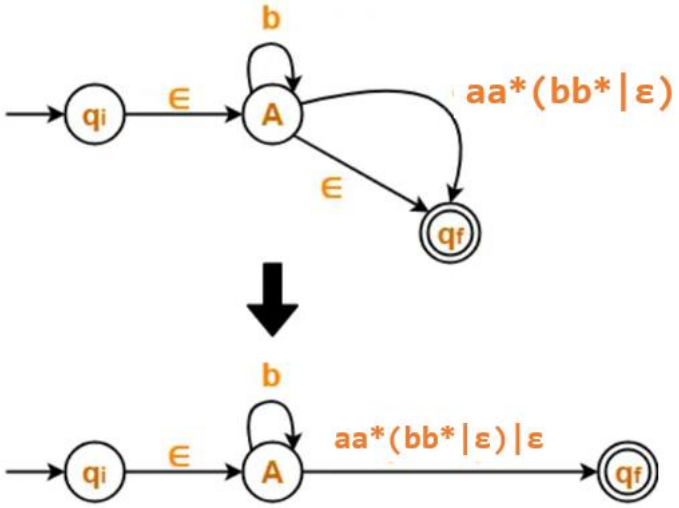


Step-05:

Now, let us eliminate state B.

- There is a path going from state A to state q_f via state B.
- So, after eliminating state B, we put a direct path from state A to state q_f having cost a.a*.(bb*|ε) = aa*(bb*|ε)

Eliminating state B, we get

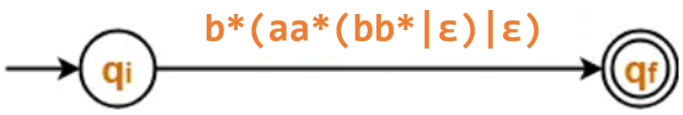


Step-06:

Now, let us eliminate state A.

- There is a path going from state q_i to state q_f via state A.
- So, after eliminating state A, we put a direct path from state q_i to state q_f having cost ε.b*.(aa*(bb*|ε)|ε) = b*(aa*(bb*|ε)|ε)

Eliminating state A, we get



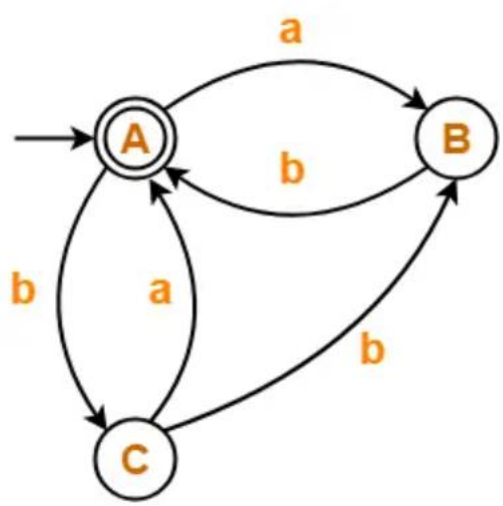
Regular Expression = b*(aa*(bb*|ε)|ε)

We know, bb* | ε = b* So, we can also write

Regular Expression = b*(aa*b*|ε)

5) Find regular expression for the following DFA using state elimination method.

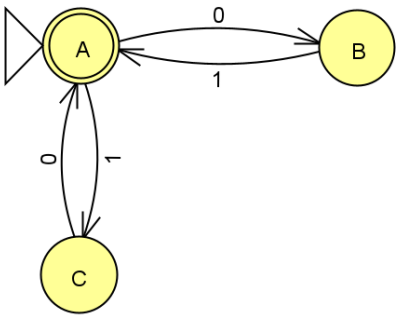
Note: The State elimination order is B, C and then A.



Regular Expression = (ab | b(a|bb))*

6) Find regular expression for the following DFA using state elimination method.

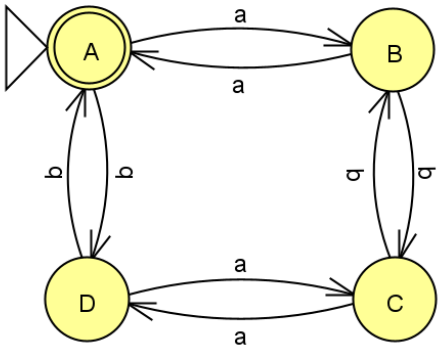
Note: The State elimination order is B, C and then A.



Regular Expression = (01|10)*

7) Find regular expression for the following DFA using state elimination method.

Note: The State elimination order is B, D, C and then A.



Regular Expression = (((ab|ba) (aa|bb)* (ab|ba)) | (aa|bb))*

Rules for regular expressions:

The set of regular expressions is defined by the following rules.

- **Every letter of ∑ can be made into a regular expression, null string, ε itself is a regular expression.**
- **If r1 and r2 are regular expressions, then (r1), r1r2, r1|r2, r1* , r^+ are also regular expressions.**

Example: ∑ = {a, b} and r is a regular expression of language made using these symbols

Regular expression	Regular language
Ø (empty set)	L(Ø) = Ø
ε (empty string)	L(ε) = {ε}
a* (Kleene closure)	L(a*) = {ε, a, aa, aaa .....}
a|b (Union)	L(a|b) = L(a) U L(b) = {a, b}
ab (concatenation)	L(ab) = L(a) L(b) = {ab}
a* | ba	L(a* | ba)= L(a*) U L(ba) = {ε, a, aa, aaa,..... , ba}

Algebraic properties of regular expressions:

Kleene closure is an unary operator and Union(|) and concatenation operator(. or no-character) are binary operators.

1. Closure:

If r1 and r2 are regular expressions (REs), then

- r1* is a RE
- r1|r2 **or** r1∪r2 is a RE
- r1r2 is a RE

Note: In mathematics, the closure property states that when **any operation is performed on elements within a set, the result is also within the same set**.
Closure property states that when a set of numbers is closed under any arithmetic operation such as addition, subtraction, multiplication, and division, it means that when the operation is performed on any two numbers of the set with the answer being another number from the set itself.

2. Closure laws

- **$(r^*)^* = r^*$** (closing an expression that is already closed does not change the language)
- **$\varepsilon^* = \varepsilon$** (a string formed by concatenating any number of copies of an empty string is empty itself)
- **$r^+ = rr^* = r^*r$,** as $r^* = \varepsilon \mid r \mid rr \mid rrr$ .... and $rr^* = r \mid rr \mid rrr$ .....
- **$r^* = r^+ \mid \varepsilon$**
- **$(r_1|r_2)^*$ = $(r_1^* \mid r_2^*)^*$**

$= (r_1^* r_2^*)^*$

$= (r_1 | r_2^*)^*$

$= (r_1^* | r_2)^*$

$= r_1^*(r_2 r_1^*)^*$

$= r_1^*(r_1 r_2^*)^*$

3. Associativity

If r1, r2, r3 are RE, then

i) r1| (r2|r3) = (r1|r2) |r3

- **For example**: r1 = a , r2 = b , r3 = c, then
- The resultant regular expression in LHS becomes a|(b| c) and the regular set for the corresponding RE is {a, b, c}.
- for the RE in RHS becomes (a| b) | c and the regular set for this RE is {a, b, c}, which is same in both cases. Therefore, **the associativity property holds for union operator**.

ii) r1(r2r3) = (r1r2)r3

- **For example:** r1 = a , r2 = b , r3 = c
- Then the string accepted by RE a(bc) is only abc.
- The string accepted by RE in RHS is (ab)c is only abc ,which is same in both cases. Therefore, the **associativity property holds for concatenation operator.**

iii) Associativity property does not holds for Kleene star(*) and Kleene plus(+) because they are unary operators.

4. Identity

- **In the case of alternation (or union) operator**

  if r|∅ = r as **r ∪ ∅= r**, therefore ∅ is the identity for |

  Therefore, **∅ is the identity element for a union operator.**
- **In the case of concatenation operator**

  rε = r ⇒ **ε is the identity element for concatenation operator.**

6. Commutative property

If r1, r2 are RE, then

- **r1|r2 = r2|r1** For example, for r1 =a and r2 =b, then RE a| b and b| a are equal.
- **r1r2 ≠ r2r1** For example, for r1 = a and r2 = b, then **RE ab is not equal to ba.**

7. Distributive property

If r1, r2, r3 are regular expressions, then

- (r1|r2)r3 = r1r3 | r2r3 i.e. Right distribution
- r1(r2| r3) = r1r2 | r1r3 i.e. left distribution

- **(r1r2) |r3 ≠ (r1|r3)(r2|r3)**

8. Idempotent law

- **r1 | r1 = r1** ⇒ r1 ∪ r1 = r1 , therefore the union operator satisfies idempotent property.
- **rr ≠ r** ⇒ concatenation operator does not satisfy idempotent property.

Note: In mathematics, a number that is idempotent keeps the same value when operated by itself

9. Identities for regular expression

There are many identities for the regular expression. Let p, q and r be regular expressions.

- **∅ | r = r | ∅ = r**

  ∅ ∪ r = r ∪ ∅ = r i.e., Union of the null set to any other language will not change it.
- ∅r = r∅ = ∅ i.e., we cannot concatenate non empty language L(r) with a language ∅ containing no string (as there is no string to concatenate in ∅), it yields empty language.
- **εr = rε = r** i.e., Concatenating the blank (i.e., empty) string to any string will not change it.
- **ε* = ε**
- **∅* = ε**

  Note: For any language L, by definition

  $$L^* = \bigcup_{i=0}^{\infty} L^i,$$

  where a word in L^i is the concatenation of i words from L. In particular, $\mathbf{L^0}=\{\epsilon\}$ since ϵ is the **concatenation of zero words from L**. It doesn't matter if L is empty or not, since we are choosing *zero* words from L.
  - r | r = r
  - r*r* = r*
  - rr* = r*r = r+
  - (r*)* = r*
  - ε |rr* = r*
  - (pq)*p = p(qp)*
  - (p|q)* = (p*q*)* = (p*|q*)*
  - (p|q)r= pr|qr and r(p|q) = rp | rq

Exercise-1: Find regular expressions for the following languages:

1. The set of all strings with an even number of 0's
2. The set of all strings of even length (length multiple of k)
3. The set of all strings that begin with 110
4. The set of all strings containing exactly three 1's
5. The set of all strings divisible by 2

6. The set of strings where third last symbol is 1

Exercise-2: Write regular expression to denote a language L

a) String which begin or end with either 00 or 11.

b) The set of all strings, when viewed as binary representation of integers, that are divisible by 2.

c) The set of all strings containing 00.

d) String not containing the substring 110.

Answer:

a) (00|11) (0|1)* | (0|1)* (00|11)

b) (0|1)*0 i.e all strings ending with 0

c) (0|1)*00 (0|1)*

d) (0|10) *1*

Regex and its Language

Look for each regular expression e which language L(e) does it produce.

(1) What about the regex *a* (only the letter)? Its language is
L(a) = {a}, just the single word/character "a".

(2) For the regex e1 | e2, where e1 and e2 are regexs themselves,
L(e1 | e2) = L(e1) U L(e2).
Example: L(a|b) = {a, b}.

(3) **For the regex *e1e2* (concatenation), where e1 and e2 are regexs themselves,**
L(e1e2) = all words w such that we can write w = w1w2 with w1 in L(e1) and w2 in L(e2).

(4) **What about a regular expression e*, where e might be a regular expression itself?**
Intuitively, a word is in L(e*) if it has the form w1w2w3w4...wn, with wi in L(e) for each i. So

L(e*) = all words w such that we can write
w = w1w2…wn
for n >= 0 with all w_i in L(e) (for i = 1, 2, ..., n)

So, what about **L((a* | b*))?**

L((a* | b*)) = L(a*) U L(b*)
= { ε, a, aa, aaa, aaaa, ...} U { ε, b, bb, bbb, bbbb}
= all strings that have either only a's OR only b's in it (including ε)

Similarly for (a* b*):

L((a* b*)) = all words w = w1w2 with w1 in L(a*) and w2 in L(b*)
= { ε, a, aa, aaa, ..., b, bb, bbb, …, ab, aab, abb, aabb, …}
= all strings that have zero or more a's, zero or more b's, a's followed by b's

Equivalence of two regular expressions

We say that two regular expressions R and S are equivalent if they describe the same language. In other words, if L(R) = L(S) for two regular expressions R and S then R = S.

We can determine the equivalence of two regular expressions R and S using:

1) **A Formal method**
2) **An Informal method**

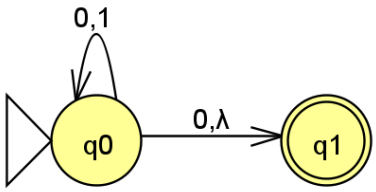
1) Formal method to prove equivalence of two regular expressions

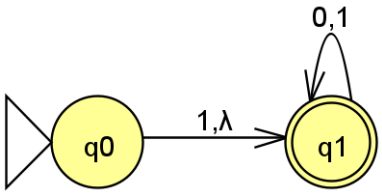
1) Let R and S be two regular expressions.
2) Construct NFA M(R) for regular expression **R** and NFA M(S) for regular expression **S.**
3) Convert both the NFAs M(R) and M(s) to respective DFAs using subset construction method.
4) Next, find the Minimal DFAs for the two DFAs constructed.
5) If two Minimal DFAs are equivalent, then regular expressions **R** and **S** are equivalent else **R** and **S** are not equivalent.

Example:

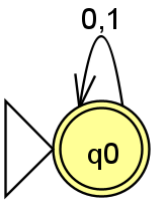
Consider the two regular expressions **R** = **(0|1)* (0|λ)** and **S** = **(1|λ) (0|1)***

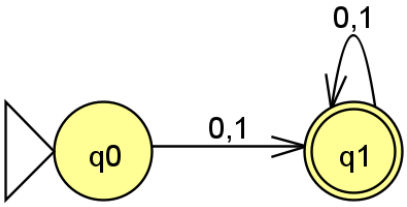
NFAs:



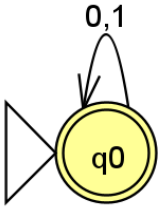


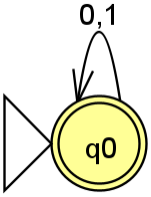
DFAs:





Minimal DFAs:





The minimal DFAs are equivalent, then regular expressions **R** and **S** are equivalent.

2) Informal method to prove equivalence of two regular expressions

How can we determine whether two regular expressions denote the same language or not?

- Find a string that belongs to one regex and does not belong to the other regex to show that they are not equivalent.
- Faster, but based on hit and trail.
- Can only be used to prove inequality.

Example:

1) Are **R = a* (ba* ba*)*** and **S = a* (ba* b)* a*** equivalent?

Solution:

Find a string that belongs to one regex and does not belong to the other regex

bbaabb ∈ R

bbaabb ∉ S

Hence, R and S are not equivalent.

2) Are **R = a(a | b)*** and **S = (a(a | b))*** equivalent?

Solution:

Find a string that belongs to one regex and does not belong to the other regex

λ ∉ R

λ ∈ S

Hence, R and S are not equivalent.

3) Are **R = (0+λ) (11*0)* (1+λ)** and **S = (1+λ) (011*) (0+λ)** equivalent?

Solution:

Find a string that belongs to one regex and does not belong to the other regex

011 ∉ R

011 ∈ S

Hence, R and S are not equivalent.

Exercise:

Can you answer whether the two regular expressions are equivalent or not?

1) (1+λ) (00*1)* 0* and (0+λ) (11*0)1*

2) 0*(10*)* and (1*0)*1*

References:

- Michael Sipser, Introduction to the Theory of Computation, 3rd Edition, Cengage Learning, June 27th 2012, ISBN: 9781285401065, 1285401069
- https://www.gatevidyalay.com/dfa-to-regular-expression-examples-automata/
- https://www3.ntu.edu.sg/home/ehchua/programming/howto/Regexe.html

Additional Info

Generalized NFA (GNFA)

In the theory of computation, a generalized nondeterministic finite automaton (GNFA), also known as an expression automaton or a generalized nondeterministic finite state machine, is a variation of a nondeterministic finite automaton (NFA) where **each transition is labeled with any regular expression**. The GNFA reads blocks of symbols from the input which constitute a string as defined by the regular expression on the transition.

There are several differences between a standard finite state machine and a generalized nondeterministic finite state machine.

- **A GNFA must have only one start state and one accept state, and these cannot be the same state, whereas an NFA or DFA both may have several accept states, and the start state can be an accept state.**
- A GNFA must have only one transition between any two states, whereas a NFA or DFA both allow for numerous transitions between states.
- In a GNFA, a state has a single transition to every state in the machine, although often it is a convention to ignore the transitions that are labelled with the empty set when drawing generalized nondeterministic finite state machines.

Generalized Transition Graph (GTG)

A generalized transition graph (GTG) is a transition graph whose **edges are labeled with regular expressions** or string of input alphabets rest part of the graph is same as the usual transition graph.

The label of any walk from initial state to a final state is the concatenation of several regular expressions, and hence itself a regular expression. The strings denoted by such regular expression are a subset of the language accepted by the generalized transition graph, with the full language being the union of all such generated subsets.

Formal definition of generalized transition graph:

A generalized transition graph is defined by a 5-tuple, are as follows;

- Q : A finite set of states.
- Σ : A finite set of input symbols.
- S : A non-empty set of start states, $S \subseteq Q$
- F : A set of final or accepting states $F \subseteq Q$
- A finite set, δ of transitions, (directed edge labels) (u, s, v), where $u, v \in Q$ and s is a regular expression over Σ.

Generalized transition graphs are nondeterministic: Given a state and a [partially consumed] input, there may be more than 1 possible successor state.

Informally a generalized transition graph is a collection of three things are as follows;

1. GTG consists finite number of states, **at least one of which is start state** and **some (maybe none) final states.**

2. GTG consist finite set of input letters (Σ) from which input strings are formed.
3. Directed edges in GTG connecting some pair of states **labeled with regular expression**.

It may be noted that in GTG, the labels of transition edges are corresponding regular expressions.

Example:

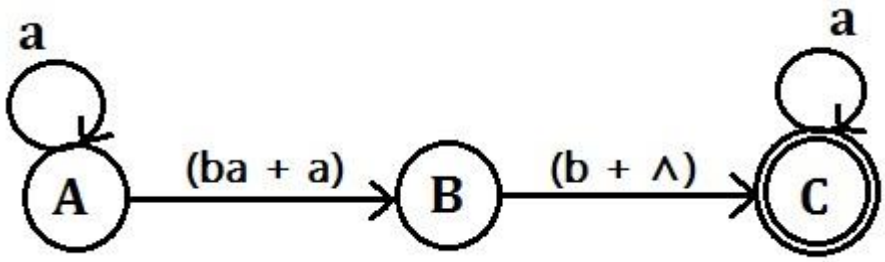


Figure 1 : Generalized Transition Graph (GTG) Example

This GTG accepts all strings without a double b. Note that the word containing the single letter b can take the free ride along the ∧-edge from start to middle, and then have letter b read to reach to the final state. The first edge should be labeled (ba + a) as in the figure above, not (ab + a).

Note: There is no difference between the Kleene star (*) closure for regular expressions and a loop in transition graphs, as illustrated in the following figure.

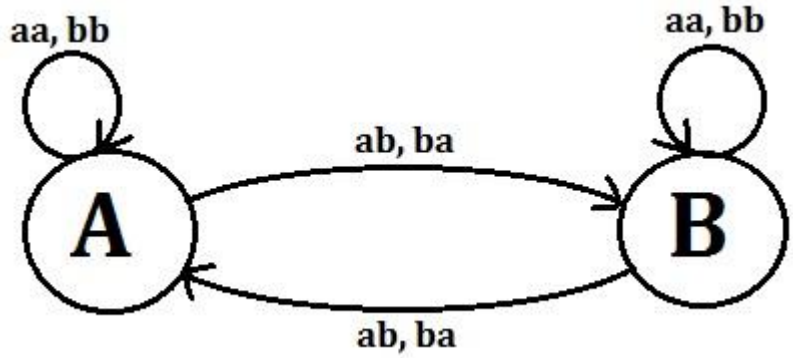


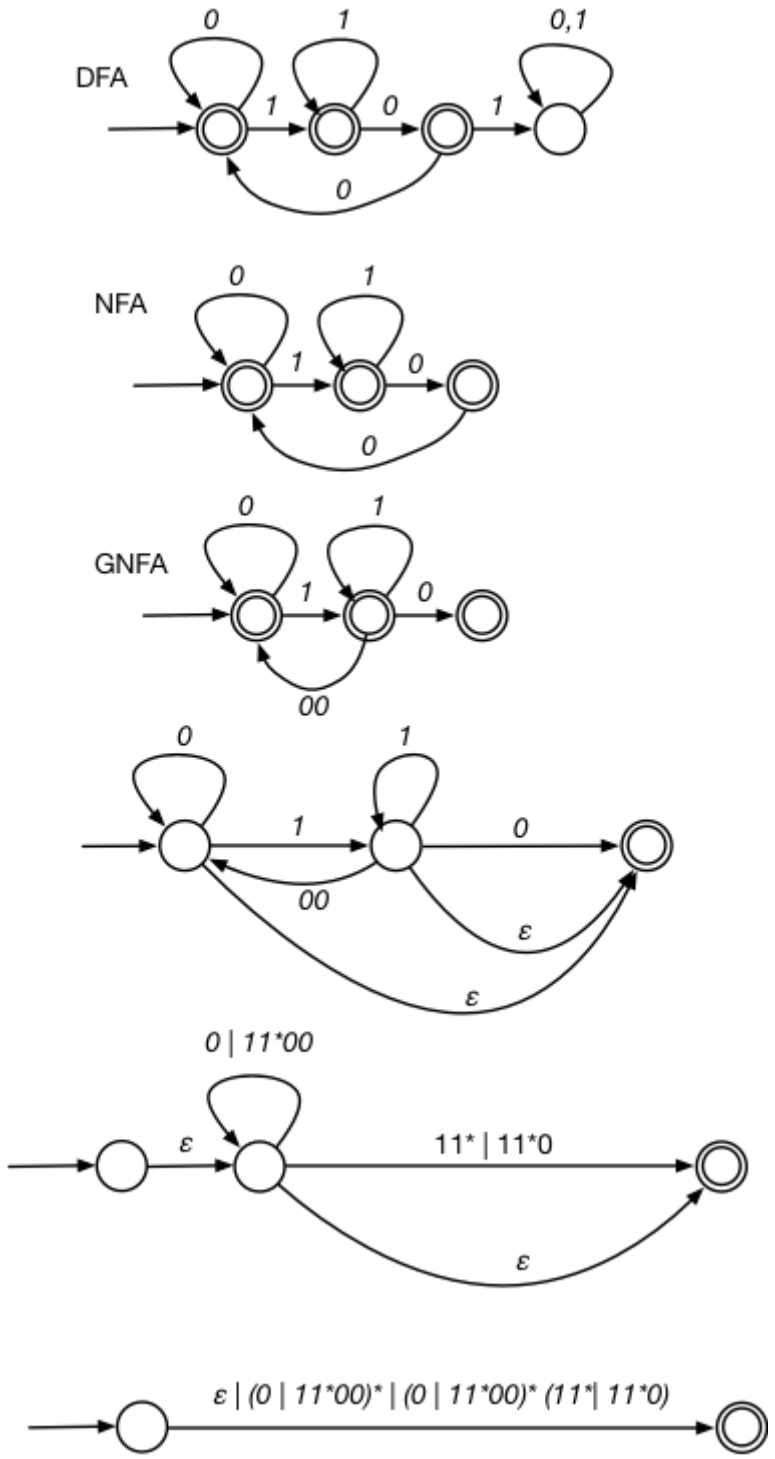
Figure 2 : Generalized Transition Graph (GTG) Example

Consider the even-even language, defined over Σ = {a, b}. As discussed earlier that even-even language can be expressed by a regular expression (aa+bb+(ab+ba)(aa+bb)*(ab+ba))* The language even-even may be accepted by the following GTG.

---

1) What is the regular expression of a string not containing 101 as a substring?

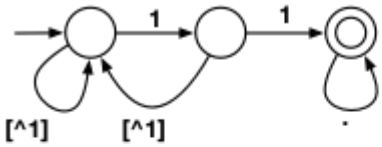
In the following construction I start with a DFA for $(101)^c$ which I then convert to a DFA and then into a sequence of GNFA's. The final GNFA yields the regular expression (which may not be the shortest possible).



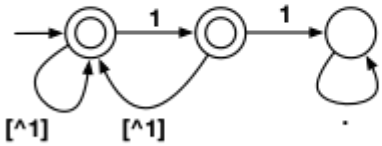
2) What is the regular expression for the string which doesn't contain 11 as a substring?

([^1] | (1[^1])*)* or simply **([^1] | 1[^1])***

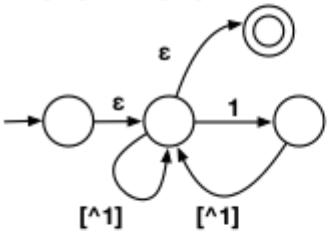
I arrived at this regular expression by first building a DFA for *L*= all strings that contain 11 as a substring. Then I built an DFA for the complement of *L*. Then there is a well known constriction for converting this to a regular expression.



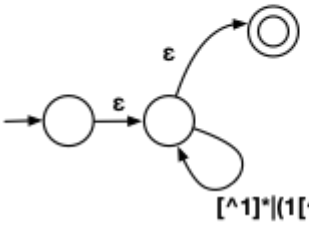
DFA for L = { .*11.* }



DFA for $\overline{L}$



NFA for $\overline{L}$



GNFA for $\overline{L}$

Greediness, Laziness and Backtracking for Repetition Operators

Greediness of Repetition Operators *, +, ?, {m,n}: The repetition operators are *greedy operators*, and by default grasp as many characters as possible for a match. For example, the regex xy{2,4} try to match for "xyyyy", then "xyyy", and then "xyy".

Lazy Quantifiers *?, +?, ??, {m,n}?, {m,}?, : You can put an extra ? after the repetition operators to curb its greediness (i.e., stop at the shortest match).

Backtracking: If a regex reaches a state where a match cannot be completed, it backtracks by unwinding one character from the greedy match. For example, if the regex z*zzz is matched against the string "zzzz", the z* first matches "zzzz"; unwinds to match "zzz"; unwinds to match "zz"; and finally unwinds to match "z", such that the rest of the patterns can find a match.

Possessive Quantifiers *+, ++, ?+, {m,n}+, {m,}+: You can put an extra + to the repetition operators to disable backtracking, even it may result in match failure. e.g, z++z will not match "zzzz". This feature might not be supported in some languages.

AUTOMATA FORMAL LANGUAGES AND LOGIC

Lecture notes

Regular Expression in Practice

Prepared by:

Kavitha K N

Assistant Professor

Department of Computer Science & Engineering

PES UNIVERSITY

(Established under Karnataka Act No.16 of 2013)

100-ft Ring Road, BSK III Stage, Bangalore – 560 085

Table of Contents:

Regular Expression in practice

1. Introduction

Regular expressions focus on solving a simple problem that is defining strings to match other strings. In simple terms a regular expression is equivalent to locating some substring within a document . For Example,expression is a valid regular expression that would match the word “expression ” in this sentence.Despite that the ability of regular expressions are further away than matching a single word.

Regular expressions can be used to match complex patterns of symbols or validate the format of input data for example your PAN card number, Adhaar, Credit card number and so on. The Programmers use regex all the time to find out what variable they are using and in what part of the program. Various programming languages provide the module to help write efficient code for example re module in python and Java String Basically regular expressions are used in search tools. For example To search a file in unix we use grep command and so on.

2. Special Characters

Since the regular expression can do more than just a search of a literal piece of a text we have few special characters reserved for the purpose of special use. Following are the meta characters that we will be discussing in detail :

#	Metacharacter	Short Description
1	.	Matches any single character except newline
2	*	Repeats preceding pattern 0 or more no. of times
3	+	Repeats preceding pattern 1 or more no. of times
4	?	Makes preceding pattern optional
5	[ ]	Matches a single character from the one’s specified in [ ]
6	^	Anchor for the start of the String or Negation symbol if used in [ ]
7	$	Marks end of the String
8	{ }	Specifies no. of repetitions to be made
9	\d	Matches a single digit from [0-9]

10	\s	Matches space character

Explanation of Metacharacters using Examples:

a) . (dot)

In regular expressions, the dot or period is one of the most commonly used metacharacters. The dot matches a single character, without caring what that character is. The only exception are newline characters

b) * (star)

It Causes the resulting RE to match 0 or more repetitions of the preceding RE

Example: a* matches { λ , a, aa,,aaa, ….}

c) + (plus)

Is a metacharacter that means to match 1 or more repetitions of the preceding RE

Example a+ matches 1 or more no. of a's

d) ? (question mark)

It matches a preceding regex at most once, in effect making it optional.

Example: ab?c will match either the strings "abc" or "ac" because the b is considered optional.

e) [ ] (square braces)

Define a character class to match a single character.it matches a single character. For example , if we have "cat" in a square bracket then it will either match c or a or t. Only a single character.

Example:

-> "[a-c]" is the same as "[abc]" and matches "a", "b" or "c"

-> "[A-Z]" matches any upper-case characters in the alphabet.

f) ^ (caret)

Caret is a special character used in regular expression, represented as cap symbol.caret is the anchor for the start of the string, or the negation symbol if used in a character class.

Example: "^abc" matches "a" at the start of the string.

When a caret appears as the first character inside square brackets, it negates the pattern. Here This pattern matches any character except a or b or c.

g) $ (dollar)

The dollar sign is the anchor for the end of the string.

Example:

-> regex$: Finds regex that must match at the end of the line

-> "xya$" matches a "a" at the end of a line.

-> "^$" matches the empty string.

h) { } (Curly braces) {m,n} : m to n repetitions:

This allows you to specify how many times a token can be repeated.

The syntax is {min,max},

where min is a positive integer number indicating the minimum number of matches, and max is an integer equal to or greater than min indicating the maximum number of matches.If the comma is present but max is omitted, the maximum number of matches is infinite.

So a{0, } is the same as a*, and

A{1, } is the same as a+. Omitting both the comma and max tells the engine to repeat the token exactly min times.

Example:

"a{2,3}" matches "aa" or "aaa".

a{3} will match a character exactly three times.

3. Examples of Regular Expressions in Practice

In this section we will see how to write regular expressions to validate the following:

a) PAN Card
b) AADHAR Number
c) Indian Mobile Number
d) Dates in 2020
e) Email Address

a) **PAN CARD:**

The valid PAN Card number must satisfy the following conditions:

- It should be 10 characters long.
- The first five characters should be any upper case alphabets.
- The next four-characters should be any number from 0 to 9.
- The last(tenth) character should be any upper case alphabet.
- It should not contain any white spaces.

PAN CARD Regex - ^[A-Z]{5}[0-9]{4}[A-Z]$

Input: str = "BNZAA2318J"
Output: true
Explanation:
The given string satisfies all the above mentioned conditions.

Input: str = "23ZAABN18J"
Output: false
Explanation:
The given string does not start with upper case alphabets, therefore it is not a valid PAN Card number.

b) **AADHAR NUMBER**

The valid Aadhar number must satisfy the following conditions:

- It should have 12 digits.
- It should not start with 0 and 1.
- It should not contain any alphabet and special characters.
- It should have white space after every 4 digits.

The regular expression is:
^[2-9]{1}[0-9]{3}\\s[0-9]{4}\\s[0-9]{4}$

It says

- The Aadhar number should start with a digit between 2 to 9 followed any 3 digits between 0 to 9 followed by \s, since \ is having a special meaning in RE, its special meaning is escaped by escape sequence,
- Followed by four digits between 0 to 9
- Followed by \s

- Followed by four digits between 0 to 9 $ says it should end with a digit
- So this is one of the regular expression to validate the Aadhar number

Input: str = “3675 9834 6012”
Output: true
Explanation: The given string satisfies all the above mentioned conditions. Therefore it is a valid Aadhar number.

Input: str = “3675 9834 6012 8”
Output: false
Explanation: The given string contains 13 digits. Therefore it is not a valid Aadhar number.

c) Regular expression to validate Indian Mobile Number

The valid Mobile number must satisfy the following conditions:

- It is a 10 digits number
- The first digit should contain numbers between 6 to 9.
- The remaining 9 digits can contain any number between 0 to 9.
- The mobile number can have 11 digits also by including 0 at the starting.
- The mobile number can be of 13 digits also by including +91 at the starting

MOBILE Number Regex - ^(0|”+91”)?[6-9]\d{9}$

Input: str = “7080059987”
Output: true
Explanation: The given string satisfies all the above mentioned conditions. Therefore it is a valid Indian mobile number.

Input: str = “2457864578”
Output: false
Explanation: The given string contains 2 as first digit. Therefore it is not a valid Indian Mobile number.

d) The regular expression to validate the date in the format mm/dd/yy
Regular expression to validate Dates in the year 2020

- It is a leap year
- Let the format be DD-MM-YY

- Months with 31 days are : Jan, Mar, May, July, Aug, Oct, Dec
  : [1, 3, 5, 7, 8, 10, 12]
- Months with 30 days are : Apr, June, Sep, Nov
  : [4, 6, 9, 11]
- Month with 29 days : Feb
  : [2]
- Year : 2020

Regular expression for date:
(0[1-9]|[10-31]”-”(0[13578]|1[0|2])|[1-30]”-”(0[469]|11)|[1-29]”-””02”)”-””2020”

Input: str = “4-11-2020”
Output: true
Explanation: The given string satisfies all the above mentioned conditions. Therefore it is a valid date in the year 2020

Input: str = “42-78-2020”
Output: false
Explanation: The given string contains digits which does not satisfy the given condition. Therefore it is not a valid date in the year 2020

e) Regular expression to validate Email address

An email is a string (a subset of ASCII characters) separated into two parts by @ symbol. a "personal_info" and a domain, that is personal_info@domain.
The length of the personal_info part may be up to 64 characters long and domain name may be up to 253 characters.

The personal_info part contains the following ASCII characters.

- Uppercase (A-Z) and lowercase (a-z) English letters.
- Digits (0-9).
- Characters ! # $ % & ' * + - / = ? ^ _ ` { | } ~
- Character . (period, dot or full stop) provided that it is not the first or last character and it will not come one after the other.
- The domain name [for example com, org, net, in, us, info] part contains letters, digits, hyphens, and dots.

Example of valid email id

- mysite@ourearth.com
- my.ownsite@ourearth.org

- mysite@you.net

Example of invalid email id

- mysite.ourearth.com [@ is not present]
- mysite@.com.my [tld (Top Level domain) can not start with dot "."]
- @you.me.net [No character before @]

Regular Expression:
^[_a-zA-Z0-9-]+(\.[_a-zA-Z0-9-]+)*@[a-zA-Z0-9-]+(\.[a-zA-Z0-9-]+)*\.(([0-9]{1,3})|([a-zA-Z]{2,3})|(com|edu|info|museum|name))$

f) Regular expression to validate simple URL

Regular expression:
^((ht|f)tp(s?))\://([0-9a-zA-Z\-]+\.)+[a-zA-Z]{2,6}(\:[0-9]+)?(/\S*)?$

- The URL can start with ht or f followed by tp
- Followed by zero or one s
- Followed by ":" symbol
- Followed by //
- Followed by any alphanumeric character
- Followed by - or .
- Followed by combination of uppercase or lowercase characters from 2 to 6 times
- Followed by : and digits 0 to nine one or more times

g) Regular expression in JAVA programming language

Regular expressions are used in validity checking for example :
Does the given input match the pattern specified using the Regular expression. We should use Java String Library(Use input.matches(regexp) for basic RE matching.
The simple JAVA code snippet shown below specifies how JAVA is used to validate the string.

```
public class validate
{
        public static void main(String[] args)
        {
        String regexp = args[0];
        String Input = args[1];
        StdOut.println(input.matches(regexp));
```

```
        }
    }
```

This code snippet takes 2 command line argument, one is the input string that has to be validated, and the other input is the regular expression

Agrs[0] contains the regular expression and args[1] contains the string to be validated against the regular expression

The method input.matches(regexp) compares the input with the regular expression and prints true if the string matches with the regular expression or False otherwise on to the terminal.
If we compile the code withy the command below:
-> java validate "[$+A-Za-z][A-Za-z0-9]*" ident123

The first argument is the regular expression to validate the Legal JAVA identifier and the second argument is one of the valid JAVA identifiers.
The regular expression says the identifier can start with any combination of uppercase or lowercase letters any number of times followed by combination of anu uppercase, lowercase letters or digits from 0 to 9
The given input string is a valid identifier so if prints TRUE on to the console.

-> java validate "[a-z]+@([a-z]+\.)+(edu/com)" abcd.pes.edu

This example is to validate the simple email address.The regular expression says the email address should start with the combination of any lowercase characters (at least one character should be there)followed by @ character and followed by the combination of any lowercase characters (at least one character should be there) followed by edu or com. The given input string is a valid email id so the code outputs true on to the terminal.

AUTOMATA FORMAL LANGUAGES AND LOGIC

Lecture notes on Converting Regular Expression to Finite Automata

Prepared by

Ms. Sangeeta V I
Assistant Professor

Ms. Preet Kanwal
Associate Professor

Department of Computer Science & Engineering

PES UNIVERSITY

(Established under Karnataka Act No.16 of 2013)
100-ft Ring Road, BSK III Stage, Bangalore – 560 085

Table of Contents:

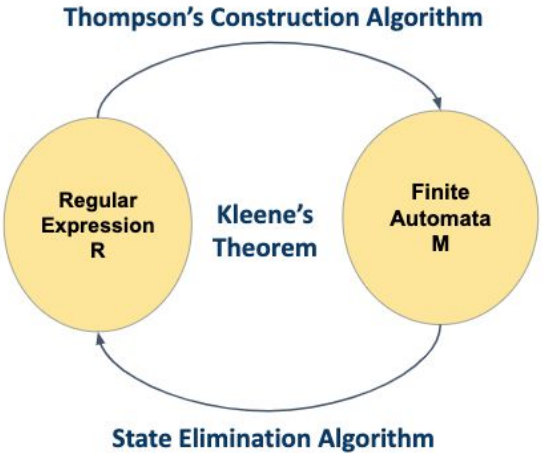
Examples Solved:

1. Regular Expression to Finite Automata

Regular expressions and finite automata have equivalent expressive power:

Many software tools work by matching regular expressions against text. Text processing utilities use regular expressions to describe advanced search patterns, but NFAs are better suited for execution on a computer.

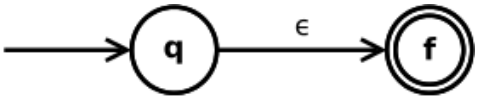
The equivalence of regular expressions and finite automata is known as Kleene's theorem. We use Thompson Construction algorithm to convert a RE to NFA . This algorithm is of practical interest, since it can compile regular expressions into NFAs. We use a State Elimination method in order to convert a finite automata to Regex.



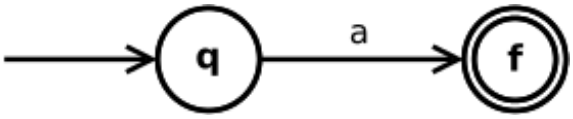
Converting regular expressions to finite automata using Thompson Construction Method:

The algorithm works recursively by splitting an expression into its constituent subexpressions, from which the NFA will be constructed using a set of rules.
Following are the rules :

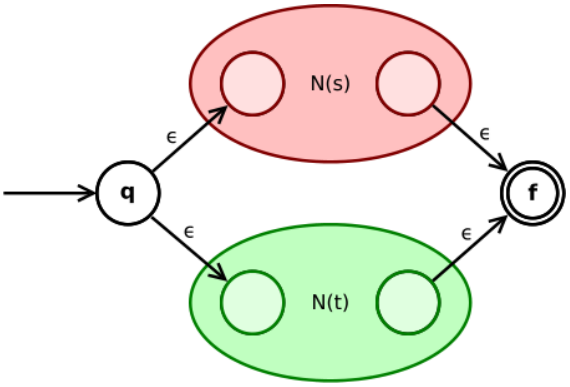
- If the operand is epsilon, then our FA has two states, q (the start state) and F (the final, accepting state), and an epsilon transition from q to F.



- If the operand is a character a, then our FA has two states, q (the start state) and F (the final, accepting state), and a transition from q to F with label a.

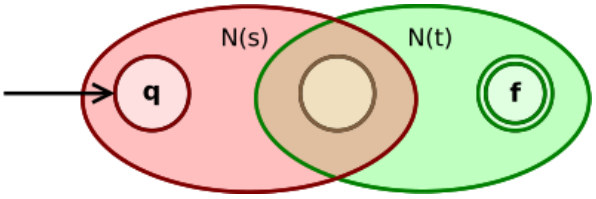


- The **union expression** *s*|*t* is converted to



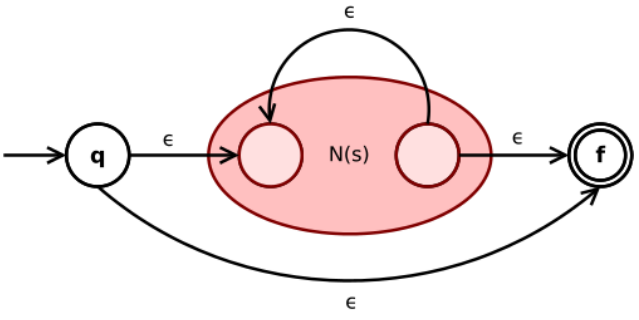
State *q* goes via ε either to the initial state of Automata of s and t . *N*(*s*) or *N*(*t*). Their final states become intermediate states of the whole NFA and merge via two ε-transitions into the final state of the NFA.

- The **concatenation expression** *st* is converted to



The initial state of *N*(*s*) is the initial state of the whole NFA. The final state of *N*(*s*) becomes the initial state of *N*(*t*). The final state of *N*(*t*) is the final state of the whole NFA.

- The **Kleene star expression** s^* is converted to



An ε-transition connects the initial and final state of the NFA with the sub-NFA .

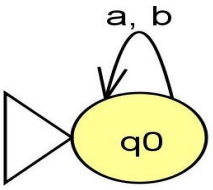
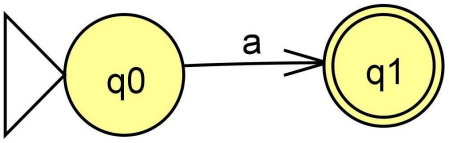
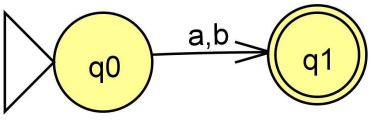
N(*s*) Automata for s in between. Another ε-transition from the inner final to the inner initial state of *N*(*s*) allows for repetition of expression *s* according to the star operator.

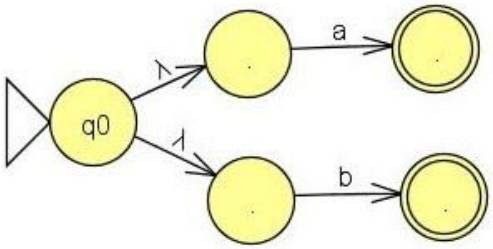
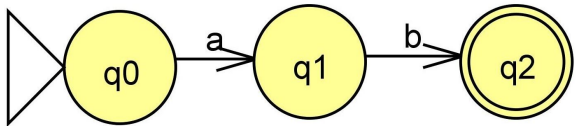
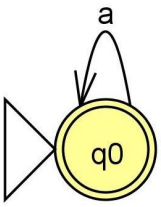
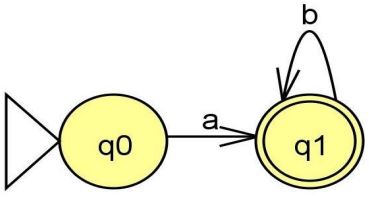
- The **parenthesized expression** (*s*) is converted to *N*(*s*) itself.

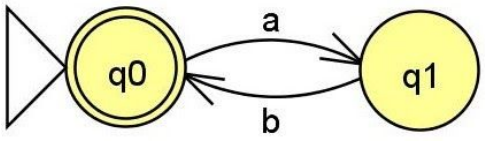
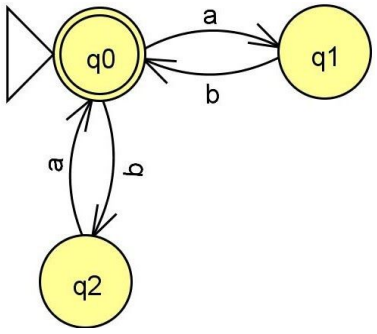
With these rules, using the **empty expression** and **symbol** rules as base cases, it is possible to prove with mathematical induction that any regular expression may be converted into an equivalent NFA.

1.1. Basic Regular Expressions to NFA/ λ -NFA

Let us start with some basic regular expressions.
Assume Σ={a,b}.

Regular Expression	**NFA/ λ -NFA**
ϕ	 ϕ represents not accepting anything that is,no accepting/final state . In state q0,any input symbol is rejected.
a	 In state q0 ,we will wait for input symbol 'a'. Once we see 'a' in state q0 we will move to state q1 and accept it.
a+b	 We accept either a or b. In state q0,if we see 'a' we move to state q1 and accept it or

	if we see ‘b’ we move to state q1 and accept it. a+b can also be represented by treating a and b as two different regular expressions ,we get the automata as shown below: 
a.b	 a followed by b and accept the string.
a*	 a* represents any number of ‘a’s including λ (empty string). So we make q0 as the final state. When we see * we put a self loop.
ab*	 ab*. ‘a’ is the minimum string. In state q0 we look for the symbol ‘a’ . Once we see the symbol ‘a’ in q0 we move to state q1 and

	accept it . In state q1 we can accept any number of b's. When we see a * we put a loop.
(ab)*	 (ab)*=sequences of ab= λ ,ab,abab,ababab,..... We look for 'a' followed by a 'b' . In state q0 we look for 'a' and move to state q1. In state q1 we see a 'b' and move back to state q0 and accept it. Since there is a *,we make q0 as the final state.
(ab+ba)*	 Strings accepted by automata=Sequences of "ab" and/or sequences of "ba".

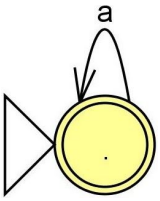
1.2: Higher end Problems on converting Regular Expression to Finite Automata

Problem #1:

Convert the regular expression a*+b*+c* to finite automata.

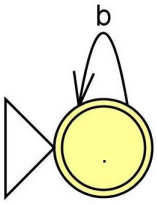
Solution :

Automata accepting a* :



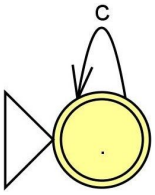
- a* indicates any number of a's including λ .
- When we see * we put a loop.

Automata accepting b*:



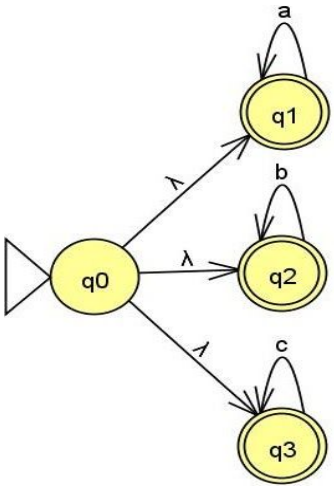
- b* indicates any number of b's including λ .
- When we see * we put a loop.

Automata accepting c*:



- c* indicates any number of c's including λ .
- When we see * we put a loop.

We can combine the automata accepting a*,b* and c* by introducing a new start state q0 with λ ,-transitions connecting to the old start states of a*,b* and c*.

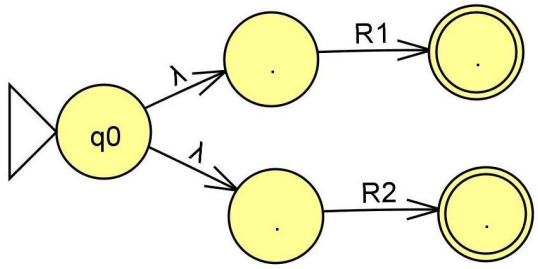


Problem #2:

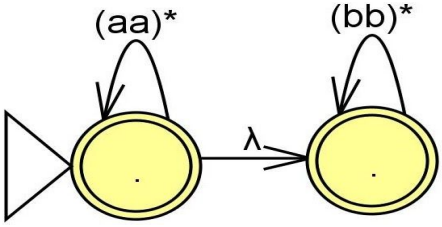
Convert the regular expression (aa)*(bb)*+a(aa)*b(bb)* to finite automata.

Let,

- R1=(aa)*(bb)*
- R2=a(aa)*b(bb)*.
- Construct accepter/automata for R1
- Construct accepter/automata for R2.
- Introducing a new start state on λ we reach the start state of R1 and R2.

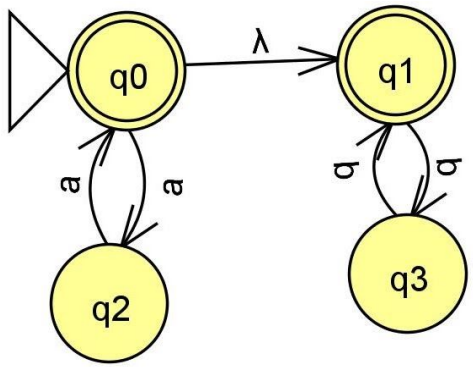


Brief diagram of automata for R1=(aa)*(bb)*



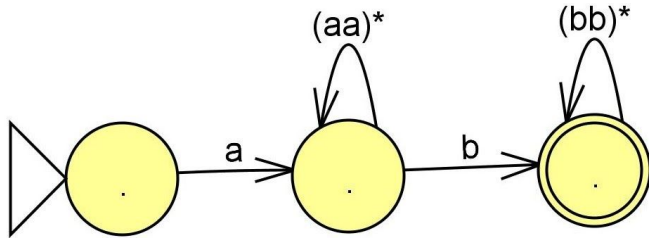
We look for sequences of any number of “aa” ‘s followed by sequences of any number of “bb”’s.

Expanded form of automata for R1=(aa)*(bb)*



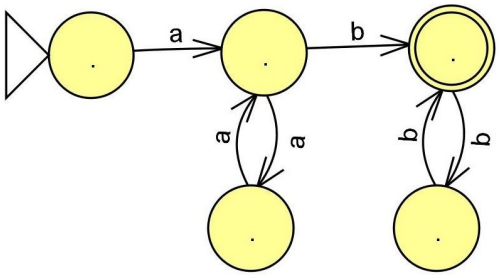
In state q0 we look for ‘a’ followed by symbol ‘a’ or ‘b’ followed by symbol ‘b’.

Brief diagram of automata for R2=a(aa)*b(bb)*

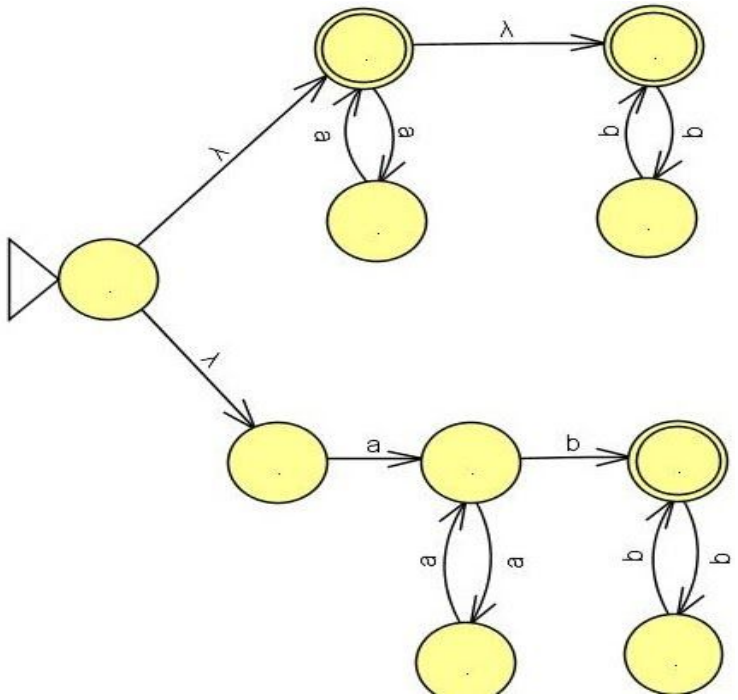


We look for “a “ followed by any number of sequences of “aa” ,followed by a ”b” and sequences of any number of “bb”. When we see a star we put a loop.

Expanded form of automata for R2=a(aa)*b(bb)*



Automata for the regular expression R1+R2= (aa)*(bb)*+a(aa)*b(bb)*.
We will combine the automata for R1 and automata for R2 by introducing a new start state on λ we reach the start state of R1 and R2.



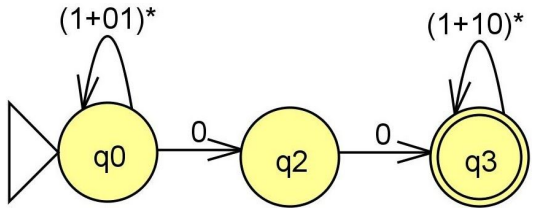
Problem #3:

Convert the regular expression (1+01)*00(1+10)* to finite automata.

Solution :

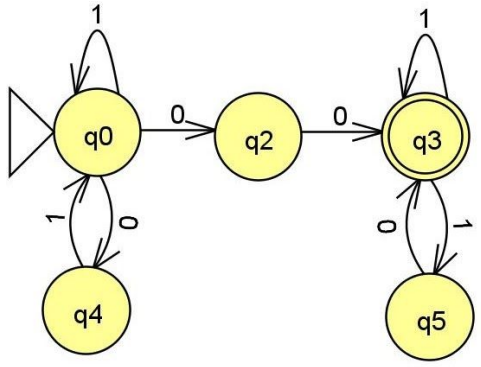
(1+01)*00(1+10)* is the concatenation of (1+01)* , 00 and (1+10)* .

Brief diagram of automata for (1+01)*00(1+10)*



- When we see a * we put a loop.

Expanded form of automata for (1+01)*00(1+10)*



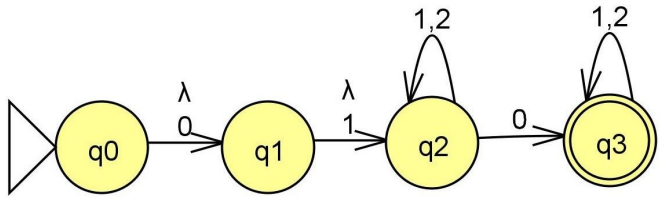
Problem #4:

Convert the regular expression (0+ λ)(1+ λ)(1+2)*0(2+1)* to finite automata.

Solution :

(0+ λ)(1+ λ)(1+2)*0(2+1)* is the concatenation of (0+ λ) , (1+ λ) , (1+2)*, 0 and (2+1)*.

Automata for (0+ λ)(1+ λ)(1+2)*0(2+1)*



When we see * we put a self loop.

Problem #5:

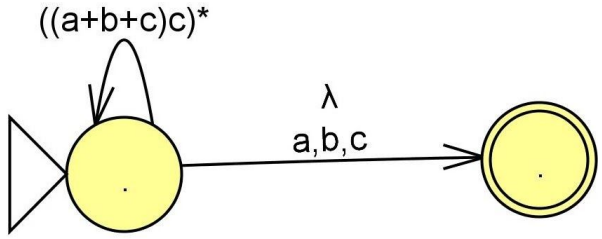
Convert the regular expression ((a+b+c)c)*(a+b+c+ λ) to finite automata.

Solution :

((a+b+c)c) indicates a or b or c followed by a c ,these are strings of length 2.

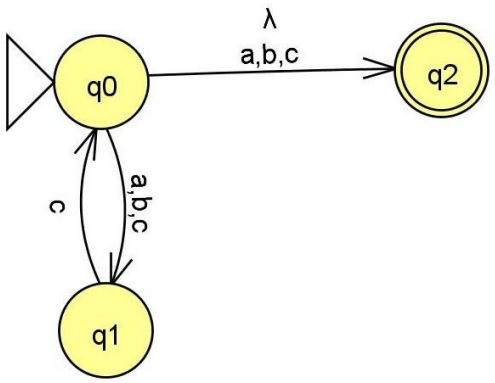
((a+b+c)c)* strings of length 2 repeated any number of times.

((a+b+c)c)*(a+b+c+ λ) strings of length 2 repeated any number of times. Followed by 'a' or 'b' or 'c' or nothing (λ).



When we see "* " we put a self loop.

Expanded form of automata for ((a+b+c)c)*(a+b+c+ λ)



- In q0,when we see ‘ a’ or ‘b’ or ‘c’ it has to be followed by “‘c“ . That is strings of length 2 (‘a’ or ‘b’ or ‘c’ followed by ‘c’)repeated any number of times.
- Followed by ‘a’ or ’b’ or ‘c’ or nothing (λ).

AUTOMATA FORMAL LANGUAGES AND LOGIC

Lecture notes on Converting Finite Automata to Regular Expression

Prepared by

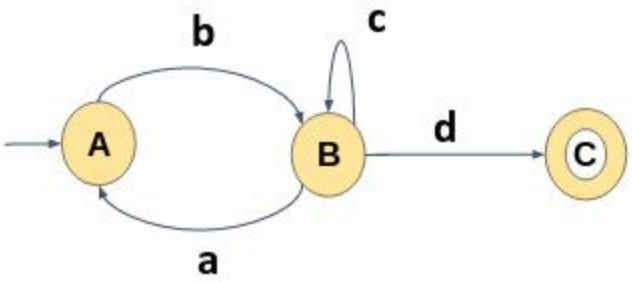
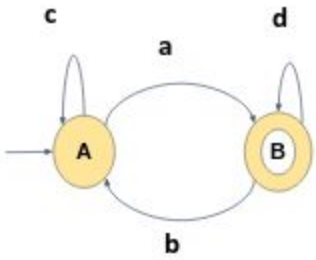
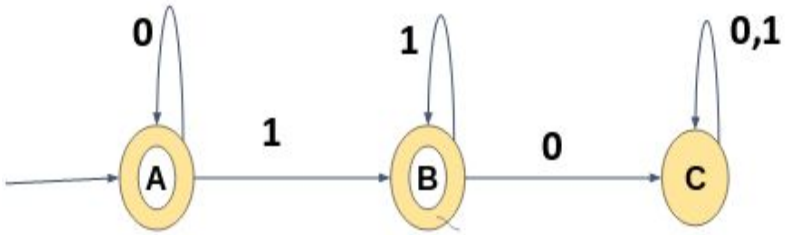
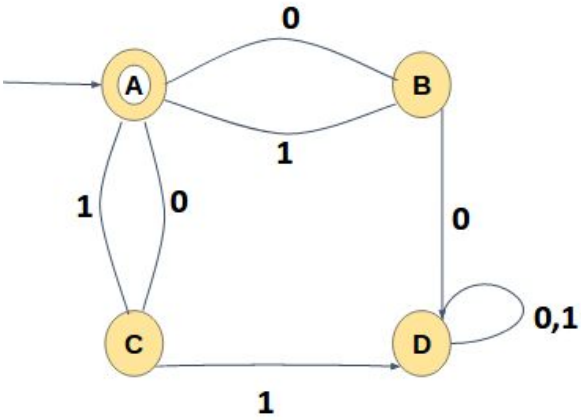
Ms. Kavitha K N
Assistant Professor

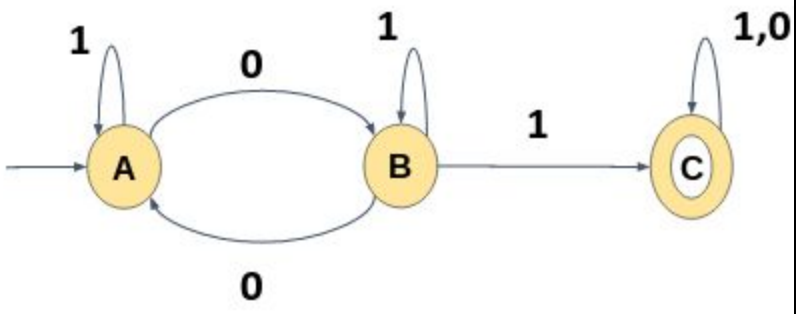
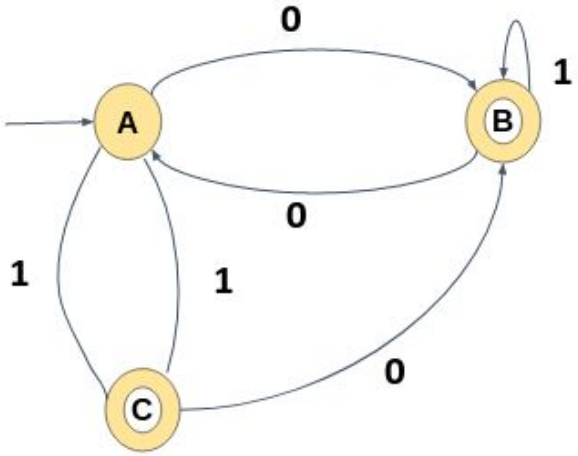
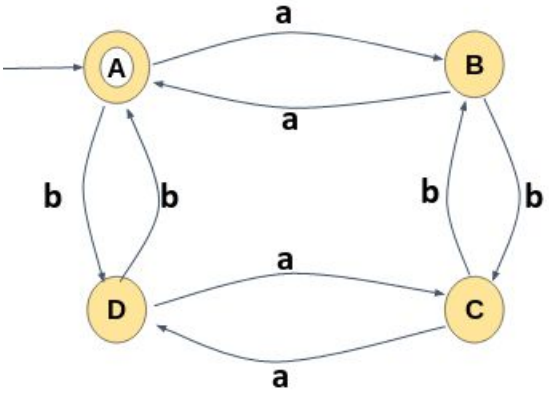
Ms. Preet Kanwal
Associate Professor

Department of Computer Science & Engineering
PES UNIVERSITY
(Established under Karnataka Act No.16 of 2013)
100-ft Ring Road, BSK III Stage, Bangalore – 560 085

Table of Contents:

Examples Solved:

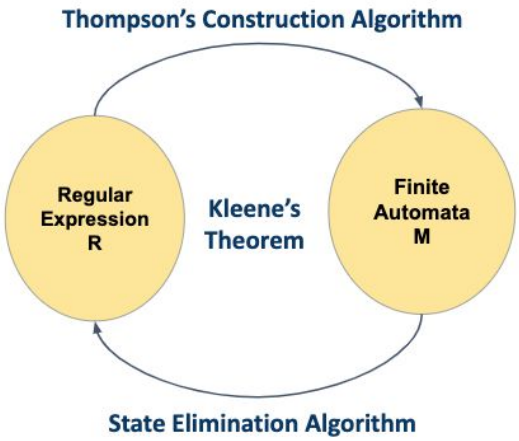
3		**10**
4		**11**
5		**17**
6		**21**

7	 1 0 1 1,0 1 A B C 0	**24**
8	 0 1 A B 0 1 1 0 C	**27**
9	 a A B a b b b b a D C a	**31**

1. **Introduction:**

   Given any DFA M, we have to construct a regular expression r (over the alphabet of input symbols of M) satisfying L(r) =L(M).

   To construct a regular expression from a DFA , we make use of the State Elimination Algorithm in which we replace each state in the DFA one by one with a corresponding regular expression.



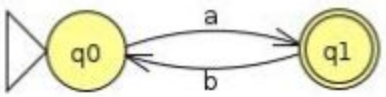
2. **State Elimination method:**

   This method is used for constructing Regular Expression from the Finite Automata
   In this method states will be removed from the automaton, restoring the labels of arc's , so that after removing all the states except first and last state, at the end only a single regular expression is formed.
   This single regular expression will be the label on an arc that goes from start state to end state, and this will be the only arc in the automata and the label will be the final regular expression for the given automaton
   This method follows the following simple steps in constructing Regular Expression from the Automaton
   Note: Before eliminating the states, Dead state/ trap states if any need to be removed

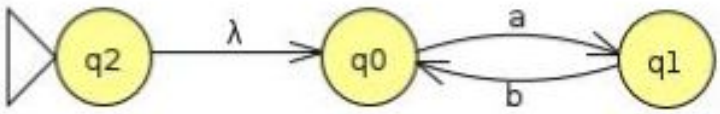
   Start with an FA for the language L.

- **Add a new start state qs to the FA.**

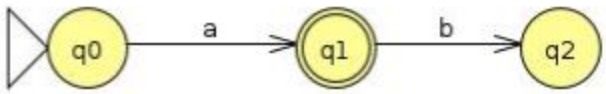
  If the start state itself is an accepting state or has any transition in(incoming edge) then create a new start state having no transition in and make a transition from new start state to an old start state with λ transition
  Example:



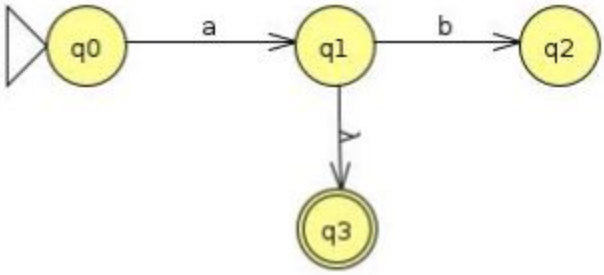
The start state q0 have incoming edge, This will be modified as below after adding new start state q2 with λ transition



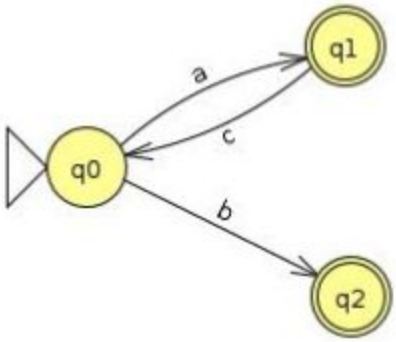
- **Add a new accept state qf to the FA**
  If there exists any outgoing edge from the final state, then create a new final state having no outgoing edge from it.



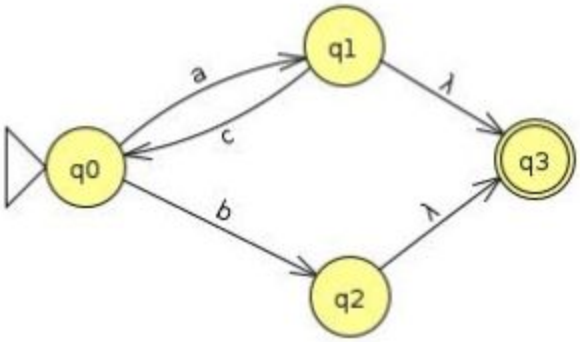
The final state q1 has an outgoing edge, so create a new final state q3 and make q1 as a non final state and have λ transition from q to q3 as shown below.



- **Add λ-transitions from each original accepting state to qf , then mark them as not accepting**.
  If there exists multiple final states in the FA, create an equivalent automaton that has a single accepting state with no transitions out.



The above automata have two accepting states, create a new single accepting state q3 and have a λ transition from q1 and q2 to q3 by changing them to non final state. The automata after introducing new final state is shown below:

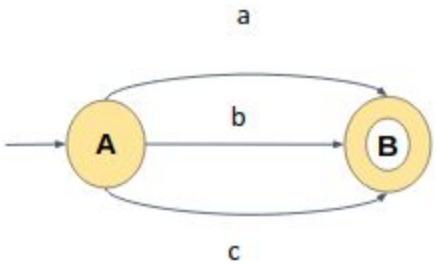


- **Repeatedly remove states in any order other than start state and final state from the FA by "shortcutting" them until only two states remain: start and final state .**

- **The transition from qs to qf is then a regular expression for the FA.**

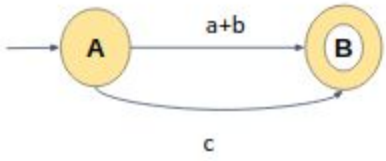
Example:

1) Conversion of Finite Automata to Regular Expression

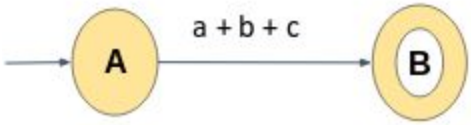


Solution:

Start state does not have any incoming edge and Accepting state does not have any outgoing edge so there is no need to introduce any new start or final state, there are three direct transition for state A to state B with the input a, or with the input b, or with the input c,So the edges can be replaced by a single edge as shown below, input a or b be can replaced with a single edge with the expression a+b.

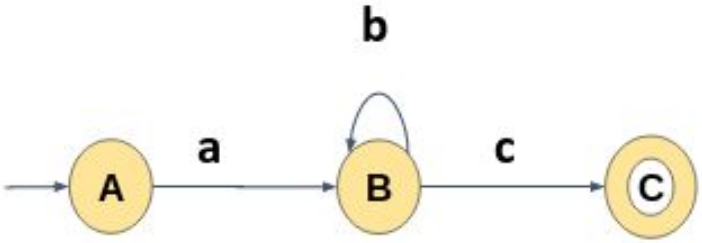


then we have two edges (transition), one for (a+b) and the other for c can be replaced with a single edge with the label (expression (a+b+c) as shown below:

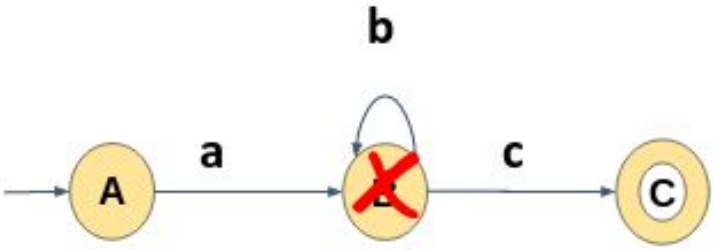


The final regular expression for the given automaton is **(a+b+c)**

2) Convert the following FA to RE



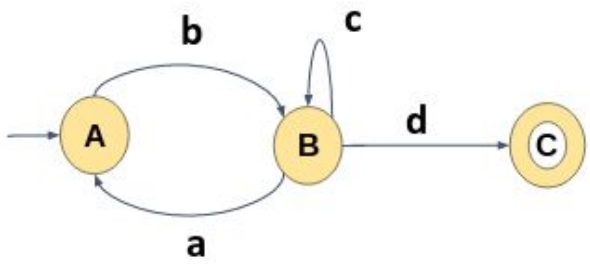
For the given DFA there is no incoming edge to the start state and no outgoing edge from the final state, se we can start eliminating the intermediate states(states between start and final state)
The only state between start state A and the final state C is B, so let us eliminate the state B



there is a path going from state A to state C through B, and State B have a self loop,
So after deleting state B, there will be a direct path from state A to state B having the cost (ab*c)

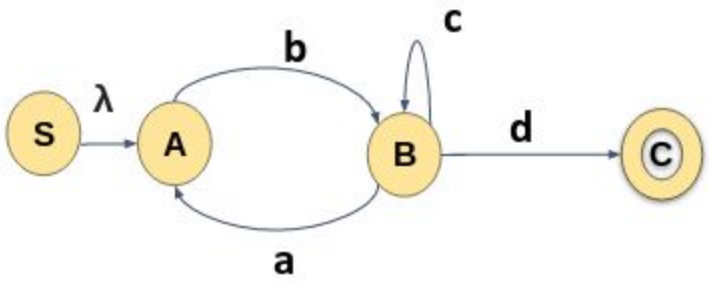
The final Regular Expression is **(ab*c)**

3) Convert the following Finite Automata to Regular Expression



Start state A is having an incoming transition, so we create a new start state with the name S, (from now state S will be the start state)

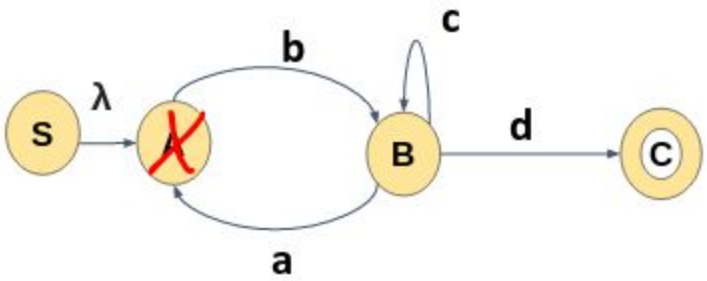
Make a transition form new start state to state A with λ as shown



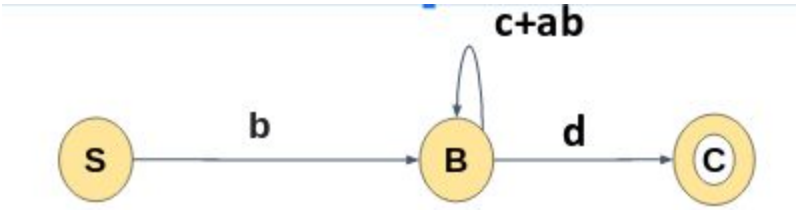
We first eliminate state A,

-> there is a path going from state S to state B through state A , so after eliminating state A there will be direct path form state A to state B with the cost/expresion ' λ .b = b'

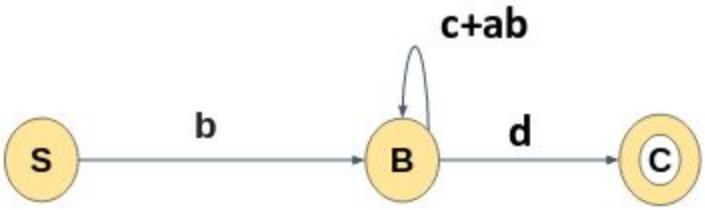
-> There is a loop on state B through state A, So after deleting state A, state B will have direct path with the cost / expression ab+c



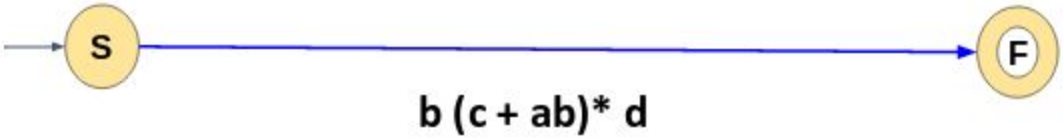
After Eliminating state A, the automata will look like this:



Next, we will eliminate state B, the only intermediate state
-> there is a path from state S to state C through B and B have self loop, so after eliminating state B there will be a direct path from state A to state C with the cost / expression b(c+ab)*d

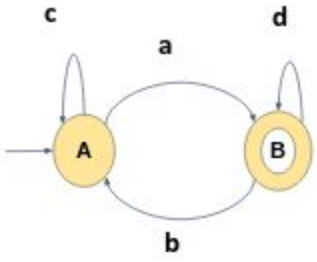


After eliminating state B the automata will look like this, there are no intermediate states to eliminate

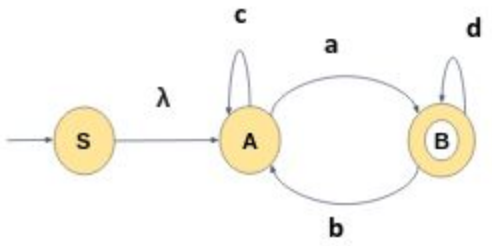


The final regular expression for the given finite automata is **b(c+ab)*d**

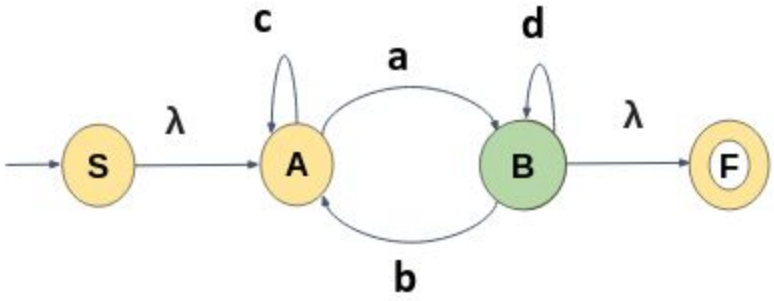
4) Convert the following Finite Automata to Regular Expression



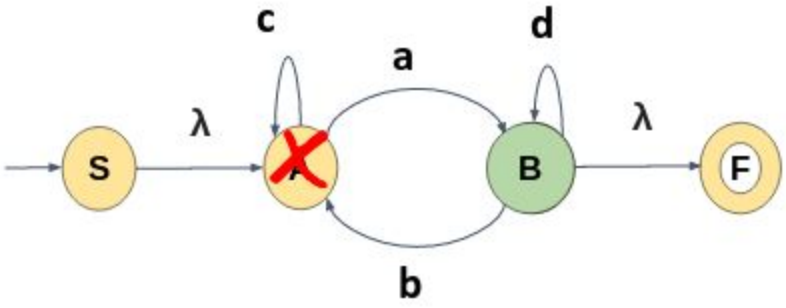
Start state A is having an incoming transition, so we create a new start state with the name S, (from now state S will be the start state). Make a transition form new start state to state A with λ as shown



Final state B has an outgoing edge so, we create a new final state F and make the state B as non final state, also have a transition from state B to state F with λ as shown below:



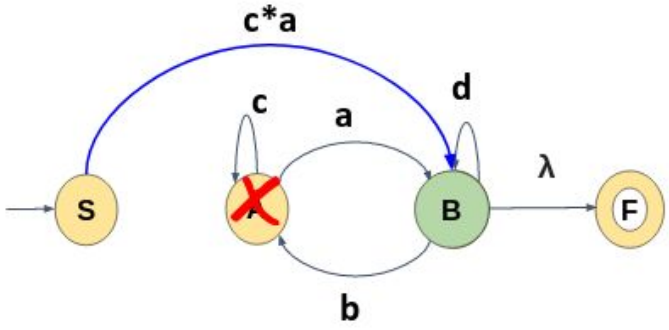
First eliminate state A,



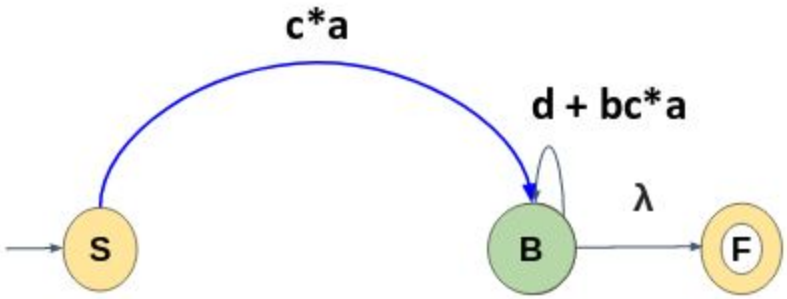
-> There is a path going from state S to state B through state A and state A have a self loop.

So after deleting state A, We will have a direct path from State S to state B having the cost **λ.c*a = c*a.**

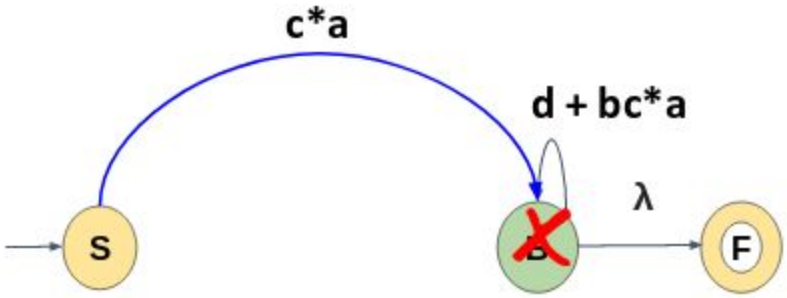
The direct path what we get after deleting A is shown below
-> It also have a loop on state B through state A , B also have a self loop, and state A also have a self loop



So after deleting state A , we get a direct self loop on state B having the expression d+bc*a as shown below:

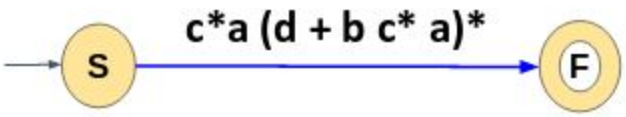


Next eliminate state B,



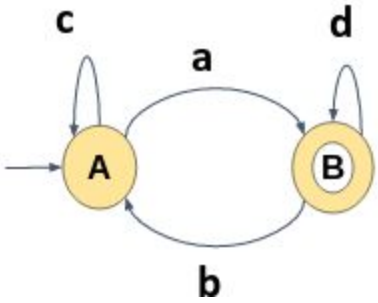
There is a path from state S to state F through B, and B have a self loop
So after eliminating there will be a direct path from state S to state F with the cost c*a.(d+bc*a)*. λ = c*a.(d+bc*a)*

We have only two states remaining and the label on the path from state S to state F is the final regular expression.

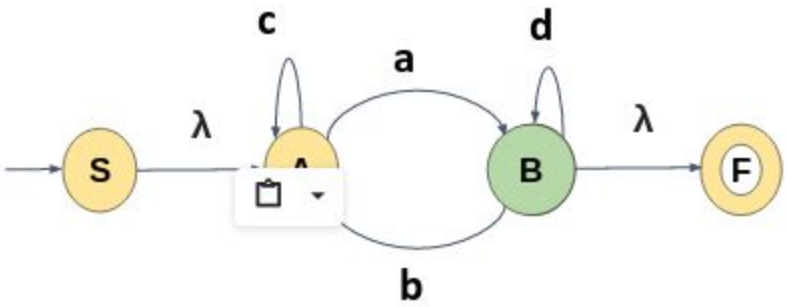


The final regular expression for the given Automaton is : **c*a (d + b c* a)***

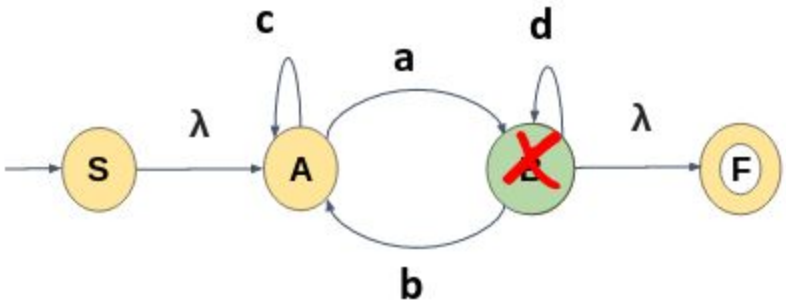
-> Consider the same example but different order in deleting the states



Adding a new start state and final state remain the same.



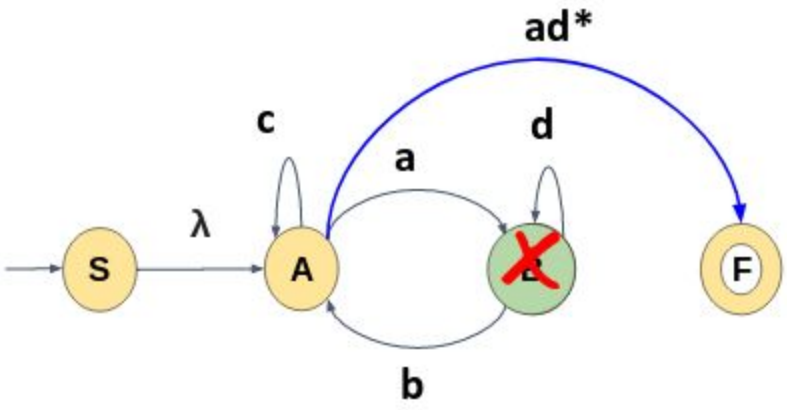
Now first eliminate state B instead of state A,



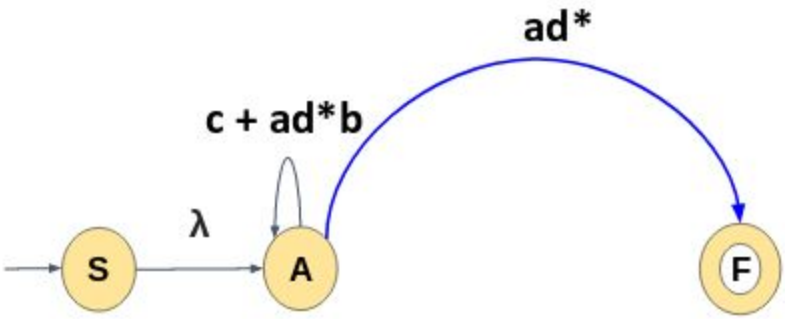
-> There is a transition from state A to state F through B and B has a self loop,

So, after eliminating state B there will be a direct path from state A to state F having a cost **(ad*. λ) = ad***

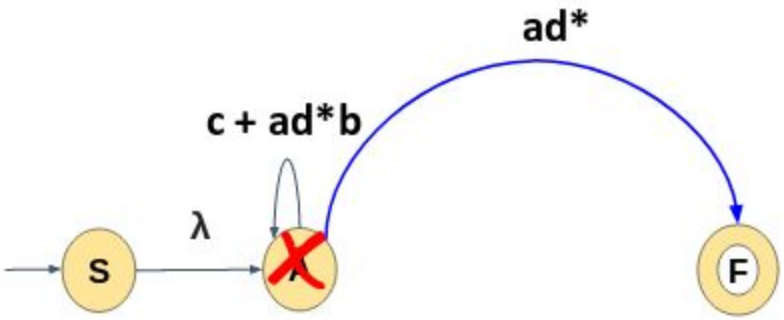
Direct path from state A to state F is shown below
-> there is a loop on state A through state B , A also have a self loop, and B also have a self loop,
so after eliminating state B there will be a direct self loop on state A having the cost **c*+ad*b**



Path after deleting state B is shown below along with the expressions

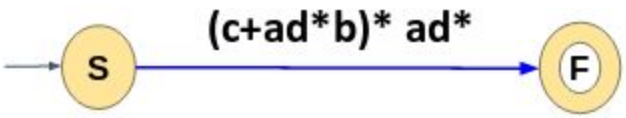


-> next eliminate the only intermediate state A(state between start state and accepting state)

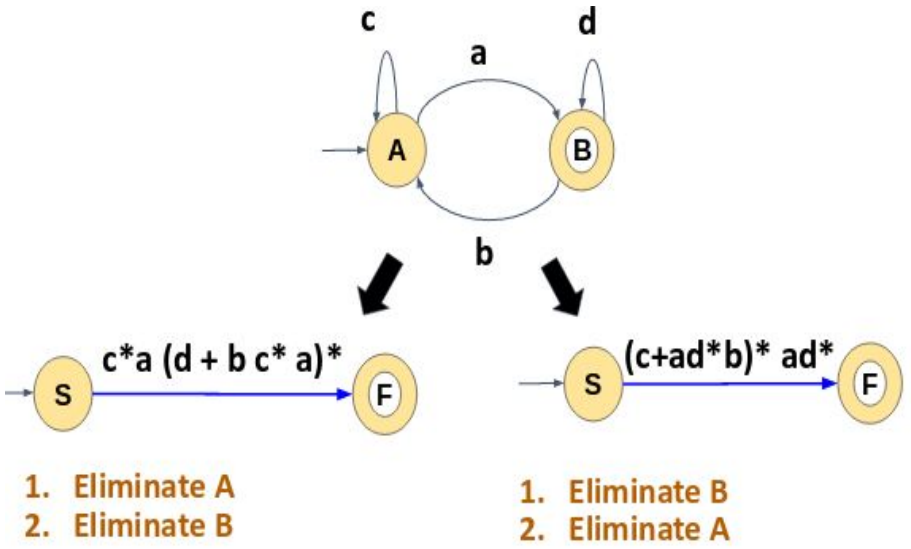


There is a path from state S to state F through state A and state A have a self loop, ,so after deleting state A, we get a direct path from state S to state F having the cost λ . **(c+ad*b)* ad* = (c+ad*b)* ad***

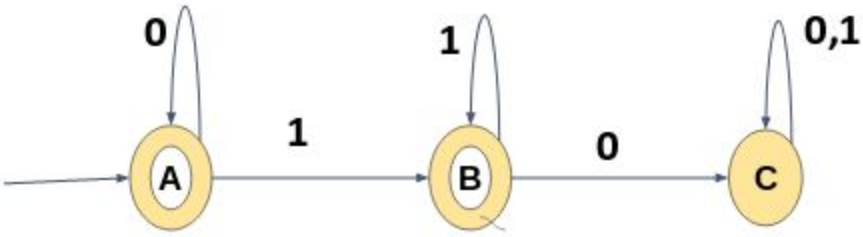
We have only two states remaining and the label on the path from state S to state F is the final regular expression



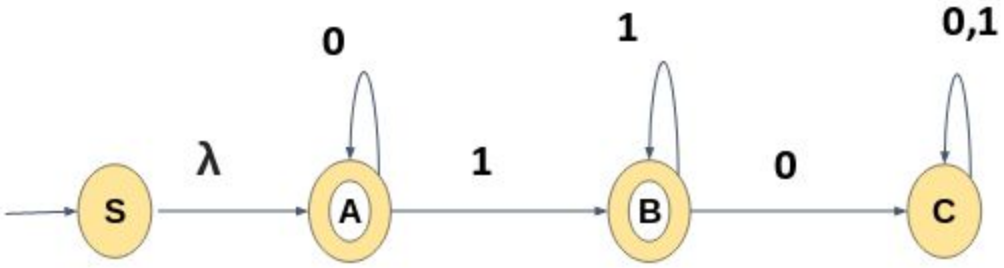
The final expression is **(c+ad*b)* ad***
The below are the two different regular expression obtained based on the order of eliminating the states



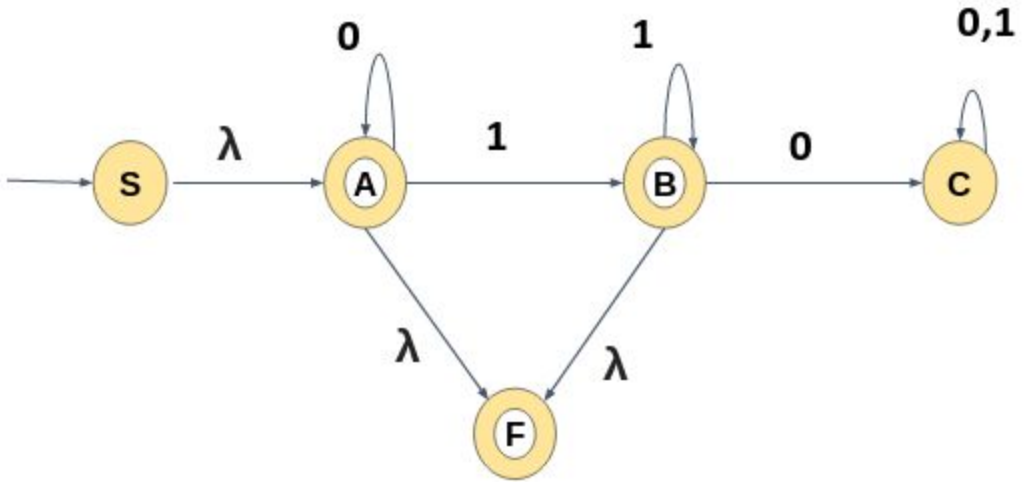
5) Convert the following Finite Automata to Regular Expression



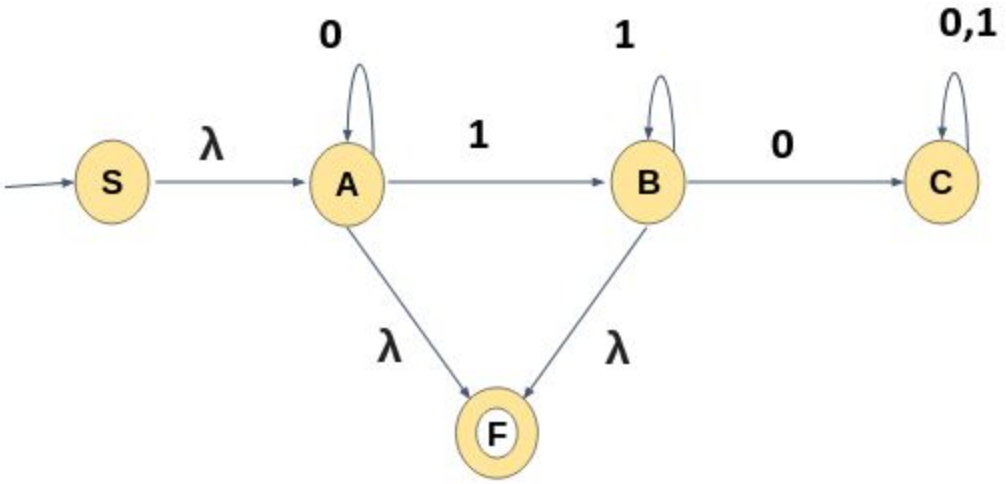
Start state A is having an incoming transition, so we create a new start state with the name S,Make a transition form new start state to state A with λ as shown:



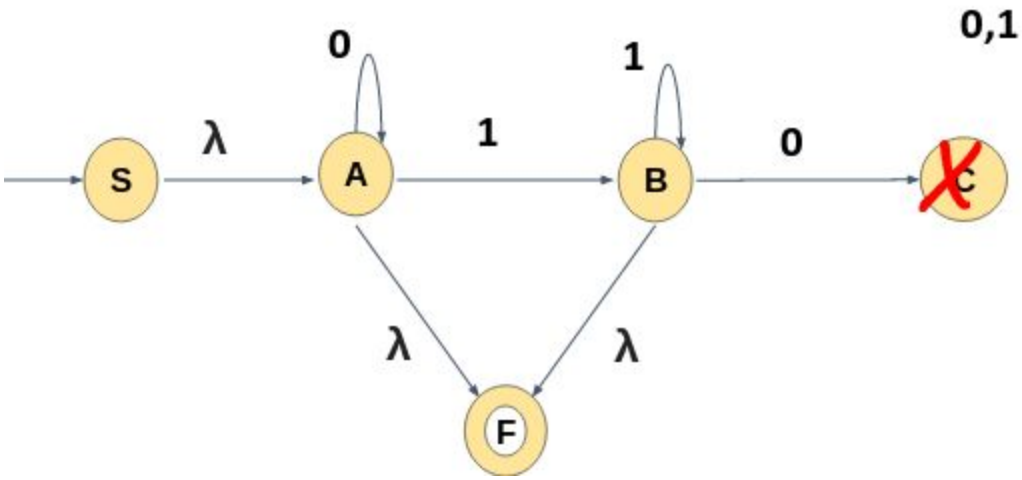
Since there are two final states and an outgoing edge , we create an equivalent automaton that has a single accepting state with no transitions out and Make a transition from old final states to new final sate with λ .



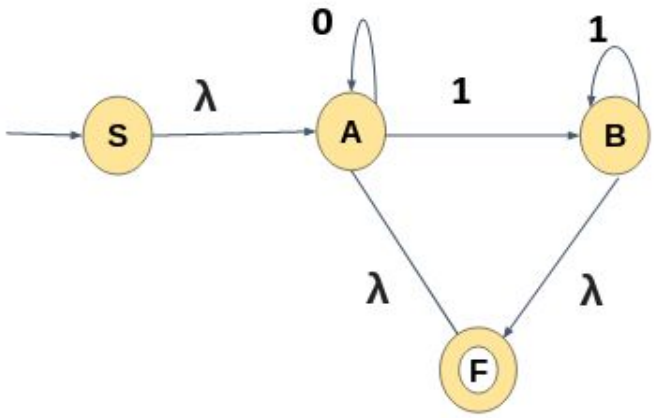
Old final states are made as non final states. once we have a single start state and final state, eliminating the intermediate state.



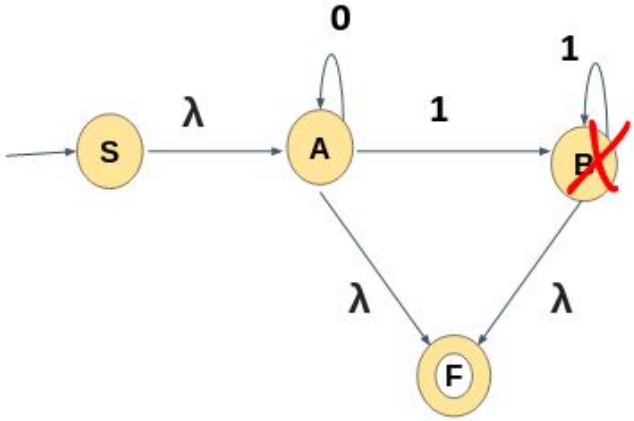
First eliminate state C. Before choosing the order of states to eliminate , it is better to check whether we have any dead /trap state, if we have one eliminate that state first. Here State C is a dead state, so delete state C and its corresponding transition.



The Automata after deleting state C will look like below:

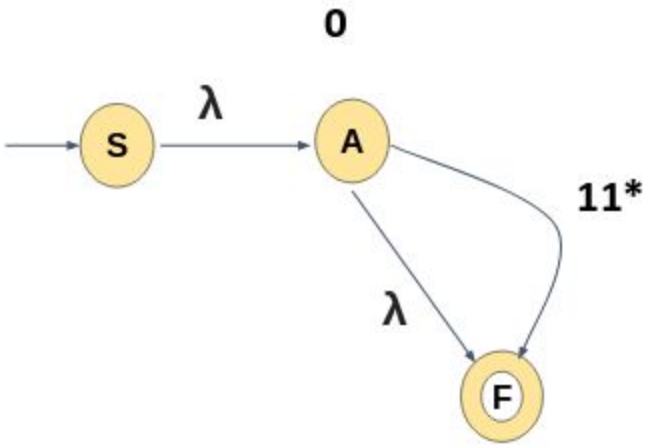


->Next eliminate state B.

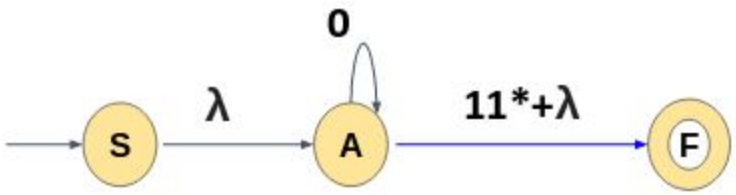


There is a transition from state A to state F through state B, and state B have a self loop. so after deleting state B, there will be a direct path from state A to state F with the cost **11*.**

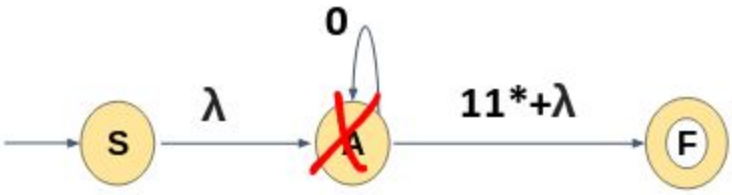
Now, there is two path from state A to state F, one is through λ and the other one is through 11*. we can represent this with a single path with the expression $\boldsymbol{\lambda + 11^*}$



DFA will look below this after the eliminating state B



-> Next will eliminate state A(the only intermediate state)



There is a path form state S to state F through state A and State A have a self loop, so after eliminating state A, we get a direct path from state S to state F wit the cost $\boldsymbol{\lambda}$.

$0^* 11^*. \lambda = 0^* 11^*$

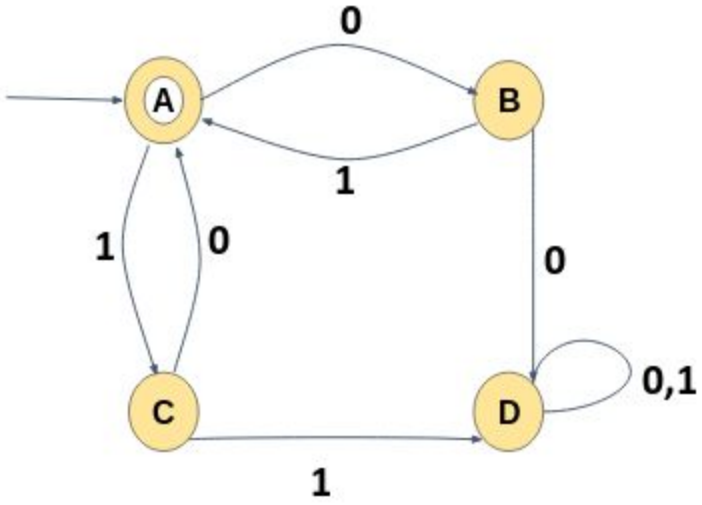
$11^* = 1+$

$(1+\lambda) = 1^*$

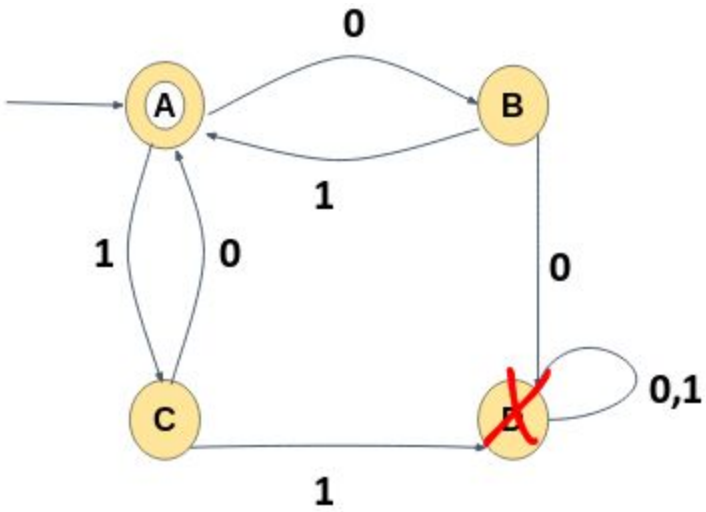
so, $0^*(11^*+\lambda) = 0^*1^*$

so the final regular expression for the given DFA is **0^*1^***

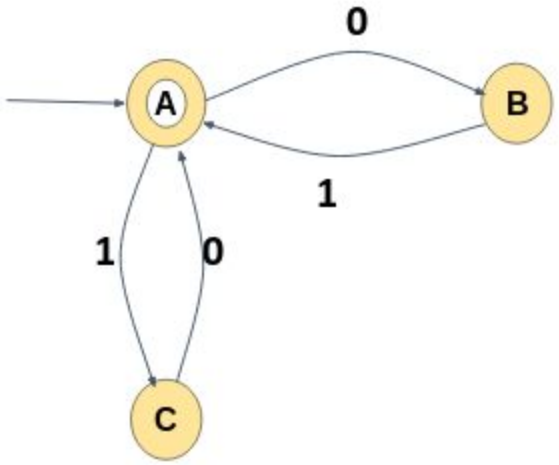
6) Convert the following Finite Automata to Regular Expression



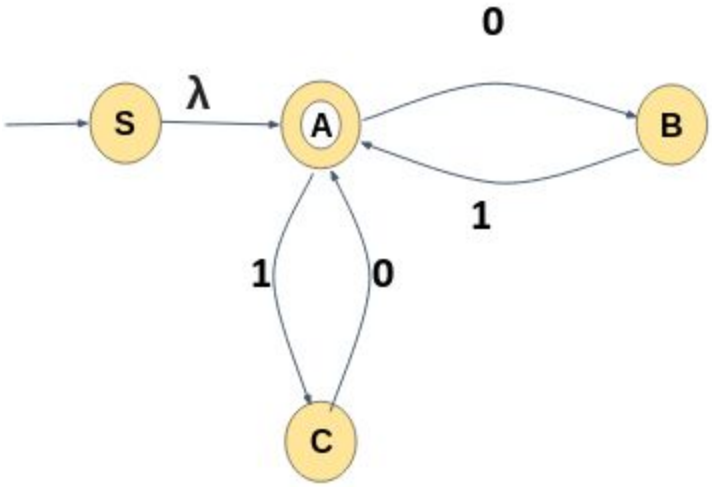
State D is a dead / trap state, so we first eliminate state D and its corresponding transition.



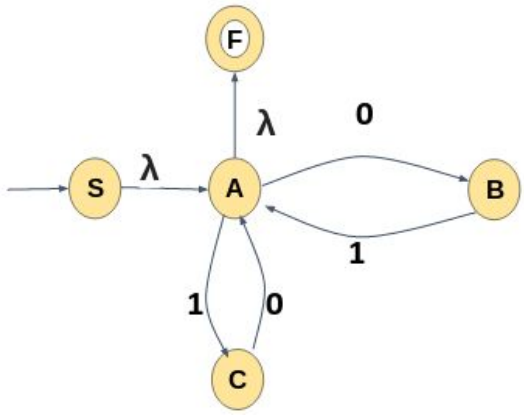
After deleting the dead state the automata will look like below.



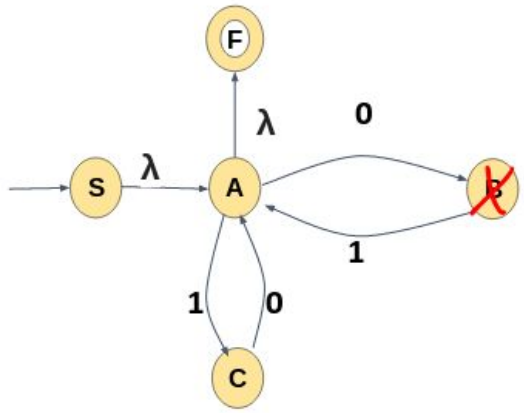
Start state A is having an incoming transition, so we create a new start state with the name S, and Make a transition from new start state S to state A with λ as shown.



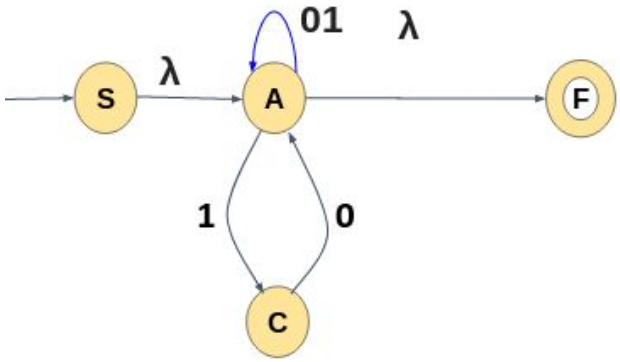
The accepting state have outgoing edge, so create a new final state with the name F and have a transition from old final state A to new final state F with λ. and make the old final state A an non final state



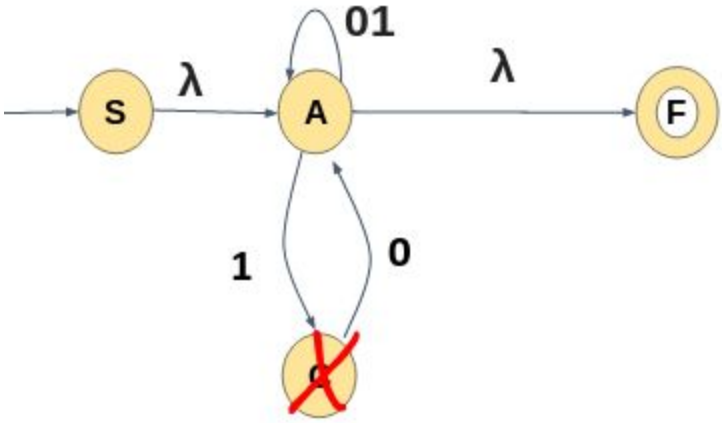
->Next eliminate state B,



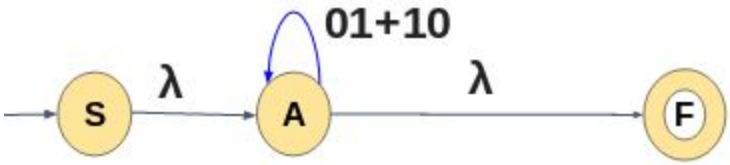
There is a loop on state A to state A through B,
After eliminating state B , State A will have a direct self loop with the cost 01
After deleting state B automata will look the following:



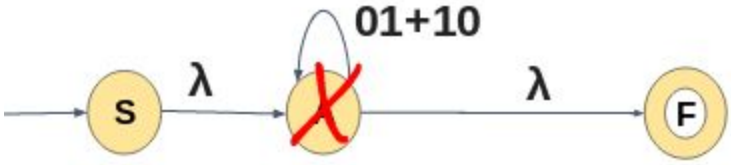
Next delete state C



There is a loop on state A through state C, and State A also have a self loop
so after eliminating state C there will be a direct self loop on state A having the cost **10+01**
After deleting state C automata will look the following:



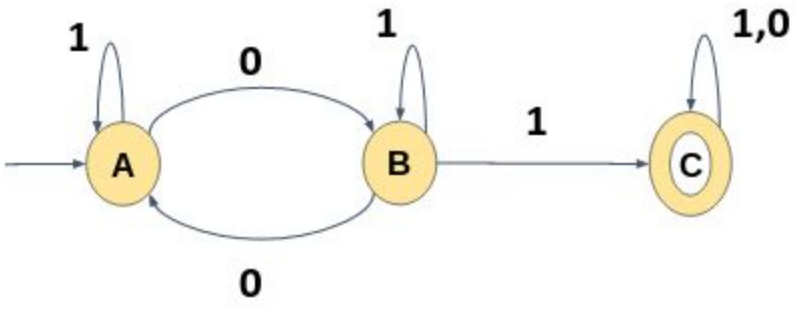
-> Next will eliminate state A(the only intermediate state)



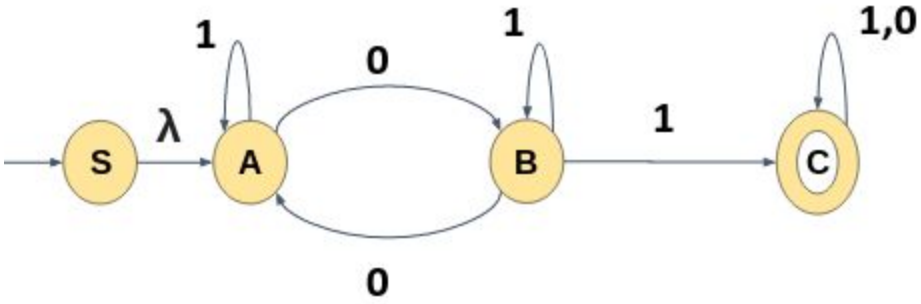
There is a path form state S to state F through state A and State A have a self loop, so after eliminating state A, we get a direct path from state S to state F wit the cost **(01+10)***

So the final expression after eliminating all the intermediate states is: **(01+10)***

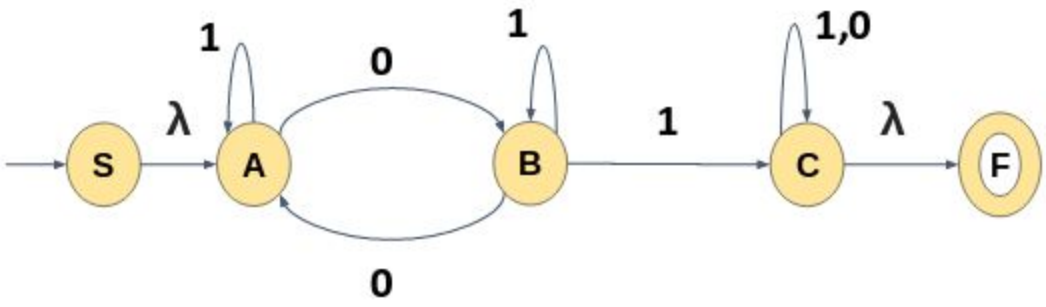
7) Convert the following FA to RE



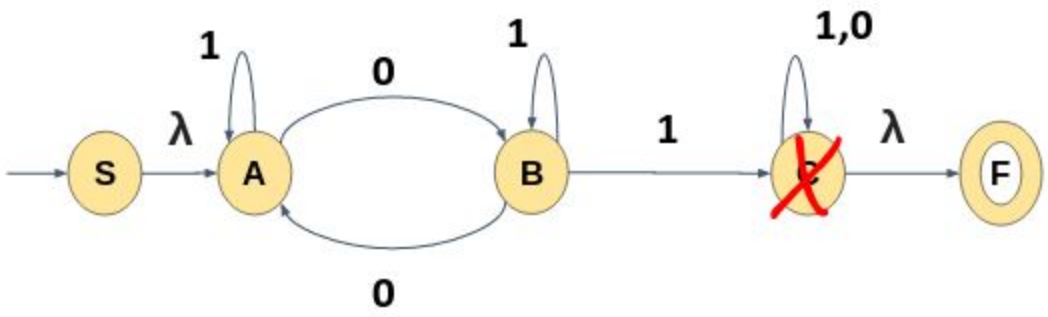
There are no dead / trap states,
Start state A is having an incoming transition, so we create a new start state with the name S,Make a transition form new start state S to state A with λ as shown



The accepting state have outgoing edge, so create a new final state with the name F and have a transition from old final state C to new final state F with λ . and make the old final state C as a non final state.

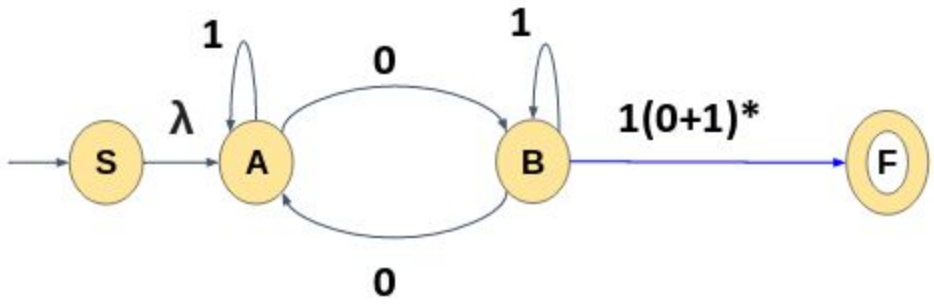


-> Eliminate state C

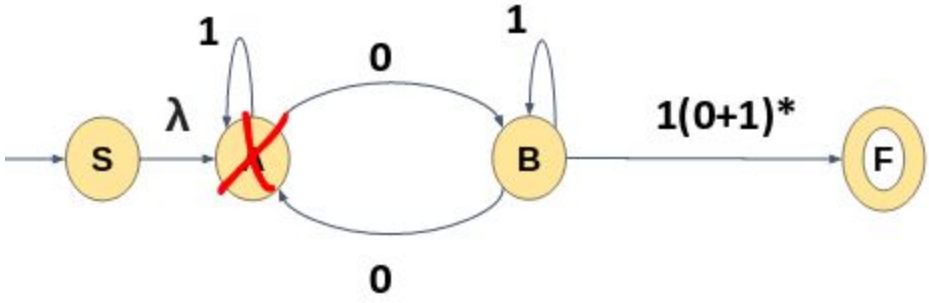


there exists a path from state A to state F through state S and state C have a self loop, So after deleting state C we will have a direct path from state B to state F with the cost $\mathbf{1 . (0+1)^* . \lambda = 1(0+1)^*}$

After eliminating state C the Automata look like the following

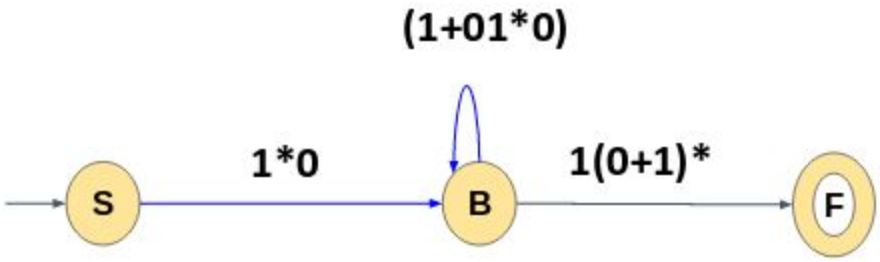


Next eliminate state A

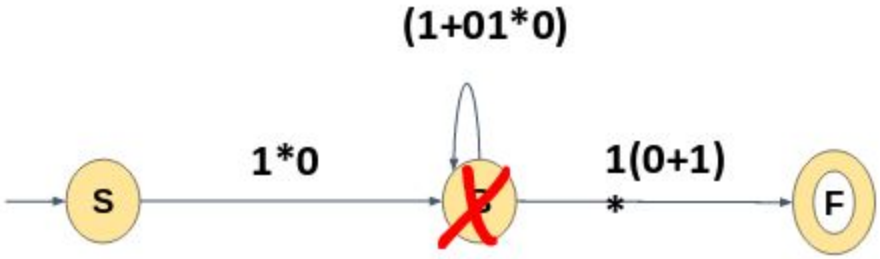


There exists a path from state S to state B through state A andstate A have a self loop So after deleting state A will have a direct path from state A to state B with the cost $\mathbf{\lambda . 1^* 0 = 1^*0}$

State B we have a loop on state B through A, B also have a self loop so,
After eliminating state A state B have a self loop with the cost **(1+01*0)**

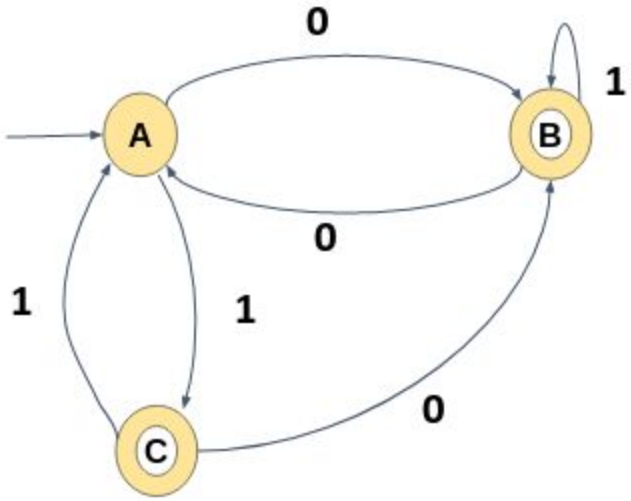


-> Next will eliminate state B(the only intermediate state)

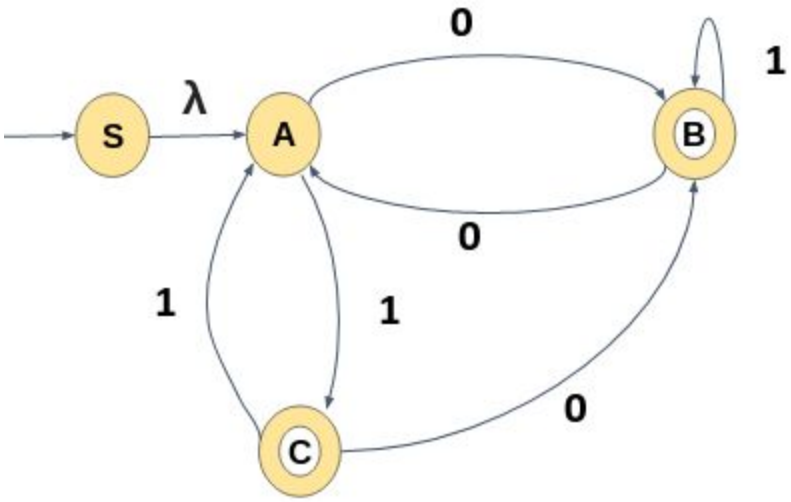


There is a path form state S to state F through state B and State B have a self loop, so after eliminating state B, we get a direct path from state S to state F wit the cost **1*0.(1+01*0)*1(0+1)**
So the final expression after eliminating all the intermediate states :
1*0(1+01*0)*1(0+1)*

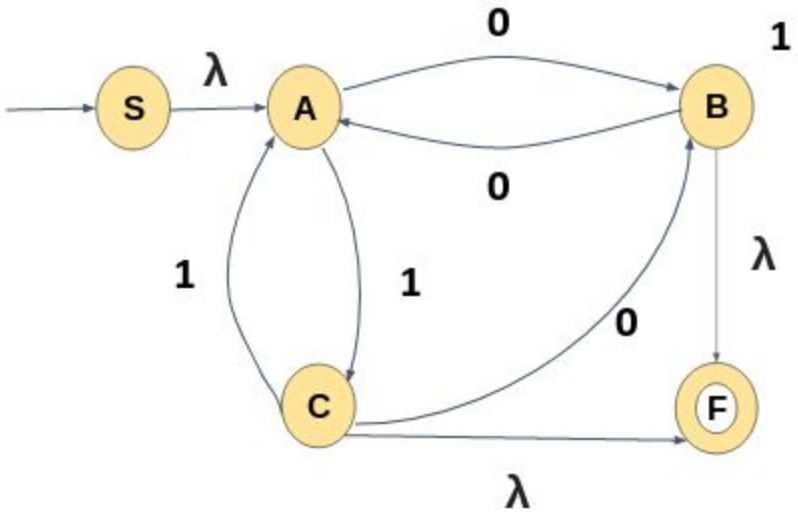
8) Convert the following FA to RE



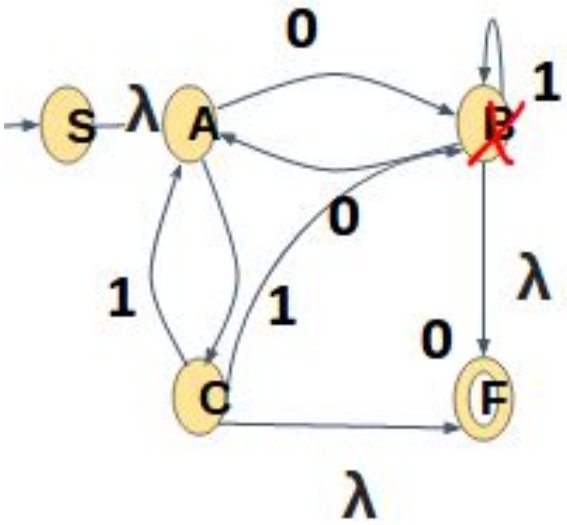
Start state A is having an incoming transition, so we create a new start state with the name S,Make a transition form new start state S to state A with λ as shown.



There are two accepting states and it has outgoing edge, so create a new final state with the name F and have a transition from old final state A and B to new final state F with λ . and make the old final state C and B as a non final state



Eliminate state B,

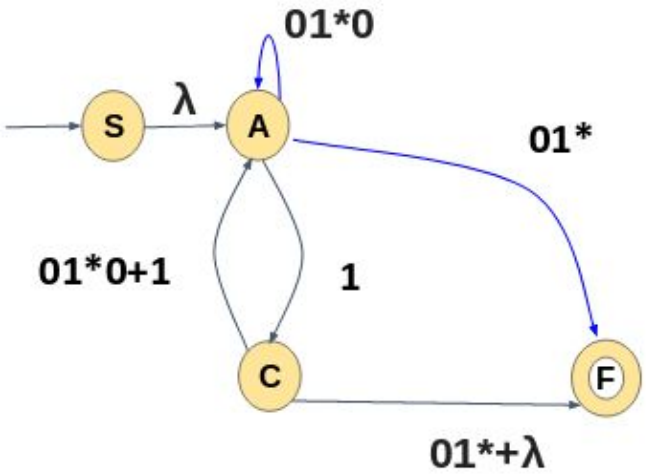


-> there is a loop on state A through B, so after deleting state B we will have a direct self loop on B with the cost **01*0**

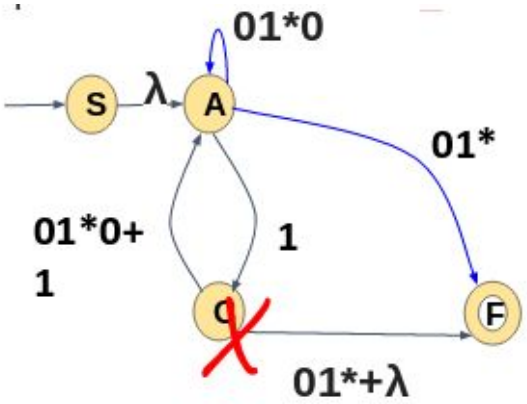
-> There is a direct path from state C to state A and indirect path to state A through state C and state B. so after eliminating B there will be a direct path from state C to state A with the cost **1+01*0 .**

-> there is a path from state A to state F through B , so after deleting state B there will be a direct path from state A to F with the case **01*. λ = 01***

After eliminating state B , Automata looks like the following



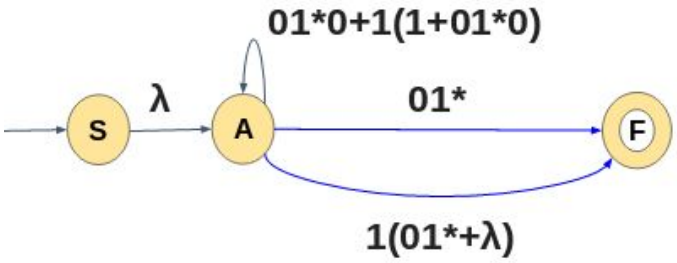
Next eliminate state C,



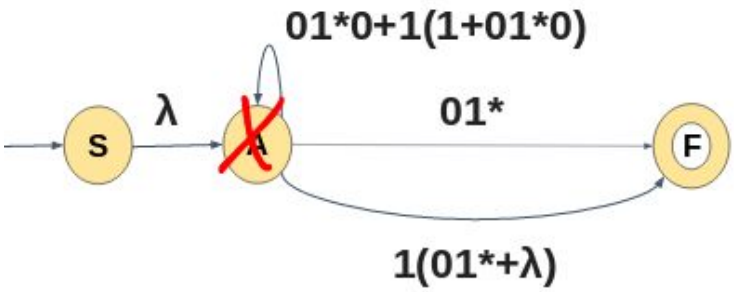
->There is a loop on state A through state C and A have a self loop, so after deleting state C we will have a direct self loop on A with the cost **01*0+1(1+01*0)**

-> There is a path from state A to state F through C, so after deleting state C there will be a direct path for state A to state F having the cost **1(01*+ λ)**

After deleting state C the automta looks like:



Next will eliminate state A(the only intermediate state)



There is a path form state S to state F through state A and State A have a self loop and have two different path from state A to state F , so after eliminating state A, we get a direct path from state S to state F with the cost **(01*0+1(1+01*0))* (01*+ 1(01*+ λ))**

The final regular expression for the given automata is:

RE=(01*0+1(1+01*0))*(01*+1(01*+ λ))

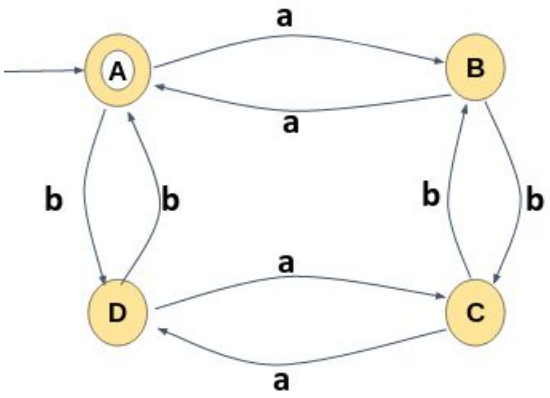
=(01*0+11+101*0)* (01*+(101*+1))

=(101*0+01*0+11)* (01*(λ +1) +1)

=(01*0(1+ λ) + 11)* (01*(λ +1) +1)

RE = (01*0(1+ λ))* (11)* (01*(λ +1) +1)

9) Convert the following FA to RE

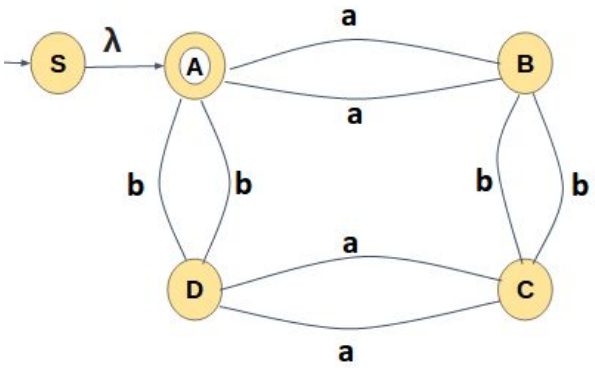


10)

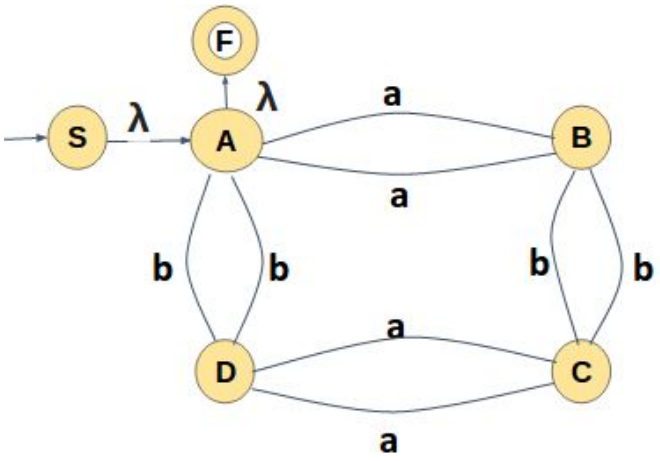
order of state elimination(B,D,C,A)

There are no dead / trap states,

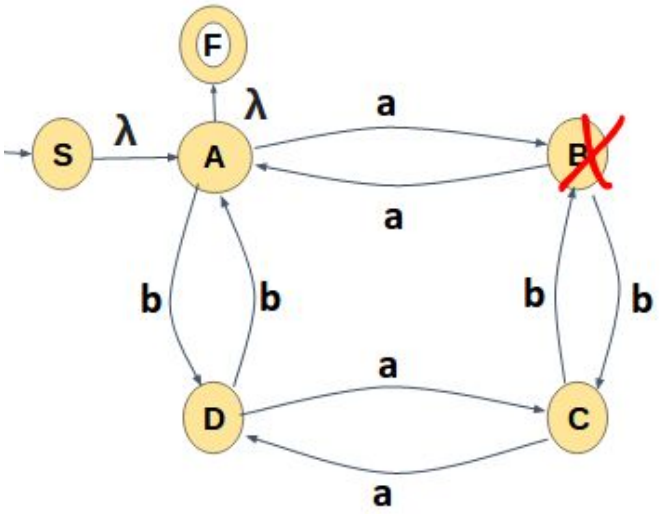
Start state A is having an incoming transition, so we create a new start state with the name S,Make a transition form new start state S to state A with λ as shown



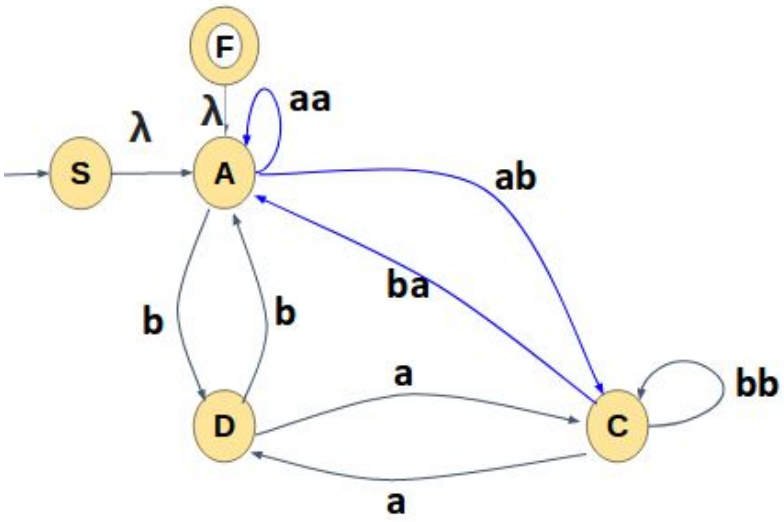
There is an outgoing edgeform accepting state, so create a new final state with the name F and have a transition from old final state A to new final state F with λ . and make the old final state A as a non final state



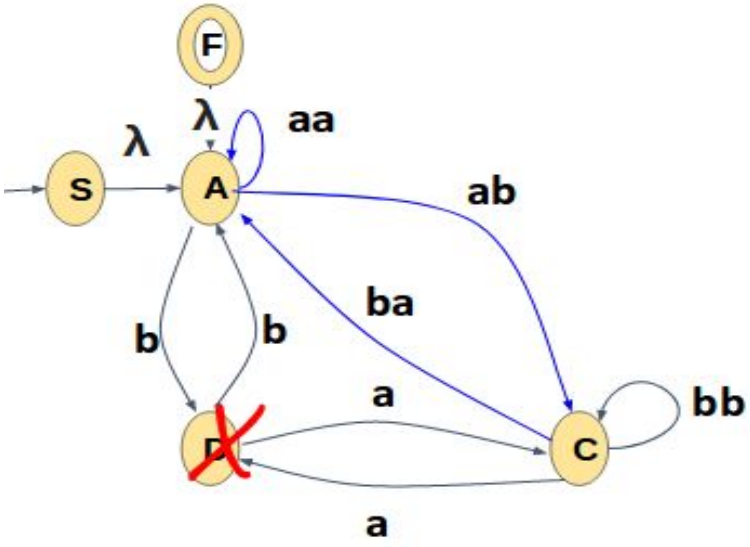
-> Eliminate state B



There is a loop on State a through B, so after deleting state B , State A will gat a direct self loop with the cost aa
There is a path from state A to state C , so after deleting state B there will be a direct path form state A to state C with the cost ab
There is a path from State C to state A through state B, so after eliminating state B there will be direct path from state C to state A with the cost ba
There is a loop on state C through B, so after eliminating state B , state C will have a self loop with the cost bb
After deleting state B the automata will like below:

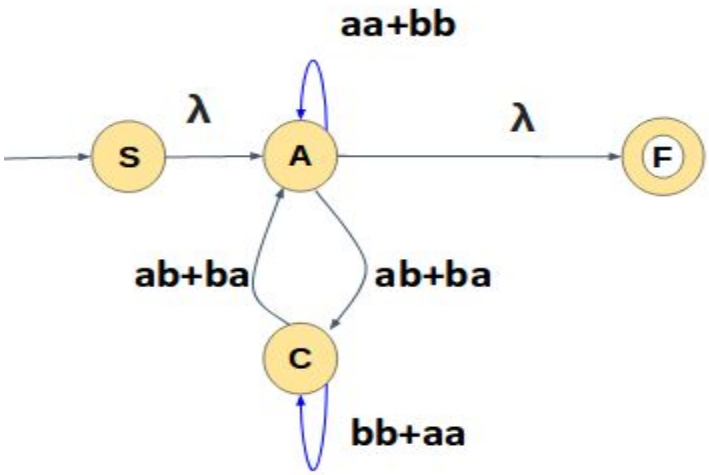


Next eliminate state D:

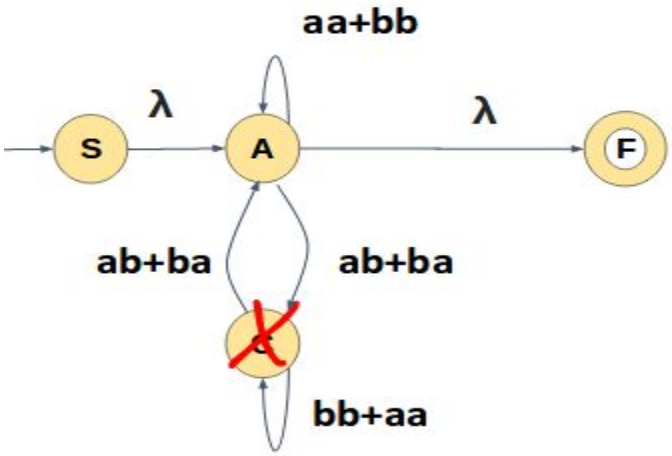


There is a loop on state A through state D and state A have a self loop,
So after deleting state D there will be a self loop on state A with the cost aa+bb
There are two paths from state A to state C, one is direct path with the cost ab and the other one is through state D with the cost ba,
So after deleting state D there will be direct path from state A to state C with the cost ab+ba
There are two paths form state C to state A, one is direct path from with the cost ba, and the other path is through state D, so after eliminating state D there will be one direct path from state C to state A with the cost **ab+ba**

After deleting state D the automata will look like :



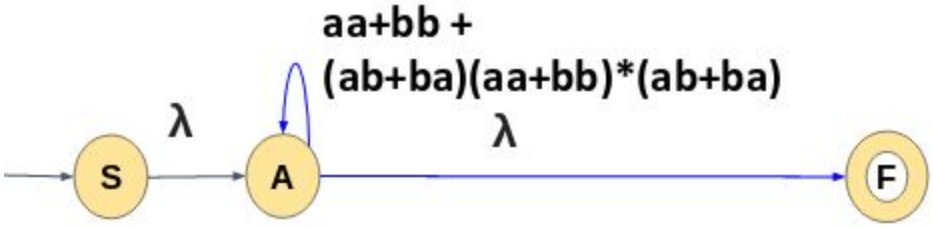
Next eliminate state C:



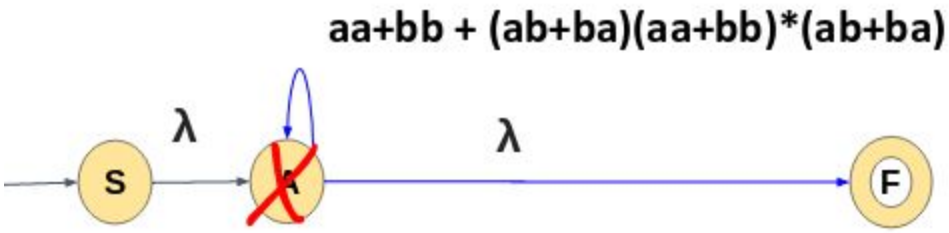
There is a loop on state A through state C abd state C have a self loop, state A also has a self loop. so after deleting state C there will be a self loop on state A with the cost

(aa+bb) + (ab+ba)(aa+aa)*(ab+ba)

After deleting state C the automata will look like:



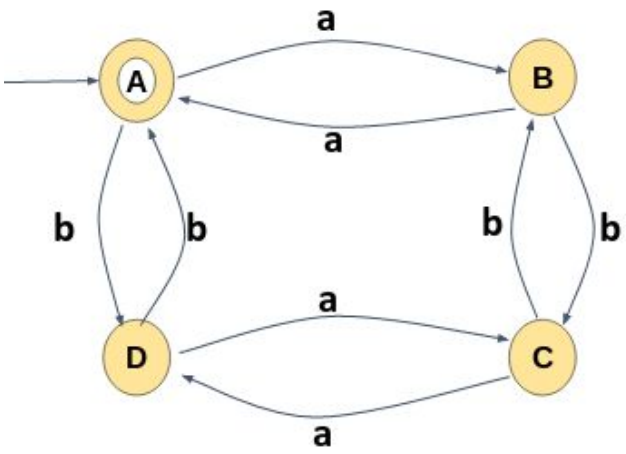
-> Next eliminate state A(the only intermediate state)



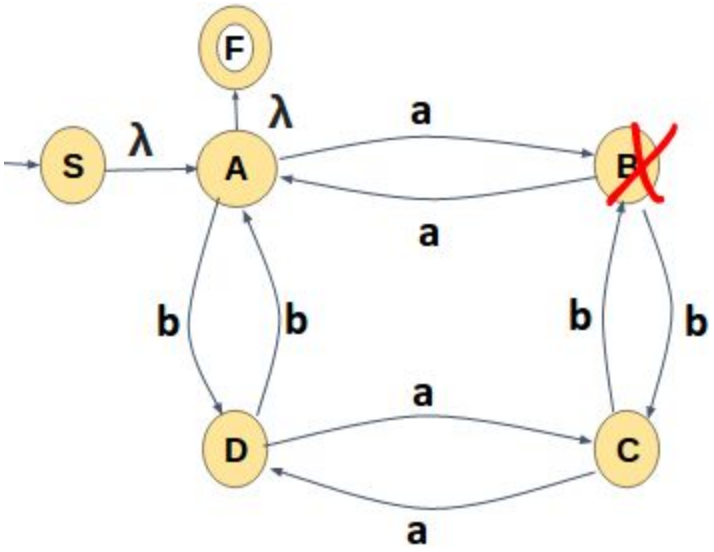
There is a path form state S to state F through state A and State A have a self loop, so after eliminating state A, we get a direct path from state S to state F wit the cost (aa+bb + (ab+ba)(aa+bb)*(ab+ba))*

The final regular expression for the given DFA is:
(aa+bb +(ab+ba)(aa+bb)*(ab+ba))*

-> Consider the same example: Order of state elimination: B,C, D ,A

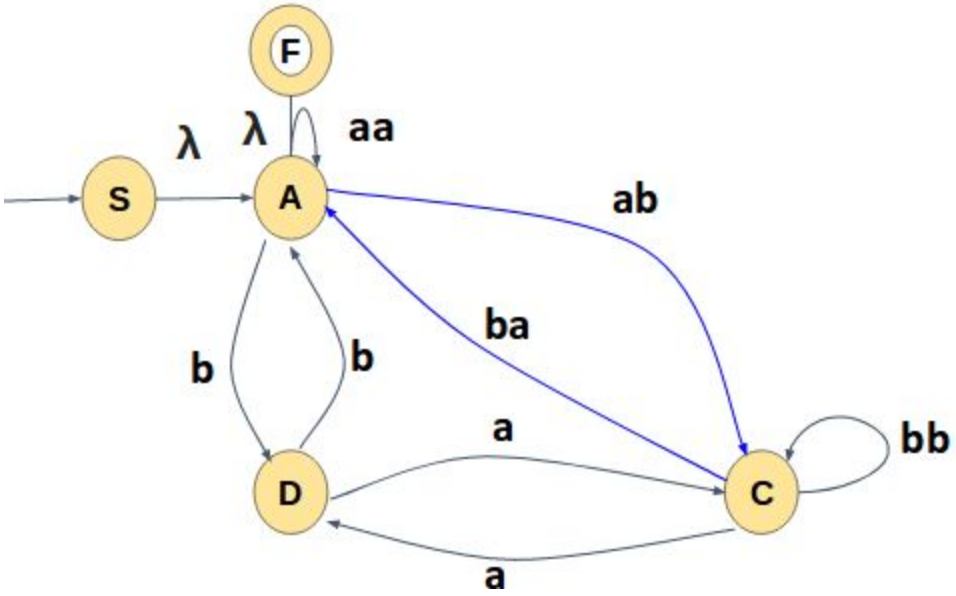


-> Eliminate state B

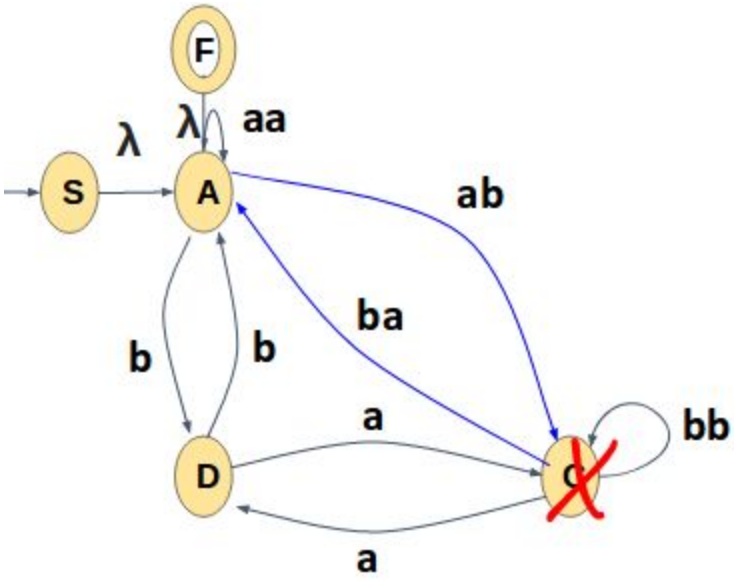


There is a loop on State a through B, so after deleting state B , State A will gat a direct self loop with the cost **aa**
There is a path from state A to state C , so after deleting state B there will be a direct path form state A to state C with the cost **ab**
There is a path from State C to state A through state B, so after eliminating state B there will be direct path from state C to state A with the cost **ba**
There is a loop on state C through B, so after eliminating state B , state C will have a self loop with the cost **bb**

After deleting state B the automata will like below:



Next eliminate state C:

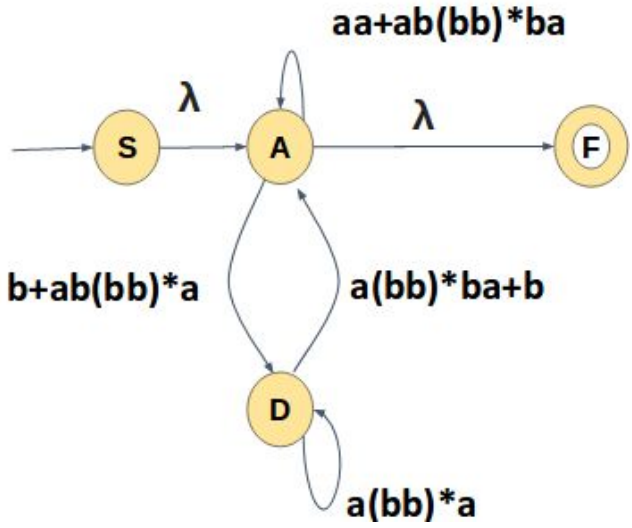


There is a loop on state A through state C , State C have a self loop, and state A also have a self loop,so after deleting state C we will have a direct self loop on state A with the cost **aa+ab(bb)*ba**

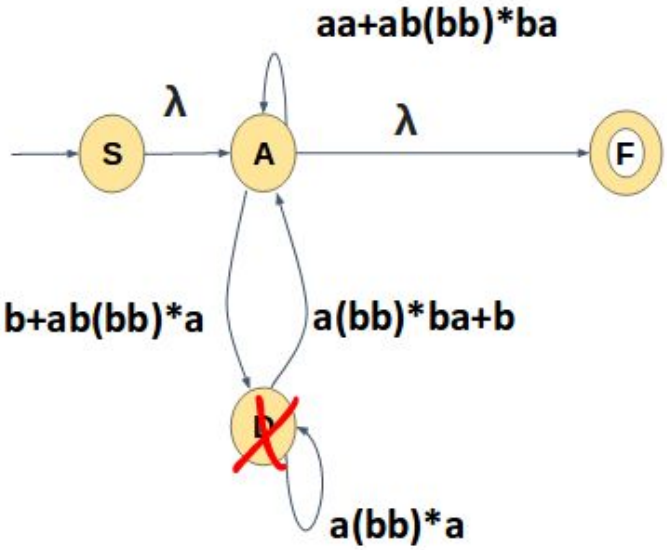
There are two paths from state D to state A, one is direct path the other one is through state C, so after deleting state C we will one direct path with the cost **b + a(bb)*ba**

There are two path from state A to state D, one is direct path and the other one is indirect through state C, So after eliminated state C there will one direct path from state A to state D with the cost **b + ab(bb)*a**

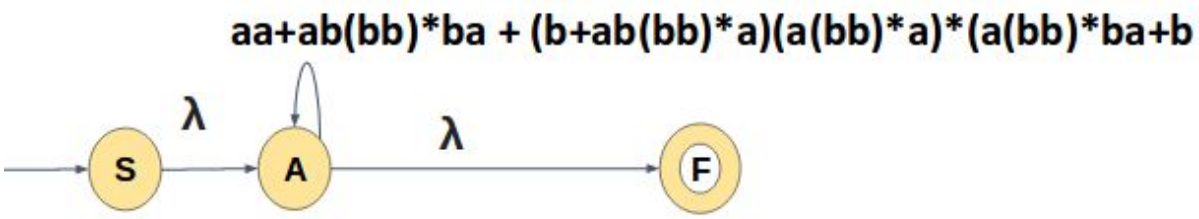
After deleting state C the automata will look like:



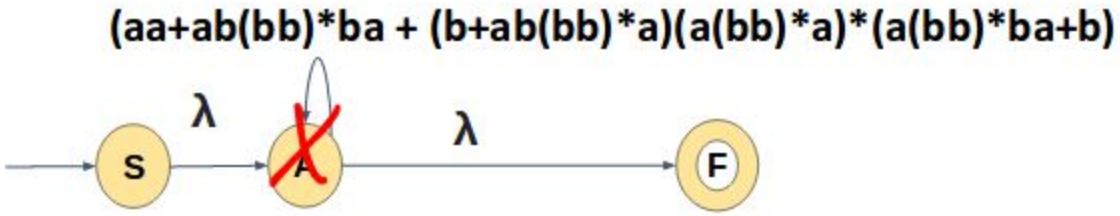
Next eliminate state D,
There is a loop on state A through state D, and state A and D have a self loop, so after deleting state D, state A will have a self loop with the cost **(b+ab(bb)*a).(a(bb)*a).(a(bb)*ba+b) + aa+ab(bb)*ba**



After deleting state D the automata will look like:



> Next will eliminate state A(the only intermediate state)

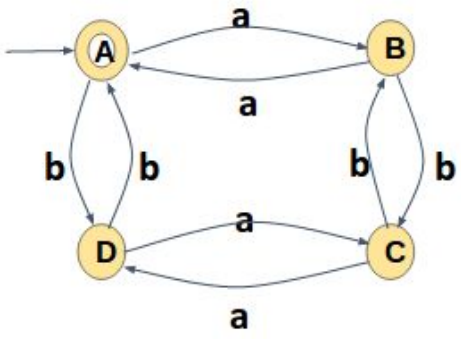


There is a path form state S to state F through state A and State A have a self loop, so after eliminating state A, we get a direct path from state S to state F wit the cost **(aa+ab(bb)*ba + (b+ab(bb)*a)(a(bb)*a)*(a(bb)*ba+b))***

Final regular expression for the given automata is :
((aa+ab(bb)*ba + (b+ab(bb)*a)(a(bb)*a)*(a(bb)*ba+b))*

We get two different regular expression for the same automata based on the order of eliminating states in the automata:
Note: We can eliminate the sate in any order and the regular expression may be different but the language accepted by the automata/ regular expression will be same

Example 9 :



(aa+bb + (ab+ba+(aa+bb)*(ab+ba))*

Eliminate : B,D,C,A

((aa+ab(bb)*ba)* +
(b+ab(bb)*a)(a(bb)*a)*(a(bb)*ba+b))*

Eliminate B,C,D,A

AUTOMATA FORMAL LANGUAGES AND LOGIC

Lecture notes on Equivalence of Regular expressions

Prepared by

Ms. Kavitha K N
Assistant Professor

Ms. Preet Kanwal
Associate Professor

Department of Computer Science & Engineering
PES UNIVERSITY
(Established under Karnataka Act No.16 of 2013)
100-ft Ring Road, BSK III Stage, Bangalore – 560 085

Table of Contents:

Examples Solved:

1. Equivalence of two regular expression

Two regular expressions(R1and R2)are equivalent(R1= R2) iff:

L(R1) = L(R2)

2. Methods to determine the equivalence of Regular Expressions:

We can determine the equivalence using two methods

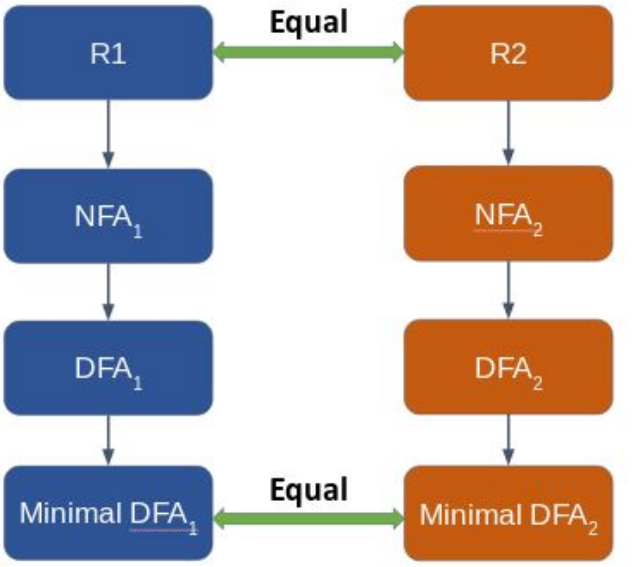
1) A Formal method

The classical approach to the problem of comparing two regular expressions α and β , i.e.,deciding ifL(R1) =L(R2), typically consists of:

1. obtain non-deterministic finite automata,N(R1) and N(R2), which accept the same language as R1 and R2, respectively using Thompson construction method.

2. convert the non-deterministic automata to equivalent deterministic ones,D(R1)≡N(R1)andD(R1)≡N(R2) using subset construction method

3. minimise both D(R1) and D(R2).Because, for a given regular language, the minimal automaton is unique

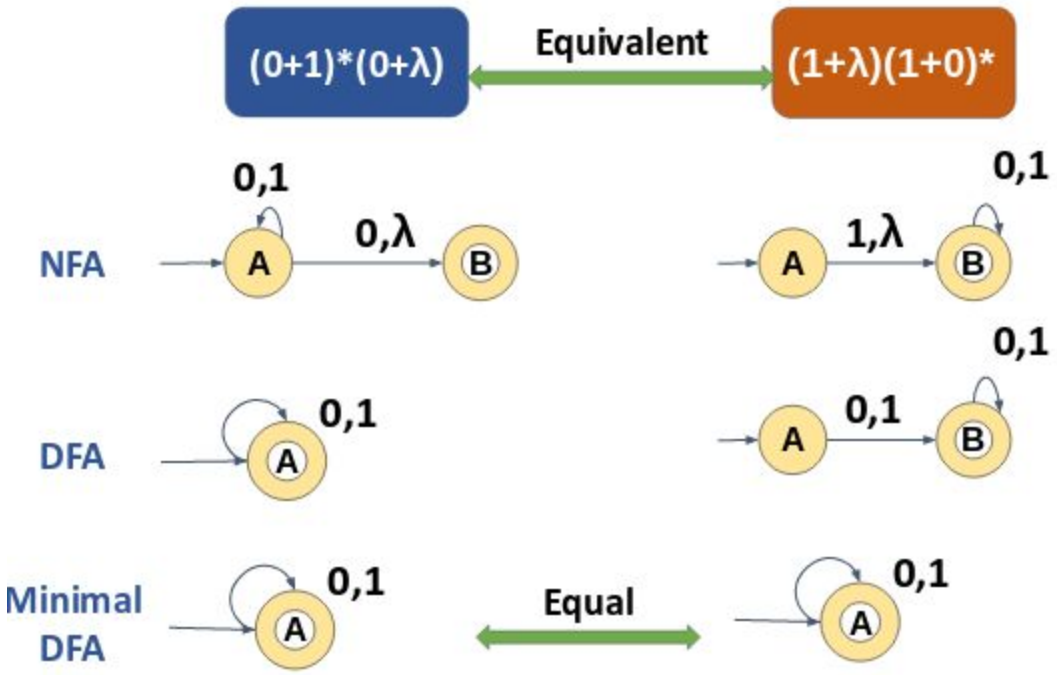


2) An Informal method

In Informal method , the aim is not to prove that the two regular expressions are equal. Instead we focus on proving that they are not equivalent.To show that two regular expressions are not equivalent we have to find a word that belongs to one expression and does not belong to the other.

3. Examples

1) Check the Equivalence of two regular expression using formal method
   R1 = (0+1)*(0+ λ) and R2 = (1+ λ)(1+0)*



Since both the minimal DFAs are equal We conclude that the given regex are equivalent.

2) Check the Equivalence of two regular expression using informal method

R1 = (0+ λ)(11*0)*(1+ λ) and R2 = (1+ λ)(011*)(0+ λ)

Consider the String 011
011 $\in$ R2
011 $\notin$ R1
So,
R1 $\neq$ R2

3) Check the Equivalence of two regular expression using informal method
R1 = (1+ λ) (00*1)* 0* and R2 = (0+ λ) (11*0)1*

Consider the String 00
00 $\in$ R1
00 $\notin$ R1
So,
R1 $\neq$ R2

4) Check the Equivalence of two regular expression using informal method
R1 = 0*(10*)* and R2 = (1*0)*1*

R1 can generate all strings generated by R2 and vice versa , So they are equivalent.
Example string :
b, aba, bb, ...

AUTOMATA FORMAL LANGUAGES AND LOGIC

Lecture notes on Regular grammar and Parsing

Prepared by:

Prof.Sangeeta V I

Assistant Professor

Department of Computer Science & Engineering

PES UNIVERSITY

(Established under Karnataka Act No.16 of 2013)

100-ft Ring Road, BSK III Stage, Bangalore – 560 085

Table of contents

Examples Solved:

Examples Solved:

Examples Solved:

Examples Solved:

1. Introduction to Grammar

Language is a set of strings constructed over an alphabet which satisfies certain properties or follows a certain set of rules. These rules can be encoded using the concept of Grammar. Grammars are used to compactly express and generate languages.

Grammars denote syntactic rules that means it is concerned only with the syntax of a string and not the meaning .

Grammar is another way to represent a set of strings. Rather than define the set through a notion of which strings are accepted or rejected, like we do with a machine model, we define the set by describing a collection of strings that the grammar can generate.

For example:She is a boy, is grammatically correct but semantically wrong.

There are various applications of regular grammars for example in compiler design the concept of grammar is used construct syntax of a programming language

Formal grammar or just grammar is a set of rules that describe how strings that are valid according to the language's syntax. can be generated from the language's alphabet.

2. Grammar definition

A grammar is a 4-tuple **G = (V,T,P,S)**, where

- **V** is a set of variables/non-terminals. We can have
  a. Lowercase names such as expr,var,op,stmt. or
  b. Capital letter near the beginning of the alphabet. A,B,C,D or Uppercase letters near the end of alphabet X,Y,Z. as non-terminals
  c. Usually Variable S is used as the-start symbol.
  d. Lowercase letters near the end of the alphabet. U,v,w,x,y,z are used to represent the entire string.
- **T** is the set terminal symbols
  a. These are basically the symbols from the input alphabet for example could be a Keywords such as :
  b. If ,else,then ,for ,while,do while etc
  c. Or Digits 0 t0 9
  d. Or Lowercase alphabets such as a,b,c,d etc
  e. Or Bold faced letters
  f. Or Symbols such as + , - , * , / , : , ; etc

  We can use Greek letters-α,β,γ to denote a grammar symbol which could either be a either terminal or a non-terminal.
- **P** is a finite set of production rules (also called simply rules or productions) of form $\alpha \rightarrow \beta$ where α is always a non terminal and cannot be λ. And β is a string over $(V \cup T)^*$
- **S** is the start symbol.

- The set of strings that can be generated or derived from Start symbol S of a grammar is called Language of the Grammar denoted by L(G).

3. Sentence and Sentential form

Sentence or the word string are used alternatively and are made up only of terminals.

Example :

For the Grammar S->aS | a |λ

Sentence (or string) is { a or λ or aa or aaaa)

A **sentential form** is made up over set of terminals and non-terminals (VUT)*

Example:

For the Grammar S->aS | a |λ

Sentential form is { a or λ or aS or aa or or aaS aaaa)

4. Chomsky Hierarchy

Noam Chomsky in 1956 described Chomsky hierarchy of grammars

Chomsky Hierarchy represents the class of languages that are accepted by the different machines. Chomsky classified grammars on the basis of the form of their productions. This is a hierarchy. Therefore every language of type 3 is also of type 2, 1 and 0. Similarly, every language of type 2 is also of type 1 and type 0, etc

Type 3 Grammar is known as Regular Grammar.

5. Linear and Non-Linear grammar

We can broadly categorize grammars into two categories as being linear and non-linear

Linear grammar is the one which has at most one non terminal in the RHS of the production.

For Examples S -> aSa | aS | Sa | ϵ

here we have only one nonterminal/variable in the right hand side of each of its productions.

In non linear grammar there is no restriction on the number non terminals that can appear on the RHS of a production for example : S -> SS | aSS | a | ϵ

Here we have two non terminals in the RHS .

A regular grammar has more restriction and is a subset of linear grammar.

5.1 Right Linear grammar and Left Linear Grammar

A regular grammar can either be right linear or left linear.

A grammar is **left linear** iff the non-terminal on RHS of a production appears on the leftmost side. That means the non-terminal is the first grammar symbol on the RHS of a production

For example

S -> Sa | ϵ (left linear)

A grammar is **right linear** iff the non-terminal on RHS of a production appears on the rightmost end. That means the non-terminal is the last grammar symbol on the RHS of a production

For example

S -> aS | ϵ (right linear)

Since both the forms are linear, we can have only one non terminal on RHS of a production.

The right- and left-linear grammars are equivalent. That means for a given language we can have both right and left linear grammar constructed. We could also convert one form to another.

6. Constructing regular grammar for given language descriptions.

Example 1:

Construct a regular grammar for the language L = {a}.

Language	Regular Expression	Regular Grammar
L = {a}	a	S⟶a

We start the regular grammar with a start symbol "S".

We can convert regular expressions to regular grammar as the regular expressions are a form of representing regular languages.

Here ,since 'a' is the only symbol in the language ,the regular grammar from S generates only 'a', S->a (production rule)

Example 2:

Construct a regular grammar for the language L = {a, b}.

Language	Regular Expression	Regular Grammar
L = {a, b}	a+b	S⟶a | b

Here ,since 'a' or 'b' is a symbol in the language ,the regular grammar from S generates either 'a' or 'b' , S->a or S->b.

Example 3:

.**Construct a regular grammar for the language L = {ab}.**

Language	Regular Expression	Regular Grammar
L = {ab}	a.b	S→aA A→b

Here ,since 'a.b' is symbol in the language ,the regular grammar produces S->ab

Example 4:.

Construct a regular grammar for the language L = {ab,ba}.

Language	Regular Expression	Regular Grammar
L = {ab}	a.b	S→ab|ba

Example 5:

Construct a regular grammar for the language L={a^n\|n>=0}

Language	Regular Expression	Regular Grammar
L = {**λ**,a,aa,aaa,...}	a*	S→aS|**λ**

The language with any number of a's including empty string .

The regular grammar from S generates any number of a's with the production S->aS or empty string with production S-> **λ**

We combine the productions,

S-aS| **λ**

Example 6:

Construct a regular grammar for the language L={a^n\|n>=1}

Language	Regular Expression	Regular Grammar
L = {a,aa,aaa,...}	a^+	S→aS|a

The language with any number of a's but not an empty string ,minimum string is one 'a'.

The regular grammar from S generates any number of a's with the production S→ aS or single 'a' with production S→ a

We combine the productions,

S→ aS| a

Example 7:

Construct a regular grammar for the regular expression (a+b)*

Language	Regular Expression	Regular Grammar
L = {**λ**,a,b,abb,baaaab,.....} L={any number of 'as and b's}	(a+b)*	S→aS|bS|**λ**

The language with any number of a's and any number of b's including an empty string.

The regular grammar from S generates any number of a's with the production S→ aS and any number of b's with the production S->bS and empty string with production

S→ **λ**.

We combine the productions,

S→aS|bS|**λ**

Example 8 :

Construct a regular grammar for the regular expression (a+b)$^+$

Language	**Regular Expression**	**Regular Grammar**
L = {λ,a,b,abb,baaaab,.....} **L={any number of 'a's and b's}**	**$(a+b)^+$**	**S→aS|bS|a|b**

The language with any number of a's and any number of b's but not an empty string.

The regular grammar from S generates any number of a's with the production S->aS and any number of b's with the production S->bS ,single 'a' with production S-> a and single 'b' with production S-> b.

We combine the productions,

S→aS|bS|a|b

Example 9:

Construct a regular grammar for the regular expression (ab)*

Language	Regular Expression	Regular Grammar
L = {λ,,ab,abababababab,.....} L={sequences of ab's}	(ab)*	S -> abS | λ

The language with sequences of ab's including empty string.

Example 10:

Construct a regular grammar for the regular expression (ab+ba)*

Language	Regular Expression	Regular Grammar
L = {**λ**,,ab,abababababab,.....} L={sequences of ab's}	(ab)*	S -> abS | baS | **λ**

The language with sequences of ab's or ba's including empty string.

Example 11:

Construct a regular grammar for the language $L=\{a^m b^n \mid n,m>=0\}$

Language	Regular Expression	Regular Grammar
L={**λ**,a,b,bb,abb,,.....} L={any number of a's followed by any number of b's}	a*b*	S -> aS|A A->bA |**λ**

aS generates as many a's as we want and when we go ahead in the pattern we introduce a new non-terminal **A** which generates only b's.

Example 12:

Construct a regular grammar for the given language with an even number of a's. L={a^{2n} |n>=0}

Language	Regular Expression	Regular Grammar
L={λ, aa, aaaa, aaaaaa ….	(aa)*	S -> aaS | λ

Example 13:

Construct a regular grammar for the given language with an odd number of a's. L={a^{2n+1} |n>=0}.

Language	Regular Expression	Regular Grammar
L={a, aaa, aaaaa, aaaaaaa ….	(aa)*a	S -> aaS | a

Example 14:

Construct a regular grammar for the given language with a number of a's as multiples of 4. L={a^{4n} |n>=0}

Language	Regular Expression	Regular Grammar
L={λ, aaaa, aaaaaaaa, aaaaaaaaaaaa, ….	(aaaa)*	S -> aaaaS | λ

Example 15:

Convert finite automata to regular grammar for the language to accept at least one 'a' over the alphabet Σ={a,b}

Language	Regular Expression	Regular Grammar
L={a, bb... a, ba any # of a's & b's	b* a (a+b)*	S -> bS | aA A -> aA | bA | λ

7. Aspects of a Grammar

There are two Aspects of a Grammar.

First is the Generative(derivation) aspect that means given a Grammar G describing language we can generate all strings that belong to the language of the Grammar.

Second is theAnalytical(parsing) aspect where given a string w we can make a check whether w belongs to the language of the Grammar or not.

8. Parse tree/ derivation tree

We all know what a tree looks like. It has leaves, branches and a root.

Technically we specify the tree upside down. That is we specify the root first and then leaves where branches will help connect root to each of the leaves.

Parse/Derivation tree is a graphical representation for the derivation of the given production rules .

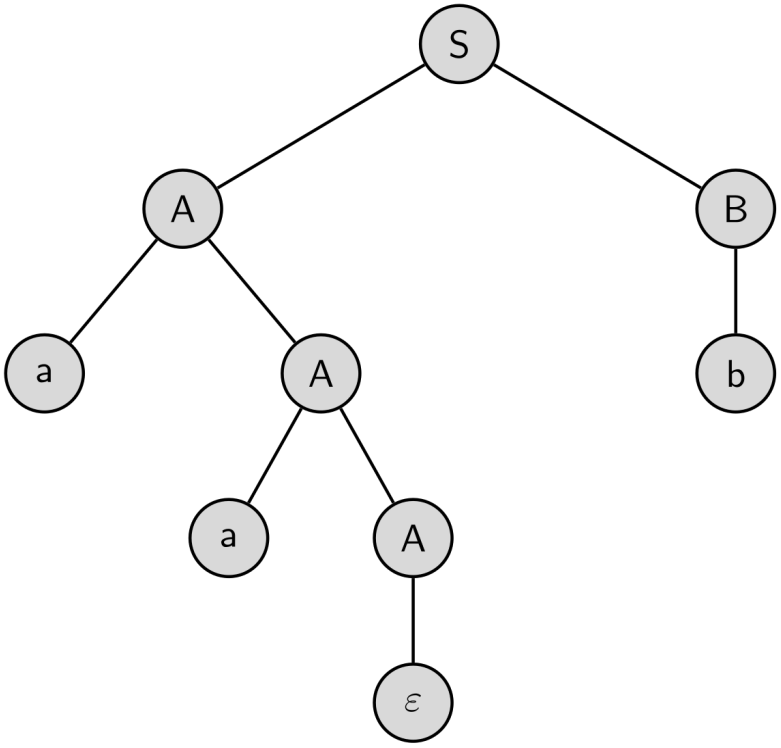
A parse tree/derivation tree contains the following properties:

- The root node is always a node indicating start symbols.
- The interior nodes are always non-terminal.
- The leaf node is always represented by a terminal.
- The derivation is read from left to right. Yield or output of the parse tree is the string derived.

9.1 Tree and its representation

- A tree is nothing but a graph.
- It is made up of a finite set of nodes.
- Nodes are usually denoted by circles or ovals .
- We use the word Edge instead of Branch.
- Edges are the connections between the nodes. It connects two nodes.
- Edges are usually represented by lines, or lines with arrows.
- Tree is a graph without cycles. That is When following the graph from node to node, we will never visit the same node twice.
- From the root node we can reach every other node. So, tree is completely connected.
- Hence we can say a tree is a connected acyclic graph.

Representation of a tree using data



S is the root note .

V is the set of non -terminals which are the interior nodes.

T is the set of terminal symbols ,which are the leaf nodes.

Yield of the parse tree is abb.

9.2 Parsing

Parsing is the process of determining whether a String w belongs to the language of the Grammar or not. Stated another way, we can say whether the String w can be derived from the Grammar or not.

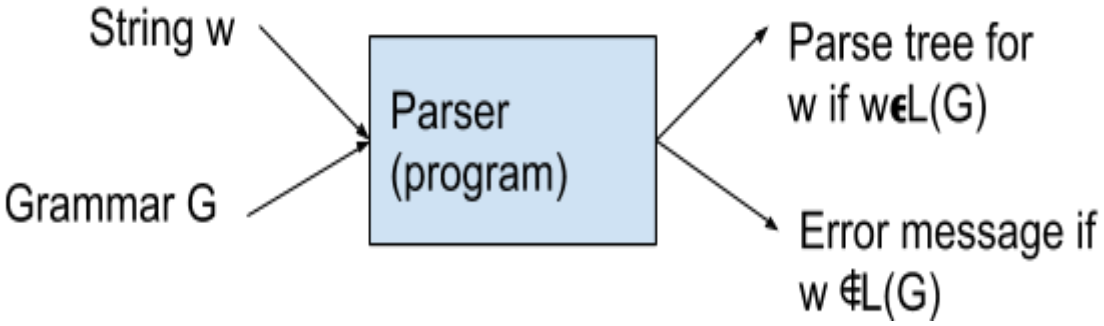
There are two ways in which parsing can be performed:

a) Top Down Parsing
b) Bottom up Parsing

9.2.1 Top-down Parsing and bottom up Parsing

Top-down Parsing is a parsing technique that first looks at the highest level of the parse tree which is the Start Symbol and works down the parse tree by using the rules of grammar while Bottom-up Parsing is a parsing technique that first looks at the lowest level of the parse tree that means the string w and works up the parse tree till the Start symbol by using the rules of grammar

Parser is a program which takes as input the string w and grammar g and produces as output parse tree for w if w belongs to lang of the grammar otherwise outputs an error message.



9.2.2 Generating a parse tree for a given String w and

Grammar G.

Example 1:

Generate parse tree for the sting w=1010 given the grammar .

S→ 0S|1S|0

Derivation: w=1010

S⇒ 1S (using S→ 1S)

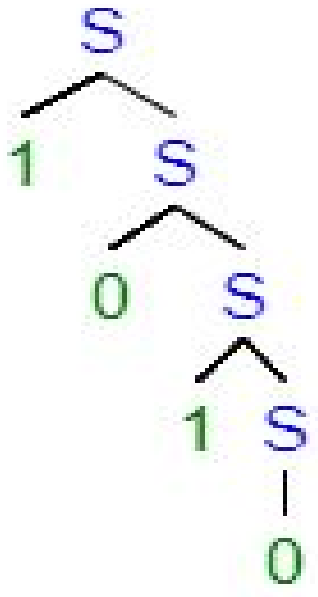
⇒ 10S (using S→ 0S)

⇒101S (using S→ 1S)

⇒ 1010 (using S→ 0)

Sentence or string =1010

Parse Tree: w=1010



Yield of the tree=1010

Example 2:

Generate parse tree for the sting w=aaabbbb given the grammar .

S→aaB

B→ aB|bbbE

E→ bE |λ

Derivation: w=aaabbbb

S⇒ aaB (using S→ aaB)

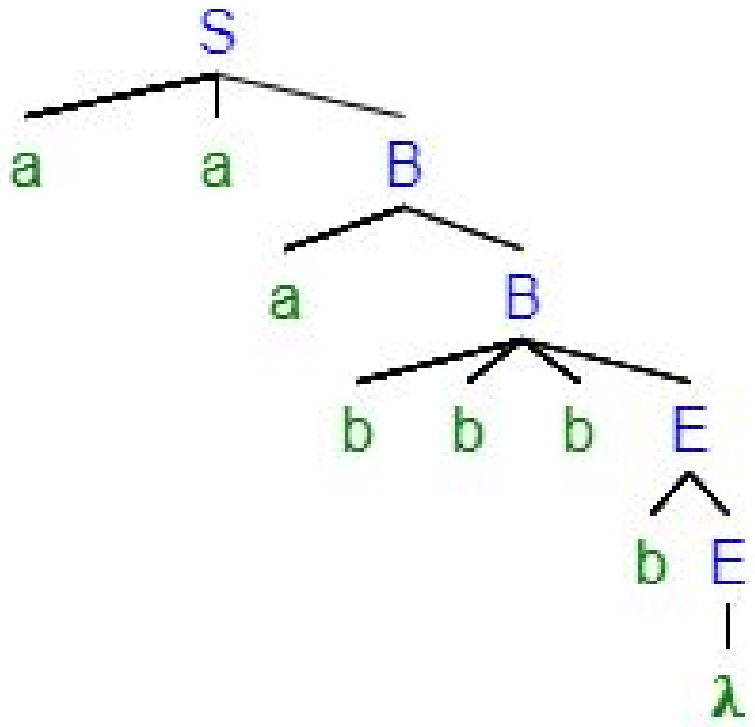
⇒ aaaB (using S→ aB)

⇒ aaabbbE (using B→ bbbE)

⇒ aaabbbbE (using E→ bE)

⇒ aaabbbb (using E→ λ)

Parse Tree: w=aaabbbb



Yield of the tree:aaabbbb

9.3 Constructing a left linear Grammar

There is another form in which regular grammars could possibly be written and it is known as the Left linear form. That means the non-terminal on RHS of a production rule must be present at the leftmost side or should be the first symbol on RHS of the production rule.

Let us consider the right linear grammar for the lang where all the strings must start with an 'a'.

Right Linear Grammar

$A \rightarrow aB$

$B \rightarrow aB|bB|\lambda$

L={starting with 'a'}

You might think that reversing all the symbols on RHS of every production rule will get us an equivalent Left linear grammar for the language L.

But surprisingly, the language represented by this grammar is the reverse of the language represented by the Right linear form.

When we reverse all the symbols in RHS of every production rule what we get is a language where every string ends with an a.

So, the conversion or construction of LLG is not as straightforward. We need to work our way via finite automata.

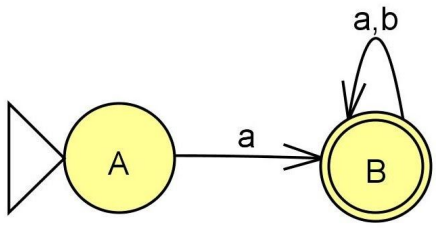
9.4 Constructing a left linear grammar from the finite automata.

- Step 1: Consider the Finite automata for language "L".
- Step 2: Rever the finite automata L .
    - Steps to reverse:
        - Change initial to final state.
        - Final state to initial state.
        - Reverse the directions of the transitions.
        - We get L^R.
- Step 3: Construct a right linear grammar for this language which is reverse of L L^R.
- Step 4: Now Reverse the symbols in RHS of every production rule.
    - Now with this step we have got a left linear grammar for a language which is a reverse of the reversed language, thereby constructing a left linear grammar for L.

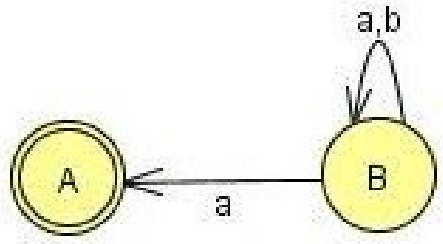
Example 1:

Constructing a left linear grammar for L={aw,w∈{a,b}*}.

Step 1: Finite automata for the language where the strings must start with an a,L={aw,w∈{a,b}*}



Step 2: Reversing directions in this automata represents automata for the reverse of the language L which will be a set of strings ending with an a.



Here, B is the start state.

Step 3: We will now generate the right linear grammar for this reversed language. We very well know how to convert finite automata to regular grammar.

B→ aB|bB|aA
A→ λ

Step 4: In order to get LLG for the language L is to reverse the symbols in RHS of every production rule of right linear grammar.

B→ Ba|Bb|Aa
A→ λ

9.5 Converting to left linear grammar to finite automata

This is exactly a reverse process of conversion from FA to Left linear grammar.

- Step 1: Given the Left linear grammar for Language "L",reverse the symbols on RHS of each production rule in order to get a right linear grammar for reversal of the language.

- Step 2: Construct finite automata for reverse of the language using right linear grammar.
- Step 3: Reverse this finite automata. This automata will now accept the language L.

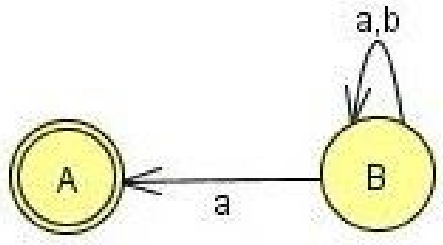
Example 1:

Convert a given left linear grammar to finite automata
Left Linear grammar
B→ Ba|Bb|Aa
A→ λ

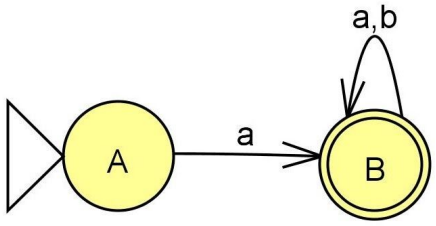
- Step 1: Reverse the symbols on RHS of each production rule in order to get a right linear grammar for reversal of the language.

  B→ aB|bB|aA
  A→ λ

- Step 2: Construct finite automata for reverse of the language using right linear grammar.



- Step 3: Reverse this finite automata. This automata will now accept the language L.

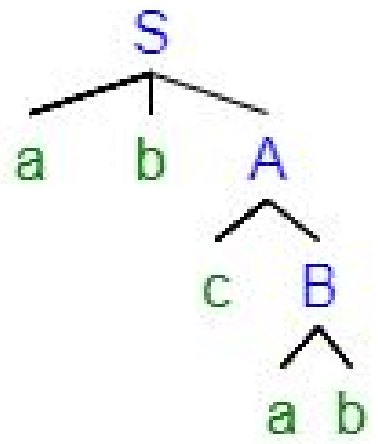


Here ,B is the start state.

9.6 Which one is easier left linear or right linear?

Given the right linear grammar,
S→ abA
A→ cB|aC
B→ ab
C→ b

Parse tree for the string abcab



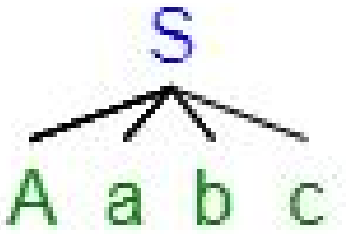
We start with the Start Symbol S. Since there is only one alternative and the first symbol in our string is also a, we will use the production rule S→ abA and expand our parse tree.
So we have successfully got a match for the first two symbols.
The next symbol to be matched is c and we have the non-terminal A.
Therefore we will expand the parse tree using the alternative A → cB.
Next we must match a and the non-terminal to be expanded is B. We use B→ab to expand the tree which will also match the last symbol in our string which is b.

We see the process of derivation of string is easy. We just need to see which input symbol is next and pick the alternative for non-terminal which will match that symbol.
It is quite easy as we scan the string left to right and also derive the string left to right.

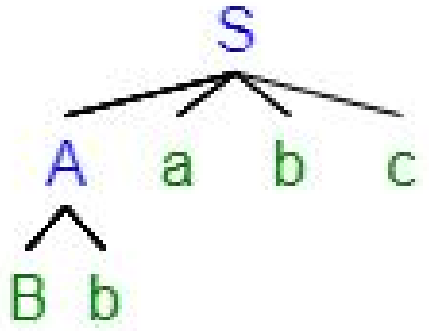
Consider this Left linear grammar
S→ Aabc
A→ Bb|C
B→ a
C→ b

Parse tree for the string ababc

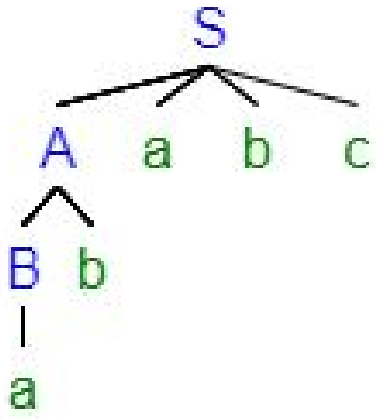
Can the rule S→Aabc recognize the string ababc?
We see that the string is ending with abc and it matches with the last part of the rule.



Now,which rule do we choose to A→Bb or A->C
We choose A->Bb as we see from the input string that the next symbol to be parsed is b
.



Then we choose B->a which completes parsing the string.



This way of parsing although possible is not very intuitive as we need to match the symbols in the string backwards.

Hence right linear form is always preferred over left linear form.

Department of Computer Science and Engineering
PES University, Bangalore, India

UE23CS243A: Automata Formal Language and Logic

Prakash C O, Associate Professor, Department of CSE

Regular Grammar (type-3 grammar):

Why do we care about regular grammars?

Programs are composed of tokens:

- Identifiers
- Literals (Integer literals, floating point literals, character literals, string literals, …)
- Keywords
- Operators and
- Special symbols (i.e., Punctuation symbols, …)

Each of these can be defined by regular grammars.

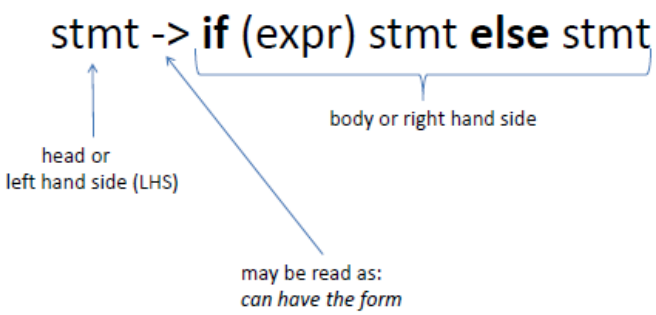
Formal Definition of a Grammar

Formally, a grammar is a 4-tuple G = (V, T, P, S)

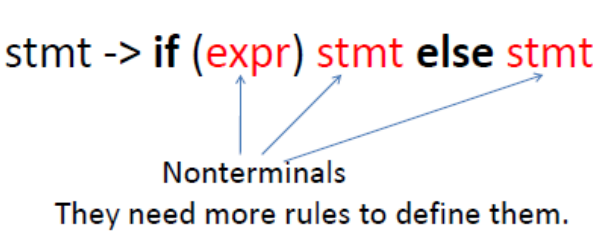
where:

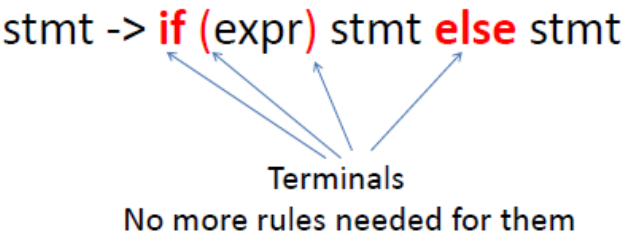
- **V - Set of nonterminals (or variables)**
- **T - Set of terminal symbols**
- **P - Set of productions(or rules)**
  - **The head is nonterminal**
  - **The body is a sequence of teminals and/or nonterminals**
- **S – Start Symbol (Designation of one nonterminal as starting symbol)**

Example:



➢ **Production rules.**





Note: All the variables in your grammar must be reachable from the start symbol of the grammar and all variables must derive something (or end)

Derivation:

A derivation in compiler design is **the successive application of production rules to produce the desired input string.**

- Given the grammar (i.e. productions)
- begin with the start symbol
- repeatedly replacing nonterminal by the body
- We obtain the language string/statement defined by the grammar (i.e. group of terminal strings)

There are two types of derivation in compiler design:

1) **Left-most Derivation**
2) **Right-most Derivation**

Left-most Derivation

The left-most derivation is a method of transforming an input string according to the grammar rules of a programming language. **The leftmost non-terminal is selected at each stage of left-most derivation.**

Grammar:

S → ABC
A → a
B → b
C → c

Input string: abc

Left-most derivation: S ⇒ ABC ⇒ aBC ⇒ abC ⇒ abc

Right-most Derivation

The right-most derivation is a method for transforming an input text depending on the grammar rules of a programming language.

The rightmost non-terminal is selected for expansion at each step, which is regulated by the production rule associated with that non-terminal.

Grammar:

S → ABC
A → a
B → b
C → c

Input string: **abc**

Right-most derivation: **S ⇒ ABC ⇒ ABc ⇒ Abc ⇒ abc**

Parse Tree: Parse Tree is the geometrical representation of a derivation.

- Parse tree is the hierarchical representation of terminals or non-terminals.
- These symbols (terminals or non-terminals) represent the derivation of the grammar to yield input strings.
- In parsing, the string springs using the beginning symbol.
- The starting symbol of the grammar must be used as the root of the Parse Tree.
- Leaves of parse tree represent terminals.
- Each interior node represents non-terminals (or productions) of a grammar.

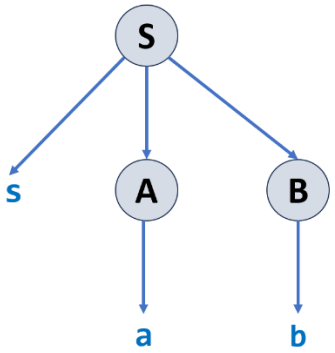
Example 1:

```
S → sAB
A → a
B → b
```

The input string is "**sab**",

Derivation: S ⇒ sAB ⇒ saB ⇒ sab

then the Parse Tree is:



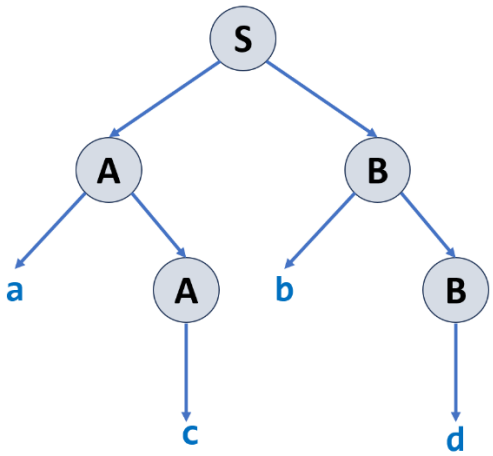
Example-2:

```
S → AB
A → c | aA
B → d | bB
```

The input string is "**acbd**",

Derivation: S ⇒ AB ⇒ aAB ⇒ acB ⇒ acbB ⇒ acbd

then the Parse Tree is as follows:



Sentential Form:

A sentential form is any string consisting of non-terminals and/or terminals that is derived from a start symbol. Therefore, every sentence is a sentential form, but only **sentential forms without non-terminals are called sentences.**

Example: Grammar: **S → aSb | λ**

$$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aaaSbbb \Rightarrow aaabbb$$

Sentential Forms sentence

We write:

$$S \overset{*}{\Rightarrow} aaabbb$$

Instead of:

$$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aaaSbbb \Rightarrow aaabbb$$

Linear Grammar

Grammars with at most one variable (or Nonterminal) at the right hand side of a production.

Example-1:

```
S → aSb
S → λ
```

Example-2:

```
S → Ab
A → aAb
A → λ
```

Non-Linear Grammar

Grammars with more than one variable (or Nonterminal) at the right hand side of a production.

Example:

```
S → SS
```

S → λ
S → aSb
S → bSa

$L(G) = \{ w \mid n_a(w)=n_b(w) \}$

Right-Linear Grammar

- **Right linear grammar:** The non-terminal symbol should be at the right end of the production body. All productions have the form:

  S → wS or S → wA
  S → w A → w

  where *S* and *A* are non-terminals in *V* and *w* is a string of terminals i.e., w ∈ Σ^*
- **Right linear grammar: The grammars, in which all rules are of the form $A \rightarrow w\alpha$ where *w* is a string of terminals and α is either empty or a single nonterminal.**

Examples:

1) S → abS
   S → a

2) S → 00B | 11S
   B → 0B | 1B | 0 | 1

3) S → cS
   S → λ

Left-Linear Grammar

- **Left linear grammar:** The non-terminal symbol should be at the left end of the production body. All productions have the form:

  S → Sw or S → Aw
  S → w A → w

  where *S* and *A* are non-terminals in *V* and *w* is a string of terminals i.e., w ∈ Σ^*
- **Left linear grammar: The grammars, in which all rules are of the form $A \rightarrow \alpha w$ where α is either empty or a single nonterminal and *w* is a string of terminals.**

Example:

S → Aab
A → Aab | B
B → a

Regular Grammar

- **A regular language can be described by a special kind of grammar called regular grammar.**
- **A regular grammar is a grammar that is left-linear or right-linear.**
- **Regular grammars generate regular languages.**

Example: Construct a regular grammar for the language of the regular expression (a|b)* *a*

Regular Grammar is:

S → aS
S → bS
} or S → aS | bS | a

$S \rightarrow a$

Does this sentence (or string) **baaba** conform to the written grammar. YES

Derivation: $S \Rightarrow bS \Rightarrow baS \Rightarrow baaS \Rightarrow baabS \Rightarrow baaba$

Construct a Regular Grammar for the given language description.

Hint: Easy way is to construct NFA(or λ-NFA with state names as uppercase letters) and then writing the regular grammar.

1	**L = {a}**
2	**L = {ab}**
3	**L = { w | w ∈ {a, b}* and |w| = 2 }**
4	**L = { w | w ∈ {a, b}* and |w| ≤ 2 }**
5	**L = {aaaa}**
6	**L = {a, b}**
7	**L = {ab, ba}**
8	**L = { λ, a, aa, aaa, … }**
9	**L = { a, aa, aaa, … }**
10	**L = { λ, ab, abab, … }**
11	**L = { λ, a, b, ab, ba, aaa, bbb, bba, bab, abb, aab, aba, baa, … }**
12	**L = { a, b, ab, ba, aaa, bbb, bba, bab, abb, aab, aba, baa, … }**
13	**Strings with zero or more a's only and Strings with zero or more b's only.**
14	**Strings with one or more a's only and Strings with one or more b's only.**
15	**Strings begins with zero or more a's followed by single b.**
16	**Strings begins with a and followed by zero or more b's.**
17	**L={ a^{2n} | n≥0 }**
17a	**L={ a^{2n} | n≥1 }**
18	**L={ a^{2n+1} | n≥0 }**
19	**L={ a^{4n} | n≥0 }**
20	**L = { w_1aw_2 | $w_1 \in \{b\}^*$ and $w_2 \in \{a, b\}^*$}**
21	**L={ $a^m b^n$ | m≥0 and n>0 }**
22	**L={ $a^m b^n$ | m>0 and n≥0 }**
23	**L={ $a^m b^n$ | m>0 and n>0 }**
24	**L={ $a^m b^n$ | m≥0 and n≥0}** L = { λ, a, aa, aaa, … , b, bb, bbb, bbbb, … , ab, aab, abb, abbb, aabb, aaab, … }
25	**0 or more a's, followed by 0 or more b's, followed by 0 or more c's. L = {ε, a, b, c, aa, ab, ac, bb, bc, cc, aaa, …}**
26	**L={ $a^m b^n$ | m>0 or n>0 }**
27	**Strings ending with abb. L = {abb, aabb, babb, aaabb, ababb, baabb, bbabb, …}**
28	**Strings starting with ab. L = {ab, aba, abb, abaa, abab, abba, abbb, …}**

29	Strings that contains aa. L = {aa, aaa, baa, aab, ...}
30	1 or more a's, followed by 1 or more b's, followed by 1 or more c's. L = {abc, aabc, abbc, abcc, aabbc, aabcc, abbcc, ...}
31	Strings that end with a or bb. L = {a, bb, aa, abb, ba, bbb, ...}
32	Strings with even number of a's followed by odd number of b's. L = {b, aab, bbb, aabbb, ...}
33	Binary strings ending with 3 0's. L = {000, 0000, 1000, 00000, 01000, 10000, 11000, ...}
34	Strings with even number of 1's. L = {ε, 11, 1111, 111111, ...}

Solutions:

Sl. No.	Language	Regex	Regular Grammar
1	L = {a}	a	S → a
2	L = {ab}	ab	S → ab
3	L = { w | w ∈ {a, b}* and |w| = 2 }	(a|b)(a|b) or [ab][ab]	S → aA | bA A → a | b
4	L = { w | w ∈ {a, b}* and |w| ≤ 2 }	(a|b)? (a|b)? or (ε|a|b)(ε|a|b)	S → aA | bA | λ A → a | b | λ
5	L = {aaaa}	aaaa	S → aaaa
6	L = {a, b}	a|b	S → a | b
7	L = {ab, ba}	ab|ba	S → ab | ba
8	L = { λ, a, aa, aaa, ... }	a*	S → aS | λ
9	L = { a, aa, aaa, ... }	a^+	S → aS | a
10	L = { λ, ab, abab, ... }	(ab)*	S → abS | λ
11	L = { λ, a, b, ab, ba, aaa, bbb, bba, bab, abb, aab, aba, baa, ... }	(a|b)*	S → aS | bS | λ
12	L = { a, b, ab, ba, aaa, bbb, bba, bab, abb, aab, aba, baa, ... }	$(a\|b)^+$	S → aS | bS | a | b
13	Strings with zero or more a's only and Strings with zero or more b's only.	a*|b*	S → A | B A → aA | λ B → bB | λ or S → aA | bB | λ A → aA | λ B → bB | λ
14	Strings with one or more a's only and Strings	$a^+\|b^+$	S → A | B

	with one or more b's only.	or aa*|bb*	A → aA | a B → bB | b or S → aA | bB A → aA | λ B → bB | λ
15	Strings begins with zero or more a's and ends with single b.	a*b	S → aS | b or S → Ab A → Aa | λ
16	Strings begins with a and followed by zero or more b's.	ab*	S → aB B → bB | λ or S → a | Sb
17	L={ a^{2n} | n≥0 }	(aa)*	S → aA | λ A → aS or S → aaS | λ
17a	L={ a^{2n} | n≥1 }	(aa)+	S → aA A → aS | a or S → aaS | aa
18	L={ a^{2n+1} | n≥0 }	(aa)*a	S → aaS | a or S → aA | a A → aS or S → aA A → aS | λ
19	L={ a^{4n} | n≥0 }	(aaaa)*	S → aaaaS | λ or S → aA | λ A → aB B → aA A → aS
20	L = { w_1aw_2 | $w_1 \in \{b\}^*$ and $w_2 \in \{a, b\}^*$}	b*a(a|b)*	S → bS | aA | a A → aA | bA | λ
21	L={ a^mb^n | m≥0 and n>0 } L = { b, bb, ab, aab, abb, bbb, ...}	a*bb* or a^*b^+	S → aS | b | bB B → bB | λ
22	L={ a^mb^n | m>0 and n≥0 } L = { a, aa, ab, aab, abb, aaa, ...}	aa*b* or a^+b^*	S → aA A → aA | λ | bB B → bB | λ

23	L={ $a^m b^n$ | m>0 and n>0 }	aa*bb* or a^+b^+	S → aA A → aA | bB B → λ | bB
24	L={ $a^m b^n$ | m≥0 and n≥0} L = { λ, a, aa, aaa, … , b, bb, bbb, bbbb, … , ab, aab, abb, abbb, aabb, aaab, … }	a*b*	S → aS | bB | λ B → bB | λ
25	0 or more a's, followed by 0 or more b's, followed by 0 or more c's. L = {ε, a, b, c, aa, ab, ac, ba, bb, bc, ca, cb, cc, aaa, …}	a*b*c*	S → aS | bB | cC | ε B → bB | cC | ε C → cC | ε
26	L={ $a^m b^n$ | m>0 or n>0 }	aa*b*| a*b*b or a^+b^*| a*b^+	S → aS | aA | bA A → bA| λ
27	Strings ending with abb. L = {abb, aabb, babb, aaabb, ababb, baabb, bbabb, …}	(a|b)*abb	S → aS | bS | aB B → bA A → b
28	Strings starting with ab. L = {ab, aba, abb, abaa, abab, abba, abbb, …}	ab(a|b)*	S → aB B → bA A → aA | bA | ε
29	Strings that contains aa. L = {aa, aaa, baa, aab, …}	(a|b)*aa(a|b)*	S → aS | bS | aA A → aB B → aB | bB | ε
30	1 or more a's, followed by 1 or more b's, followed by 1 or more c's. L = {abc, aabc, abbc, abcc, aabbc, aabcc, abbcc, …}	$a^+b^+c^+$ or aa*bb*cc*	S → aA A → aA | bB B → bB | cC C → cC | ε
31	Strings that end with a or bb. L = {a, bb, aa, abb, ba, bbb, …}	(a|b)*(a|bb)	S → aS | bS | a | bB B → b
32	Strings with even number of a's followed by odd number of b's. L = {b, aab, bbb, aabbb, …}	(aa)*(bb)*b	S → aA | bB | b A → aC C → aA | bB | ε B → bD | ε D → bB
33	Binary strings ending with 3 0's. L = {000, 0000, 1000, 00000, 01000, 10000, 11000, …}	(0+1)*000	S → 0S | 1S | 0A A → 0B B → 0
34	Strings with even number of 1's. L = {ε, 11, 1111, 111111, …}	(11)*	S → 1A | ε A → 1S
35	Strings with atmost 3 a's and Σ ∈ {a, b}	b*a?b*| b*ab*ab*| b*ab*ab*ab*	S → bS | aA | λ A → bA | aB | λ B → bB | aC| λ C → bC | λ

Note:

Every regular language can be specified by a finite state automaton, a regular expression, or a regular grammar. Furthermore, all of these specifications are equivalent in the sense that they all capture the class of regular languages.

Finite State Automaton	**Recognizer**	Determines whether a particular input string in the regular language is recognized by the FSA or not.
Regular Expression	**Expresser**	Expresses a regular language as a pattern to which every string in the regular language conforms.
Regular Grammar	**Generator**	Provides grammar rules for generating strings in the regular language

Note: **Regular grammar is a formal specification of a regular language by way of grammar rules.**

Exercises:

Construct a Regular Grammar for the given language description and Regular expression.

1. Strings of a's and b's of length 2. Regex = aa | ab | ba | bb OR (a|b)(a|b)

2. Strings of a's and b's of length ≤ 2. Regex = ε|a|b|aa|ab|ba|bb OR (ε | a | b)(ε | a | b) OR (a|b)? (a|b)?

3. Strings of a's and b's of length ≤ 5. Regex = $(\varepsilon \mid a \mid b)^5$

4. Even-lengthed strings of a's and b's. Regex = (aa | ab | ba | bb)* OR ((a|b)(a|b))*

5. Odd-lengthed strings of a's and b's. Regex = (a|b) ((a|b)(a|b))*

6. L(R) = { w : w ∈ {0,1}* with at least three consecutive 0's } Regex = (0|1)* 000 (0|1)*

7. Strings of 0's and 1's with no two consecutive 0's. Regex = (1 | 01)* (0 | ε)

8. Strings of a's and b's starting with a and ending with b. Regex = a (a|b)* b

9. Strings of a's and b's whose second last symbol is a. Regex = (a|b)* a (a|b)

10. Strings of a's and b's whose third last symbol is a and fourth last symbol is b.

Regex = (a|b)* b a (a|b) (a|b)

11. Strings of a's and b's whose first and last symbols are the same. Regex = (a (a|b)* a) | (b (a|b)* a)

12. Strings of a's and b's whose first and last symbols are different. Regex = (a (a|b)* b) | (b (a|b)* a)

13. Strings of a's and b's whose last and second last symbols are same. Regex = (a|b)* (aa | bb)

14. Strings of a's and b's whose length is even or a multiple of 3 or both.

Regex = R1 | R2 where R1 = ((a|b)(a|b))* and R2 = ((a|b)(a|b)(a|b))*

15. Strings of a's and b's such that every block of 4 consecutive symbols has at least 2 a's.

Regex = (aaxx | axax | axxa | xaax | xaxa | xxaa)* where x = (a|b)

16. L = $\{a^n b^m : n \geq 0, m \geq 0\}$ Regex = a* b*

17. $L = \{a^n b^m : n > 0, m > 0\}$ Regex = aa* bb* OR a^+b^+

18. $L = \{a^n b^m : n \mid m \text{ is even}\}$ Regex =aa* bb* | a(aa)* b(bb)*

19. $L = \{a^{2n} b^{2m} : n \geq 0, m \geq 0\}$ Regex = (aa)* (bb)*

20. Strings of a's and b's containing not more than three a's. Regex = b* (ε | a) b* (ε | a) b* (ε | a) b*

21. $L = \{a^n b^m : n \geq 3, m \leq 3\}$ Regex =aaa a* (ε | b) (ε | b) (ε | b)

22. $L = \{ w : |w| \bmod 3 = 0 \text{ and } w \in \{a,b\}^* \}$ Regex = ((a|b)(a|b)(a|b))*

23. $L = \{ w : n_a(w) \bmod 3 = 0 \text{ and } w \in \{a,b\}^* \}$ Regex =b* a b* a b* a b*

24. Strings of 0's and 1's that do not end with 01. Regex = (0|1)* (00 | 10 | 11)

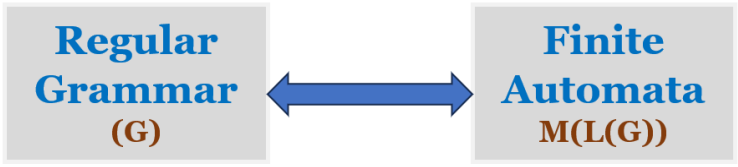
25. $L = \{ vuv : u, v \in \{a,b\}^* \text{ and } |v| = 2\}$ Regex = (aa | ab| ba | bb) (a|b)* (aa | ab | ba | bb)

26. Strings of a's and b's that end with ab or ba. Regex = (a|b)* (ab | ba)

27. $L = \{a^n b^m : m,n \geq 1 \text{ and } mn \geq 3\}$ Regex = a bbb b* | aaa a* b | aa a* bb b*

Equivalence of Regular Grammar and Finite Automata

The relationship of regular grammar and finite automata is shown below:



If G is a regular grammar, then L(G) is a regular language accepted by Finite automata.

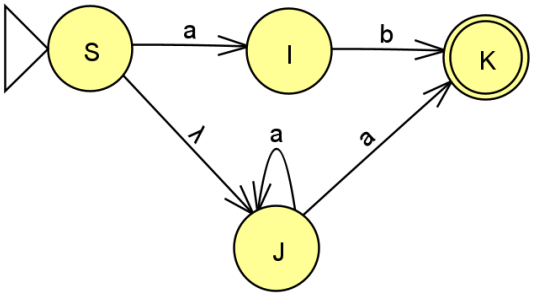
Converting Finite Automata to Regular Grammars

Algorithm: Finite Automata to Regular Grammar

Perform the following steps to construct a regular grammar that generates the language of a given Finite automata:

1. Rename the states to a set of uppercase letters (if the state names are named with $q_0,q_1,q_2 \ldots$ or 1,2,3,...)
2. The start symbol of the grammar is the Finite Machines start state.
3. For each state transition from I to J labelled with a, create the production **I → aJ**
4. For each state transition from I to J labelled with λ, create the production **I → J**
5. For each final state K, create a null production **K → λ**

Example: Convert the following finite automata to regular grammar.

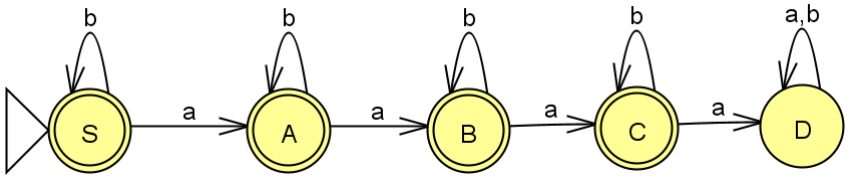


Regular Grammar:

- **S → aI**
- **S → J**
- **I → bK**
- **J → aJ**
- **J → aK**
- **K → λ**

Exercise:

1. Convert the following finite automata to regular grammar for the language to accept at most 3 'a's over the alphabet Σ={a, b}.



Solution:

S → bS | aA | λ
A → bA | aB | λ
B → bB | aC| λ
C → bC| aD | λ

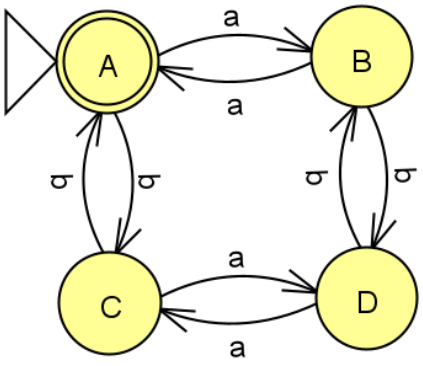
State D is a trap (or dead) state and its rules are

D → aD | bD **(Not used for string derivation, it's an useless production)**

Note: The start symbol of the grammar is S, because the start state is S.

2. Convert the following finite automata to regular grammar.

Where L = { n_a(w)mod 2=0 and **n_b(w)mod 2=0}**



Solution:

A → aB | bC | λ

B → aA | bD

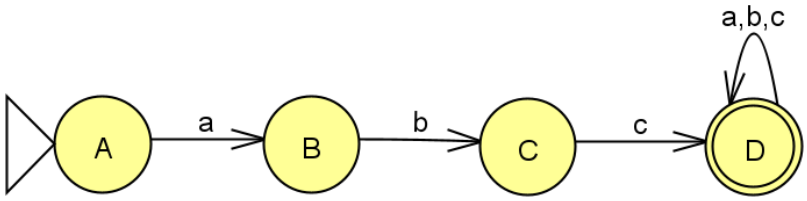
C → aD | bA

D → aC | bB

Note: The start symbol of the grammar is A, because the start state is A.

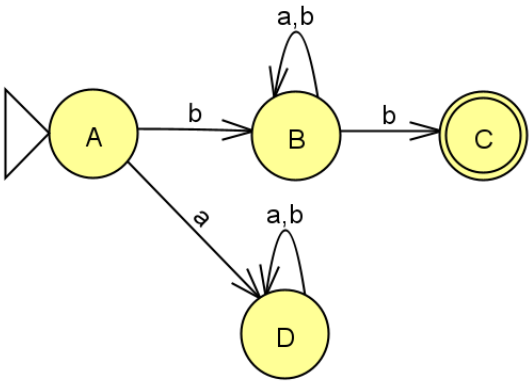
3. Convert the following finite automata to regular grammar.

Where L = { abcw | where w∈{a,b,c}*}



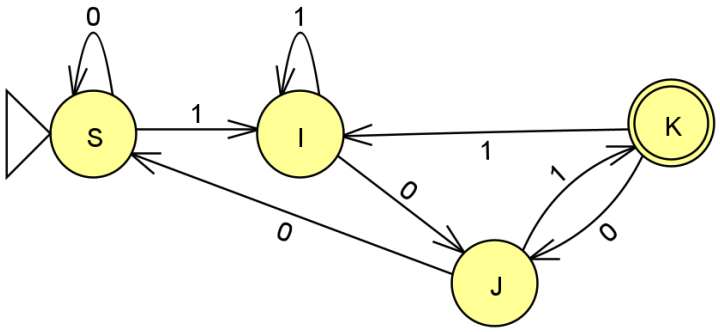
4. Convert the following finite automata to regular grammar.

Where L = { bwb | where w∈{a,b}*}



5. Convert the following finite automata to regular grammar.

Where language contains binary strings ends with 101.



Converting Regular Grammars to Finite Automata

Algorithm-01: Regular Grammar to Finite Automata

Perform the following steps to construct an Automata that accepts the language of a given regular grammar:

1. If necessary, transform the grammar so that all productions have the form A→x or A → xB, where x is either a single letter (i.e., terminal symbol) or λ
2. The start state of the Automata is the grammar's start symbol.
3. For each production I → aJ, construct a state transition from I to J labelled with the letter a.
4. For each production I → J, construct a state transition from I to J labelled with λ.
5. If there are productions of the form I → a for some letter a, then create a single new state symbol F. For each production I → a, construct a state transition from I to F labelled with a.
6. If there is a production of the form I → λ or I → ε (i.e., Null production), then state I is a final state.

Examples:

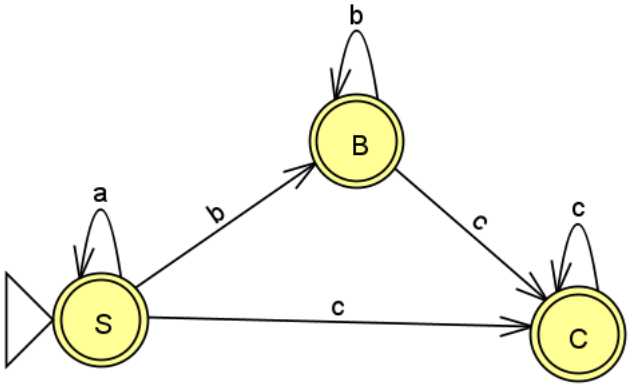
1) Convert the following regular grammar to finite automata.

S → aS | bB | cC | ε

B → bB | cC | ε

C → cC | ε

Solution:



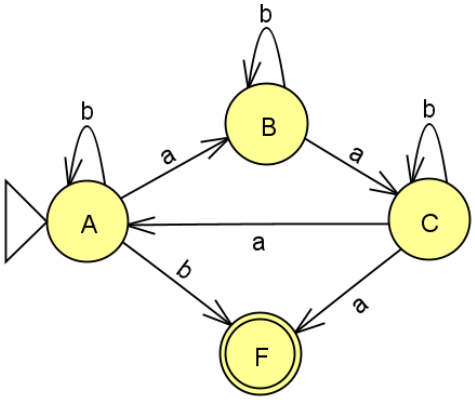
2) Convert the following regular grammar to finite automata.

A → aB | bA | b

B → aC | bB

C → aA | bC | a

Solution:



3) Convert the following regular grammar to finite automata.

S → bS | aA

A → aA | aB |bA

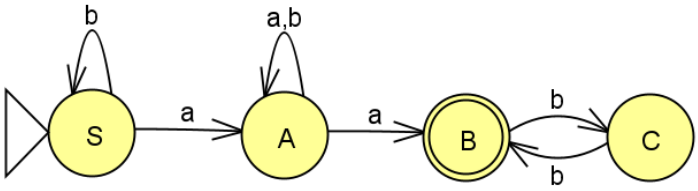
B → bbB

B → λ

Solution:

Transform the given grammar so that all productions have the form A→x or A → xB, where x is either a single letter (i.e., terminal symbol) or λ

S→bS | aA
A→aA | aB | bA
B→bC
C→bB
B→λ



Algorithm-02: Regular Grammar to Automata

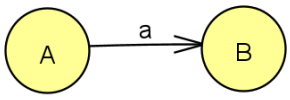
Assume that a regular grammar is given in its right-linear form, this grammar may be easily converted to a Automata. A right-linear grammar, defined by G = (V, T, P, S), may be converted to a automata, defined by M = (Q, Σ, δ, q0, F) by:

1. Create a state for each variable.

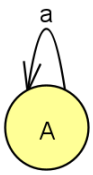
2. Convert each production rule into a transition.

a) If the production rule is of the form $\mathbf{V_i \rightarrow aV_j}$, where a ∈ T, add the transition $\mathbf{\delta(V_i, a) = V_j}$ to automata M.

i) For example, **A → aB** becomes:

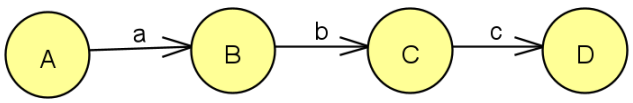


ii) For example, **A → aA** becomes:



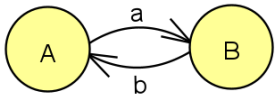
b) If the production rule is of the form $\mathbf{V_i \rightarrow wV_j}$, where w ∈ T*, create a series of states which derive w and end in $\mathbf{V_j}$. Add the states in between to set Q.

i) For example, **A → abcD** becomes:



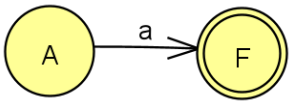
Note: **For intermediate states, give new names (i.e., names that are not there in the Non-terminals list V of the given grammar).**

ii) For example, **A → abA** becomes:

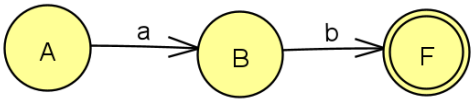


c) If the production rule is of the form $\mathbf{V_i \rightarrow w}$, where w ∈ T*, create a series of states which derive w and end in a final state.

i) For example, **A → a** becomes:



ii) For example, **A → ab** becomes:



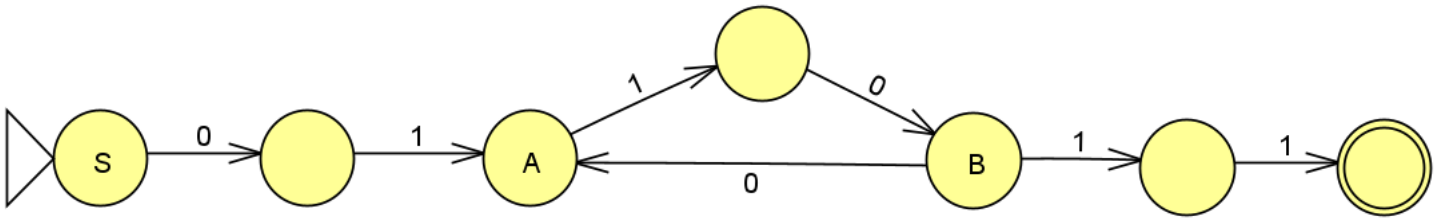
d) If the production rule is of the form $\mathbf{V_i \rightarrow \lambda}$ **or** $\mathbf{V_i \rightarrow \varepsilon}$ (i.e., Null production), then **state $\mathbf{V_i}$ is a final state.**

Examples:

1) Convert the following regular grammar to Automata.

S→01A
A→10B
B→0A|11

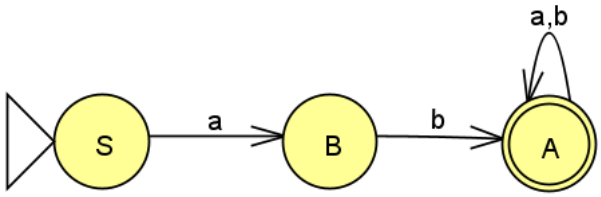
Solution:



2) Convert the following regular grammar to Automata.

S → aB
B → bA
A → aA | bA | ε

Solution:



Reverse of a Regular Language

If L is a regular language, then L^R is also regular

- Let L be a regular language, then there exists an automata M such that L = L(M).
  M = (Q, ∑, δ, q_0, F) (where machine M has a single final state)
- Construct a new machine M^R (i.e., L^R=L(M^R)) by toggling initial and final state and by reversing the arrows (i.e., swapping initial and final state and changing the directions of the edges).
- The reverse of a regular language i.e., L^R is the language accepted by automata M^R.

Converting Right Linear Grammar to Left Linear Grammar

If G is a Right Linear Grammar, then there is a Left Linear Grammar G'' such that L(G) = L(G'')

Conversion outline:

a) From G, construct M
b) From M construct M^R for L^R
c) Generate G' a Right Linear Grammar for L^R
d) Generate G'' a Left Linear Grammar for $(L^R)^R$ =L (i.e., getting G'' from G' by reversing all symbols on RHS of productions)

Example:

1) Convert the following Right Linear Grammar to Left Linear Grammar

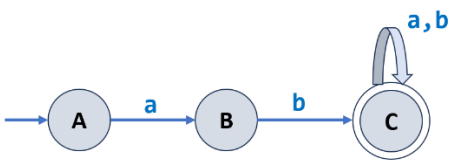
The Right Linear Grammar G is:

A → aB
B → bC
C → aC | bC | ε

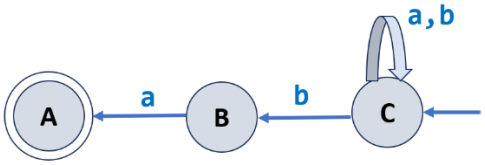
Where L(G) = { abw | w ∈ {a,b}*}

Solution:

a) From G, construct automata M



b) From M construct M^R for L^R



c) Generate G' a Right Linear Grammar from M^R for L^R

C → aC | bC | bB
B → aA
A → ε

d) Generate G'' a Left Linear Grammar for $(L^R)^R = L$ (i.e., getting G'' from G' by reversing all symbols on RHS of productions)

C → Ca | Cb | Bb
B → Aa
A → ε

Note: L(G) = L(G'')

Converting Left Linear Grammar to Right Linear Grammar

If G is a Left Linear Grammar, then there is a Right Linear Grammar G'' such that L(G) = L(G'')

Conversion outline:

a) From G, Generate G' a Right Linear Grammar for L^R (i.e., getting G' from G by reversing all symbols on RHS of productions)
b) From G' construct M for L^R
c) Construct M^R from M for $(L^R)^R$
d) Generate G'' a right Linear Grammar for $(L^R)^R = L$

Example:

1) Convert the following Left Linear Grammar to Right Linear Grammar

Left Linear Grammar G is:

C → Ca | Cb | Bb
B → Aa
A → ε

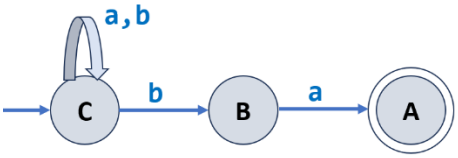
Where L(G) = { abw | w ∈ {a,b}*}

Solution:

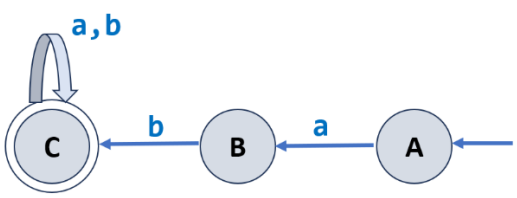
a) From G, Generate G' a Right Linear Grammar for L^R (i.e., getting G' from G by reversing all symbols on RHS of productions)

C → aC | bC | bB
B → aA
A → ε

b) From G' construct M for L^R



c) Construct M^R from M for $(L^R)^R$



d) Generate G'' a right Linear Grammar for $(L^R)^R = L$

A → aB
B → bC
C → aC | bC | ε

-------***-------

AUTOMATA FORMAL LANGUAGES AND LOGIC

Lecture notes on equivalence of Regular Grammar & Finite Automata

Prepared by:

Prof.Sangeeta V I

Assistant Professor

Department of Computer Science & Engineering

PES UNIVERSITY

(Established under Karnataka Act No.16 of 2013)

100-ft Ring Road, BSK III Stage, Bangalore – 560 085

Table of contents

Examples Solved:

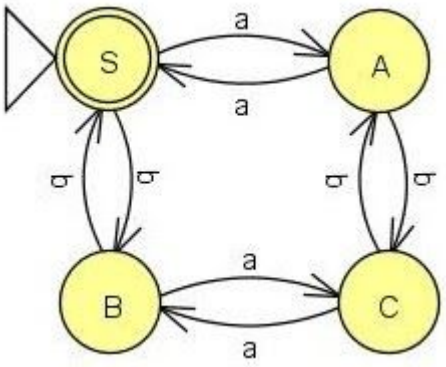
Examples Solved:

1. Conversion from Finite automata to regular grammar.

Regular grammar and Finite Automata are equivalent in power.

Example 1:

Converting a given finite automata accepting L={n_a(w)mod 2=0 and n_b(w)mod 2=0 } to regular grammar.



L={even number of a's and b's}

Start state of automata will be the start symbol of the grammar.

We start with S,S on seeing terminal a it moves to state A(S→ aA) and on seeing terminal b it moves to state B (S→ bB).

Since S is also the final state,we introduce the production S→ λ.

S→ aA|bB| λ

We will repeat the same for other states and terminal symbols.

A→ aS|bC B→ aC|bS C→ aB|bA

So the grammar is,

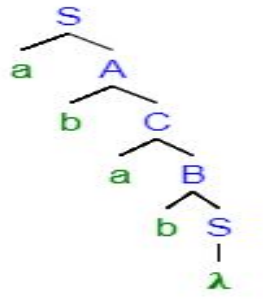
S→ aA|bB| λ

A→ aS|bC

B→ aC|bS

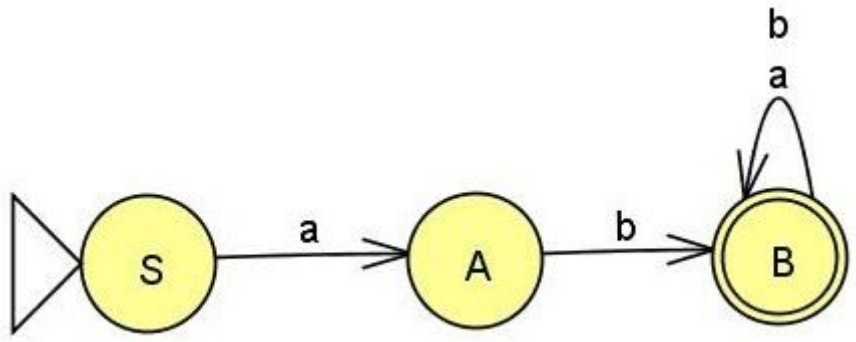
C→ aB|bA

Parse tree for the string abab.



Example 2:

Converting a given finite automata accepting L={abw,w∈{a,b}*} to regular grammar.



Start state of automata will be the start symbol of the grammar.

State S on seeing terminal a it goes to state A. So we introduce the production S→ aA

State A on seeing terminal a it goes to state B. So we introduce the production S→ bB

State B on seeing terminal a,b remains in state B. So we introduce the production B→ aB|bB.

Any final state we introduce the production B→ λ.

So the grammar is,

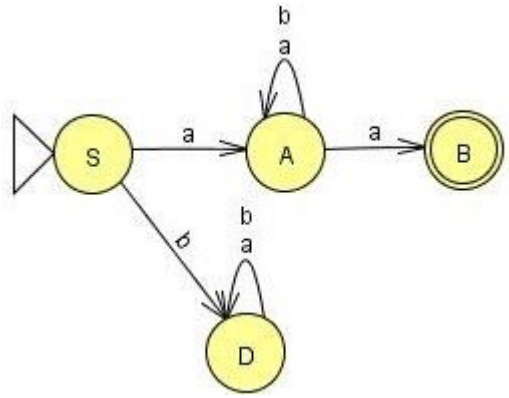
S→ aA

A→ bB

B→ aB|bB|λ

Example 3:

Converting given finite automata accepting L={awa,w∈{a,b}*} to regular grammar.



Start state of automata will be the start symbol of the grammar.

State S on seeing terminal a it goes to state A. So we introduce the production S→ aA

State A on seeing terminal a,b remains in state A. So ,we introduce the production

A→ aA|bA.

State A on seeing terminal a it goes to state B. So we introduce the production A→ aB

Since B is the final state we introduce the production B→ λ**.**

We will not encode the production to state D ,as D is a dead state and it will never lead us to the terminal.It is an useless production.

So the grammar is,

S→ aA

A→ aA|bA|aB

B→ λ

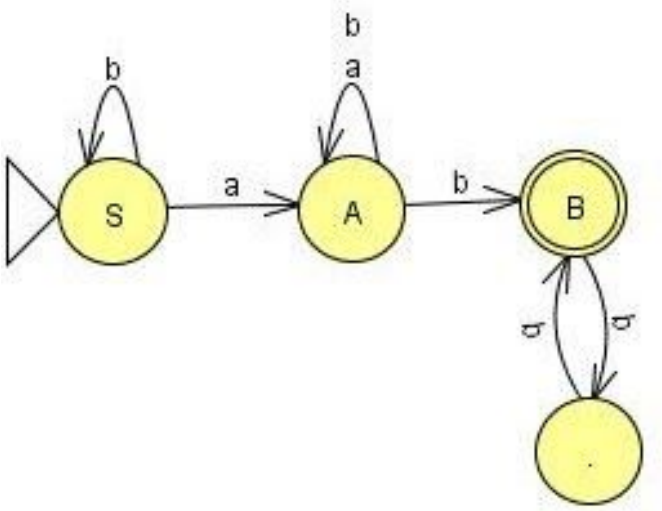
2. Converting Regular grammar to finite automata.

Example 1:

Convert given regular grammar to finite automata.

S→ bS|aA
A→ aA|bA|aB
B→ bbB| λ

State S on seeing b remains in S,we will have a self loop on S.
State S on seeing a moves to state A.
A on seeing a and b remains in state A ,we will have a self loop on A.
A on seeing a also moves to state B.
B loops on bb,we introduce a new state and loop on bb.
B→ λ indicates B is a final state.



Example 2:

Convert given regular grammar to finite automata.

S→ 01A

A→ 10B

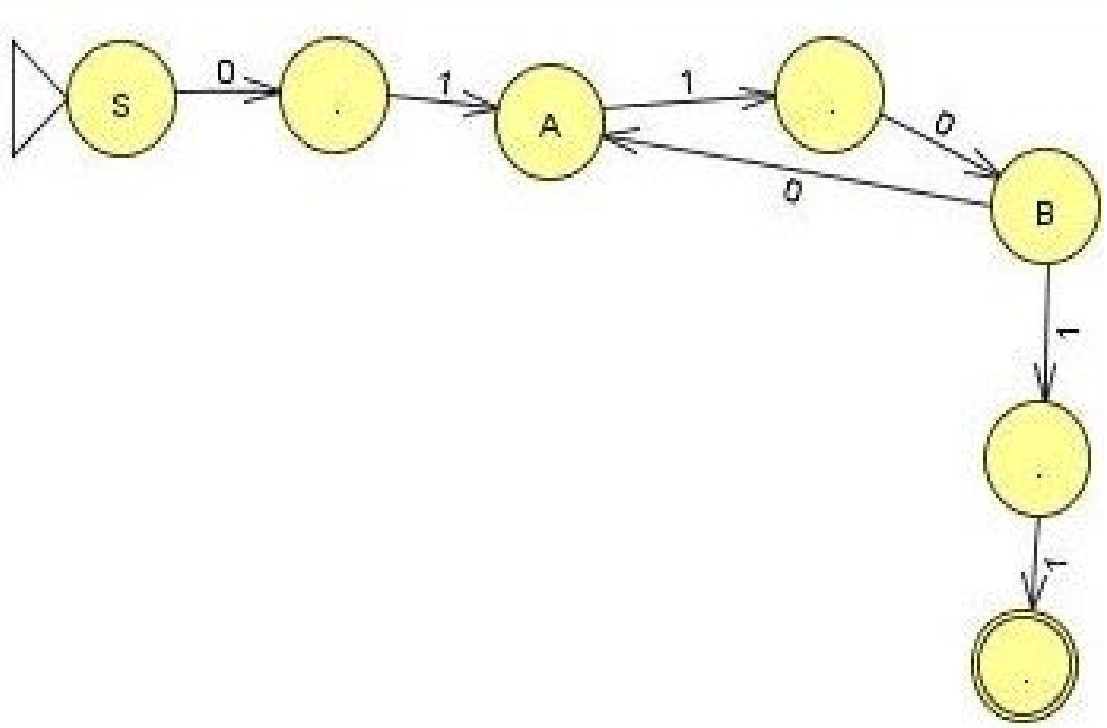
B→ 0A|11

State S on seeing 01 is moving to state A.

State A on seeing 10 is moving to state B.

State B on seeing 0 moves to state A.

B ends with 11.



Example 3:

Convert given regular grammar to finite automata.

A→ aB|bA|b

B→ aC|bB

C→ aA|bC|a

State A seeing terminal a is moving to state B.

State A seeing terminal b remains in state B.

State B seeing terminal a is moving to state C.

State B seeing terminal b remains in state B.

State A accepts only b as well.

State C on a moves to final state.